```
In [2]: numbers = np.arange(1,   6)

In [3]: numbers
Out[3]: array([1, 2, 3, 4, 5])

In [4]: numbers * 2
Out[4]: array([ 2,  4,  6,  8, 10])

In [5]: numbers ** 3
Out[5]: array([  1,   8,  27,  64, 125])

In [6]: numbers  # numbers is   unchanged by the arithmetic operators
Out[6]: array([1, 2, 3, 4, 5])
```

Snippet `[6]` shows that the arithmetic operators did not modify `numbers`. Operators `+` and `*` are *commutative*, so snippet `[4]` could also be written as `2 * numbers`.

Augmented assignments *modify* every element in the left operand.

lick here to view code image

```
In [7]: numbers += 10

In [8]: numbers
Out[8]: array([11, 12, 13, 14, 15])
```

Broadcasting

Normally, the arithmetic operations require as operands two `array`s of the *same size and shape*. When one operand is a single value, called a **scalar**, NumPy performs the element-wise calculations as if the scalar were an `array` of the same shape as the other operand, but with the scalar value in all its elements. This is called **broadcasting**. Snippets `[4]`, `[5]` and `[7]` each use this capability. For example, snippet `[4]` is equivalent to:

lick here to view code image

```
numbers * [2, 2, 2, 2, 2]
```

Broadcasting also can be applied between `array`s of different sizes and shapes, enabling some concise and powerful manipulations. We'll show more examples of broadcasting later in the chapter when we introduce NumPy's universal functions.

C

Arithmetic Operations Between `array`s

You may perform arithmetic operations and augmented assignments between `array`s of the *same* shape. Let's multiply the one-dimensional `array`s `numbers` and `numbers2` (created below) that each contain five elements:

lick here to view code image

```
In [9]: numbers2 = np.linspace(1.1,   5.5, 5)

In [10]: numbers2
Out[10]: array([ 1.1,  2.2,  3.3,  4.4,  5.5])

In [11]: numbers * numbers2
Out[11]: array([ 12.1,  26.4,  42.9,  61.6,  82.5])
```

The result is a new array formed by multiplying the `array`s *element-wise* in each operand—`11 * 1.1`, `12 * 2.2`, `13 * 3.3`, etc. Arithmetic between `array`s of integers and floating-point numbers results in an `array` of floating-point numbers.

Comparing `array`s

You can compare `array`s with individual values and with other `array`s. Comparisons are performed *element-wise*. Such comparisons produce `array`s of Boolean values in which each element's `True` or `False` value indicates the comparison result:

lick here to view code image

```
In [12]: numbers
Out[12]: array([11, 12, 13, 14, 15])

In [13]: numbers >= 13
Out[13]: array([False, False,  True,  True,  True])

In [14]: numbers2
Out[14]: array([ 1.1,  2.2,  3.3,  4.4,  5.5])

In [15]: numbers2 < numbers
Out[15]: array([ True,  True,  True,  True,  True])

In [16]: numbers == numbers2
Out[16]: array([False, False, False, False, False])

In [17]: numbers == numbers
Out[17]: array([ True,  True,  True,  True,  True])
```

C

Snippet `[13]` uses broadcasting to determine whether each element of `numbers` is greater than or equal to `13`. The remaining snippets compare the corresponding elements of each `array` operand.

7.8 NUMPY CALCULATION METHODS

An `array` has various methods that perform calculations using its contents. By default, these methods ignore the `array`'s shape and use *all* the elements in the calculations. For example, calculating the mean of an `array` totals all of its elements regardless of its shape, then divides by the total number of elements. You can perform these calculations on each dimension as well. For example, in a two-dimensional array, you can calculate each row's mean and each column's mean.

Consider an `array` representing four students' grades on three exams:

lick here to view code image

```
In [1]: import numpy as np

In [2]: grades = np.array([[87,  96, 70], [100, 87, 90],
   ...:                    [94,  77, 90], [100, 81, 82]])
   ...:

In [3]: grades
Out[3]:
array([[ 87,  96,  70],
       [100,  87,  90],
       [ 94,  77,  90],
       [100,  81,  82]])
```

We can use methods to calculate **`sum`**, **`min`**, **`max`**, **`mean`**, **`std`** (standard deviation) and **`var`** (variance)—each is a functional-style programming *reduction*:

lick here to view code image

```
In [4]: grades.sum()
Out[4]: 1054

In [5]: grades.min()
Out[5]: 70

In [6]: grades.max()
Out[6]: 100

In [7]: grades.mean()
```

C

```
Out[7]: 87.83333333333333

In [8]: grades.std()
Out[8]: 8.792357792739987

In [9]: grades.var()
Out[9]: 77.30555555555556
```

Calculations by Row or Column

Many calculation methods can be performed on specific `array` dimensions, known as the `array`'s *axes*. These methods receive an `axis` keyword argument that specifies which dimension to use in the calculation, giving you a quick way to perform calculations by row or column in a two-dimensional `array`.

Assume that you want to calculate the average grade on each *exam*, represented by the columns of `grades`. Specifying `axis=0` performs the calculation on all the *row* values within each column:

lick here to view code image

```
In [10]: grades.mean(axis=0)
Out[10]: array([95.25, 85.25, 83.  ])
```

So `95.25` above is the average of the first column's grades (`87`, `100`, `94` and `100`), `85.25` is the average of the second column's grades (`96`, `87`, `77` and `81`) and `83` is the average of the third column's grades (`70`, `90`, `90` and `82`). Again, NumPy does *not* display trailing `0`s to the right of the decimal point in `'83.'`. Also note that it *does* display all element values in the same field width, which is why `'83.'` is followed by two spaces.

Similarly, specifying `axis=1` performs the calculation on all the *column* values within each individual row. To calculate each student's average grade for all exams, we can use:

lick here to view code image

```
In [11]: grades.mean(axis=1)
Out[11]: array([84.33333333, 92.33333333, 87.        ,  87.66666667])
```

This produces four averages—one each for the values in each row. So `84.33333333` is

C

the average of row 0's grades (87, 96 and 70), and the other averages are for the remaining rows.

NumPy arrays have many more calculation methods. For the complete list, see

```
ttps://docs.scipy.org/doc/numpy/reference/arrays.ndarray.html
```

7.9 UNIVERSAL FUNCTIONS

NumPy offers dozens of standalone **universal functions** (or **ufuncs**) that perform various element-wise operations. Each performs its task using one or two array or array-like (such as lists) arguments. Some of these functions are called when you use operators like + and * on arrays. Each returns a new array containing the results.

Let's create an array and calculate the square root of its values, using the **sqrt universal function**:

lick here to view code image

```
In [1]: import numpy as np

In [2]: numbers = np.array([1,   4, 9, 16, 25, 36])

In [3]: np.sqrt(numbers)
Out[3]: array([1., 2., 3., 4., 5., 6.])
```

Let's add two arrays with the same shape, using the **add universal function:**

lick here to view code image

```
In [4]: numbers2 = np.arange(1,   7) * 10

In [5]: numbers2
Out[5]: array([10, 20, 30, 40, 50, 60])

In [6]: np.add(numbers, numbers2)
Out[6]: array([11, 24, 39, 56, 75, 96])
```

The expression np.add(numbers, numbers2) is equivalent to:

```
numbers + numbers2
```

€

Broadcasting with Universal Functions

Let's use the **`multiply` universal function** to multiply every element of `numbers2` by the scalar value `5`:

lick here to view code image

```
In [7]: np.multiply(numbers2, 5)
Out[7]: array([ 50, 100, 150, 200, 250, 300])
```

The expression `np.multiply(numbers2, 5)` is equivalent to:

```
numbers2 * 5
```

Let's reshape `numbers2` into a 2-by-3 `array`, then multiply its values by a one-dimensional `array` of three elements:

lick here to view code image

```
In [8]: numbers3 = numbers2.reshape(2,   3)

In [9]: numbers3
Out[9]:
array([[10, 20, 30],
       [40, 50, 60]])

In [10]: numbers4 = np.array([2,   4, 6])

In [11]: np.multiply(numbers3, numbers4)
Out[11]:
array([[ 20,  80, 180],
       [ 80, 200, 360]])
```

This works because `numbers4` has the same length as each row of `numbers3`, so NumPy can apply the multiply operation by treating `numbers4` as if it were the following `array`:

```
array([[2, 4, 6],
       [2, 4, 6]])
```

If a universal function receives two `array`s of different shapes that do not support broadcasting, a `ValueError` occurs. You can view the broadcasting rules at:

C

```
ttps://docs.scipy.org/doc/numpy/user/basics.broadcasting.html
```

Other Universal Functions

The NumPy documentation lists universal functions in five categories—math, trigonometry, bit manipulation, comparison and floating point. The following table lists some functions from each category. You can view the complete list, their descriptions and more information about universal functions at:

```
ttps://docs.scipy.org/doc/numpy/reference/ufuncs.html
```

NumPy universal functions

Math—`add`, `subtract`, `multiply`, `divide`, `remainder`, `exp`, `log`, `sqrt`, `power`, and more.

Trigonometry—`sin`, `cos`, `tan`, `hypot`, `arcsin`, `arccos`, `arctan`, and more.

Bit manipulation—`bitwise_and`, `bitwise_or`, `bitwise_xor`, `invert`, `left_shift` and `right_shift`-.

Comparison—`greater`, `greater_equal`, `less`, `less_equal`, `equal`, `not_equal`, `logical_and`, `logical_or`-, `logical_xor`, `logical_not`, `minimum`, `maximum`, and more.

Floating point—`floor`, `ceil`, `isinf`, `isnan`, `fabs`, `trunc`, and more.

7.10 INDEXING AND SLICING

One-dimensional `array`s can be indexed and sliced using the same syntax and techniques we demonstrated in the “Sequences: Lists and Tuples” chapter. Here, we focus on `array`-specific indexing and slicing capabilities.

Indexing with Two-Dimensional `array`s

h

To select an element in a two-dimensional `array`, specify a tuple containing the element's row and column indices in square brackets (as in snippet `[4]`):

lick here to view code image

```
In [1]: import numpy as np

In [2]: grades = np.array([[87,    96, 70], [100, 87, 90],
   ...:                    [94,    77, 90], [100, 81, 82]])
   ...:

In [3]: grades
Out[3]:
array([[ 87,  96,  70],
       [100,  87,  90],
       [ 94,  77,  90],
       [100,  81,  82]])

In [4]: grades[0, 1]  # row 0, column 1
Out[4]: 96
```

Selecting a Subset of a Two-Dimensional `array`'s Rows

To select a single row, specify only one index in square brackets:

lick here to view code image

```
In [5]: grades[1]
Out[5]: array([100,  87,  90])
```

To select multiple sequential rows, use slice notation:

lick here to view code image

```
In [6]: grades[0:2]
Out[6]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

To select multiple non-sequential rows, use a list of row indices:

```
In [7]: grades[[1, 3]]
Out[7]:
array([[100,  87,  90],
       [100,  81,  82]])
```

C

Selecting a Subset of a Two-Dimensional `array`'s Columns

You can select subsets of the columns by providing a tuple specifying the row(s) and column(s) to select. Each can be a specific index, a slice or a list. Let's select only the elements in the first column:

lick here to view code image

```
In [8]: grades[:, 0]
Out[8]: array([ 87, 100,  94, 100])
```

The `0` after the comma indicates that we're selecting only column `0`. The : before the comma indicates which rows within that column to select. In this case, : is a *slice* representing *all* rows. This also could be a specific row number, a slice representing a subset of the rows or a list of specific row indices to select, as in snippets `[5]`–`[7]`.

You can select consecutive columns using a slice:

```
In [9]: grades[:, 1:3]
Out[9]:
array([[96, 70],
       [87, 90],
       [77, 90],
       [81, 82]])
```

or specific columns using a *list* of column indices:

```
In [10]: grades[:, [0, 2]]
Out[10]:
array([[ 87,  70],
       [100,  90],
       [ 94,  90],
       [100,  82]])
```

7.11 VIEWS: SHALLOW COPIES

The previous chapter introduced *view objects*—that is, objects that "see" the data in other objects, rather than having their own copies of the data. Views are shallow copies. Various array methods and slicing operations produce views of an `array`'s data.

The `array` method **`view`** returns a *new* array object with a *view* of the original `array`

C

object's data. First, let's create an `array` and a view of that `array`:

lick here to view code image

```
In [1]: import numpy as np

In [2]: numbers = np.arange(1,   6)

In [3]: numbers
Out[3]: array([1, 2, 3, 4, 5])

In [4]: numbers2 = numbers.view()

In [5]: numbers2
Out[5]: array([1, 2, 3, 4, 5])
```

We can use the built-in `id` function to see that `numbers` and `numbers2` are *different* objects:

```
In [6]: id(numbers)
Out[6]: 4462958592

In [7]: id(numbers2)
Out[7]: 4590846240
```

To prove that `numbers2` views the *same* data as `numbers`, let's modify an element in `numbers`, then display both `array`s:

lick here to view code image

```
In [8]: numbers[1] *= 10

In [9]: numbers2
Out[9]: array([ 1, 20,  3,  4,  5])

In [10]: numbers
Out[10]: array([ 1, 20,  3,  4,  5])
```

Similarly, changing a value in the view also changes that value in the original `array`:

```
In [11]: numbers2[1] /= 10

In [12]: numbers
Out[12]: array([1, 2, 3, 4, 5])
```

C

```
In [13]: numbers2
Out[13]: array([1, 2, 3, 4, 5])
```

Slice Views

Slices also create views. Let's make `numbers2` a slice that views only the first three elements of `numbers`:

```
In [14]: numbers2 = numbers[0:3]
In [15]: numbers2
Out[15]: array([1, 2, 3])
```

Again, we can confirm that `numbers` and `numbers2` are different objects with `id`:

```
In [16]: id(numbers)
Out[16]: 4462958592

In [17]: id(numbers2)
Out[17]: 4590848000
```

We can confirm that `numbers2` is a view of *only* the first *three* `numbers` elements by attempting to access `numbers2[3]`, which produces an `IndexError`:

lick here to view code image

```
In [18]: numbers2[3]
---------------------------------------------------------------------------
IndexError                                Traceback   (most recent call last
 ipython-input-18-582053f52daa> in <module>()
----> 1 numbers2[3]

IndexError: index 3 is out   of bounds for axis 0 with size 3
```

Now, let's modify an element both `array`s share, then display them. Again, we see that `numbers2` is a view of `numbers`:

lick here to view code image

```
In [19]: numbers[1] *= 20

In [20]: numbers
Out[20]: array([1, 2, 3, 4, 5])
```

e

```
In [21]: numbers2
Out[21]: array([ 1, 40,  3])
```

7.12 DEEP COPIES

Though views are *separate* `array` objects, they save memory by sharing element data from other `array`s. However, when sharing *mutable* values, sometimes it's necessary to create a **deep copy** with *independent* copies of the original data. This is especially important in multi-core programming, where separate parts of your program could attempt to modify your data at the same time, possibly corrupting it.

The **`array` method `copy`** returns a new array object with a *deep copy* of the original `array` object's data. First, let's create an `array` and a deep copy of that `array`:

lick here to view code image

```
In [1]: import numpy as np

In [2]: numbers = np.arange(1,   6)

In [3]: numbers
Out[3]: array([1, 2, 3, 4, 5])

In [4]: numbers2 = numbers.copy()

In [5]: numbers2
Out[5]: array([1, 2, 3, 4, 5])
```

To prove that `numbers2` has a separate copy of the data in `numbers`, let's modify an element in `numbers`, then display both `array`s:

lick here to view code image

```
In [6]: numbers[1] *= 10

In [7]: numbers
Out[7]: array([ 1, 20,  3,  4,  5])

In [8]: numbers2
Out[8]: array([ 1, 2,  3,  4,  5])
```

As you can see, the change appears only in `numbers`.

C

Module `copy`—Shallow vs. Deep Copies for Other Types of Python Objects

In previous chapters, we covered *shallow copying*. In this chapter, we've covered how to *deep copy* `array` objects using their `copy` method. If you need deep copies of other types of Python objects, pass them to the **`copy`** module's **`deepcopy`** function.

7.13 RESHAPING AND TRANSPOSING

We've used `array` method `reshape` to produce two-dimensional arrays from one-dimensional ranges. NumPy provides various other ways to reshape `array`s.

`reshape` vs. `resize`

The array methods `reshape` and `resize` both enable you to change an array's dimensions. Method `reshape` returns a *view* (shallow copy) of the original `array` with the new dimensions. It does *not* modify the original `array`:

lick here to view code image

```
In [1]: import numpy as np

In [2]: grades = np.array([[87,   96, 70], [100, 87, 90]])

In [3]: grades
Out[3]:
array([[ 87,  96,  70],
       [100,  87,  90]])

In [4]: grades.reshape(1, 6)
Out[4]: array([[ 87,  96,  70, 100,  87,  90]])

In [5]: grades
Out[5]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

Method **`resize`** *modifies the original* `array`*'s shape*:

lick here to view code image

```
In [6]: grades.resize(1, 6)

In [7]: grades
Out[7]: array([[ 87,  96,  70, 100,  87,  90]])
```

C

`flatten` vs. `ravel`

You can take a multidimensional array and flatten it into a single dimension with the methods **`flatten`** and **`ravel`**. Method `flatten` *deep copies* the original array's data:

lick here to view code image

```
In [8]: grades = np.array([[87,   96, 70], [100, 87, 90]])

In [9]: grades
Out[9]:
array([[ 87,  96,  70],
       [100,  87,  90]])

In [10]: flattened = grades.flatten()

In [11]: flattened
Out[11]: array([ 87,  96,  70, 100,  87,  90])

In [12]: grades
Out[12]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

To confirm that `grades` and `flattened` do *not* share the data, let's modify an element of `flattened`, then display both `array`s:

lick here to view code image

```
In [13]: flattened[0] = 100

In [14]: flattened
Out[14]: array([100,  96,  70, 100,  87,  90])

In [15]: grades
Out[15]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

Method `ravel` produces a *view* of the original `array`, which *shares* the `grades array`'s data:

lick here to view code image

```
In [16]: raveled = grades.ravel()
```

C

```
In [17]: raveled
Out[17]: array([ 87,  96,  70, 100,  87,  90])

In [18]: grades
Out[18]:
array([[ 87,  96,  70],
       [100,  87,  90]])
```

To confirm that `grades` and `raveled` *share* the same data, let's modify an element of `raveled`, then display both `array`s:

lick here to view code image

```
In [19]: raveled[0] = 100

In [20]: raveled
Out[20]: array([100,  96,  70, 100,  87,  90])

In [21]: grades
Out[21]:
array([[100,  96,  70],
       [100,  87,  90]])
```

Transposing Rows and Columns

You can quickly **transpose** an `array`'s rows and columns—that is "flip" the `array`, so the rows become the columns and the columns become the rows. The **`T` attribute** returns a transposed *view* (shallow copy) of the `array`. The original `grades array` represents two students' grades (the rows) on three exams (the columns). Let's transpose the rows and columns to view the data as the grades on three exams (the rows) for two students (the columns):

```
In [22]: grades.T
Out[22]:
array([[100, 100],
       [ 96,  87],
       [ 70,  90]])
```

Transposing does *not* modify the original `array`:

```
In [23]: grades
Out[23]:
array([[100,  96,  70],
       [100,  87,  90]])
```

C

Horizontal and Vertical Stacking

You can combine arrays by adding more columns or more rows—known as *horizontal stacking* and *vertical stacking*. Let's create another 2-by-3 array of grades:

lick here to view code image

```
In [24]: grades2 = np.array([[94,   77, 90], [100, 81, 82]])
```

Let's assume `grades2` represents three additional exam grades for the two students in the `grades` array. We can combine `grades` and `grades2` with NumPy's **`hstack` (horizontal stack) function** by passing a tuple containing the arrays to combine. The extra parentheses are required because `hstack` expects one argument:

lick here to view code image

```
In [25]: np.hstack((grades, grades2))
Out[25]:
array([[100,  96,  70,  94,  77,  90],
       [100,  87,  90, 100,  81,  82]])
```

Next, let's assume that `grades2` represents two more students' grades on three exams. In this case, we can combine `grades` and `grades2` with NumPy's **`vstack` (vertical stack) function**:

lick here to view code image

```
In [26]: np.vstack((grades, grades2))
Out[26]:
array([[100,  96,  70],
       [100,  87,  90],
       [ 94,  77,  90],
       [100,  81,  82]])
```

7.14 INTRO TO DATA SCIENCE: PANDAS `SERIES` AND `DATAFRAME`S

NumPy's `array` is optimized for homogeneous numeric data that's accessed via integer indices. Data science presents unique demands for which more customized data structures are required. Big data applications must support mixed data types, customized indexing, missing data, data that's not structured consistently and data that

C

eeds to be manipulated into forms appropriate for the databases and data analysis packages you use.

Pandas is the most popular library for dealing with such data. It provides two key collections that you'll use in several of our Intro to Data Science sections and throughout the data science case studies—**`Series`** for one-dimensional collections and **`DataFrames`** for two-dimensional collections. You can use pandas' **`MultiIndex`** to manipulate multi-dimensional data in the context of `Series` and `DataFrame`s.

Wes McKinney created pandas in 2008 while working in industry. The name pandas is derived from the term "panel data," which is data for measurements over time, such as stock prices or historical temperature readings. McKinney needed a library in which the same data structures could handle both time- and non-time-based data with support for data alignment, missing data, common database-style data manipulations, and more. [4]

[4] McKinney, Wes. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*, pp. 123165. Sebastopol, CA: OReilly Media, 2018.

NumPy and pandas are intimately related. `Series` and `DataFrame`s use `array`s "under the hood." `Series` and `DataFrame`s are valid arguments to many NumPy operations. Similarly, `array`s are valid arguments to many `Series` and `DataFrame` operations.

Pandas is a massive topic—the PDF of its documentation [5] is over 2000 pages. In this and the next chapters' Intro to Data Science sections, we present an introduction to pandas. We discuss its `Series` and `DataFrame`s collections, and use them in support of data preparation. You'll see that `Series` and `DataFrame`s make it easy for you to perform common tasks like selecting elements a variety of ways, filter/map/reduce operations (central to functional-style programming and big data), mathematical operations, visualization and more.

[5] For the latest pandas documentation, see `ttp://pandas.pydata.org/pandas-docs/stable/`.

7.14.1 pandas `Series`

A **`Series`** is an enhanced one-dimensional `array`. Whereas `array`s use only zero-based integer indices, `Series` support custom indexing, including even non-integer indices like strings. `Series` also offer additional capabilities that make them more

h

convenient for many data-science oriented tasks. For example, `Series` may have missing data, and many `Series` operations ignore missing data by default.

Creating a `Series` with Default Indices

By default, a `Series` has integer indices numbered sequentially from 0. The following creates a `Series` of student grades from a list of integers:

lick here to view code image

```
In [1]: import pandas as pd

In [2]: grades = pd.Series([87, 100, 94])
```

The initializer also may be a tuple, a dictionary, an `array`, another `Series` or a single value. We'll show a single value momentarily.

Displaying a `Series`

Pandas displays a `Series` in two-column format with the indices *left aligned* in the left column and the values *right aligned* in the right column. After listing the `Series` elements, pandas shows the data type (`dtype`) of the underlying `array`'s elements:

```
In [3]: grades
Out[3]:
0     87
1    100
2     94
dtype: int64
```

Note how easy it is to display a `Series` in this format, compared to the corresponding code for displaying a list in the same two-column format.

Creating a `Series` with All Elements Having the Same Value

You can create a `Series` of elements that all have the same value:

lick here to view code image

```
In [4]: pd.Series(98.6, range(3))
Out[4]:
0    98.6
1    98.6
```

C

```
2    98.6
dtype: float64
```

The second argument is a one-dimensional iterable object (such as a list, an `array` or a `range`) containing the `Series`' indices. The number of indices determines the number of elements.

Accessing a `Series`' Elements

You can access a `Series`'s elements by via square brackets containing an index:

```
In [5]: grades[0]
Out[5]: 87
```

Producing Descriptive Statistics for a Series

`Series` provides many methods for common tasks including producing various descriptive statistics. Here we show `count`, `mean`, `min`, `max` and `std` (standard deviation):

lick here to view code image

```
In [6]: grades.count()
Out[6]: 3

In [7]: grades.mean()
Out[7]: 93.66666666666667

In [8]: grades.min()
Out[8]: 87

In [9]: grades.max()
Out[9]: 100

In [10]: grades.std()
Out[10]: 6.506407098647712
```

Each of these is a functional-style reduction. Calling `Series` method **`describe`** produces all these stats and more:

lick here to view code image

```
In [11]: grades.describe()
Out[11]:
```

C

```
count      3.000000
mean      93.666667
std        6.506407
min       87.000000
25%       90.500000
50%       94.000000
75%       97.000000
max      100.000000
dtype: float64
```

The `25%`, `50%` and `75%` are **quartiles**:

- `50%` represents the median of the sorted values.
- `25%` represents the median of the first half of the sorted values.
- `75%` represents the median of the second half of the sorted values.

For the quartiles, if there are two middle elements, then their average is that quartile's median. We have only three values in our `Series`, so the 25% quartile is the average of 87 and 94, and the 75% quartile is the average of 94 and 100. Together, the **interquartile range** is the 75% quartile minus the 25% quartile, which is another measure of dispersion, like standard deviation and variance. Of course, quartiles and interquartile range are more useful in larger datasets.

Creating a `Series` with Custom Indices

You can specify *custom* indices with the `index` keyword argument:

lick here to view code image

```
In [12]: grades = pd.Series([87,   100, 94], index=['Wally', 'Eva', 'Sam']

 n [13]: grades
Out[13]:
Wally     87
Eva      100
Sam       94
dtype: int64
```

In this case, we used string indices, but you can use other immutable types, including integers not beginning at 0 and nonconsecutive integers. Again, notice how nicely and concisely pandas formats a `Series` for display.

€

Dictionary Initializers

If you initialize a `Series` with a dictionary, its keys become the `Series`' indices, and its values become the `Series`' element values:

lick here to view code image

```
In [14]: grades = pd.Series({'Wally':   87, 'Eva': 100, 'Sam': 94})

In [15]: grades
Out[15]:
Wally     87
Eva      100
Sam       94
dtype: int64
```

Accessing Elements of a `Series` Via Custom Indices

In a `Series` with custom indices, you can access individual elements via square brackets containing a custom index value:

```
In [16]: grades['Eva']
Out[16]: 100
```

If the custom indices are strings that could represent valid Python identifiers, pandas automatically adds them to the `Series` as attributes that you can access via a dot (`.`), as in:

```
In [17]: grades.Wally
Out[17]: 87
```

`Series` also has *built-in* attributes. For example, the **`dtype` attribute** returns the underlying `array`'s element type:

```
In [18]: grades.dtype
Out[18]: dtype('int64')
```

and the **`values` attribute** returns the underlying `array`:

lick here to view code image

```
In [19]: grades.values
```

C

```
Out[19]: array([ 87, 100,  94])
```

Creating a Series of Strings

If a `Series` contains strings, you can use its **`str` attribute** to call string methods on the elements. First, let's create a `Series` of hardware-related strings:

lick here to view code image

```
In [20]: hardware = pd.Series(['Hammer',   'Saw', 'Wrench'])

In [21]: hardware
Out[21]:
0    Hammer
1       Saw
2    Wrench
dtype: object
```

Note that pandas also *right-aligns* string element values and that the `dtype` for strings is `object`.

Let's call string method `contains` on each element to determine whether the value of each element contains a lowercase `'a'`:

lick here to view code image

```
In [22]: hardware.str.contains('a')
Out[22]:
0     True
1     True
2    False
dtype: bool
```

Pandas returns a `Series` containing `bool` values indicating the `contains` method's result for each element—the element at index `2` (`'Wrench'`) does not contain an `'a'`, so its element in the resulting `Series` is `False`. Note that pandas handles the iteration internally for you—another example of functional-style programming. The `str` attribute provides many string-processing methods that are similar to those in Python's string type. For a list, see: `ttps://pandas.pydata.org/pandas-ocs/stable/api.html#string-handling`.

The following uses string method `upper` to produce a *new* `Series` containing the

uppercase versions of each element in `hardware`:

lick here to view code image

```
In [23]: hardware.str.upper()
Out[23]:
0    HAMMER
1       SAW
2    WRENCH
dtype: object
```

7.14.2 DataFrames

A **DataFrame** is an enhanced two-dimensional `array`. Like `Series`, `DataFrame`s can have custom row and column indices, and offer additional operations and capabilities that make them more convenient for many data-science oriented tasks. `DataFrame`s also support missing data. Each column in a `DataFrame` is a `Series`. The `Series` representing each column may contain different element types, as you'll soon see when we discuss loading datasets into `DataFrame`s.

Creating a DataFrame from a Dictionary

Let's create a `DataFrame` from a dictionary that represents student grades on three exams:

lick here to view code image

```
In [1]: import pandas as pd

In [2]: grades_dict = {'Wally':   [87, 96, 70], 'Eva': [100, 87, 90],
   ...:                'Sam':   [94, 77, 90], 'Katie': [100, 81, 82],
   ...:                'Bob':   [83, 65, 85]}
   ...:

In [3]: grades = pd.DataFrame(grades_dict)

In [4]: grades
Out[4]:
   Wally  Eva  Sam  Katie  Bob
0     87  100   94    100   83
1     96   87   77     81   65
2     70   90   90     82   85
```

Pandas displays `DataFrame`s in tabular format with the indices *left aligned* in the

C

index column and the remaining columns' values *right aligned*. The dictionary's *keys* become the column names and the *values* associated with each key become the element values in the corresponding column. Shortly, we'll show how to "flip" the rows and columns. By default, the row indices are auto-generated integers starting from 0.

Customizing a `DataFrame`'s Indices with the `index` Attribute

We could have specified custom indices with the `index` keyword argument when we created the `DataFrame`, as in:

lick here to view code image

```
pd.DataFrame(grades_dict, index=['Test1', 'Test2', 'Test3'])
```

Let's use the **`index` attribute** to change the `DataFrame`'s indices from sequential integers to labels:

lick here to view code image

```
In [5]: grades.index = ['Test1',   'Test2', 'Test3']

In [6]: grades
Out[6]:
       Wally  Eva  Sam  Katie  Bob
Test1     87  100   94    100   83
Test2     96   87   77     81   65
Test3     70   90   90     82   85
```

When specifying the indices, you must provide a one-dimensional collection that has the same number of elements as there are *rows* in the `DataFrame`; otherwise, a `ValueError` occurs. `Series` also provides an **`index` attribute** for changing an existing `Series`' indices.

Accessing a `DataFrame`'s Columns

One benefit of pandas is that you can quickly and conveniently look at your data in many different ways, including selecting portions of the data. Let's start by getting `Eva`'s grades by name, which displays her column as a `Series`:

```
In [7]: grades['Eva']
Out[7]:
Test1    100
```

C

```
Test2     87
Test3     90
Name: Eva, dtype: int64
```

If a `DataFrame`'s column-name strings are valid Python identifiers, you can use them as attributes. Let's get `Sam`'s grades with the `Sam` *attribute*:

```
In [8]: grades.Sam
Out[8]:
Test1    94
Test2    77
Test3    90
Name: Sam, dtype: int64
```

Selecting Rows via the `loc` and `iloc` Attributes

Though `DataFrame`s support indexing capabilities with `[]`, the pandas documentation recommends using the attributes `loc`, `iloc`, `at` and `iat`, which are optimized to access `DataFrame`s and also provide additional capabilities beyond what you can do only with `[]`. Also, the documentation states that indexing with `[]` *often* produces a copy of the data, which is a logic error if you attempt to assign new values to the `DataFrame` by assigning to the result of the `[]` operation.

You can access a row by its label via the `DataFrame`'s **`loc` attribute**. The following lists all the grades in the row `'Test1'`:

lick here to view code image

```
In [9]: grades.loc['Test1']
Out[9]:
Wally     87
Eva      100
Sam       94
Katie    100
Bob       83
Name: Test1, dtype: int64
```

You also can access rows by integer zero-based indices using the **`iloc` attribute** (the `i` in `iloc` means that it's used with integer indices). The following lists all the grades in the second row:

lick here to view code image

C

```
In [10]: grades.iloc[1]
Out[10]:
Wally    96
Eva      87
Sam      77
Katie    81
Bob      65
Name: Test2, dtype: int64
```

Selecting Rows via Slices and Lists with the `loc` and `iloc` Attributes

The index can be a *slice*. When using slices containing labels with `loc`, the range specified *includes* the high index (`'Test3'`):

lick here to view code image

```
In [11]: grades.loc['Test1':'Test3']
Out[11]:
       Wally  Eva  Sam  Katie  Bob
Test1     87  100   94    100   83
Test2     96   87   77     81   65
Test3     70   90   90     82   85
```

When using slices containing integer indices with `iloc`, the range you specify *excludes* the high index (`2`):

lick here to view code image

```
In [12]: grades.iloc[0:2]
Out[12]:
       Wally  Eva  Sam  Katie  Bob
Test1     87  100   94    100   83
Test2     96   87   77     81   65
```

To select *specific rows*, use a *list* rather than slice notation with `loc` or `iloc`:

lick here to view code image

```
In [13]: grades.loc[['Test1', 'Test3']]
Out[13]:
       Wally  Eva  Sam  Katie  Bob
Test1     87  100   94    100   83
Test3     70   90   90     82   85

In [14]: grades.iloc[[0, 2]]
```

C

```
Out[14]:
       Wally  Eva  Sam  Katie  Bob
Test1     87  100   94    100   83
Test3     70   90   90     82   85
```

Selecting Subsets of the Rows and Columns

So far, we've selected only *entire* rows. You can focus on small subsets of a `DataFrame` by selecting rows *and* columns using two slices, two lists or a combination of slices and lists.

Suppose you want to view only `Eva`'s and `Katie`'s grades on `Test1` and `Test2`. We can do that by using `loc` with a slice for the two consecutive rows and a list for the two non-consecutive columns:

lick here to view code image

```
In [15]: grades.loc['Test1':'Test2', ['Eva', 'Katie']]
Out[15]:
       Eva  Katie
Test1  100    100
Test2   87     81
```

The slice `'Test1':'Test2'` selects the rows for `Test1` and `Test2`. The list `['Eva', 'Katie']` selects only the corresponding grades from those two columns.

Let's use `iloc` with a list and a slice to select the first and third tests and the first three columns for those tests:

lick here to view code image

```
In [16]: grades.iloc[[0, 2], 0:3]
Out[16]:
       Wally  Eva  Sam
Test1     87  100   94
Test3     70   90   90
```

Boolean Indexing

One of pandas' more powerful selection capabilities is **Boolean indexing**. For example, let's select all the A grades—that is, those that are greater than or equal to 90:

lick here to view code image

C

```
In [17]: grades[grades >= 90]
Out[17]:
       Wally    Eva   Sam  Katie  Bob
Test1    NaN  100.0  94.0  100.0  NaN
Test2   96.0    NaN   NaN    NaN  NaN
Test3    NaN   90.0  90.0    NaN  NaN
```

Pandas checks every grade to determine whether its value is greater than or equal to 90 and, if so, includes it in the new `DataFrame`. Grades for which the condition is `False` are represented as **`NaN` (not a number)** in the new `DataFrame`. `NaN` is pandas' notation for missing values.

Let's select all the B grades in the range 80–89:

lick here to view code image

```
In [18]: grades[(grades >= 80)   & (grades < 90)]
Out[18]:
       Wally   Eva  Sam  Katie   Bob
Test1   87.0   NaN  NaN    NaN  83.0
Test2    NaN  87.0  NaN   81.0   NaN
Test3    NaN   NaN  NaN   82.0  85.0
```

Pandas Boolean indices combine multiple conditions with the Python operator `&` (bitwise AND), *not* the `and` Boolean operator. For `or` conditions, use `|` (bitwise OR). NumPy also supports Boolean indexing for `array`s, but always returns a one-dimensional array containing only the values that satisfy the condition.

Accessing a Specific `DataFrame` Cell by Row and Column

You can use a `DataFrame`'s **`at`** and **`iat`** attributes to get a single value from a `DataFrame`. Like `loc` and `iloc`, `at` uses labels and `iat` uses integer indices. In each case, the row and column indices must be separated by a comma. Let's select `Eva`'s `Test2` grade (`87`) and `Wally`'s `Test3` grade (`70`)

lick here to view code image

```
In [19]: grades.at['Test2', 'Eva']
Out[19]: 87

In [20]: grades.iat[2, 0]
Out[20]: 70
```

C

You also can assign new values to specific elements. Let's change `Eva`'s `Test2` grade to `100` using `at`, then change it back to `87` using `iat`:

lick here to view code image

```
In [21]: grades.at['Test2', 'Eva'] = 100

In [22]: grades.at['Test2', 'Eva']
Out[22]: 100

In [23]: grades.iat[1, 2] = 87

In [24]: grades.iat[1, 2]
Out[24]: 87.0
```

Descriptive Statistics

Both `Series` and `DataFrame`s have a **`describe` method** that calculates basic descriptive statistics for the data and returns them as a `DataFrame`. In a `DataFrame`, the statistics are calculated by column (again, soon you'll see how to flip rows and columns):

lick here to view code image

```
In [25]: grades.describe()
Out[25]:
            Wally         Eva         Sam       Katie         Bob
count    3.000000    3.000000    3.000000    3.000000    3.000000
mean    84.333333   92.333333   87.000000   87.666667   77.666667
std     13.203535    6.806859    8.888194   10.692677   11.015141
min     70.000000   87.000000   77.000000   81.000000   65.000000
25%     78.500000   88.500000   83.500000   81.500000   74.000000
50%     87.000000   90.000000   90.000000   82.000000   83.000000
75%     91.500000   95.000000   92.000000   91.000000   84.000000
max     96.000000  100.000000   94.000000  100.000000   85.000000
```

As you can see, `describe` gives you a quick way to summarize your data. It nicely demonstrates the power of array-oriented programming with a clean, concise functional-style call. Pandas handles internally all the details of calculating these statistics for each column. You might be interested in seeing similar statistics on test-by-test basis so you can see how all the students performs on Tests 1, 2 and 3—we'll show how to do that shortly.

By default, pandas calculates the descriptive statistics with floating-point values and

C

displays them with six digits of precision. You can control the precision and other default settings with pandas' **`set_option` function**:

lick here to view code image

```
In [26]: pd.set_option('precision',   2)

In [27]: grades.describe()
Out[27]:
       Wally     Eva    Sam   Katie    Bob
count   3.00    3.00   3.00    3.00   3.00
mean   84.33   92.33  87.00   87.67  77.67
std    13.20    6.81   8.89   10.69  11.02
min    70.00   87.00  77.00   81.00  65.00
25%    78.50   88.50  83.50   81.50  74.00
50%    87.00   90.00  90.00   82.00  83.00
75%    91.50   95.00  92.00   91.00  84.00
max    96.00  100.00  94.00  100.00  85.00
```

For student grades, the most important of these statistics is probably the mean. You can calculate that for each student simply by calling `mean` on the `DataFrame`:

```
In [28]: grades.mean()
Out[28]:
Wally    84.33
Eva      92.33
Sam      87.00
Katie    87.67
Bob      77.67
dtype: float64
```

In a moment, we'll show how to get the average of all the students' grades on each test in one line of additional code.

Transposing the `DataFrame` with the `T` Attribute

You can quickly **transpose** the rows and columns—so the rows become the columns, and the columns become the rows—by using the **`T` attribute**:

lick here to view code image

```
In [29]: grades.T
Out[29]:
       Test1  Test2  Test3
Wally     87     96     70
Eva      100     87     90
```

C

```
Sam        94      77      90
Katie     100      81      82
Bob        83      65      85
```

`T` returns a transposed *view* (not a copy) of the `DataFrame`.

Let's assume that rather than getting the summary statistics by student, you want to get them by test. Simply call `describe` on `grades.T`, as in:

lick here to view code image

```
In [30]: grades.T.describe()
Out[30]:
        Test1  Test2  Test3
count    5.00   5.00   5.00
mean    92.80  81.20  83.40
std      7.66  11.54   8.23
min     83.00  65.00  70.00
25%     87.00  77.00  82.00
50%     94.00  81.00  85.00
75%    100.00  87.00  90.00
max    100.00  96.00  90.00
```

To see the average of all the students' grades on each test, just call `mean` on the `T` attribute:

```
In [31]: grades.T.mean()
Out[31]:
Test1    92.8
Test2    81.2
Test3    83.4
dtype: float64
```

Sorting by Rows by Their Indices

You'll often sort data for easier readability. You can sort a `DataFrame` by its rows or columns, based on their indices or values. Let's sort the rows by their *indices* in *descending* order using **`sort_index`** and its keyword argument `ascending=False` (the default is to sort in *ascending* order). This returns a new `DataFrame` containing the sorted data:

lick here to view code image

```
In [32]: grades.sort_index(ascending=False)
```

C

```
Out[32]:
       Wally  Eva  Sam  Katie  Bob
Test3     70   90   90     82   85
Test2     96   87   77     81   65
Test1     87  100   94    100   83
```

Sorting by Column Indices

Now let's sort the columns into ascending order (left-to-right) by their column names. Passing the **`axis=1` keyword argument** indicates that we wish to sort the *column* indices, rather than the row indices—`axis=0` (the default) sorts the *row* indices:

lick here to view code image

```
In [33]: grades.sort_index(axis=1)
Out[33]:
       Bob  Eva  Katie  Sam  Wally
Test1   83  100    100   94     87
Test2   65   87     81   77     96
Test3   85   90     82   90     70
```

Sorting by Column Values

Let's assume we want to see `Test1`'s grades in descending order so we can see the students' names in highest-to-lowest grade order. We can call the method **`sort_values`** as follows:

lick here to view code image

```
In [34]: grades.sort_values(by='Test1',   axis=1, ascending=False)
Out[34]:
       Eva  Katie  Sam  Wally  Bob
Test1  100    100   94     87   83
Test2   87     81   77     96   65
Test3   90     82   90     70   85
```

The `by` and `axis` keyword arguments work together to determine which values will be sorted. In this case, we sort based on the column values (`axis=1`) for `Test1`.

Of course, it might be easier to read the grades and names if they were in a column, so we can sort the transposed `DataFrame` instead. Here, we did not need to specify the `axis` keyword argument, because `sort_values` sorts data in a specified column by default:

C

lick here to view code image

```
In [35]: grades.T.sort_values(by='Test1',   ascending=False)
Out[35]:
       Test1  Test2  Test3
Eva      100     87     90
Katie    100     81     82
Sam       94     77     90
Wally     87     96     70
Bob       83     65     85
```

Finally, since you're sorting only `Test1`'s grades, you might not want to see the other tests at all. So, let's combine selection with sorting:

lick here to view code image

```
In [36]: grades.loc['Test1'].sort_values(ascending=False)
Out[36]:
Katie    100
Eva      100
Sam       94
Wally     87
Bob       83
Name: Test1, dtype: int64
```

Copy vs. In-Place Sorting

By default the `sort_index` and `sort_values` return a *copy* of the original `DataFrame`, which could require substantial memory in a big data application. You can sort the `DataFrame` *in place*, rather than *copying* the data. To do so, pass the keyword argument `inplace=True` to either `sort_index` or `sort_values`.

We've shown many pandas `Series` and `DataFrame` features. In the next chapter's Intro to Data Science section, we'll use `Series` and `DataFrame`s for *data munging*—cleaning and preparing data for use in your database or analytics software.

7.15 WRAP-UP

This chapter explored the use of NumPy's high-performance `ndarray`s for storing and retrieving data, and for performing common data manipulations concisely and with reduced chance of errors with functional-style programming. We refer to `ndarray`s simply by their synonym, `array`s.

C

he chapter examples demonstrated how to create, initialize and refer to individual elements of one- and two-dimensional `array`s. We used attributes to determine an `array`'s size, shape and element type. We showed functions that create `array`s of 0s, 1s, specific values or ranges values. We compared list and `array` performance with the IPython `%timeit` magic and saw that `array`s are up to two orders of magnitude faster.

We used `array` operators and NumPy universal functions to perform element-wise calculations on every element of `array`s that have the same shape. You also saw that NumPy uses broadcasting to perform element-wise operations between `array`s and scalar values, and between `array`s of different shapes. We introduced various built-in `array` methods for performing calculations using all elements of an `array`, and we showed how to perform those calculations row-by-row or column-by-column. We demonstrated various `array` slicing and indexing capabilities that are more powerful than those provided by Python's built-in collections. We demonstrated various ways to reshape `array`s. We discussed how to shallow copy and deep copy `array`s and other Python objects.

In the Intro to Data Science section, we began our multisection introduction to the popular pandas library that you'll use in many of the data science case study chapters. You saw that many big data applications need more flexible collections than NumPy's `array`s, collections that support mixed data types, custom indexing, missing data, data that's not structured consistently and data that needs to be manipulated into forms appropriate for the databases and data analysis packages you use.

We showed how to create and manipulate pandas array-like one-dimensional `Series` and two-dimensional `DataFrames`. We customized `Series` and `DataFrame` indices. You saw pandas' nicely formatted outputs and customized the precision of floating-point values. We showed various ways to access and select data in `Series` and `DataFrame`s. We used method `describe` to calculate basic descriptive statistics for `Series` and `DataFrame`s. We showed how to transpose `DataFrame` rows and columns via the `T` attribute. You saw several ways to sort `DataFrame`s using their index values, their column names, the data in their rows and the data in their columns. You're now familiar with four powerful array-like collections—lists, `array`s, `Series` and `DataFrame`s—and the contexts in which to use them. We'll add a fifth—tensors—in the "Deep Learning" chapter.

In the next chapter, we take a deeper look at strings, string formatting and string methods. We also introduce regular expressions, which we'll use to match patterns in text. The capabilities you'll see will help you prepare for the "atural Language

rocessing (NLP)" chapter and other key data science chapters. In the next chapter's Intro to Data Science section, we'll introduce pandas *data munging*—preparing data for use in your database or analytics software. In subsequent chapters, we'll use pandas for basic time-series analysis and introduce pandas visualization capabilities.

P

. Strings: A Deeper Look

Objectives

In this chapter, you'll:

- Understand text processing.
- Use string methods.
- Format string content.
- Concatenate and repeat strings.
- Strip whitespace from the ends of strings.
- Change characters from lowercase to uppercase and vice versa.
- Compare strings with the comparison operators.
- Search strings for substrings and replace substrings.
- Split strings into tokens.
- Concatenate strings into a single string with a specified separator between items.
- Create and use regular expressions to match patterns in strings, replace substrings and validate data.
- Use regular expression metacharacters, quantifiers, character classes and grouping.
- Understand how critical string manipulations are to natural language processing.
- Understand the data science terms data munging, data wrangling and data cleaning, and use regular expressions to munge data into preferred formats.

Outline

.1 INTRODUCTION

We've introduced strings, basic string formatting and several string operators and methods. You saw that strings support many of the same sequence operations as lists and tuples, and that strings, like tuples, are immutable. Now, we take a deeper look at strings and introduce regular expressions and the `re` module, which we'll use to match patterns [1] in text. Regular expressions are particularly important in today's data rich applications. The capabilities presented here will help you prepare for the "atural anguage Processing (NLP)" chapter and other key data science chapters. In the NLP chapter, we'll look at other ways to have computers manipulate and even "understand" text. The table below shows many string-processing and NLP-related applications. In the Intro to Data Science section, we briefly introduce data cleaning/munging/wrangling with Pandas `Series` and `DataFrame`s.

[1] Well see in the data science case study chapters that searching for patterns in text is a crucial part of machine learning.

String and NLP applications		
Anagrams		
Automated grading of written homework	Inter-language translation	
Automated teaching systems	Legal document preparation	Spam classification
Categorizing articles	Monitoring social media posts	Speech-to-text engines
Chatbots	Natural language understanding-	Spell checkers
Compilers and	Opinion analysis	Steganography
		Text editors

interpreters	Page-composition software	Text-to-speech engines
Creative writing	Palindromes	Web scraping
Cryptography	Parts-of-speech tagging	Who authored Shakespeare's works?
Document classification	Project Gutenberg free books	Word clouds
Document similarity	Reading books, articles, documentation and absorbing knowledge	Word games
Document summarization	Search engines	Writing medical diagnoses from x-rays, scans, blood tests
Electronic book readers	Sentiment analysis	and many more
Fraud detection		
Grammar checkers		

8.2 FORMATTING STRINGS

Proper text formatting makes data easier to read and understand. Here, we present many text-formatting capabilities.

8.2.1 Presentation Types

You've seen basic string formatting with f-strings. When you specify a placeholder for a value in an f-string, Python assumes the value should be displayed as a string unless you specify another type. In some cases, the type is required. For example, let's format the `float` value `17.489` rounded to the hundredths position:

```
In [1]: f'{17.489:.2f}'
Out[1]: '17.49'
```

Python supports precision *only* for floating-point and `Decimal` values. Formatting is *type dependent*—if you try to use `.2f` to format a string like `'hello'`, a `ValueError` occurs. So the **presentation type** `f` in the *format specifier* `.2f` is required. It indicates what type is being formatted so Python can determine whether the other formatting information is allowed for that type. Here, we show some common presentation types. You can view the complete list at

```
ttps://docs.python.org/3/library/string.html#formatspec
```

Integers

The **`d` presentation type** formats integer values as strings:

```
In [2]: f'{10:d}'
Out[2]: '10'
```

There also are integer presentation types (`b`, `o` and `x` or `X`) that format integers using the binary, octal or hexadecimal number systems. [2]

[2] See the online appendix Number Systems for information about the binary, octal and hexadecimal number systems.

Characters

The **`c` presentation type** formats an integer character code as the corresponding character:

lick here to view code image

```
In [3]: f'{65:c} {97:c}'
Out[3]: 'A a'
```

Strings

The **`s` presentation type** is the default. If you specify `s` explicitly, the value to format must be a variable that references a string, an expression that produces a string or a string literal, as in the first placeholder below. If you do not specify a presentation type, as in the second placeholder below, non-string values like the integer `7` are converted to strings:

lick here to view code image

€

```
In [4]: f'{"hello":s} {7}'
Out[4]: 'hello 7'
```

In this snippet, `"hello"` is enclosed in double quotes. Recall that you cannot place single quotes inside a single-quoted string.

Floating-Point and Decimal Values

You've used the `f` presentation type to format floating-point and `Decimal` values. For extremely large and small values of these types, **Exponential (scientific) notation** can be used to format the values more compactly. Let's show the difference between `f` and `e` for a large value, each with three digits of precision to the right of the decimal point:

lick here to view code image

```
In [5]: from decimal import Decimal

In [6]: f'{Decimal("10000000000000000000000000.0"):.3f}'
Out[6]: '10000000000000000000000000.000'

In [7]: f'{Decimal("10000000000000000000000000.0"):.3e}'
Out[7]: '1.000e+25'
```

For the **`e` presentation type** in snippet `[5]`, the formatted value `1.000e+25` is equivalent to

```
1.000 x 10^25
```

If you prefer a capital `E` for the exponent, use the **`E` presentation type** rather than `e`.

8.2.2 Field Widths and Alignment

Previously you used *field widths* to format text in a specified number of character positions. By default, Python *right-aligns* numbers and *left-aligns* other values such as strings—we enclose the results below in brackets (`[]`) so you can see how the values align in the field:

lick here to view code image

```
In [1]: f'[{27:10d}]'
```

C

```
Out[1]: '[        27]'

In [2]: f'[{3.5:10f}]'
Out[2]: '[  3.500000]'

In [3]: f'[{"hello":10}]'
Out[3]: '[hello     ]'
```

Snippet [2] shows that Python formats `float` values with six digits of precision to the right of the decimal point by default. For values that have fewer characters than the field width, the remaining character positions are filled with spaces. Values with more characters than the field width use as many character positions as they need.

Explicitly Specifying Left and Right Alignment in a Field

Recall that you can specify left and right alignment with < and >:

lick here to view code image

```
In [4]: f'[{27:<15d}]'
Out[4]: '[27             ]'

In [5]: f'[{3.5:<15f}]'
Out[5]: '[3.500000       ]'

In [6]: f'[{"hello":>15}]'
Out[6]: '[          hello]'
```

Centering a Value in a Field

In addition, you can *center* values:

lick here to view code image

```
In [7]: f'[{27:^7d}]'
Out[7]: '[  27   ]'

In [8]: f'[{3.5:^7.1f}]'
Out[8]: '[  3.5  ]'

In [9]: f'[{"hello":^7}]'
Out[9]: '[ hello ]'
```

Centering attempts to spread the remaining unoccupied character positions equally to the left and right of the formatted value. Python places the extra space to the right if an

C

odd number of character positions remain.

8.2.3 Numeric Formatting

There are a variety of numeric formatting capabilities.

Formatting Positive Numbers with Signs

Sometimes it's desirable to force the sign on a positive number:

```
In [1]: f'[{27:+10d}]'
Out[1]: '[       +27]'
```

The + before the field width specifies that a positive number should be preceded by a +. A negative number always starts with a -. To fill the remaining characters of the field with 0s rather than spaces, place a 0 before the field width (and *after* the + if there is one):

```
In [2]: f'[{27:+010d}]'
Out[2]: '[+000000027]'
```

Using a Space Where a + Sign Would Appear in a Positive Value

A space indicates that positive numbers should show a space character in the sign position. This is useful for aligning positive and negative values for display purposes:

lick here to view code image

```
In [3]: print(f'{27:d}\n{27: d}\n{-27: d}')
27
 27
-27
```

Note that the two numbers with a space in their format specifiers align. If a field width is specified, the space should appear *before* the field width.

Grouping Digits

You can format numbers with **thousands separators** by using a **comma (,)**, as follows:

```
In [4]: f'{12345678:,d}'
```

C

```
Out[4]: '12,345,678'

In [5]: f'{123456.78:,.2f}'
Out[5]: '123,456.78'
```

8.2.4 String's `format` Method

Python's f-strings were added to the language in version 3.6. Before that, formatting was performed with the string method **`format`**. In fact, f-string formatting is based on the `format` method's capabilities. We show you the `format` method here because you'll encounter it in code written prior to Python 3.6. You'll often see the `format` method in the Python documentation and in the many Python books and articles written before f-strings were introduced. However, we recommend using the newer f-string formatting that we've presented to this point.

You call method `format` on a *format string* containing curly brace (`{}`) *placeholders*, possibly with format specifiers. You pass to the method the values to be formatted. Let's format the `float` value `17.489` rounded to the hundredths position:

lick here to view code image

```
In [1]: '{:.2f}'.format(17.489)
Out[1]: '17.49'
```

In a placeholder, if there's a format specifier, you precede it by a colon (`:`), as in f-strings. The result of the `format` call is a new string containing the formatted results.

Multiple Placeholders

A format string may contain multiple placeholders, in which case the `format` method's arguments correspond to the placeholders from left to right:

lick here to view code image

```
In [2]: '{} {}'.format('Amanda', 'Cyan')
Out[2]: 'Amanda Cyan'
```

Referencing Arguments By Position Number

The format string can reference specific arguments by their position in the `format` method's argument list, starting with position 0:

C

lick here to view code image

```
In [3]: '{0} {0} {1}'.format('Happy', 'Birthday')
Out[3]: 'Happy Happy Birthday'
```

Note that we used the position number `0` (`'Happy'`) twice—you can reference each argument as often as you like and in any order.

Referencing Keyword Arguments

You can reference keyword arguments by their keys in the placeholders:

lick here to view code image

```
In [4]: '{first} {last}'.format(first='Amanda',   last='Gray')
Out[4]: 'Amanda Gray'

In [5]: '{last} {first}'.format(first='Amanda',   last='Gray')
Out[5]: 'Gray Amanda'
```

8.3 CONCATENATING AND REPEATING STRINGS

In earlier chapters, we used the + operator to concatenate strings and the `*` operator to repeat strings. You also can perform these operations with augmented assignments. Strings are immutable, so each operation assigns a new string object to the variable:

lick here to view code image

```
In [1]: s1 = 'happy'

In [2]: s2 = 'birthday'

In [3]: s1 += ' ' + s2

In [4]: s1
Out[4]: 'happy birthday'

In [5]: symbol = '>'

In [6]: symbol *= 5

In [7]: symbol
Out[7]: '>>>>>'
```

C

8.4 STRIPPING WHITESPACE FROM STRINGS

There are several string methods for removing whitespace from the ends of a string. Each returns a new string leaving the original unmodified. Strings are immutable, so each method that appears to modify a string returns a new one.

Removing Leading and Trailing Whitespace

Let's use string method **`strip`** to remove the leading and trailing whitespace from a string:

lick here to view code image

```
In [1]: sentence = '\t  \n  This is a   test string. \t\t \n'

In [2]: sentence.strip()
Out[2]: 'This is a test string.'
```

Removing Leading Whitespace

Method **`lstrip`** removes only leading whitespace:

lick here to view code image

```
In [3]: sentence.lstrip()
Out[3]: 'This is a test string. \t\t \n'
```

Removing Trailing Whitespace

Method **`rstrip`** removes only trailing whitespace:

lick here to view code image

```
In [4]: sentence.rstrip()
Out[4]: '\t  \n  This is a test string.'
```

As the outputs demonstrate, these methods remove all kinds of whitespace, including spaces, newlines and tabs.

8.5 CHANGING CHARACTER CASE

In earlier chapters, you used string methods `lower` and `upper` to convert strings to all

C

lowercase or all uppercase letters. You also can change a string's capitalization with methods `capitalize` and `title`.

Capitalizing Only a String's First Character

Method **`capitalize`** copies the original string and returns a new string with only the first letter capitalized (this is sometimes called *sentence capitalization*):

lick here to view code image

```
In [1]: 'happy birthday'.capitalize()
Out[1]: 'Happy birthday'
```

Capitalizing the First Character of Every Word in a String

Method **`title`** copies the original string and returns a new string with only the first character of each word capitalized (this is sometimes called *book-title capitalization*):

lick here to view code image

```
In [2]: 'strings: a deeper look'.title()
Out[2]: 'Strings: A Deeper Look'
```

8.6 COMPARISON OPERATORS FOR STRINGS

Strings may be compared with the comparison operators. Recall that strings are compared based on their underlying integer numeric values. So uppercase letters compare as less than lowercase letters because uppercase letters have lower integer values. For example, `'A'` is `65` and `'a'` is `97`. You've seen that you can check character codes with `ord`:

lick here to view code image

```
In [1]: print(f'A: {ord("A")}; a: {ord("a")}')
A: 65; a: 97
```

Let's compare the strings `'Orange'` and `'orange'` using the comparison operators:

lick here to view code image

```
In [2]: 'Orange' == 'orange'
```

C

```
Out[2]: False

In [3]: 'Orange' != 'orange'
Out[3]: True

In [4]: 'Orange' < 'orange'
Out[4]: True

In [5]: 'Orange' <= 'orange'
Out[5]: True

In [6]: 'Orange' > 'orange'
Out[6]: False

In [7]: 'Orange' >= 'orange'
Out[7]: False
```

8.7 SEARCHING FOR SUBSTRINGS

You can search in a string for one or more adjacent characters—known as a *substring*—to count the number of occurrences, determine whether a string contains a substring, or determine the index at which a substring resides in a string. Each method shown in this section compares characters lexicographically using their underlying numeric values.

Counting Occurrences

String method **`count`** returns the number of times its argument occurs in the string on which the method is called:

lick here to view code image

```
In [1]: sentence = 'to be or not to   be that is the question'

In [2]: sentence.count('to')
Out[2]: 2
```

If you specify as the second argument a *start_index*, `count` searches only the slice *string*`[`*start_index*`:]`—that is, from *start_index* through end of the string:

lick here to view code image

```
In [3]: sentence.count('to', 12)
Out[3]: 1
```

C

If you specify as the second and third arguments the *start_index* and *end_index*, `count` searches only the slice *string*`[`*start_index*`:`*end_index*`]`—that is, from *start_index* up to, but not including, *end_index*:

lick here to view code image

```
In [4]: sentence.count('that',   12, 25)
Out[4]: 1
```

Like `count`, each of the other string methods presented in this section has *start_index* and *end_index* arguments for searching only a slice of the original string.

Locating a Substring in a String

String method **`index`** searches for a substring within a string and returns the first index at which the substring is found; otherwise, a `ValueError` occurs:

lick here to view code image

```
In [5]: sentence.index('be')
Out[5]: 3
```

String method **`rindex`** performs the same operation as `index`, but searches from the end of the string and returns the *last* index at which the substring is found; otherwise, a `Value-Error` occurs:

lick here to view code image

```
In [6]: sentence.rindex('be')
Out[6]: 16
```

String methods **`find`** and **`rfind`** perform the same tasks as `index` and `rindex` but, if the substring is not found, return `-1` rather than causing a `Value-Error`.

Determining Whether a String Contains a Substring

If you need to know only whether a string contains a substring, use operator `in` or `not in`:

lick here to view code image

C

```
In [7]: 'that' in sentence
Out[7]: True

In [8]: 'THAT' in sentence
Out[8]: False

In [9]: 'THAT' not in sentence
Out[9]: True
```

Locating a Substring at the Beginning or End of a String

String methods **`startswith`** and **`endswith`** return `True` if the string starts with or ends with a specified substring:

lick here to view code image

```
In [10]: sentence.startswith('to')
Out[10]: True

In [11]: sentence.startswith('be')
Out[11]: False

In [12]: sentence.endswith('question')
Out[12]: True

In [13]: sentence.endswith('quest')
Out[13]: False
```

8.8 REPLACING SUBSTRINGS

A common text manipulation is to locate a substring and replace its value. Method **`replace`** takes two substrings. It searches a string for the substring in its first argument and replaces *each* occurrence with the substring in its second argument. The method returns a new string containing the results. Let's replace tab characters with commas:

lick here to view code image

```
In [1]: values = '1\t2\t3\t4\t5'

In [2]: values.replace('\t', ',')
Out[2]: '1,2,3,4,5'
```

Method `replace` can receive an optional third argument specifying the maximum

C

number of replacements to perform.

8.9 SPLITTING AND JOINING STRINGS

When you read a sentence, your brain breaks it into individual words, or **tokens,** each of which conveys meaning. Interpreters like IPython tokenize statements, breaking them into individual components such as keywords, identifiers, operators and other elements of a programming language. Tokens typically are separated by whitespace characters such as blank, tab and newline, though other characters may be used—the separators are known as **delimiters**.

Splitting Strings

We showed previously that string method `split` with *no* arguments tokenizes a string by breaking it into substrings at each whitespace character, then returns a list of tokens. To tokenize a string at a custom delimiter (such as each comma-and-space pair), specify the delimiter string (such as, `', '`) that `split` uses to tokenize the string:

lick here to view code image

```
In [1]: letters = 'A, B, C, D'

In [2]: letters.split(', ')
Out[2]: ['A', 'B', 'C', 'D']
```

If you provide an integer as the second argument, it specifies the maximum number of splits. The last token is the remainder of the string after the maximum number of splits:

lick here to view code image

```
In [3]: letters.split(', ', 2)
Out[3]: ['A', 'B', 'C, D']
```

There is also an **`rsplit`** method that performs the same task as `split` but processes the maximum number of splits from the end of the string toward the beginning.

Joining Strings

String method **`join`** concatenates the strings in its argument, which must be an iterable containing only string values; otherwise, a `TypeError` occurs. The separator between the concatenated items is the string on which you call `join`. The following

C

code creates strings containing comma-separated lists of values:

lick here to view code image

```
In [4]: letters_list = ['A', 'B', 'C', 'D']
In [5]: ','.join(letters_list)
Out[5]: 'A,B,C,D'
```

The next snippet joins the results of a list comprehension that creates a list of strings:

lick here to view code image

```
In [6]: ','.join([str(i) for i in range(10)])
Out[6]: '0,1,2,3,4,5,6,7,8,9'
```

In the "Files and Exceptions" chapter, you'll see how to work with files that contain comma-separated values. These are known as **CSV files** and are a common format for storing data that can be loaded by spreadsheet applications like Microsoft Excel or Google Sheets. In the data science case study chapters, you'll see that many key libraries, such as NumPy, Pandas and Seaborn, provide built-in capabilities for working with CSV data.

String Methods `partition` and `rpartition`

String method **`partition`** splits a string into a tuple of three strings based on the method's *separator* argument. The three strings are

- the part of the original string before the separator,
- the separator itself, and
- the part of the string after the separator.

This might be useful for splitting more complex strings. Consider a string representing a student's name and grades:

```
'Amanda: 89, 97, 92'
```

Let's split the original string into the student's name, the separator `': '` and a string representing the list of grades:

C

lick here to view code image

```
In [7]: 'Amanda: 89, 97, 92'.partition(': ')
Out[7]: ('Amanda', ': ', '89, 97, 92')
```

To search for the separator from the end of the string instead, use method **`rpartition`** to split. For example, consider the following URL string:

```
' ttp://www.deitel.com/books/PyCDS/table_of_contents.html'
```

Let's use `rpartition` split `'table_of_contents.html'` from the rest of the URL:

lick here to view code image

```
In [8]: url = 'http://www.deitel.com/books/PyCDS/table_of_contents.html'

In [9]: rest_of_url, separator, document = url.rpartition('/')

In [10]: document
Out[10]: 'table_of_contents.html'

In [11]: rest_of_url
Out[11]: 'http://www.deitel.com/books/PyCDS'
```

String Method `splitlines`

In the "Files and Exceptions" chapter, you'll read text from a file. If you read large amounts of text into a string, you might want to split the string into a list of lines based on newline characters. Method **`splitlines`** returns a list of new strings representing the lines of text split at each newline character in the original string. Recall that Python stores multiline strings with embedded `\n` characters to represent the line breaks, as shown in snippet `[13]`:

lick here to view code image

```
In [12]: lines = """This   is line 1
    ...: This is line2
    ...: This is   line3"""

In [13]: lines
Out[13]: 'This is line 1\nThis is line2\nThis is line3'
```

€

```
In [14]: lines.splitlines()
Out[14]: ['This is line 1', 'This is line2', 'This is   line3']
```

Passing `True` to `splitlines` keeps the newlines at the end of each string:

lick here to view code image

```
In [15]: lines.splitlines(True)
Out[15]: ['This is line 1\n', 'This is line2\n', 'This is   line3']
```

8.10 CHARACTERS AND CHARACTER-TESTING METHODS

Many programming languages have separate string and character types. In Python, a character is simply a one-character string.

Python provides string methods for testing whether a string matches certain characteristics. For example, string method **`isdigit`** returns `True` if the string on which you call the method contains only the digit characters (`0`–`9`). You might use this when validating user input that must contain only digits:

lick here to view code image

```
In [1]: '-27'.isdigit()
Out[1]: False

In [2]: '27'.isdigit()
Out[2]: True
```

and the string method `isalnum` returns `True` if the string on which you call the method is alphanumeric—that is, it contains only digits and letters:

lick here to view code image

```
In [3]: 'A9876'.isalnum()
Out[3]: True

In [4]: '123 Main Street'.isalnum()
Out[4]: False
```

The table below shows many of the character-testing methods. Each method returns

C

`alse` if the condition described is not satisfied:

String Method	Description
`isalnum()`	Returns `True` if the string contains only *alphanumeric* characters (i.e., digits and letters).
`isalpha()`	Returns `True` if the string contains only *alphabetic* characters (i.e., letters).
`isdecimal()`	Returns `True` if the string contains only *decimal integer* characters (that is, base 10 integers) and does not contain a + or – sign.
`isdigit()`	Returns `True` if the string contains only digits (e.g., `'0'`, `'1'`, `'2'`).
`isidentifier()`	Returns `True` if the string represents a valid *identifier*.
`islower()`	Returns `True` if all alphabetic characters in the string are *lowercase* characters (e.g., `'a'`, `'b'`, `'c'`).
`isnumeric()`	Returns `True` if the characters in the string represent a *numeric value* without a + or – sign and without a decimal point.
`isspace()`	Returns `True` if the string contains only *whitespace* characters.

F

`istitle()`	Returns `True` if the first character of each word in the string is the only *uppercase* character in the word.
`isupper()`	Returns `True` if all alphabetic characters in the string are *uppercase* characters (e.g., `'A'`, `'B'`, `'C'`).

8.11 RAW STRINGS

Recall that backslash characters in strings introduce *escape sequences*—like `\n` for newline and `\t` for tab. So, if you wish to include a backslash in a string, you must use two back-slash characters `\\`. This makes some strings difficult to read. For example, Microsoft Windows uses backslashes to separate folder names when specifying a file's location. To represent a file's location on Windows, you might write:

lick here to view code image

```
In [1]: file_path = 'C:\\MyFolder\\MySubFolder\\MyFile.txt'

In [2]: file_path
Out[2]: 'C:\\MyFolder\\MySubFolder\\MyFile.txt'
```

For such cases, **raw strings**—preceded by the character `r`—are more convenient. They treat each backslash as a regular character, rather than the beginning of an escape sequence:

lick here to view code image

```
In [3]: file_path = r'C:\MyFolder\MySubFolder\MyFile.txt'

In [4]: file_path
Out[4]: 'C:\\MyFolder\\MySubFolder\\MyFile.txt'
```

Python converts the raw string to a regular string that still uses the two backslash characters in its internal representation, as shown in the last snippet. Raw strings can make your code more readable, particularly when using the regular expressions that we discuss in the next section. Regular expressions often contain many backslash characters.

C

.12 INTRODUCTION TO REGULAR EXPRESSIONS

Sometimes you'll need to recognize *patterns* in text, like phone numbers, e-mail addresses, ZIP Codes, web page addresses, Social Security numbers and more. A **regular expression** string describes a *search pattern* for *matching* characters in other strings.

Regular expressions can help you extract data from unstructured text, such as social media posts. They're also important for ensuring that data is in the correct format before you attempt to process it. [3]

[3] The topic of regular expressions might feel more challenging than most other Python features youve used. After mastering this subject, youll often write more concise code than with conventional string-processing techniques, speeding the code-development process. Youll also deal with fringe cases you might not ordinarily think about, possibly avoiding subtle bugs.

Validating Data

Before working with text data, you'll often use regular expressions to *validate the data*. For example, you can check that:

- A U.S. ZIP Code consists of five digits (such as 02215) or five digits followed by a hyphen and four more digits (such as 02215-4775).
- A string last name contains only letters, spaces, apostrophes and hyphens.
- An e-mail address contains only the allowed characters in the allowed order.
- A U.S. Social Security number contains three digits, a hyphen, two digits, a hyphen and four digits, and adheres to other rules about the specific numbers that can be used in each group of digits.

You'll rarely need to create your own regular expressions for common items like these. Websites like

- `ttps://regex101.com`
- `ttp://www.regexlib.com`
- `ttps://www.regular-expressions.info`

and others offer repositories of existing regular expressions that you can copy and use. Many sites like these also provide interfaces in which you can test regular expressions to determine whether they'll meet your needs.

Other Uses of Regular Expressions

In addition to validating data, regular expressions often are used to:

- Extract data from text (sometimes known as *scraping*)—For example, locating all URLs in a web page. [You might prefer tools like BeautifulSoup, XPath and lxml.]
- Clean data—For example, removing data that's not required, removing duplicate data, handling incomplete data, fixing typos, ensuring consistent data formats, dealing with outliers and more.
- Transform data into other formats—For example, reformatting data that was collected as tab-separated or space-separated values into comma-separated values (CSV) for an application that requires data to be in CSV format.

8.12.1 `re` Module and Function `fullmatch`

To use regular expressions, import the Python Standard Library's **`re`** module:

```
In [1]: import re
```

One of the simplest regular expression functions is **`fullmatch`**, which checks whether the *entire* string in its second argument matches the pattern in its first argument.

Matching Literal Characters

Let's begin by matching *literal characters*—that is, characters that match themselves:

lick here to view code image

```
In [2]: pattern = '02215'

In [3]: 'Match' if re.fullmatch(pattern, '02215')   else 'No match'
Out[3]: 'Match'

In [4]: 'Match' if re.fullmatch(pattern, '51220')   else 'No match'
Out[4]: 'No match'
```

C

The function's first argument is the regular expression pattern to match. Any string can be a regular expression. The variable `pattern`'s value, `'02215'`, contains only *literal digits* that match *themselves* in the specified order. The second argument is the string that should entirely match the pattern.

If the second argument matches the pattern in the first argument, `fullmatch` returns an object containing the matching text, which evaluates to `True`. We'll say more about this object later. In snippet `[4]`, even though the second argument contains the *same digits* as the regular expression, they're in a *different* order. So there's no match, and `fullmatch` returns `None`, which evaluates to `False`.

Metacharacters, Character Classes and Quantifiers

Regular expressions typically contain various special symbols called **metacharacters**, which are shown in the table below:

Regular expression metacharacters
`[] {} () \ * + ^ $ ? . |`

The **\ metacharacter** begins each of the predefined **character classes**, each matching a specific set of characters. Let's validate a five-digit ZIP Code:

lick here to view code image

```
In [5]: 'Valid' if re.fullmatch(r'\d{5}', '02215') else 'Invalid'
Out[5]: 'Valid'

In [6]: 'Valid' if re.fullmatch(r'\d{5}', '9876') else 'Invalid'
Out[6]: 'Invalid'
```

In the regular expression `\d{5}`, **`\d`** is a character class representing a digit (0–9). A character class is a *regular expression escape sequence* that matches *one* character. To match more than one, follow the character class with a **quantifier**. The quantifier `{5}` repeats `\d` five times, as if we had written `\d\d\d\d\d`, to match five consecutive digits. In snippet `[6]`, `fullmatch` returns `None` because `'9876'` contains only four consecutive digit characters.

C

Other Predefined Character Classes

The table below shows some common predefined character classes and the groups of characters they match. To match any metacharacter as its *literal* value, precede it by a backslash (`\`). For example, `\\` matches a backslash (`\`) and `\$` matches a dollar sign (`$`).

Character class	Matches
`\d`	Any digit (0–9).
`\D`	Any character that is *not* a digit.
`\s`	Any whitespace character (such as spaces, tabs and newlines).
`\S`	Any character that is *not* a whitespace character.
`\w`	Any **word character** (also called an **alphanumeric character**)—that is, any uppercase or lowercase letter, any digit or an underscore
`\W`	Any character that is *not* a word character.

Custom Character Classes

Square brackets, `[]`, define a **custom character class** that matches a *single* character. For example, `[aeiou]` matches a lowercase vowel, `[A-Z]` matches an uppercase letter, `[a-z]` matches a lowercase letter and `[a-zA-Z]` matches any lowercase or uppercase letter.

Let's validate a simple first name with no spaces or punctuation. We'll ensure that it begins with an uppercase letter (`A–Z`) followed by any number of lowercase letters (`a–z`):

lick here to view code image

```
In [7]: 'Valid' if re.fullmatch('[A-Z][a-z]*',   'Wally') else 'Invalid'
Out[7]: 'Valid'

In [8]: 'Valid' if re.fullmatch('[A-Z][a-z]*',   'eva') else 'Invalid'
Out[8]: 'Invalid'
```

A first name might contain many letters. The *** quantifier** matches *zero or more occurrences* of the subexpression to its left (in this case, `[a-z]`). So `[A-Z][a-z]*` matches an uppercase letter followed by *zero or more* lowercase letters, such as `'Amanda'`, `'Bo'` or even `'E'`.

When a custom character class starts with a **caret (^)**, the class matches any character that's *not* specified. So `[^a-z]` matches any character that's *not* a lowercase letter:

lick here to view code image

```
In [9]: 'Match' if re.fullmatch('[^a-z]', 'A') else 'No match'
Out[9]: 'Match'

In [10]: 'Match' if re.fullmatch('[^a-z]', 'a') else 'No match'
Out[10]: 'No match'
```

Metacharacters in a custom character class are treated as literal characters—that is, the characters themselves. So `[*+$]` matches a *single* `*`, `+` or `$` character:

lick here to view code image

```
In [11]: 'Match' if re.fullmatch('[*+$]', '*') else 'No match'
Out[11]: 'Match'

In [12]: 'Match' if re.fullmatch('[*+$]', '!') else 'No match'
Out[12]: 'No match'
```

* vs. + Quantifier

If you want to require *at least one* lowercase letter in a first name, you can replace the `*`

C

quantifier in snippet [7] with **+**, which matches *at least one occurrence* of a subexpression:

lick here to view code image

```
In [13]: 'Valid' if re.fullmatch('[A-Z][a-z]+',   'Wally') else 'Invalid'
Out[13]: 'Valid'

In [14]: 'Valid' if re.fullmatch('[A-Z][a-z]+',   'E') else 'Invalid'
Out[14]: 'Invalid'
```

Both * and + are **greedy**—they match as many characters as possible. So the regular expression [A-Z][a-z]+ matches 'Al', 'Eva', 'Samantha', 'Benjamin' and any other words that begin with a capital letter followed at least one lowercase letter.

Other Quantifiers

The **? quantifier** matches *zero or one occurrences* of a subexpression:

lick here to view code image

```
In [15]: 'Match' if re.fullmatch('labell?ed', 'labelled') else 'No match'
Out[15]: 'Match'

In [16]: 'Match' if re.fullmatch('labell?ed',   'labeled') else 'No match'
Out[16]: 'Match'

In [17]: 'Match' if re.fullmatch('labell?ed',   'labellled') else 'No matc
 ut[17]: 'No match'
```

The regular expression labell?ed matches labelled (the U.K. English spelling) and labeled (the U.S. English spelling), but not the misspelled word labellled. In each snippet above, the first five literal characters in the regular expression (label) match the first five characters of the second arguments. Then l? indicates that there can be *zero or one more* l characters before the remaining literal ed characters.

You can match *at least n occurrences* of a subexpression with the **{*n*,} quantifier**. The following regular expression matches strings containing *at least* three digits:

lick here to view code image

```
In [18]: 'Match' if re.fullmatch(r'\d{3,}',   '123') else 'No match'
Out[18]: 'Match'

In [19]: 'Match' if re.fullmatch(r'\d{3,}',   '1234567890') else 'No match
 ut[19]: 'Match'

In [20]: 'Match' if re.fullmatch(r'\d{3,}', '12') else 'No match'
Out[20]: 'No match'
```

You can match *between n and m (inclusive) occurrences* of a subexpression with the **`{n,m}` quantifier**. The following regular expression matches strings containing 3 to 6 digits:

lick here to view code image

```
In [21]: 'Match' if re.fullmatch(r'\d{3,6}',   '123') else 'No match'
Out[21]: 'Match'

In [22]: 'Match' if re.fullmatch(r'\d{3,6}',   '123456') else 'No match'
Out[22]: 'Match'

In [23]: 'Match' if re.fullmatch(r'\d{3,6}', '1234567') else 'No match'
Out[23]: 'No match'

In [24]: 'Match' if re.fullmatch(r'\d{3,6}', '12') else 'No match'
Out[24]: 'No match'
```

8.12.2 Replacing Substrings and Splitting Strings

The `re` module provides function `sub` for replacing patterns in a string, and function `split` for breaking a string into pieces, based on patterns.

Function `sub`—Replacing Patterns

By default, the `re` module's **`sub` function** replaces *all* occurrences of a pattern with the replacement text you specify. Let's convert a tab-delimited string to comma-delimited:

lick here to view code image

```
In [1]: import re

In [2]: re.sub(r'\t', ', ', '1\t2\t3\t4')
```

C

```
Out[2]: '1, 2, 3, 4'
```

The `sub` function receives three required arguments:

- the *pattern to match* (the tab character `'\t'`)
- the *replacement text* (`', '`) and
- the *string to be searched* (`'1\t2\t3\t4'`)

and returns a new string. The keyword argument `count` can be used to specify the maximum number of replacements:

lick here to view code image

```
In [3]: re.sub(r'\t', ', ', '1\t2\t3\t4',    count=2)
Out[3]: '1, 2, 3\t4'
```

Function `split`

The **`split` function** *tokenizes* a string, using a regular expression to specify the *delimiter*, and returns a list of strings. Let's tokenize a string by splitting it at any comma that's followed by 0 or more whitespace characters—`\s` is the whitespace character class and `*` indicates *zero or more* occurrences of the preceding subexpression:

lick here to view code image

```
In [4]: re.split(r',\s*', '1,  2,  3,4,    5,6,7,8')
Out[4]: ['1', '2', '3', '4', '5', '6', '7', '8']
```

Use the keyword argument `maxsplit` to specify the maximum number of splits:

lick here to view code image

```
In [5]: re.split(r',\s*', '1,  2,  3,4,    5,6,7,8', maxsplit=3)
Out[5]: ['1', '2', '3', '4,    5,6,7,8']
```

In this case, after the `3` splits, the fourth string contains the rest of the original string.

C

8.12.3 Other Search Functions; Accessing Matches

Earlier we used the `fullmatch` function to determine whether an *entire* string matched a regular expression. There are several other searching functions. Here, we discuss the `search`, `match`, `findall` and `finditer` functions, and show how to access the matching substrings.

Function `search`—Finding the First Match Anywhere in a String

Function **`search`** looks in a string for the *first* occurrence of a substring that matches a regular expression and returns a **match object** (of type **`SRE_Match`**) that contains the matching substring. The match object's **`group`** method returns that substring:

lick here to view code image

```
In [1]: import re

In [2]: result = re.search('Python',   'Python is fun')

In [3]: result.group() if result   else 'not found'
Out[3]: 'Python'
```

Function `search` returns `None` if the string does *not* contain the pattern:

lick here to view code image

```
In [4]: result2 = re.search('fun!',   'Python is fun')

In [5]: result2.group() if   result2 else 'not found'
Out[5]: 'not found'
```

You can search for a match only at the *beginning* of a string with function **`match`**.

Ignoring Case with the Optional `flags` Keyword Argument

Many `re` module functions receive an optional `flags` keyword argument that changes how regular expressions are matched. For example, matches are *case sensitive* by default, but by using the `re` module's `IGNORECASE` constant, you can perform a *case-insensitive* search:

lick here to view code image

```
In [6]: result3 = re.search('Sam',   'SAM WHITE', flags=re.IGNORECASE)
```

C

```
In [7]: result3.group() if   result3 else 'not found'
Out[7]: 'SAM'
```

Here, `'SAM'` matches the pattern `'Sam'` because both have the same letters, even though `'SAM'` contains only uppercase letters.

Metacharacters That Restrict Matches to the Beginning or End of a String

The **^ metacharacter** at the beginning of a regular expression (and not inside square brackets) is an anchor indicating that the expression matches only the *beginning* of a string:

lick here to view code image

```
In [8]: result = re.search('^Python',   'Python is fun')

In [9]: result.group() if result   else 'not found'
Out[9]: 'Python'

In [10]: result = re.search('^fun',   'Python is fun')

In [11]: result.group() if   result else 'not found'
Out[11]: 'not found'
```

Similarly, the **$ metacharacter** at the end of a regular expression is an anchor indicating that the expression matches only the *end* of a string:

lick here to view code image

```
In [12]: result = re.search('Python$',   'Python is fun')

In [13]: result.group() if   result else 'not found'
Out[13]: 'not found'

In [14]: result = re.search('fun$', 'Python is fun')

In [15]: result.group() if   result else 'not found'
Out[15]: 'fun'
```

Function `findall` and `finditer`—Finding All Matches in a String

Function **`findall`** finds *every* matching substring in a string and returns a list of the matching substrings. Let's extract all the U.S. phone numbers from a string. For simplicity we'll assume that U.S. phone numbers have the form `###-###-####`:

C

lick here to view code image

```
In [16]: contact = 'Wally White,   Home: 555-555-1234, Work: 555-555-4321'

In [17]: re.findall(r'\d{3}-\d{3}-\d{4}',   contact)
Out[17]: ['555-555-1234', '555-555-4321']
```

Function **`finditer`** works like `findall`, but returns a lazy *iterable* of match objects. For large numbers of matches, using `finditer` can save memory because it returns one match at a time, whereas `findall` returns all the matches at once:

lick here to view code image

```
In [18]: for phone in re.finditer(r'\d{3}-\d{3}-\d{4}',   contact):
    ...:     print(phone.group())
    ...:
555-555-1234
555-555-4321
```

Capturing Substrings in a Match

You can use **parentheses metacharacters**—`(` and `)`—to capture substrings in a match. For example, let's capture as separate substrings the name and e-mail address in the string `text`:

lick here to view code image

```
In [19]: text = 'Charlie Cyan,   e-mail: demo1@deitel.com'

In [20]: pattern = r'([A-Z][a-z]+   [A-Z][a-z]+), e-mail: (\w+@\w+\.\w{3})

 n [21]: result = re.search(pattern, text)
```

The regular expression specifies two substrings to capture, each denoted by the metacharacters `(` and `)`. These metacharacters do *not* affect whether the `pattern` is found in the string `text`—the `match` function returns a match object *only* if the *entire* `pattern` is found in the string `text`.

Let's consider the regular expression:

- `'([A-Z][a-z]+ [A-Z][a-z]+)'` matches two words separated by a space. Each

word must have an initial capital letter.

- `', e-mail: '` contains literal characters that match themselves.
- `(\w+@\w+\.\w{3})` matches a *simple* e-mail address consisting of one or more alphanumeric characters (`\w+`), the `@` character, one or more alphanumeric characters (`\w+`), a dot (`\.`) and three alphanumeric characters (`\w{3}`). We preceded the dot with `\` because a dot (`.`) is a regular expression metacharacter that matches one character.

The match object's **`groups`** method returns a tuple of the captured substrings:

lick here to view code image

```
In [22]: result.groups()
Out[22]: ('Charlie Cyan', 'demo1@deitel.com')
```

The match object's `group` method returns the *entire* match as a single string:

lick here to view code image

```
In [23]: result.group()
Out[23]: 'Charlie Cyan, e-mail: demo1@deitel.com'
```

You can access each captured substring by passing an integer to the `group` method. The captured substrings are *numbered from* `1` (unlike list indices, which start at `0`):

lick here to view code image

```
In [24]: result.group(1)
Out[24]: 'Charlie Cyan'

In [25]: result.group(2)
Out[25]: 'demo1@deitel.com'
```

8.13 INTRO TO DATA SCIENCE: PANDAS, REGULAR EXPRESSIONS AND DATA MUNGING

Data does not always come in forms ready for analysis. It could, for example, be in the wrong format, incorrect or even missing. Industry experience has shown that data

C

cientists can spend as much as 75% of their time preparing data before they begin their studies. Preparing data for analysis is called **data munging** or **data wrangling**. These are synonyms—from this point forward, we'll say data munging.

Two of the most important steps in data munging are *data cleaning* and *transforming data* into the optimal formats for your database systems and analytics software. Some common data cleaning examples are:

- deleting observations with missing values,
- substituting reasonable values for missing values,
- deleting observations with bad values,
- substituting reasonable values for bad values,
- tossing outliers (although sometimes you'll want to keep them),
- duplicate elimination (although sometimes duplicates are valid),
- dealing with inconsistent data,
- and more.

You're probably already thinking that data cleaning is a difficult and messy process where you could easily make bad decisions that would negatively impact your results. This is correct. When you get to the data science case studies in the later chapters, you'll see that data science is more of an **empirical science**, like medicine, and less of a theoretical science, like theoretical physics. Empirical sciences base their conclusions on observations and experience. For example, many medicines that effectively solve medical problems today were developed by observing the effects that early versions of these medicines had on lab animals and eventually humans, and gradually refining ingredients and dosages. The actions data scientists take can vary per project, be based on the quality and nature of the data and be affected by evolving organization and professional standards.

Some common data transformations include:

- removing unnecessary data and *features* (we'll say more about features in the data science case studies),

s

- combining related features,
- sampling data to obtain a representative subset (we'll see in the data science case studies that *random sampling* is particularly effective for this and we'll say why),
- standardizing data formats,
- grouping data,
- and more.

It's always wise to hold onto your original data. We'll show simple examples of cleaning and transforming data in the context of Pandas `Series` and `DataFrame`s.

Cleaning Your Data

Bad data values and missing values can significantly impact data analysis. Some data scientists advise against any attempts to insert "reasonable values." Instead, they advocate clearly marking missing data and leaving it up to the data analytics package to handle the issue. Others offer strong cautions. [4]

[4] This footnote was abstracted from a comment sent to us July 20, 2018 by one of the books reviewers, Dr. Alison Sanchez of the University of San Diego School of Business. She commented: Be cautious when mentioning 'substituting reasonable values' for missing or bad values. A stern warning: 'Substituting' values that increase statistical significance or give more 'reasonable' or 'better' results is not permitted. 'Substituting' data should not turn into 'fudging' data. The first rule readers should learn is not to eliminate or change values that contradict their hypotheses. 'Substituting reasonable values' does not mean readers should feel free to change values to get the results they want.

Let's consider a hospital that records patients' temperatures (and probably other vital signs) four times per day. Assume that the data consists of a name and four `float` values, such as

```
['Brown, Sue', 98.6, 98.4, 98.7, 0.0]
```

The preceding patient's first three recorded temperatures are 99.7, 98.4 and 98.7. The last temperature was missing and recorded as 0.0, perhaps because the sensor malfunctioned. The average of the first three values is 98.57, which is close to normal.

However, if you calculate the average temperature *including* the missing value for which 0.0 was substituted, the average is only 73.93, clearly a questionable result. Certainly, doctors would not want to take drastic remedial action on this patient—it's crucial to "get the data right."

One common way to clean the data is to substitute a *reasonable* value for the missing temperature, such as the average of the patient's other readings. Had we done that above, then the patient's average temperature would remain 98.57—a much more likely average temperature, based on the other readings.

Data Validation

Let's begin by creating a `Series` of five-digit ZIP Codes from a dictionary of city-name/five-digit-ZIP-Code key–value pairs. We intentionally entered an invalid ZIP Code for Miami:

lick here to view code image

```
In [1]: import pandas as pd

In [2]: zips = pd.Series({'Boston':   '02215', 'Miami': '3310'})

In [3]: zips
Out[3]:
Boston     02215
Miami       3310
dtype: object
```

Though `zips` looks like a two-dimensional array, it's actually one-dimensional. The "second column" represents the `Series`' ZIP Code *values* (from the dictionary's values), and the "first column" represents their *indices* (from the dictionary's keys).

We can use regular expressions with Pandas to validate data. The **`str` attribute** of a `Series` provides string-processing and various regular expression methods. Let's use the `str` attribute's **`match` method** to check whether each ZIP Code is valid:

lick here to view code image

```
In [4]: zips.str.match(r'\d{5}')
Out[4]:
Boston      True
Miami      False
dtype: bool
```

C

Method `match` applies the regular expression `\d{5}` to *each* `Series` element, attempting to ensure that the element is comprised of exactly five digits. You do not need to loop explicitly through all the ZIP Codes—`match` does this for you. This is another example of functional-style programming with internal rather than external iteration. The method returns a new `Series` containing `True` for each valid element. In this case, the ZIP Code for Miami did *not* match, so its element is `False`.

There are several ways to deal with invalid data. One is to catch it at its source and interact with the source to correct the value. That's not always possible. For example, the data could be coming from high-speed sensors in the Internet of Things. In that case, we would not be able to correct it at the source, so we could apply data cleaning techniques. In the case of the bad Miami ZIP Code of `3310`, we might look for Miami ZIP Codes beginning with `3310`. There are two—`33101` and `33109`—and we could pick one of those.

Sometimes, rather than matching an *entire* value to a pattern, you'll want to know whether a value contains a *substring* that matches the pattern. In this case, use method **`contains`**- instead of `match`. Let's create a `Series` of strings, each containing a U.S. city, state and ZIP Code, then determine whether each string contains a substring matching the pattern `' [A-Z]{2} '` (a space, followed by two uppercase letters, followed by a space):

lick here to view code image

```
In [5]: cities = pd.Series(['Boston,   MA 02215', 'Miami, FL 33101'])

In [6]: cities
Out[6]:
0    Boston, MA 02215
1     Miami, FL 33101
dtype: object

In [7]: cities.str.contains(r'   [A-Z]{2} ')
Out[7]:
0    True
1    True
dtype: bool

In [8]: cities.str.match(r' [A-Z]{2}   ')
Out[8]:
0    False
1    False
dtype: bool
```

C

We did not specify the index values, so the `Series` uses zero-based indexes by default (snippet `[6]`). Snippet `[7]` uses `contains` to show that both `Series` elements contain substrings that match `' [A-Z]{2} '`. Snippet `[8]` uses `match` to show that neither element's value matches that pattern in its entirety, because each has other characters in its complete value.

Reformatting Your Data

We've discussed data cleaning. Now let's consider munging data into a different format. As a simple example, assume that an application requires U.S. phone numbers in the format `###-###-####`, with hyphens separating each group of digits. The phone numbers have been provided to us as 10-digit strings without hyphens. Let's create the `DataFrame`:

lick here to view code image

```
In [9]: contacts = [['Mike Green',   'demo1@deitel.com', '5555555555'],
   ...:             ['Sue Brown',   'demo2@deitel.com', '5555551234']]
   ...:

In [10]: contactsdf = pd.DataFrame(contacts,
    ...:                           columns=['Name', 'Email', 'Phone'])
    ...:

In [11]: contactsdf
Out[11]:
         Name              Email       Phone
0  Mike Green  demo1@deitel.com  5555555555
1   Sue Brown  demo2@deitel.com  5555551234
```

In this `DataFrame`, we specified column indices via the `columns` keyword argument but did *not* specify row indices, so the rows are indexed from `0`. Also, the output shows the column values right aligned by default. This differs from Python formatting in which numbers in a field are *right aligned* by default but non-numeric values are *left aligned* by default.

Now, let's munge the data with a little more functional-style programming. We can *map* the phone numbers to the proper format by calling the `Series` method **`map`** on the `DataFrame`'s `'Phone'` column. Method `map`'s argument is a *function* that receives a value and returns the *mapped* value. The function `get_formatted_phone` maps 10 consecutive digits into the format `###-###-####`:

lick here to view code image

C

```
 n [12]: import re

In [13]: def   get_formatted_phone(value):
    ...:     result = re.fullmatch(r'(\d{3})(\d{3})(\d{4})',   value)
    ...:     return '-'.join(result.groups()) if   result else value
    ...:
    ...:
```

The regular expression in the block's first statement matches *only* 10 consecutive digits. It captures substrings containing the first three digits, the next three digits and the last four digits. The `return` statement operates as follows:

- If `result` is `None`, we simply return `value` unmodified.
- Otherwise, we call `result.groups()` to get a tuple containing the captured substrings and pass that tuple to string method `join` to concatenate the elements, separating each from the next with `'-'` to form the mapped phone number.

`Series` method `map` returns a new `Series` containing the results of calling its function argument for each value in the column. Snippet `[15]` displays the result, including the column's name and type:

lick here to view code image

```
In [14]: formatted_phone = contactsdf['Phone'].map(get_formatted_phone)

In [15]: formatted_phone
0      555-555-5555
1      555-555-1234
Name: Phone, dtype: object
```

Once you've confirmed that the data is in the correct format, you can update it in the original `DataFrame` by assigning the new `Series` to the `'Phone'` column:

lick here to view code image

```
In [16]: contactsdf['Phone'] =   formatted_phone

In [17]: contactsdf
Out[17]:
         Name              Email         Phone
0  Mike Green  demo1@deitel.com  555-555-5555
1   Sue Brown  demo2@deitel.com  555-555-1234
```

C

e'll continue our pandas discussion in the next chapter's Intro to Data Science section, and we'll use pandas in several later chapters.

8.14 WRAP-UP

In this chapter, we presented various string formatting and processing capabilities. You formatted data in f-strings and with the string method `format`. We showed the augmented assignments for concatenating and repeating strings. You used string methods to remove whitespace from the beginning and end of strings and to change their case. We discussed additional methods for splitting strings and for joining iterables of strings. We introduced various character-testing methods.

We showed raw strings that treat backslashes (`\`) as literal characters rather than the beginning of escape sequences. These were particularly useful for defining regular expressions, which often contain many backslashes.

Next, we introduced the powerful pattern-matching capabilities of regular expressions with functions from the `re` module. We used the `fullmatch` function to ensure that an entire string matched a pattern, which is useful for validating data. We showed how to use the `replace` function to search for and replace substrings. We used the `split` function to tokenize strings based on delimiters that match a regular expression pattern. Then we showed various ways to search for patterns in strings and to access the resulting matches.

In the Intro to Data Science section, we introduced the synonyms data munging and data wrangling and showed q sample data munging operation, namely transforming data. We continued our discussion of Panda's `Series` and `DataFrame`s by using regular expressions to validate and munge data.

In the next chapter, we'll continue using various string-processing capabilities as we introduce reading text from files and writing text to files. We'll use the `csv` module for manipulating comma-separated value (CSV) files. We'll also introduce exception handling so we can process exceptions as they occur, rather than displaying a traceback.

W

. Files and Exceptions

Objectives

In this chapter, you'll:

- Understand the notions of files and persistent data.
- Read, write and update files.
- Read and write CSV files, a common format for machine-learning datasets.
- Serialize objects into the JSON data-interchange format—commonly used to transmit over the Internet—and deserialize JSON into objects.
- Use the `with` statement to ensure that resources are properly released, avoiding "resource leaks".
- Use the `try` statement to delimit code in which exceptions may occur and handle those exceptions with associated `except` clauses.
- Use the `try` statement's `else` clause to execute code when no exceptions occur in the `try` suite.
- Use the `try` statement's `finally` clause to execute code regardless of whether an exception occurs in the `try`.
- `raise` exceptions to indicate runtime problems.
- Understand the traceback of functions and methods that led to an exception.
- Use pandas to load into a `DataFrame` and process the Titanic Disaster CSV dataset.

Outline

9.1 INTRODUCTION

Variables, lists, tuples, dictionaries, sets, arrays, pandas `Series` and pandas `DataFrame`s offer only *temporary* data storage. The data is lost when a local variable "goes out of scope" or when the program terminates. **Files** provide long-term retention of typically large amounts of data, even after the program that created the data terminates, so data maintained in files is persistent. Computers store files on secondary storage devices, including solid-state drives, hard disks and more. In this chapter, we explain how Python programs create, update and process data files.

We consider text files in several popular formats—plain text, JSON (JavaScript Object Notation) and CSV (comma-separated values). We'll use JSON to serialize and deserialize objects to facilitate saving those objects to secondary storage and transmitting them over the Internet. Be sure to read this chapter's Intro to Data Science section in which we'll use both the Python Standard Library's `csv` module and pandas to load and manipulate CSV data. In particular, we'll look at the CSV version of the Titanic disaster dataset. We'll use many popular datasets in upcoming data-science case-study chapters on natural language processing, data mining Twitter, IBM Watson, machine learning, deep learning and big data.

As part of our continuing emphasis on Python security, we'll discuss the security vulnerabilities of serializing and deserializing data with the Python Standard Library's `pickle` module. We recommend JSON serialization in preference to `pickle`.

We also introduce **exception handling**. An exception indicates an execution-time problem. You've seen exceptions of types `ZeroDivisionError`, `NameError`, `ValueError`, `StatisticsError`, `TypeError`, `IndexError`, `KeyError` and `RuntimeError`. We'll show how to deal with exceptions as they occur by using `try` statements and associated `except` clauses to *handle* exceptions. We'll also discuss the `try` statement's `else` and `finally` clauses. The features presented here help you write *robust*, *fault-tolerant* programs that can deal with problems and continue executing or *terminate gracefully*.

rograms typically request and release resources (such as files) during program execution. Often, these are in limited supply or can be used only by one program at a time. We show how to guarantee that after a program uses a resource, it's released for use by other programs, even if an exception has occurred. You'll use the `with` statement for this purpose.

9.2 FILES

Python views a **text file** as a sequence of characters and a **binary file** (for images, videos and more) as a sequence of bytes. As in lists and arrays, the first character in a text file and byte in a binary file is located at position 0, so in a file of *n* characters or bytes, the highest position number is *n* – 1. The diagram below shows a conceptual view of a file:

For each file you **open**, Python creates a **file object** that you'll use to interact with the file.

End of File

Every operating system provides a mechanism to denote the end of a file. Some represent it with an **end-of-file marker** (as in the preceding figure), while others might maintain a count of the total characters or bytes in the file. Programming languages generally hide these operating-system details from you.

Standard File Objects

When a Python program begins execution, it creates three **standard file objects**:

- `sys.stdin`—the **standard input file object**
- `sys.stdout`—the **standard output file object**, and
- `sys.stderr`—the **standard error file object**.

Though these are considered file objects, they do not read from or write to files by default. The `input` function implicitly uses `sys.stdin` to get user input from the keyboard. Function `print` implicitly outputs to `sys.stdout`, which appears in the command line. Python implicitly outputs program errors and tracebacks to

P

`sys.stderr`, which also appears in the command line. You must import the `sys` module if you need to refer to these objects explicitly in your code, but this is rare.

9.3 TEXT-FILE PROCESSING

In this section, we'll write a simple text file that might be used by an accounts-receivable system to track the money owed by a company's clients. We'll then read that text file to confirm that it contains the data. For each client, we'll store the client's account number, last name and account balance owed to the company. Together, these data fields represent a client **record**. Python imposes no structure on a file, so notions such as records do not exist natively in Python. Programmers must structure files to meet their applications' requirements. We'll create and maintain this file in order by account number. In this sense, the account number may be thought of as a **record key**. For this chapter, we assume that you launch IPython from the `ch09` examples folder.

9.3.1 Writing to a Text File: Introducing the `with` Statement

Let's create an `accounts.txt` file and write five client records to the file. Generally, records in text files are stored one per line, so we end each record with a newline character:

lick here to view code image

```
In [1]: with open('accounts.txt', mode='w')   as accounts:
   ...:     accounts.write('100 Jones   24.98\n')
   ...:     accounts.write('200 Doe   345.67\n')
   ...:     accounts.write('300 White   0.00\n')
   ...:     accounts.write('400 Stone   -42.16\n')
   ...:     accounts.write('500 Rich   224.62\n')
   ...:
```

You can also write to a file with `print` (which automatically outputs a `\n`), as in

lick here to view code image

```
print('100 Jones 24.98', file=accounts)
```

The `with` Statement

Many applications *acquire* resources, such as files, network connections, database

C

connections and more. You should *release* resources as soon as they're no longer needed. This practice ensures that other applications can use the resources. Python's **`with` statement**:

- acquires a resource (in this case, the file object for `accounts.txt`) and assigns its corresponding object to a variable (`accounts` in this example),
- allows the application to use the resource via that variable, and
- calls the resource object's `close` method to release the resource when program control reaches the end of the `with` statement's suite.

Built-In Function `open`

The built-in **`open` function** opens the file `accounts.txt` and associates it with a file object. The `mode` argument specifies the **file-open mode**, indicating whether to open a file for reading from the file, for writing to the file or both. The mode `'w'` opens the file for *writing*, creating the file if it does not exist. If you do not specify a path to the file, Python creates it in the current folder (`ch09`). Be careful—opening a file for writing *deletes* all the existing data in the file. By convention, the **`.txt` file extension** indicates a plain text file.

Writing to the File

The `with` statement assigns the object returned by `open` to the variable `accounts` in the **`as` clause**. In the `with` statement's suite, we use the variable `accounts` to interact with the file. In this case, we call the file object's **`write` method** five times to write five records to the file, each as a separate line of text ending in a newline. At the end of the `with` statement's suite, the `with` statement *implicitly* calls the file object's **`close`** method to close the file.

Contents of `accounts.txt` File

After executing the previous snippet, your `ch09` directory contains the file `accounts.txt` with the following contents, which you can view by opening the file in a text editor:

```
100 Jones 24.98
200 Doe 345.67
300 White 0.00
400 Stone -42.16
500 Rich 224.62
```

In the next section, you'll read the file and display its contents.

9.3.2 Reading Data from a Text File

We just created the text file `accounts.txt` and wrote data to it. Now let's read that data from the file sequentially from beginning to end. The following session reads records from the file `accounts.txt` and displays the contents of each record in columns with the `Account` and `Name` columns *left aligned* and the `Balance` column *right aligned*, so the decimal points align vertically:

lick here to view code image

```
In [1]: with open('accounts.txt', mode='r')   as accounts:
   ...:     print(f'{"Account":<10}{"Name":<10}{"Balance":>10}')
   ...:     for record in accounts:
   ...:         account, name, balance = record.split()
   ...:         print(f'{account:<10}{name:<10}{balance:>10}')
   ...:
Account   Name         Balance
100       Jones          24.98
200       Doe           345.67
300       White           0.00
400       Stone         -42.16
500       Rich          224.62
```

If the contents of a file should not be modified, open the file for reading only. This prevents the program from accidentally modifying the file. You open a file for reading by passing the `'r'` file-open mode as function `open`'s second argument. If you do not specify the folder in which to store the file, `open` assumes the file is in the current folder.

Iterating through a file object, as shown in the preceding `for` statement, reads one line at a time from the file and returns it as a string. For each `record` (that is, line) in the file, string method `split` returns tokens in the line as a list, which we unpack into the variables `account`, `name` and `balance`. [1] The last statement in the `for` statement's suite displays these variables in columns using field widths.

[1] When splitting strings on spaces (the default), `split` automatically discards the newline character.

File Method `readlines`

C

The file object's **`readlines`** method also can be used to read an *entire* text file. The method returns each line as a string in a list of strings. For small files, this works well, but iterating over the lines in a file object, as shown above, can be more efficient. [2] Calling `readlines` for a large file can be a time-consuming operation, which must complete before you can begin using the list of strings. Using the file object in a `for` statement enables your program to process each text line as it's read.

[2] ttps://docs.python.org/3/tutorial/inputoutput.html#methods-f-file-objects.

Seeking to a Specific File Position

While reading through a file, the system maintains a **file-position pointer** representing the location of the next character to read. Sometimes it's necessary to process a file sequentially from the beginning *several times* during a program's execution. Each time, you must reposition the file-position pointer to the beginning of the file, which you can do either by closing and reopening the file, or by calling the file object's **`seek`** method, as in

```
file_object.seek(0)
```

The latter approach is faster.

9.4 UPDATING TEXT FILES

Formatted data written to a text file cannot be modified without the risk of destroying other data. If the name `'White'` needs to be changed to `'Williams'` in `accounts.txt`, the old name cannot simply be overwritten. The original record for `White` is stored as

```
300 White 0.00
```

If you overwrite the name `'White'` with the name `'Williams'`, the record becomes

```
300 Williams00
```

The new last name contains three more characters than the original one, so the characters beyond the second "`i`" in `'Williams'` overwrite other characters in the line. The problem is that in the formatted input–output model, records and their fields

can vary in size. For example, 7, 14, –117, 2074 and 27383 are all integers and are stored in the same number of "raw data" bytes internally (typically 4 or 8 bytes in today's systems). However, when these integers are output as formatted text, they become different-sized fields. For example, 7 is one character, 14 is two characters and 27383 is five characters.

To make the preceding name change, we can:

- copy the records before `300 White 0.00` into a temporary file,
- write the updated and correctly formatted record for account 300 to this file,
- copy the records after `300 White 0.00` to the temporary file,
- delete the old file and
- rename the temporary file to use the original file's name.

This can be cumbersome because it requires processing *every* record in the file, even if you need to update only one record. Updating a file as described above is more efficient when an application needs to update many records in one pass of the file. [3]

[3] n the chapter, Big Data: Hadoop, Spark, NoSQL and IoT, youll see that database systems solve this update in place problem efficiently.

Updating `accounts.txt`

Let's use a `with` statement to update the `accounts.txt` file to change account 300's name from `'White'` to `'Williams'` as described above:

lick here to view code image

```
 n [1]: accounts = open('accounts.txt',   'r')

In [2]: temp_file = open('temp_file.txt',   'w')

In [3]: with accounts,   temp_file:
   ...:     for record in accounts:
   ...:         account, name, balance = record.split()
   ...:         if account != '300':
   ...:             temp_file.write(record)
   ...:         else:
   ...:             new_record = ' '.join([account,   'Williams', balance]
   ...:             temp_file.write(new_record + '\n')
```

C

```
    ...:
```

or readability, we opened the file objects (snippets `[1]` and `[2]`), then specified their variable names in the first line of snippet `[3]`. This `with` statement manages two resource objects, specified in a comma-separated list after `with`. The `for` statement unpacks each record into `account`, `name` and `balance`. If the `account` is not `'300'`, we write `record` (which contains a newline) to `temp_file`. Otherwise, we assemble the new record containing `'Williams'` in place of `'White'` and write it to the file. After snippet `[3]`, `temp_file.txt` contains:

```
100 Jones 24.98
200 Doe 345.67
300 Williams 0.00
400 Stone -42.16
500 Rich 224.62
```

`os` Module File-Processing Functions

At this point, we have the old `accounts.txt` file and the new `temp_file.txt`. To complete the update, let's delete the old `accounts.txt` file, then rename `temp_file.txt` as `accounts.txt`. The **`os` module** [4] provides functions for interacting with the operating system, including several that manipulate your system's files and directories. Now that we've created the temporary file, let's use the **`remove` function** [5] to delete the original file:

[4] `ttps://docs.python.org/3/library/os.html`.

[5] Use `remove` with cautionit does not warn you that youre *permanently* deleting the file.

lick here to view code image

```
In [4]: import os

In [5]: os.remove('accounts.txt')
```

Next, let's use the **`rename` function** to rename the temporary file as `'accounts.txt'`:

lick here to view code image

E

```
In [6]: os.rename('temp_file.txt',   'accounts.txt')
```

9.5 SERIALIZATION WITH JSON

Many libraries we'll use to interact with cloud-based services, such as Twitter, IBM Watson and others, communicate with your applications via JSON objects. **JSON (JavaScript Object Notation)** is a text-based, human-and-computer-readable, data-interchange format used to represent objects as collections of name–value pairs. JSON can even represent objects of custom classes like those you'll build in the next chapter.

JSON has become the preferred data format for transmitting objects across platforms. This is especially true for invoking cloud-based web services, which are functions and methods that you call over the Internet. You'll become proficient at working with JSON data. In the "Data Mining Twitter" chapter, you'll access JSON objects containing tweets and their metadata. In the "IBM Watson and Cognitive Computing" chapter, you'll access data in the JSON responses returned by Watson services. In the " ig Data: adoop, Spark, NoSQL and IoT" chapter, we'll store JSON tweet objects that we obtain from Twitter in MongoDB, a popular NoSQL database. In that chapter, we'll also work with other web services that send and receive data as JSON objects.

JSON Data Format

JSON objects are similar to Python dictionaries. Each JSON object contains a comma-separated list of *property names* and *values*, in curly braces. For example, the following key–value pairs might represent a client record:

```
{"account": 100, "name": "Jones", "balance": 24.98}
```

JSON also supports arrays which, like Python lists, are comma-separated values in square brackets. For example, the following is an acceptable JSON array of numbers:

```
[100, 200, 300]
```

Values in JSON objects and arrays can be:

- strings in *double quotes* (like `"Jones"`),
- numbers (like `100` or `24.98`),

- JSON Boolean values (represented as `true` or `false` in JSON),
- `null` (to represent no value, like `None` in Python),
- arrays (like `[100, 200, 300]`), and
- other JSON objects.

Python Standard Library Module `json`

The **`json` module** enables you to convert objects to JSON (JavaScript Object Notation) text format. This is known as **serializing** the data. Consider the following dictionary, which contains one key–value pair consisting of the key `'accounts'` with its associated value being a list of dictionaries representing two accounts. Each account dictionary contains three key–value pairs for the account number, name and balance:

lick here to view code image

```
In [1]: accounts_dict = {'accounts':   [
   ...:     {'account': 100, 'name': 'Jones', 'balance': 24.98},
   ...:     {'account': 200, 'name': 'Doe', 'balance': 345.67}]}
```

Serializing an Object to JSON

Let's write that object in JSON format to a file:

lick here to view code image

```
In [2]: import json

In [3]: with open('accounts.json', 'w') as accounts:
   ...:     json.dump(accounts_dict, accounts)
   ...:
```

Snippet `[3]` opens the file `accounts.json` and uses the `json` module's **`dump` function** to serialize the dictionary `accounts_dict` into the file. The resulting file contains the following text, which we reformatted slightly for readability:

lick here to view code image

```
{"accounts":
   [{"account": 100, "name": "Jones", "balance": 24.98},
    {"account": 200, "name": "Doe", "balance": 345.67}]}
```

C

Note that JSON delimits strings with *double-quote characters*.

Deserializing the JSON Text

The `json` module's **`load` function** reads the entire JSON contents of its file object argument and converts the JSON into a Python object. This is known as **deserializing** the data. Let's reconstruct the original Python object from this JSON text:

lick here to view code image

```
In [4]: with open('accounts.json', 'r') as accounts:
   ...:     accounts_json = json.load(accounts)
   ...:
   ...:
```

We can now interact with the loaded object. For example, we can display the dictionary:

lick here to view code image

```
In [5]: accounts_json
Out[5]:
{'accounts': [{'account': 100, 'name': 'Jones', 'balance':   24.98},
  {'account': 200, 'name': 'Doe', 'balance': 345.67}]}
```

As you'd expect, you can access the dictionary's contents. Let's get the list of dictionaries associated with the `'accounts'` key:

lick here to view code image

```
In [6]: accounts_json['accounts']
Out[6]:
[{'account': 100, 'name': 'Jones', 'balance': 24.98},
 {'account': 200, 'name': 'Doe', 'balance': 345.67}]
```

Now, let's get the individual account dictionaries:

lick here to view code image

```
In [7]: accounts_json['accounts'][0]
Out[7]: {'account': 100, 'name': 'Jones', 'balance': 24.98}

In [8]: accounts_json['accounts'][1]
```

C

```
Out[8]: {'account': 200, 'name': 'Doe', 'balance': 345.67}
```

Though we did not do so here, you can modify the dictionary as well. For example, you could add accounts to or remove accounts from the list, then write the dictionary back into the JSON file.

Displaying the JSON Text

The `json` module's **`dumps` function** (`dumps` is short for "dump string") returns a Python string representation of an object in JSON format. Using `dumps` with `load`, you can read the JSON from the file and display it in a nicely indented format—sometimes called "pretty printing" the JSON. When the `dumps` function call includes the `indent` keyword argument, the string contains newline characters and indentation for pretty printing—you also can use `indent` with the `dump` function when writing to a file:

lick here to view code image

```
In [9]: with open('accounts.json', 'r') as accounts:
   ...:     print(json.dumps(json.load(accounts), indent=4))
   ...:
{
    "accounts": [
        {
            "account": 100,
            "name": "Jones",
            "balance": 24.98
        },
        {
            "account": 200,
            "name": "Doe",
            "balance": 345.67
        }
    ]
}
```

9.6 FOCUS ON SECURITY: `PICKLE` SERIALIZATION AND DESERIALIZATION

The Python Standard Library's **`pickle` module** can serialize objects into in a Python-specific data format. **Caution: The Python documentation provides the following warnings about `pickle`:**

- "Pickle files can be hacked. If you receive a raw pickle file over the network, don't trust it! It could have malicious code in it, that would run arbitrary Python when

C

ou try to de-pickle it. However, if you are doing your own pickle writing and reading, you're safe (provided no one else has access to the pickle file, of course.)" [6]

[6] ttps://wiki.python.org/moin/UsingPickle.

- "Pickle is a protocol which allows the serialization of arbitrarily complex Python objects. As such, it is specific to Python and cannot be used to communicate with applications written in other languages. It is also insecure by default: deserializing pickle data coming from an untrusted source can execute arbitrary code, if the data was crafted by a skilled attacker." [7]

[7] ttps://docs.python.org/3/tutorial/inputoutput.html#reading-nd-writing-files.

We do not recommend using `pickle`, but it's been used for many years, so you're likely to encounter it in **legacy code**—old code that's often no longer supported.

9.7 ADDITIONAL NOTES REGARDING FILES

The following table summarizes the various file-open modes for text files, including the modes for reading and writing we've introduced. The *writing* and *appending* modes create the file if it does not exist. The *reading* modes raise a `FileNotFoundError` if the file does not exist. Each text-file mode has a corresponding binary-file mode specified with `b`, as in `'rb'` or `'wb+'`. You'd use these modes, for example, if you were reading or writing binary files, such as images, audio, video, compressed ZIP files and many other popular custom file formats.

Mode	Description
`'r'`	Open a text file for reading. This is the default if you do not specify the file-open mode when you call open.
`'w'`	Open a text file for writing. Existing file contents are *deleted*.

`'a'`	Open a text file for appending at the end, creating the file if it does not exist. New data is written at the end of the file.
`'r+'`	Open a text file reading and writing.
`'w+'`	Open a text file reading and writing. Existing file contents are *deleted*.
`'a+'`	Open a text file reading and appending at the end. New data is written at the end of the file. If the file does not exist, it is created.

Other File Object Methods

Here are a few more useful file-object methods.

- For a text file, the `read` method returns a string containing the number of characters specified by the method's integer argument. For a binary file, the method returns the specified number of bytes. If no argument is specified, the method returns the entire contents of the file.
- The `readline` method returns one line of text as a string, including the newline character if there is one. This method returns an empty string when it encounters the end of the file.
- The `writelines` method receives a list of strings and writes its contents to a file.

The classes that Python uses to create file objects are defined in the Python Standard Library's **`io` module** (`ttps://docs.python.org/3/library/io.html`).

9.8 HANDLING EXCEPTIONS

Various types of exceptions can occur when you work with files, including:

- A `FileNotFoundError` occurs if you attempt to open a non-existent file for reading with the `'r'` or `'r+'` modes.

h

- A **`PermissionsError`** occurs if you attempt an operation for which you do not have permission. This might occur if you try to open a file that your account is not allowed to access or create a file in a folder where your account does not have permission to write, such as where your computer's operating system is stored.
- A `ValueError` (with the error message `'I/O operation on closed file.'`) occurs when you attempt to write to a file that has already been closed.

9.8.1 Division by Zero and Invalid Input

Let's revisit two exceptions that you saw earlier in the book.

Division By Zero

Recall that attempting to divide by `0` results in a `ZeroDivisionError`:

lick here to view code image

```
In [1]: 10 / 0
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last
 ipython-input-1-a243dfbf119d> in <module>()
----> 1 10 / 0

ZeroDivisionError: division   by zero

In [2]:
```

In this case, the interpreter is said to **raise an exception** of type `ZeroDivisionError`. When an exception is raised in IPython, it:

- terminates the snippet,
- displays the exception's traceback, then
- shows the next `In []` prompt so you can input the next snippet.

If an exception occurs in a script, it terminates and IPython displays the traceback.

Invalid Input

Recall that the `int` function raises a `Value-Error` if you attempt to convert to an

nteger a string (like `'hello'`) that does not represent a number:

lick here to view code image

```
In [2]: value = int(input('Enter an   integer: '))
Enter an integer: hello
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last
 ipython-input-2-b521605464d6> in <module>()
----> 1 value = int(input('Enter   an integer: '))

ValueError: invalid literal   for int() with base 10: 'hello'

In [3]:
```

9.8.2 `try` Statements

Now let's see how to *handle* these exceptions so that you can enable code to continue processing. Consider the following script and sample execution. Its loop attempts to read two integers from the user, then display the first number divided by the second. The script uses exception handling to catch and handle (i.e., deal with) any `ZeroDivisionError`s and `ValueError`s that arise—in this case, allowing the user to re-enter the input.

lick here to view code image

```
1 # dividebyzero.py
2 """Simple exception handling   example."""
3
4 while True:
5     # attempt to convert and divide values
6     try:
7         number1   = int(input('Enter numerator: '))
8         number2   = int(input('Enter denominator: '))
9         result   = number1 / number2
10     except ValueError:  # tried to   convert non-numeric value to int
11         print('You must enter two integers\n')
12     except ZeroDivisionError:  #   denominator was 0
13         print('Attempted to divide by zero\n')
14     else:  # executes only if no   exceptions occur
15         print(f'{number1:.3f} /   {number2:.3f} = {result:.3f}')
16         break  # terminate the loop
```

lick here to view code image

```
Enter numerator: 100
Enter denominator: 0
Attempted to divide by zero

Enter numerator: 100
Enter denominator: hello
You must enter two integers

Enter numerator: 100
Enter denominator: 7
100.000 / 7.000 = 14.286
```

`try` Clause

Python uses **`try` statements** (like lines 6–16) to enable exception handling. The `try` statement's **`try` clause** (lines 6–9) begins with keyword `try`, followed by a colon (`:`) and a suite of statements that *might* raise exceptions.

`except` Clause

A `try` clause may be followed by one or more **`except` clauses** (lines 10–11 and 12–13) that immediately follow the `try` clause's suite. These also are known as *exception handlers*. Each `except` clause specifies the type of exception it handles. In this example, each exception handler just displays a message indicating the problem that occurred.

`else` Clause

After the last `except` clause, an optional **`else` clause** (lines 14–16) specifies code that should execute only if the code in the `try` suite did not raise exceptions. If no exceptions occur in this example's `try` suite, line 15 displays the division result and line 16 terminates the loop.

Flow of Control for a `ZeroDivisionError`

Now let's consider this example's flow of control, based on the first three lines of the sample output:

- First, the user enters `100` for the numerator in response to line 7 in the `try` suite.
- Next, the user enters `0` for the denominator in response to line 8 in the `try` suite.
- At this point, we have two integer values, so line 9 attempts to divide `100` by `0`,

ausing Python to raise a `ZeroDivisionError`. The point in the program at which an exception occurs is often referred to as the **raise point**.

When an exception occurs in a `try` suite, it terminates immediately. If there are any `except` handlers following the `try` suite, program control transfers to the first one. If there are no `except` handlers, a process called *stack unwinding* occurs, which we discuss later in the chapter.

In this example, there *are* `except` handlers, so the interpreter searches for the *first one* that matches the type of the raised exception:

- The `except` clause at lines 10–11 handles `ValueError`s. This does not match the type `ZeroDivisionError`, so that `except` clause's suite does not execute and program control transfers to the next `except` handler.
- The `except` clause at lines 12–13 handles `ZeroDivisionError`s. This *is* a match, so that `except` clause's suite executes, displaying `"Attempted to divide by zero"`.

When an `except` clause successfully handles the exception, program execution resumes with the `finally` clause (if there is one), then with the next statement after the `try` statement. In this example, we reach the end of the loop, so execution resumes with the next loop iteration. Note that after an exception is handled, program control does *not* return to the raise point. Rather, control resumes after the `try` statement. We'll discuss the `finally` clause shortly.

Flow of Control for a `ValueError`

Now let's consider the flow of control, based on the next three lines of the sample output:

- First, the user enters `100` for the numerator in response to line 7 in the `try` suite.
- Next, the user enters `hello` for the denominator in response to line 8 in the `try` suite. The input is not a valid integer, so the `int` function raises a `ValueError`.

The exception terminates the `try` suite and program control transfers to the first `except` handler. In this case, the `except` clause at lines 10–11 is a match, so its suite executes, displaying `"You must enter two integers"`. Then, program execution

c

resumes with the next statement after the `try` statement. Again, that's the end of the loop, so execution resumes with the next loop iteration.

Flow of Control for a Successful Division

Now let's consider the flow of control, based on the last three lines of the sample output:

- First, the user enters `100` for the numerator in response to line 7 in the `try` suite.
- Next, the user enters `7` for the denominator in response to line 8 in the `try` suite.
- At this point, we have two valid integer values and the denominator is not `0`, so line 9 successfully divides `100` by `7`.

When no exceptions occur in the `try` suite, program execution resumes with the `else` clause (if there is one); otherwise, program execution resumes with the next statement after the `try` statement. In this example's `else` clause, we display the division result, then terminate the loop, and the program terminates.

9.8.3 Catching Multiple Exceptions in One `except` Clause

It's relatively common for a `try` clause to be followed by several `except` clauses to handle various types of exceptions. If several `except` suites are identical, you can catch those exception types by specifying them as a tuple in a *single* `except` handler, as in:

```
except (type1, type2, ...) as variable_name:
```

The `as` clause is optional. Typically, programs do not need to reference the caught exception object directly. If you do, you can use the variable in the `as` clause to reference the exception object in the `except` suite.

9.8.4 What Exceptions Does a Function or Method Raise?

Exceptions may surface via statements in a `try` suite, via functions or methods called directly or indirectly from a `try` suite, or via the Python interpreter as it executes the code (for example, `ZeroDivisionError`s).

Before using any function or method, read its online API documentation, which specifies what exceptions are thrown (if any) by the function or method and indicates

easons why such exceptions may occur. Next, read the online API documentation for each exception type to see potential reasons why such an exception occurs.

9.8.5 What Code Should Be Placed in a `try` Suite?

Place in a `try` suite a significant logical section of a program in which several statements can raise exceptions, rather than wrapping a separate `try` statement around every statement that raises an exception. However, for proper exception-handling granularity, each `try` statement should enclose a section of code small enough that, when an exception occurs, the specific context is known and the `except` handlers can process the exception properly. If many statements in a `try` suite raise the same exception types, multiple `try` statements may be required to determine each exception's context.

9.9 `FINALLY` CLAUSE

Operating systems typically can prevent more than one program from manipulating a file at once. When a program finishes processing a file, the program should close it to release the resource so other programs can access it. Closing the file helps prevent a **resource leak**.

The `finally` Clause of the `try` Statement

A `try` statement may have a `finally` clause after any `except` clauses or the `else` clause. The **`finally`** clause is guaranteed to execute. [8] In other languages that have `finally`, this makes the `finally` suite an ideal location to place resource-deallocation code for resources acquired in the corresponding `try` suite. In Python, we prefer the `with` statement for this purpose and place other kinds of "clean up" code in the `finally` suite.

[8] The only reason a `finally` suite will not execute if program control enters the corresponding `try` suite is if the application terminates first, for example by calling the `sys` modules `exit` function.

Example

The following IPython session demonstrates that the `finally` clause always executes, regardless of whether an exception occurs in the corresponding `try` suite. First, let's consider a `try` statement in which no exceptions occur in the `try` suite:

lick here to view code image

€

```
In [1]: try:
   ...:     print('try suite with no   exceptions raised')
   ...: except:
   ...:     print('this will not   execute')
   ...: else:
   ...:     print('else executes   because no exceptions in the try suite'
   ...: finally:
   ...:     print('finally always   executes')
   ...:
try suite with no exceptions raised
else executes because no exceptions in the try suite
finally always executes

In [2]:
```

The preceding `try` suite displays a message but does not raise any exceptions. When program control successfully reaches the end of the `try` suite, the `except` clause is skipped, the `else` clause executes and the `finally` clause displays a message showing that it always executes. When the `finally` clause terminates, program control continues with the next statement after the `try` statement. In an IPython session, the next `In []` prompt appears.

Now let's consider a `try` statement in which an exception occurs in the `try` suite:

lick here to view code image

```
In [2]: try:
   ...:     print('try suite that   raises an exception')
   ...:     int('hello')
   ...:     print('this will not   execute')
   ...: except ValueError:
   ...:     print('a ValueError   occurred')
   ...: else:
   ...:     print('else will not   execute because an exception occurred')
   ...: finally:
   ...:     print('finally always   executes')
   ...:
try suite that raises an exception
a ValueError occurred
finally always executes

In [3]:
```

This `try` suite begins by displaying a message. The second statement attempts to convert the string `'hello'` to an integer, which causes the `int` function to raise a

C

`ValueError`. The `try` suite immediately terminates, skipping its last `print` statement. The `except` clause catches the `ValueError` exception and displays a message. The `else` clause does not execute because an exception occurred. Then, the `finally` clause displays a message showing that it always executes. When the `finally` clause terminates, program control continues with the next statement after the try statement. In an IPython session, the next `In []` prompt appears.

Combining `with` Statements and `try` `except` Statements

Most resources that require explicit release, such as files, network connections and database connections, have potential exceptions associated with processing those resources. For example, a program that processes a file might raise `IOError`s. For this reason, *robust* file-processing code normally appears in a `try` suite containing a `with` statement to guarantee that the resource gets released. The code is in a `try` suite, so you can catch in `except` handlers any exceptions that occur and you do not need a `finally` clause because the `with` statement handles resource deallocation.

To demonstrate this, first let's assume you're asking the user to supply the name of a file and they provide that name incorrectly, such as `gradez.txt` rather than the file we created earlier `grades.txt`. In this case, the `open` call raises a `FileNotFoundError` by attempting to open a non-existent file:

lick here to view code image

```
In [3]: open('gradez.txt')
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last
 ipython-input-3-b7f41b2d5969> in <module>()
----> 1 open('gradez.txt')

FileNotFoundError: [Errno 2]   No such file or directory: 'gradez.txt'
```

To catch exceptions like `FileNotFoundError` that occur when you try to open a file for reading, wrap the `with` statement in a `try` suite, as in:

lick here to view code image

```
In [4]: try:
   ...:     with open('gradez.txt', 'r') as accounts:
   ...:         print(f'{"ID":<3}{"Name":<7}{"Grade"}')
   ...:         for record in accounts:
```

e

```
    ...:             student_id, name, grade =   record.split()
    ...:             print(f'{student_id:<3}{name:<7}{grade}')
    ...: except   FileNotFoundError:
    ...:     print('The file name you   specified does not exist')
    ...:
The file name you specified does not exist
```

9.10 EXPLICITLY RAISING AN EXCEPTION

You've seen various exceptions raised by your Python code. Sometimes you might need to write functions that raise exceptions to inform callers of errors that occur. The **`raise`** statement explicitly raises an exception. The simplest form of the `raise` statement is

raise *ExceptionClassName*

The `raise` statement creates an object of the specified exception class. Optionally, the exception class name may be followed by parentheses containing arguments to initialize the exception object—typically to provide a custom error message string. Code that raises an exception first should release any resources acquired before the exception occurred. In the next section, we'll show an example of raising an exception.

In most cases, when you need to raise an exception, it's recommended that you use one of Python's many built-in exception types [9] listed at:

[9] You may be tempted to create custom exception classes that are specific to your application. Well say more about custom exceptions in the next chapter.

ttps://docs.python.org/3/library/exceptions.html

9.11 (OPTIONAL) STACK UNWINDING AND TRACEBACKS

Each exception object stores information indicating the precise series of function calls that led to the exception. This is helpful when debugging your code. Consider the following function definitions—`function1` calls `function2` and `function2` raises an `Exception`:

lick here to view code image

```
In [1]: def function1():
   ...:     function2()
```

```
    ...:

In [2]: def function2():
    ...:     raise Exception('An exception occurred')
    ...:
```

Calling `function1` results in the following traceback. For emphasis, we placed in bold the parts of the traceback indicating the lines of code that led to the exception:

lick here to view code image

```
In [3]: function1()
---------------------------------------------------------------------------
Exception                                 Traceback (most recent call last
 ipython-input-3-c0b3cafe2087> in <module>()
----> 1 function1()

<ipython-input-1-a9f4faeeeb0c> in function1()
      1 def function1():
----> 2     function2()
      3

<ipython-input-2-c65e19d6b45b> in function2()
      1 def function2():
----> 2     raise Exception('An exception occurred')

Exception: An exception   occurred
```

Traceback Details

The traceback shows the type of exception that occurred (`Exception`) followed by the complete function call stack that led to the raise point. The stack's bottom function call is listed *first* and the top is *last*, so the interpreter displays the following text as a reminder:

```
Traceback (most recent call last)
```

In this traceback, the following text indicates the bottom of the function-call stack—the `function1` call in snippet `[3]` (indicated by `ipython-input-3`):

```
<ipython-input-3-c0b3cafe2087> in <module>()
----> 1 function1()
```

Next, we see that `function1` called `function2` from line `2` in snippet `[1]`:

```
<ipython-input-1-a9f4faeeeb0c> in function1()
      1 def function1():
----> 2     function2()
      3
```

Finally, we see the *raise point*—in this case, line 2 in snippet `[2]` raised the exception:

```
<ipython-input-2-c65e19d6b45b> in function2()
      1 def function2():
----> 2     raise Exception('An exception occurred')
```

Stack Unwinding

In our previous exception-handling examples, the raise point occurred in a `try` suite, and the exception was handled in one of the `try` statement's corresponding `except` handlers. When an exception is *not* caught in a given function, **stack unwinding** occurs. Let's consider stack unwinding in the context of this example:

- In `function2`, the `raise` statement raises an exception. This is not in a `try` suite, so `function2` terminates, its stack frame is removed from the function-call stack, and control returns to the statement in `function1` that called `function2`.
- In `function1`, the statement that called `function2` is not in a `try` suite, so `function1` terminates, its stack frame is removed from the function-call stack, and control returns to the statement that called `function1`—snippet `[3]` in the IPython session.
- The call in snippet `[3]` call is not in a `try` suite, so that function call terminates. Because the exception was not caught (known as an **uncaught exception**), IPython displays the traceback, then awaits your next input. If this occurred in a typical script, the script would terminate.[o]

[o] In more advanced applications that use threads, an uncaught exception terminates only the thread in which the exception occurs, not necessarily the entire application.

Tip for Reading Tracebacks

You'll often call functions and methods that belong to libraries of code you did not

write. Sometimes those functions and methods raise exceptions. When reading a traceback, start from the end of the traceback and read the error message first. Then, read upward through the traceback, looking for the first line that indicates code you wrote in your program. Typically, this is the location in your code that led to the exception.

Exceptions in `finally` Suites

Raising an exception in a `finally` suite can lead to subtle, hard-to-find problems. If an exception occurs and is not processed by the time the `finally` suite executes, stack unwinding occurs. If the `finally` suite raises a *new* exception that the suite does not catch, the first exception is *lost*, and the *new* exception is passed to the next enclosing `try` statement. For this reason, a `finally` suite should always enclose in a `try` statement any code that may raise an exception, so that the exceptions will be processed within that suite.

9.12 INTRO TO DATA SCIENCE: WORKING WITH CSV FILES

Throughout this book, you'll work with many datasets as we present data-science concepts. **CSV (comma-separated values)** is a particularly popular file format. In this section, we'll demonstrate CSV file processing with a Python Standard Library module and pandas.

9.12.1 Python Standard Library Module `csv`

The **`csv` module**[1] provides functions for working with CSV files. Many other Python libraries also have built-in CSV support.

[1] `ttps://docs.python.org/3/library/csv.html`.

Writing to a CSV File

Let's create an `accounts.csv` file using CSV format. The `csv` module's documentation recommends opening CSV files with the additional keyword argument `newline=''` to ensure that newlines are processed properly:

lick here to view code image

```
In [1]: import csv

In [2]: with open('accounts.csv', mode='w',   newline='') as   accounts:
```

```
...:     writer = csv.writer(accounts)
...:     writer.writerow([100,   'Jones', 24.98])
...:     writer.writerow([200,   'Doe', 345.67])
...:     writer.writerow([300,   'White', 0.00])
...:     writer.writerow([400,   'Stone', -42.16])
...:     writer.writerow([500,   'Rich', 224.62])
...:
```

The **`.csv` file extension** indicates a CSV-format file. The `csv` module's **`writer` function** returns an object that writes CSV data to the specified file object. Each call to the `writer`'s **`writerow` method** receives an iterable to store in the file. Here we're using lists. By default, `writerow` delimits values with commas, but you can specify custom delimiters.[2] After the preceding snippet, `accounts.csv` contains:

[2] ttps://docs.python.org/3/library/csv.html#csv-fmt-params.

```
100,Jones,24.98
200,Doe,345.67
300,White,0.00
400,Stone,-42.16
500,Rich,224.62
```

CSV files generally do not contain spaces after commas, but some people use them to enhance readability. The `writerow` calls above can be replaced with one **`writerows`** call that outputs a comma-separated list of iterables representing the records.

If you write data that contains commas within a given string, `writerow` encloses that string in double quotes. For example, consider the following Python list:

```
[100, 'Jones, Sue', 24.98]
```

The single-quoted string `'Jones, Sue'` contains a comma separating the last name and first name. In this case, `writerow` would output the record as

```
100,"Jones, Sue",24.98
```

The quotes around `"Jones, Sue"` indicate that this is a *single* value. Programs reading this from a CSV file would break the record into *three* pieces—`100`, `'Jones, Sue'` and `24.98`.

h

Reading from a CSV File

Now let's read the CSV data from the file. The following snippet reads records from the file `accounts.csv` and displays the contents of each record, producing the same output we showed earlier:

lick here to view code image

```
In [3]: with open('accounts.csv', 'r',   newline='') as accounts:
   ...:     print(f'{"Account":<10}{"Name":<10}{"Balance":>10}')
   ...:     reader = csv.reader(accounts)
   ...:     for record in reader:
   ...:         account, name, balance = record
   ...:         print(f'{account:<10}{name:<10}{balance:>10}')
   ...:
Account   Name         Balance
100       Jones          24.98
200       Doe           345.67
300       White            0.0
400       Stone         -42.16
500       Rich          224.62
```

The `csv` module's **`reader` function** returns an object that reads CSV-format data from the specified file object. Just as you can iterate through a file object, you can iterate through the `reader` object one record of comma-delimited values at a time. The preceding `for` statement returns each `record` as a list of values, which we unpack into the variables `account`, `name` and `balance`, then display.

Caution: Commas in CSV Data Fields

Be careful when working with strings containing embedded commas, such as the name `'Jones, Sue'`. If you accidentally enter this as the two strings `'Jones'` and `'Sue'`, then `writerow` would, of course, create a CSV record with *four* fields, not *three*. Programs that read CSV files typically expect every record to have the *same* number of fields; otherwise, problems occur. For example, consider the following two lists:

```
[100, 'Jones', 'Sue', 24.98]
[200, 'Doe'  , 345.67]
```

The first list contains *four* values and the second contains only *three*. If these two records were written into the CSV file, then read into a program using the previous snippet, the following statement would fail when we attempt to unpack the four-field `record` into only three variables:

C

```
    account, name, balance = record
```

Caution: Missing Commas and Extra Commas in CSV Files

Be careful when preparing and processing CSV files. For example, suppose your file is composed of records, each with *four* comma-separated `int` values, such as:

```
    100,85,77,9
```

If you accidentally omit one of these commas, as in:

```
    100,8577,9
```

then the record has only *three* fields, one with the invalid value `8577`.

If you put two adjacent commas where only one is expected, as in:

```
    100,85,,77,9
```

then you have *five* fields rather than *four*, and one of the fields erroneously would be *empty*. Each of these comma-related errors could confuse programs trying to process the record.

9.12.2 Reading CSV Files into Pandas `DataFrame`s

In the Intro to Data Science sections in the previous two chapters, we introduced many pandas fundamentals. Here, we demonstrate pandas' ability to load files in CSV format, then perform some basic data-analysis tasks.

Datasets

In the data-science case studies, we'll use various free and open datasets to demonstrate machine learning and natural language processing concepts. There's an enormous variety of free datasets available online. The popular **Rdatasets repository** provides links to over 1100 free datasets in comma-separated values (CSV) format. These were originally provided with the R programming language for people learning about and developing statistical software, though they are not specific to R. They are now available on GitHub at:

```
ttps://vincentarelbundock.github.io/Rdatasets/datasets.html
```

This repository is so popular that there's a **`pydataset` module** specifically for accessing Rdatasets. For instructions on installing `pydataset` and accessing datasets with it, see:

```
ttps://github.com/iamaziz/PyDataset
```

Another large source of datasets is:

```
ttps://github.com/awesomedata/awesome-public-datasets
```

A commonly used machine-learning dataset for beginners is the **Titanic disaster dataset**, which lists all the passengers and whether they survived when the ship *Titanic* struck an iceberg and sank April 14–15, 1912. We'll use it here to show how to load a dataset, view some of its data and display some descriptive statistics. We'll dig deeper into a variety of popular datasets in the data-science chapters later in the book.

Working with Locally Stored CSV Files

You can load a CSV dataset into a `DataFrame` with the pandas function **`read_csv`**. The following loads and displays the CSV file `accounts.csv` that you created earlier in this chapter:

lick here to view code image

```
In [1]: import pandas as pd

In [2]: df = pd.read_csv('accounts.csv',
   ...:                  names=['account', 'name', 'balance'])
   ...:

In [3]: df
Out[3]:
   account   name  balance
0      100  Jones    24.98
1      200    Doe   345.67
2      300  White     0.00
3      400  Stone   -42.16
4      500   Rich   224.62
```

The `names` argument specifies the `DataFrame`'s column names. Without this

argument, `read_csv` assumes that the CSV file's first row is a comma-delimited list of column names.

To save a `DataFrame` to a file using CSV format, call `DataFrame` method **`to_csv`**:

lick here to view code image

```
In [4]: df.to_csv('accounts_from_dataframe.csv',    index=False)
```

The `index=False` keyword argument indicates that the row names (`0`–`4` at the left of the `DataFrame`'s output in snippet `[3]`) are not written to the file. The resulting file contains the column names as the first row:

```
account,name,balance
100,Jones,24.98
200,Doe,345.67
300,White,0.0
400,Stone,-42.16
500,Rich,224.62
```

9.12.3 Reading the Titanic Disaster Dataset

The Titanic disaster dataset is one of the most popular machine-learning datasets. The dataset is available in many formats, including CSV.

Loading the Titanic Dataset via a URL

If you have a URL representing a CSV dataset, you can load it into a `DataFrame` with `read_csv`. Let's load the Titanic Disaster dataset directly from GitHub:

lick here to view code image

```
In [1]: import pandas as pd

In [2]: titanic = pd.read_csv('https://vincentarelbundock.github.io/'    +
   ...:     'Rdatasets/csv/carData/TitanicSurvival.csv')
   ...:
```

Viewing Some of the Rows in the Titanic Dataset

This dataset contains over 1300 rows, each representing one passenger. According to Wikipedia, there were approximately 1317 passengers and 815 of them died.[3] For large

C

datasets, displaying the `DataFrame` shows only the first 30 rows, followed by "..." and the last 30 rows. To save space, let's view the first five and last five rows with `DataFrame` methods **`head`** and **`tail`**. Both methods return five rows by default, but you can specify the number of rows to display as an argument:

[3] ttps://en.wikipedia.org/wiki/Passengers_of_the_RMS_Titanic.

lick here to view code image

```
In [3]: pd.set_option('precision',   2)  # format for   floating-point val

 n [4]: titanic.head()
Out[4]:
                        Unnamed: 0 survived     sex    age   passengerClas
     Allen, Miss. Elisabeth Walton      yes  female    29.00            1s
    Allison, Master. Hudson Trevor      yes    male     0.92            1s
      Allison, Miss. Helen Loraine       no  female     2.00            1s
   Allison, Mr. Hudson Joshua Crei       no    male    30.00            1s
   Allison, Mrs. Hudson J C (Bessi       no  female    25.00            1s

 n [5]: titanic.tail()
Out[5]:
                     Unnamed: 0 survived     sex    age    passengerClass
1304       Zabour, Miss. Hileni       no  female    14.50              3rd
1305     Zabour, Miss. Thamine        no  female      NaN              3rd
1306  Zakarian, Mr. Mapriededer       no    male    26.50              3rd
1307        Zakarian, Mr. Ortin       no    male    27.00              3rd
1308         Zimmerman, Mr. Leo       no    male    29.00              3rd
```

Note that pandas adjusts each column's width, based on the widest value in the column or based on the column name, whichever is wider. Also, note the value in the age column of row `1305` is `NaN` (not a number), indicating a missing value in the dataset.

Customizing the Column Names

The first column in this dataset has a strange name (`'Unnamed: 0'`). We can clean that up by setting the column names. Let's change `'Unnamed: 0'` to `'name'` and let's shorten `'passengerClass'` to `'class'`:

lick here to view code image

```
In [6]: titanic.columns = ['name',   'survived', 'sex', 'age', 'class']

In [7]: titanic.head()
Out[7]:
```

```
                              name survived     sex    age   class
0    Allen, Miss. Elisabeth Walton      yes  female    29.00   1st
1   Allison, Master. Hudson Trevor      yes    male     0.92   1st
2     Allison, Miss. Helen Loraine       no  female     2.00   1st
3  Allison, Mr. Hudson Joshua Crei       no    male    30.00   1st
4  Allison, Mrs. Hudson J C (Bessi       no  female    25.00   1st
```

9.12.4 Simple Data Analysis with the Titanic Disaster Dataset

Now, you can use pandas to perform some simple analysis. For example, let's look at some descriptive statistics. When you call `describe` on a `DataFrame` containing both numeric and non-numeric columns, `describe` calculates these statistics *only for the numeric columns*—in this case, just the `age` column:

```
In [8]: titanic.describe()
Out[8]:
             age
count  1046.00
mean     29.88
std      14.41
min       0.17
25%      21.00
50%      28.00
75%      39.00
max      80.00
```

Note the discrepancy in the `count` (`1046`) vs. the dataset's number of rows (1309—the last row's index was `1308` when we called `tail`). Only `1046` (the `count` above) of the records contained an age value. The rest were *missing* and marked as `NaN`, as in row `1305`. When performing calculations, Pandas *ignores missing data (*`NaN`*) by default*. For the `1046` people with valid ages, the average (`mean`) age was `29.88` years old. The youngest passenger (`min`) was just over two months old (`0.17 * 12` is `2.04`), and the oldest (`max`) was 80. The median age was `28` (indicated by the `50%` quartile). The `25%` quartile is the median age in the first half of the passengers (sorted by age), and the `75%` quartile is the median of the second half of passengers.

Let's say you want to determine some statistics about people who survived. We can compare the `survived` column to `'yes'` to get a new `Series` containing `True`/`False` values, then use `describe` to summarize the results:

lick here to view code image

```
In [9]: (titanic.survived == 'yes').describe()
```

C

```
Out[9]:
count      1309
unique        2
top       False
freq        809
Name: survived, dtype: object
```

For non-numeric data, `describe` displays different descriptive statistics:

- `count` is the total number of items in the result.
- `unique` is the number of unique values (`2`) in the result—`True` (survived) and `False` (died).
- `top` is the most frequently occurring value in the result.
- `freq` is the number of occurrences of the `top` value.

9.12.5 Passenger Age Histogram

Visualization is a nice way to get to know your data. Pandas has many built-in visualization capabilities that are implemented with Matplotlib. To use them, first enable Matplotlib support in IPython:

```
In [10]: %matplotlib
```

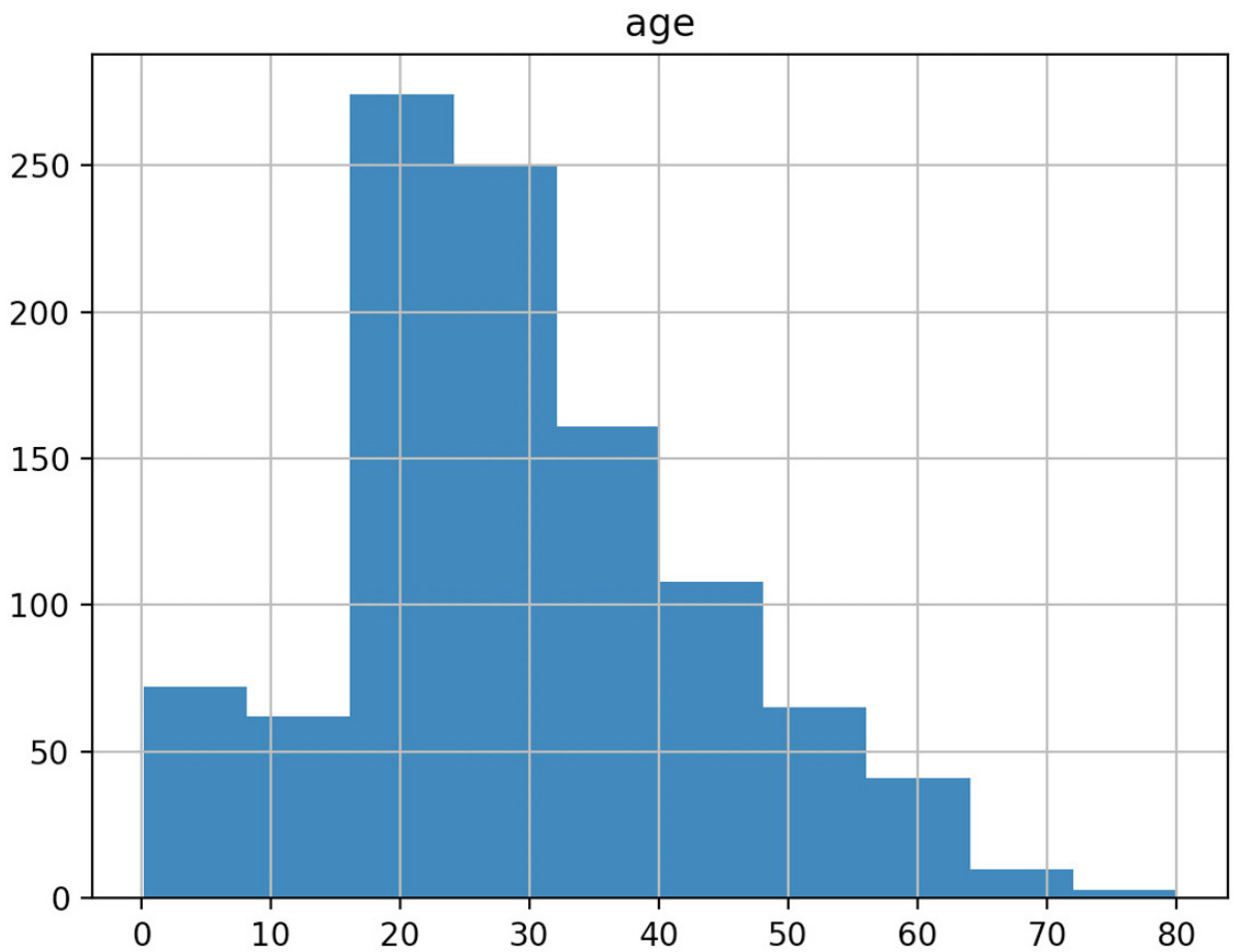
A histogram visualizes the distribution of numerical data over a range of values. A `DataFrame`'s **`hist`** method automatically analyzes each numerical column's data and produces a corresponding histogram. To view histograms of each numerical data column, call `hist` on your `DataFrame`:

lick here to view code image

```
In [11]: histogram = titanic.hist()
```

The Titanic dataset contains only one numerical data column, so the diagram shows one histogram for the age distribution. For datasets with multiple numerical columns, `hist` creates a separate histogram for each numerical column.

C



9.13 WRAP-UP

In this chapter, we introduced text-file processing and exception handling. Files are used to store data persistently. We discussed file objects and mentioned that Python views a file as a sequence of characters or bytes. We also mentioned the standard file objects that are automatically created for you when a Python program begins executing.

We showed how to create, read, write and update text files. We considered several popular file formats—plain text, JSON (JavaScript Object Notation) and CSV (comma-separated values). We used the built-in `open` function and the `with` statement to open a file, write to or read from the file and automatically close the file to prevent resource leaks when the `with` statement terminates. We used the Python Standard Library's `json` module to serialize objects into JSON format and store them in a file, load JSON objects from a file, deserialize them into Python objects and pretty-print a JSON object for readability.

We discussed how exceptions indicate execution-time problems and listed the various exceptions you've already seen. We showed how to deal with exceptions by wrapping code in `try` statements that provide `except` clauses to handle specific types of exceptions that may occur in the `try` suite, making your programs more robust and fault-tolerant.

e discussed the `try` statement's `finally` clause for executing code if program flow entered the corresponding `try` suite. You can use either the `with` statement or a `try` statement's `finally` clause for this purpose—we prefer the `with` statement.

In the Intro to Data Science section, we used both the Python Standard Library's `csv` module and capabilities of the pandas library to load, manipulate and store CSV data. Finally, we loaded the Titanic disaster dataset into a pandas `DataFrame`, changed some column names for readability, displayed the `head` and `tail` of the dataset, and performed simple analysis of the data. In the next chapter, we'll discuss Python's object-oriented programming capabilities.

W

10. Object-Oriented Programming

Objectives

In this chapter, you'll:

- Create custom classes and objects of those classes.
- Understand the benefits of crafting valuable classes.
- Control access to attributes.
- Appreciate the value of object orientation.
- Use Python special methods `__repr__`, `__str__` and `__format__` to get an object's string representations.
- Use Python special methods to overload (redefine) operators to use them with objects of new classes.
- Inherit methods, properties and attributes from existing classes into new classes, then customize those classes.
- Understand the inheritance notions of base classes (superclasses) and derived classes (subclasses).
- Understand duck typing and polymorphism that enable "programming in the general."
- Understand class `object` from which all classes inherit fundamental capabilities.
- Compare composition and inheritance.
- Build test cases into docstrings and run these tests with `doctest`,
- Understand namespaces and how they affect scope.

Outline

10.1 INTRODUCTION

ection 1.2 introduced the basic terminology and concepts of object-oriented programming. Everything in Python is an object, so you've been using objects constantly throughout this book. Just as houses are built from blueprints, objects are built from classes—one of the core technologies of object-oriented programming. Building a new object from even a large class is simple—you typically write one statement.

Crafting Valuable Classes

You've already used lots of classes created by other people. In this chapter you'll create your own *custom* classes. You'll focus on "crafting valuable classes" that help you meet the requirements of the applications you'll build. You'll use object-oriented programming with its core technologies of classes, objects, inheritance and polymorphism. Software applications are becoming larger and more richly functional. Object-oriented programming makes it easier for you to design, implement, test, debug and update such edge-of-the-practice applications. Read ections 10.1 through 0.9 for a code-intensive introduction to these technologies. Most people can skip ections 10.10 through 0.15, which provide additional perspectives on these technologies and present additional related features.

Class Libraries and Object-Based Programming

The vast majority of object-oriented programming you'll do in Python is **object-based programming** in which you primarily create and use objects of *existing* classes. You've been doing this throughout the book with built-in types like `int`, `float`, `str`, `list`, `tuple`, `dict`

nd `set`, with Python Standard Library types like `Decimal`, and with NumPy `array`s, Matplotlib `Figure`s and `Axes`, and pandas `Series` and `DataFrame`s.

To take maximum advantage of Python you must familiarize yourself with lots of preexisting classes. Over the years, the Python open-source community has crafted an enormous number of valuable classes and packaged them into class libraries. This makes it easy for you to reuse existing classes rather than "reinventing the wheel." Widely used open-source library classes are more likely to be thoroughly tested, bug free, performance tuned and portable across a wide range of devices, operating systems and Python versions. You'll find abundant Python libraries on the Internet at sites like GitHub, BitBucket, Source Forge and more—most easily installed with `conda` or `pip`. This is a key reason for Python's popularity. The vast majority of the classes you'll need are likely to be freely available in open-source libraries.

Creating Your Own Custom Classes

Classes are new data types. Each Python Standard Library class and third-party library class is a custom type built by someone else. In this chapter, you'll develop application-specific classes, like `CommissionEmployee`, `Time`, `Card`, `DeckOfCards` and more.

Most applications you'll build for your own use will commonly use either no custom classes or just a few. If you become part of a development team in industry, you may work on applications that contain hundreds, or even thousands, of classes. You can contribute your custom classes to the Python open-source community, but you are not obligated to do so. Organizations often have policies and procedures related to open-sourcing code.

Inheritance

Perhaps most exciting is the notion that new classes can be formed through inheritance and composition from classes in abundant class libraries. Eventually, software will be constructed predominantly from **standardized, reusable components** just as hardware is constructed from interchangeable parts today. This will help meet the challenges of developing ever more powerful software.

When creating a new class, instead of writing all new code, you can designate that the new class is to be formed initially by **inheriting** the attributes (variables) and methods (the class version of functions) of a previously defined **base class** (also called a **superclass**). The new class is called a **derived class** (or **subclass**). After inheriting, you then customize the derived class to meet the specific needs of your application. To minimize the customization effort, you should always try to inherit from the base class that's closest to your needs. To do that effectively, you should familiarize yourself with the class libraries that are geared to the kinds of applications you'll be building.

Polymorphism

We explain and demonstrate **polymorphism**, which enables you to conveniently program "in the general" rather than "in the specific." You simply send the *same* method call to objects possibly of many *different* types. Each object responds by "doing the right thing." So the same

a

ethod call takes on “many forms,” hence the term “poly-morphism.” We’ll explain how to implement polymorphism through inheritance and a Python feature called duck typing. We’ll explain both and show examples of each.

An Entertaining Case Study: Card-Shuffling-and-Dealing Simulation

You’ve already used a random-numbers-based die-rolling simulation and used those techniques to implement the popular dice game craps. Here, we present a card-shuffling-and-dealing simulation, which you can use to implement your favorite card games. You’ll use Matplotlib with attractive public-domain card images to display the full deck of cards both before and after the deck is shuffled.

Data Classes

Python 3.7’s new *data classes* help you build classes faster by using a more concise notation and by autogenerating portions of the classes. The Python community’s early reaction to data classes has been positive. As with any major new feature, it may take time before it’s widely used. We present class development with both the older and newer technologies.

Other Concepts Introduced in This Chapter

Other concepts we present include:

- How to specify that certain identifiers should be used only inside a class and not be accessible to clients of the class.
- Special methods for creating string representations of your classes’ objects and specifying how objects of your classes work with Python’s built-in operators (a process called *operator overloading*).
- An introduction to the Python exception class hierarchy and creating custom exception classes.
- Testing code with the Python Standard Library’s `doctest` module.
- How Python uses namespaces to determine the scopes of identifiers.

10.2 CUSTOM CLASS `ACCOUNT`

Let’s begin with a bank `Account` class that holds an account holder’s name and balance. An actual bank account class would likely include lots of other information, such as address, birth date, telephone number, account number and more. The `Account` class accepts deposits that increase the balance and withdrawals that decrease the balance.

10.2.1 Test-Driving Class `Account`

Each new class you create becomes a new *data type* that can be used to create objects. This is one reason why Python is said to be an **extensible language**. Before we look at class

m

`Account`'s definition, let's demonstrate its capabilities.

Importing Classes `Account` and `Decimal`

To use the new `Account` class, launch your IPython session from the `ch10` examples folder, then import class `Account`:

lick here to view code image

```
In [1]: from account import Account
```

Class `Account` maintains and manipulates the account balance as a `Decimal`, so we also import class `Decimal`:

lick here to view code image

```
In [2]: from decimal import Decimal
```

Create an `Account` Object with a Constructor Expression

To create a `Decimal` object, we can write:

```
value = Decimal('12.34')
```

This is known as a **constructor expression** because it builds and initializes an object of the class, similar to the way a house is constructed from a blueprint then painted with the buyer's preferred colors. Constructor expressions create new objects and initialize their data using argument(s) specified in parentheses. The parentheses following the class name are required, even if there are no arguments.

Let's use a constructor expression to create an `Account` object and initialize it with an account holder's name (a string) and balance (a `Decimal`):

lick here to view code image

```
In [3]: account1 = Account('John   Green', Decimal('50.00'))
```

Getting an `Account`'s Name and Balance

Let's access the `Account` object's `name` and `balance` attributes:

lick here to view code image

```
In [4]: account1.name
Out[4]: 'John Green'
```

C

```
In [5]: account1.balance
Out[5]: Decimal('50.00')
```

Depositing Money into an `Account`

An `Account`'s `deposit` method receives a positive dollar amount and adds it to the balance:

lick here to view code image

```
In [6]: account1.deposit(Decimal('25.53'))

In [7]: account1.balance
Out[7]: Decimal('75.53')
```

`Account` Methods Perform Validation

Class `Account`'s methods validate their arguments. For example, if a deposit amount is negative, `deposit` raises a `ValueError`:

lick here to view code image

```
In [8]: account1.deposit(Decimal('-123.45'))
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-8-27dc468365a7> in <module>()
----> 1 account1.deposit(Decimal('-123.45'))

~/Documents/examples/ch10/account.py in deposit(self, amount)
     21         # if amount is less than 0.00, raise an exception
     22         if amount < Decimal('0.00'):
---> 23             raise ValueError('Deposit amount must be positive.')
     24
     25         self.balance += amount

ValueError: Deposit amount must be positive.
```

10.2.2 `Account` Class Definition

Now, let's look at `Account`'s class definition, which is located in the file `account.py`.

Defining a Class

A class definition begins with the keyword **`class`** (line 5) followed by the class's name and a colon (`:`). This line is called the **class header**. The *Style Guide for Python Code* recommends that you begin each word in a multi-word class name with an uppercase letter (for example, `CommissionEmployee`). Every statement in a class's suite is indented.

lick here to view code image

```
1 # account.py
```

C

```
2 """Account class definition."""
3 from decimal import Decimal
4
5 class Account:
6     """Account class for maintaining a bank   account balance."""
7
```

Each class typically provides a descriptive docstring (line 6). When provided, it must appear in the line or lines immediately following the class header. To view any class's docstring in IPython, type the class name and a question mark, then press *Enter*:

lick here to view code image

```
In [9]: Account?
 nit signature: Account(name, balance)
Docstring:      Account class for maintaining a bank   account balance.
Init docstring: Initialize an Account object.
File:           ~/Documents/examples/ch10/account.py
Type:           type
```

The identifier `Account` is both the class name and the name used in a constructor expression to create an `Account` object and invoke the class's `__init__` method. For this reason, IPython's help mechanism shows both the class's docstring (`"Docstring:"`) and the `__init__` method's docstring (`"Init docstring:"`).

Initializing Account Objects: Method `__init__`

The constructor expression in snippet `[3]` from the preceding section:

lick here to view code image

```
account1 = Account('John Green', Decimal('50.00'))
```

creates a new object, then initializes its data by calling the class's **`__init__`** method. Each new class you create can provide an `__init__` method that specifies how to initialize an object's data attributes. Returning a value other than `None` from `__init__` results in a `TypeError`. Recall that `None` is returned by any function or method that does not contain a `return` statement. Class `Account`'s `__init__` method (lines 8–16) initializes an `Account` object's `name` and `balance` attributes if the `balance` is valid:

lick here to view code image

```
8     def __init__(self, name, balance):
9         """Initialize an Account   object."""
10
11         # if balance is less than 0.00, raise an exception
12         if balance < Decimal('0.00'):
13             raise ValueError('Initial balance   must be >= to 0.00.')
```

C

```
14
15          self.name    = name
16          self.balance    = balance
17
```

When you call a method for a specific object, Python implicitly passes a reference to that object as the method's first argument. For this reason, all methods of a class must specify at least one parameter. By convention most Python programmers call a method's first parameter `self`. A class's methods must use that reference (`self`) to access the object's attributes and other methods. Class `Account`'s `__init__` method also specifies parameters for the `name` and `balance`.

The `if` statement *validates* the `balance` parameter. If `balance` is less than `0.00`, `__init__` raises a `ValueError`, which terminates the `__init__` method. Otherwise, the method creates and initializes the new `Account` object's `name` and `balance` attributes.

When an object of class `Account` is created, it does not yet have any attributes. They're added *dynamically* via assignments of the form:

```
self.attribute_name = value
```

Python classes may define many **special methods**, like `__init__`, each identified by leading and trailing double-underscores (`__`) in the method name. Python class **`object`**, which we'll discuss later in this chapter, defines the special methods that are available for *all* Python objects.

Method `deposit`

The `Account` class's `deposit` method adds a positive `amount` to the account's `balance` attribute. If the `amount` argument is less than `0.00`, the method raises a `ValueError`, indicating that only positive deposit `amount`s are allowed. If the `amount` is valid, line 25 adds it to the object's `balance` attribute.

lick here to view code image

```
18      def deposit(self, amount):
19          """Deposit money to the    account."""
20
21          # if amount is less than 0.00, raise an exception
22          if amount < Decimal('0.00'):
23              raise ValueError('amount must be    positive.')
24
25          self.balance    += amount
```

10.2.3 Composition: Object References as Members of Classes

An `Account` *has a* `name`, and an `Account` *has a* `balance`. Recall that "everything in Python

C

is an object." This means that an object's attributes are references to objects of other classes. For example, an `Account` object's `name` attribute is a reference to a string object and an `Account` object's `balance` attribute is a reference to a `Decimal` object. Embedding references to objects of other types is a form of software reusability known as **composition** and is sometimes referred to as the **"has a" relationship**. Later in this chapter, we'll discuss *inheritance*, which establishes *"is a"* relationships.

10.3 CONTROLLING ACCESS TO ATTRIBUTES

Class `Account`'s methods validate their arguments to ensure that the `balance` is *always* valid—that is, always greater than or equal to `0.00`. In the previous example, we used the attributes `name` and `balance` only to *get* the values of those attributes. It turns out that we also can use those attributes to *modify* their values. Consider the `Account` object in the following IPython session:

lick here to view code image

```
In [1]: from account import Account

In [2]: from decimal import Decimal

In [3]: account1 = Account('John   Green', Decimal('50.00'))

In [4]: account1.balance
Out[4]: Decimal('50.00')
```

Initially, `account1` contains a valid balance. Now, let's set the `balance` attribute to an *invalid* negative value, then display the `balance`:

lick here to view code image

```
In [5]: account1.balance = Decimal('-1000.00')

In [6]: account1.balance
Out[6]: Decimal('-1000.00')
```

Snippet `[6]`'s output shows that `account1`'s `balance` is now negative. As you can see, unlike methods, data attributes cannot validate the values you assign to them.

Encapsulation

A class's **client code** is any code that uses objects of the class. Most object-oriented programming languages enable you to **encapsulate** (or *hide*) an object's data from the client code. Such data in these languages is said to be *private data*.

Leading Underscore (_) Naming Convention

Python does *not* have private data. Instead, you use *naming conventions* to design classes

C

that encourage correct use. By convention, Python programmers know that any attribute name beginning with an underscore (_) is for a class's *internal use only*. Client code should use the class's methods and—as you'll see in the next section—the class's properties to interact with each object's internal-use data attributes. Attributes whose identifiers do *not* begin with an underscore (_) are considered *publicly accessible* for use in client code. In the next section, we'll define a `Time` class and use these naming conventions. However, even when we use these conventions, attributes are always accessible.

10.4 PROPERTIES FOR DATA ACCESS

Let's develop a `Time` class that stores the time in 24-hour clock format with hours in the range 0–23, and minutes and seconds each in the range 0–59. For this class, we'll provide *properties*, which look like data attributes to client-code programmers, but control the manner in which they get and modify an object's data. This assumes that other programmers follow Python conventions to correctly use objects of your class.

10.4.1 Test-Driving Class `Time`

Before we look at class `Time`'s definition, let's demonstrate its capabilities. First, ensure that you're in the `ch10` folder, then `import` class `Time` from `timewithproperties.py`:

lick here to view code image

```
In [1]: from timewithproperties import Time
```

Creating a `Time` Object

Next, let's create a `Time` object. Class `Time`'s `__init__` method has `hour`, `minute` and `second` parameters, each with a default argument value of `0`. Here, we specify the `hour` and `minute`—`second` defaults to `0`:

lick here to view code image

```
In [2]: wake_up = Time(hour=6,    minute=30)
```

Displaying a `Time` Object

Class `Time` defines two methods that produce string representations of `Time` object. When you evaluate a variable in IPython as in snippet `[3]`, IPython calls the object's `__repr__` special method to produce a string representation of the object. Our `__repr__` implementation creates a string in the following format:

lick here to view code image

```
In [3]: wake_up
```

C

```
Out[3]: Time(hour=6, minute=30, second=0)
```

We'll also provide the `__str__` special method, which is called when an object is converted to a string, such as when you output the object with `print`. [1] Our `__str__` implementation creates a string in 12-hour clock format:

[1] If a class does not provide `__str__` and an object of the class is converted to a string, the classs `__repr__` method is called instead.

```
In [4]: print(wake_up)
6:30:00 AM
```

Getting an Attribute Via a Property

Class time provides `hour`, `minute` and `second` **properties**, which provide the convenience of data attributes for getting and modifying an object's data. However, as you'll see, properties are implemented as methods, so they may contain additional logic, such as specifying the format in which to return a data attribute's value or validating a new value before using it to modify a data attribute. Here, we get the `wake_up` object's `hour` value:

```
In [5]: wake_up.hour
Out[5]: 6
```

Though this snippet appears to simply get an `hour` data attribute's value, it's actually a call to an `hour` *method* that returns the value of a data attribute (which we named `_hour`, as you'll see in the next section).

Setting the `Time`

You can set a new time with the `Time` object's `set_time` method. Like method `__init__`, method `set_time` provides `hour`, `minute` and `second` parameters, each with a default of `0`:

lick here to view code image

```
In [6]: wake_up.set_time(hour=7,   minute=45)

In [7]: wake_up
Out[7]: Time(hour=7, minute=45, second=0)
```

Setting an Attribute via a Property

Class `Time` also supports setting the `hour`, `minute` and `second` values individually via its properties. Let's change the `hour` value to `6`:

lick here to view code image

C

```
In [8]: wake_up.hour = 6

In [9]: wake_up
Out[9]: Time(hour=6, minute=45, second=0)
```

Though snippet [8] appears to simply assign a value to a data attribute, it's actually a call to an `hour` method that takes `6` as an argument. The method validates the value, then assigns it to a corresponding data attribute (which we named `_hour`, as you'll see in the next section).

Attempting to Set an Invalid Value

To prove that class `Time`'s properties *validate* the values you assign to them, let's try to assign an invalid value to the `hour` property, which results in a `ValueError`:

lick here to view code image

```
In [10]: wake_up.hour = 100
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-10-1fce0716ef14> in <module>()
----> 1 wake_up.hour = 100

~/Documents/examples/ch10/timewithproperties.py in   hour(self, hour)
     20         """Set   the hour."""
     21         if not (0 <= hour < 24):
---> 22             raise   ValueError(f'Hour ({hour})   must be 0-23')
     23
     24         self._hour = hour

ValueError: Hour (100) must   be 0-23
```

10.4.2 Class `Time` Definition

Now that we've seen class `Time` in action, let's look at its definition.

Class Time: `__init__` Method with Default Parameter Values

Class `Time`'s `__init__` method specifies `hour`, `minute` and `second` parameters, each with a default argument of `0`. Similar to class `Account`'s `__init__` method, recall that the `self` parameter is a reference to the `Time` object being initialized. The statements containing `self.hour`, `self.minute` and `self.second` *appear* to create `hour`, `minute` and `second` attributes for the new `Time` object (`self`). However, these statements actually call methods that implement the class's `hour`, `minute` and `second` *properties* (lines 13–50). Those methods then create attributes named `_hour`, `_minute` and `_second` that are meant for use only inside the class:

lick here to view code image

```
1 # timewithproperties.py
2 """Class Time with read-write   properties."""
```

C

```
3
4 class Time:
5     """Class Time with read-write   properties."""
6
7     def __init__(self, hour=0,   minute=0, second=0):
8         """Initialize each   attribute."""
9         self.hour   = hour  # 0-23
10         self.minute   = minute  # 0-59
11         self.second   = second  # 0-59
12
```

Class Time: `hour` Read-Write Property

Lines 13–24 define a *publicly accessible* **read-write property** named `hour` that manipulates a data attribute named `_hour`. The single-leading-underscore (`_`) naming convention indicates that client code should not access `_hour` directly. As you saw in the previous section's snippets `[5]` and `[8]`, properties look like data attributes to programmers working with `Time` objects. However, notice that properties are implemented as *methods*. Each property defines a *getter* method which *gets* (that is, returns) a data attribute's value and can *optionally* define a *setter* method which *sets* a data attribute's value:

lick here to view code image

```
13     @property
14     def hour(self):
15         """Return the hour."""
16         return self._hour
17
18     @hour.setter
19     def hour(self, hour):
20         """Set the hour."""
21         if not (0 <= hour < 24):
22             raise ValueError(f'Hour ({hour}) must be 0-23')
23
24         self._hour   = hour
25
```

The **`@property` decorator** precedes the property's *getter* method, which receives only a `self` parameter. Behind the scenes, a decorator adds code to the decorated function—in this case to make the `hour` function work with attribute syntax. The *getter* method's name is the property name. This *getter* method returns the `_hour` data attribute's value. The following client-code expression invokes the *getter* method:

```
wake_up.hour
```

You also can use the *getter* method inside the class, as you'll see shortly.

A decorator of the form **`@property_name.setter`** (in this case, `@hour.setter`) precedes the property's *setter* method. The method receives two parameters—`self` and a parameter (`hour`) representing the value being assigned to the property. If the `hour`

C

parameter's value is *valid*, this method assigns it to the `self` object's `_hour` attribute; otherwise, the method raises a `ValueError`. The following client-code expression invokes the *setter* by assigning a value to the property:

```
wake_up.hour = 8
```

We also invoked this `setter` inside the class at line 9 of `__init__`:

```
self.hour = hour
```

Using the *setter* enabled us to *validate* `__init__`'s `hour` argument *before* creating and initializing the object's `_hour` attribute, which occurs the *first* time the `hour` property's *setter* executes as a result of line 9. A **read-write property** has both a *getter* and a *setter*. A **read-only property** has only a *getter*.

Class Time: `minute` and `second` Read-Write Properties

Lines 26–37 and 39–50 define read-write `minute` and `second` properties. Each property's `setter` ensures that its second argument is in the range `0–59` (the valid range of values for minutes and seconds):

lick here to view code image

```
26      @property
27      def minute(self):
28          """Return the   minute."""
29          return self._minute
30
31      @minute.setter
32      def minute(self, minute):
33          """Set the minute."""
34          if not (0 <= minute <   60):
35              raise ValueError(f'Minute ({minute}) must be 0-59')
36
37          self._minute   = minute
38
39      @property
40      def second(self):
41          """Return the   second."""
42          return self._second
43
44      @second.setter
45      def second(self, second):
46          """Set the second."""
47          if not (0 <= second <   60):
48              raise ValueError(f'Second ({second}) must be 0-59')
49
50          self._second   = second
51
```

Class Time: Method `set_time`

C

We provide method `set_time` as a convenient way to change *all three* attributes with a *single* method call. Lines 54–56 invoke the *setters* for the `hour`, `minute` and `second` properties:

lick here to view code image

```
52     def set_time(self, hour=0,   minute=0, second=0):
53         """Set values of hour, minute, and   second."""
54         self.hour   = hour
55         self.minute   = minute
56         self.second   = second
57
```

Class Time: Special Method `__repr__`

When you pass an object to built-in function `repr`—which happens implicitly when you evaluate a variable in an IPython session—the corresponding class's **`__repr__` special method** is called to get a string representation of the object:

lick here to view code image

```
58     def __repr__(self):
59         """Return Time string for   repr()."""
60         return (f'Time(hour={self.hour}, minute={self.minute}, '   +
61                 f'second={self.second})')
62
```

The Python documentation indicates that `__repr__` returns the "official" string representation of the object. Typically this string looks like a constructor expression that creates and initializes the object, [2] as in:

[2] `ttps://docs.python.org/3/reference/datamodel.html`.

```
'Time(hour=6, minute=30, second=0)'
```

which is similar to the constructor expression in the previous section's snippet `[2]`. Python has a built-in function **`eval`** that could receive the preceding string as an argument and use it to create and initialize a `Time` object containing values specified in the string.

Class Time: Special Method `__str__`

For our class `Time` we also define the **`__str__`** special method. This method is called implicitly when you convert an object to a string with the built-in function `str`, such as when you `print` an object or call `str` explicitly. Our implementation of `__str__` creates a string in 12-hour clock format, such as `'7:59:59 AM'` or `'12:30:45 PM'`:

lick here to view code image

```
    def __str__(self):
        """Print Time in 12-hour clock   format."""
        return (('12' if self.hour in (0, 12) else str(self.hour % 12)) +
                f':{self.minute:0>2}:{self.second:0>2}' +
                (' AM' if self.hour < 12 else ' PM'))
```

10.4.3 Class `Time` Definition Design Notes

Let's consider some class-design issues in the context of our `Time` class.

Interface of a Class

Class `Time`'s properties and methods define the class's **public interface**—that is, the set of properties and methods programmers should use to interact with objects of the class.

Attributes Are Always Accessible

Though we provided a well-defined interface, Python does *not* prevent you from directly manipulating the data attributes `_hour`, `_minute` and `_second`, as in:

lick here to view code image

```
In [1]: from timewithproperties import Time

In [2]: wake_up = Time(hour=7,   minute=45, second=30)

In [3]: wake_up._hour
Out[3]: 7

In [4]: wake_up._hour = 100

In [5]: wake_up
Out[5]: Time(hour=100, minute=45, second=30)
```

After snippet `[4]`, the `wake_up` object contains *invalid* data. Unlike many other object-oriented programming languages, such as C++, Java and C#, data attributes in Python cannot be hidden from client code. The Python tutorial says, "**nothing in Python makes it possible to enforce data hiding—it is all based upon convention**." [3]

[3] `ttps://docs.python.org/3/tutorial/classes.html#random-remarks`.

Internal Data Representation

We chose to represent the time as three integer values for hours, minutes and seconds. It would be perfectly reasonable to represent the time internally as the number of seconds since midnight. Though we'd have to reimplement the properties `hour`, `minute` and `second`, programmers could use the *same* interface and get the *same* results without being aware of these changes. We leave it to you to make this change and show that client code using `Time` objects does not need to change.

Evolving a Class's Implementation Details

When you design a class, carefully consider the class's interface before making that class available to other programmers. Ideally, you'll design the interface such that existing code will not break if you update the class's implementation details—that is, the internal data representation or how its method bodies are implemented.

If Python programmers follow convention and do not access attributes that begin with leading underscores, then class designers can evolve class implementation details without breaking client code.

Properties

It may seem that providing properties with both *setters* and *getters* has no benefit over accessing the data attributes directly, but there are subtle differences. A *getter* seems to allow clients to read the data at will, but the *getter* can control the formatting of the data. A *setter* can scrutinize attempts to modify the value of a data attribute to prevent the data from being set to an invalid value.

Utility Methods

Not all methods need to serve as part of a class's interface. Some serve as **utility methods** used only *inside* the class and are not intended to be part of the class's public interface used by client code. Such methods should be named with a single leading underscore. In other object-oriented languages like C++, Java and C#, such methods typically are implemented as private methods.

Module `datetime`

In professional Python development, rather than building your own classes to represent times and dates, you'll typically use the Python Standard Library's `datetime` module capabilities. For more details about the `datetime` module, see:

```
ttps://docs.python.org/3/library/datetime.html
```

10.5 SIMULATING "PRIVATE" ATTRIBUTES

In programming languages such as C++, Java and C#, classes state explicitly which class members are *publicly accessible*. Class members that may not be accessed outside a class definition are **private** and visible only within the class that defines them. Python programmers often use "private" attributes for data or utility methods that are essential to a class's inner workings but are not part of the class's public interface.

As you've seen, Python objects' attributes are *always* accessible. However, Python has a naming convention for "private" attributes. Suppose we want to create an object of class `Time` and to *prevent* the following assignment statement:

h

```
wake_up._hour = 100
```

that would set the hour to an invalid value. Rather than `_hour`, we can name the attribute `__hour` with *two* leading underscores. This convention indicates that `__hour` is "private" and should not be accessible to the class's clients. To help prevent clients from accessing "private" attributes, Python *renames* them by preceding the attribute name with *_ClassName*, as in `_Time__hour`. This is called **name mangling**. If you try assign to `__hour`, as in

```
wake_up.__hour = 100
```

Python raises an `AttributeError`, indicating that the class does not have an `__hour` attribute. We'll show this momentarily.

IPython Auto-Completion Shows Only "Public" Attributes

In addition, IPython does not show attributes with one or two leading underscores when you try to auto-complete an expression like

```
wake_up.
```

by pressing *Tab*. Only attributes that are part of the `wake_up` object's "public" interface are displayed in the IPython auto-completion list.

Demonstrating "Private" Attributes

To demonstrate name mangling, consider class `PrivateClass` with one "public" data attribute `public_data` and one "private" data attribute `__private_data`:

lick here to view code image

```
1 # private.py
2 """Class with public and private   attributes."""
3
4 class PrivateClass:
5     """Class with public and private   attributes."""
6
7     def __init__(self):
8         """Initialize the public and private   attributes."""
9         self.public_data   = "public"  #   public attribute
10         self.__private_data   = "private"  #   private attribute
```

Let's create an object of class `PrivateData` to demonstrate these data attributes:

lick here to view code image

```
In [1]: from private import PrivateClass
```

C

```
In [2]: my_object = PrivateClass()
```

Snippet [3] shows that we can access the public_data attribute directly:

lick here to view code image

```
In [3]: my_object.public_data
Out[3]: 'public'
```

However, when we attempt to access __private_data directly in snippet [4], we get an AttributeError indicating that the class does not have an attribute by that name:

lick here to view code image

```
In [4]: my_object.__private_data
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-4-d896bfdf2053> in <module>()
----> 1 my_object.__private_data

AttributeError:   'PrivateClass' object has no attribute '__private_data'
```

This occurs because python changed the attribute's name. Unfortunately, the attribute __private_data is still indirectly accessible.

10.6 CASE STUDY: CARD SHUFFLING AND DEALING SIMULATION

Our next example presents two custom classes that you can use to shuffle and deal a deck of cards. Class Card represents a playing card that has a face ('Ace', '2', '3',, 'Jack', 'Queen', 'King') and a suit ('Hearts', 'Diamonds', 'Clubs', 'Spades'). Class DeckOfCards represents a deck of 52 playing cards as a list of Card objects. First, we'll test-drive these classes in an IPython session to demonstrate card shuffling and dealing capabilities and displaying the cards as text. Then we'll look at the class definitions. Finally, we'll use another IPython session to display the 52 cards as images using Matplotlib. We'll show you where to get nice-looking public-domain card images.

10.6.1 Test-Driving Classes Card and DeckOfCards

Before we look at classes Card and DeckOfCards, let's use an IPython session to demonstrate their capabilities.

Creating, Shuffling and Dealing the Cards

First, import class DeckOfCards from deck.py and create an object of the class:

lick here to view code image

C

```
In [1]: from deck import DeckOfCards

In [2]: deck_of_cards = DeckOfCards()
```

`DeckOfCards` method `__init__` creates the 52 `Card` objects in order by suit and by face within each suit. You can see this by printing the `deck_of_cards` object, which calls the `DeckOfCards` class's `__str__` method to get the deck's string representation. Read each row left-to-right to confirm that all the cards are displayed in order from each suit (`Hearts`, `Diamonds`, `Clubs` and `Spades`):

lick here to view code image

```
In [3]: print(deck_of_cards)
Ace of Hearts      2 of Hearts        3 of Hearts        4   of Hearts
5 of Hearts        6 of Hearts        7 of Hearts        8   of Hearts
9 of Hearts        10 of Hearts       Jack of Hearts       Queen of Hearts
King of Hearts     Ace of Diamonds    2 of Diamonds      3   of Diamonds
4 of Diamonds      5 of Diamonds      6 of Diamonds      7   of Diamonds
8 of Diamonds      9 of Diamonds      10 of Diamonds       Jack of Diamonds
Queen of Diamonds  King of Diamonds   Ace of Clubs       2   of Clubs
3 of Clubs         4 of Clubs         5 of Clubs         6   of Clubs
7 of Clubs         8 of Clubs         9 of Clubs         10   of Clubs
Jack of Clubs      Queen of Clubs     King of Clubs        Ace of Spades
2 of Spades        3 of Spades        4 of Spades        5   of Spades
6 of Spades        7 of Spades        8 of Spades        9   of Spades
10 of Spades       Jack of Spades     Queen of Spades      King of Spades
```

Next, let's shuffle the deck and print the `deck_of_cards` object again. We did not specify a seed for reproducibility, so each time you shuffle, you'll get different results:

lick here to view code image

```
In [4]: deck_of_cards.shuffle()

In [5]: print(deck_of_cards)
King of Hearts     Queen of Clubs     Queen of Diamonds  10   of Clubs
5 of Hearts        7 of Hearts        4 of Hearts        2   of Hearts
5 of Clubs         8 of Diamonds      3 of Hearts        10   of Hearts
8 of Spades        5 of Spades        Queen of Spades      Ace of Clubs
8 of Clubs         7 of Spades        Jack of Diamonds   10   of Spades
4 of Diamonds      8 of Hearts        6 of Spades          King of Spades
9 of Hearts        4 of Spades        6 of Clubs           King of Clubs
3 of Spades        9 of Diamonds      3 of Clubs           Ace of Spades
Ace of Hearts      3 of Diamonds      2 of Diamonds      6   of Hearts
King of Diamonds   Jack of Spades     Jack of Clubs      2   of Spades
5 of Diamonds      4 of Clubs         Queen of Hearts    9   of Clubs
10 of Diamonds     2 of Clubs         Ace of Diamonds    7   of Diamonds
9 of Spades        Jack of Hearts     6 of Diamonds      7   of Clubs
```

Dealing Cards

C

We can deal one `Card` at a time by calling method `deal_card`. IPython calls the returned `Card` object's `__repr__` method to produce the string output shown in the `Out[]` prompt:

lick here to view code image

```
In [6]: deck_of_cards.deal_card()
Out[6]: Card(face='King', suit='Hearts')
```

Class Card's Other Features

To demonstrate class `Card`'s `__str__` method, let's deal another card and pass it to the built-in `str` function:

lick here to view code image

```
In [7]: card = deck_of_cards.deal_card()

In [8]: str(card)
Out[8]: 'Queen of Clubs'
```

Each `Card` has a corresponding image file name, which you can get via the `image_name` read-only property. We'll use this soon when we display the `Card`s as images:

lick here to view code image

```
In [9]: card.image_name
Out[9]: 'Queen_of_Clubs.png'
```

10.6.2 Class `Card`—Introducing Class Attributes

Each `Card` object contains three string properties representing that `Card`'s `face`, `suit` and `image_name` (a file name containing a corresponding image). As you saw in the preceding section's IPython session, class `Card` also provides methods for initializing a `Card` and for getting various string representations.

Class Attributes `FACES` and `SUITS`

Each object of a class has its own copies of the class's data attributes. For example, each `Account` object has its own `name` and `balance`. Sometimes, an attribute should be shared by *all* objects of a class. A **class attribute** (also called a **class variable**) represents *class-wide* information. It belongs to the *class*, not to a specific object of that class. Class `Card` defines two class attributes (lines 5–7):

- `FACES` is a list of the card face names.
- `SUITS` is a list of the card suit names.

C

lick here to view code image

```
1 # card.py
2 """Card class that represents a playing card   and its image file name."""
3
4 class Card:
5     FACES = ['Ace', '2', '3', '4', '5', '6',
6              '7', '8', '9', '10', 'Jack', 'Queen', 'King']
7     SUITS = ['Hearts', 'Diamonds', 'Clubs', 'Spades']
8
```

You define a class attribute by assigning a value to it inside the class's definition, but not inside any of the class's methods or properties (in which case, they'd be local variables). `FACES` and `SUITS` are *constants* that are not meant to be modified. Recall that the *Style Guide for Python Code* recommends naming your constants with all capital letters. [4]

[4] Recall that Python does not have true constants, so `FACES` and `SUITS` are still modifiable.

We'll use elements of these lists to initialize each `Card` we create. However, we do not need a separate copy of each list in every `Card` object. Class attributes can be accessed through any object of the class, but are typically accessed through the class's name (as in, `Card.FACES` or `Card.SUITS`). Class attributes exist as soon as you import their class's definition.

Card Method `__init__`

When you create a `Card` object, method `__init__` defines the object's `_face` and `_suit` data attributes:

lick here to view code image

```
9     def __init__(self, face, suit):
10         """Initialize a Card with a face and   suit."""
11         self._face   = face
12         self._suit   = suit
13
```

Read-Only Properties `face`, `suit` and `image_name`

Once a `Card` is created, its `face`, `suit` and `image_name` do not change, so we implement these as read-only properties (lines 14–17, 19–22 and 24–27). Properties `face` and `suit` return the corresponding data attributes `_face` and `_suit`. A property is not required to have a corresponding data attribute. To demonstrate this, the `Card` property `image_name`'s value is *created dynamically* by getting the `Card` object's string representation with `str(self)`, replacing any spaces with underscores and appending the `'.png'` filename extension. So, `'Ace of Spades'` becomes `'Ace_of_Spades.png'`. We'll use this file name to load a PNG-format image representing the `Card`. PNG (Portable Network Graphics) is a popular image format for web-based images.

C

lick here to view code image

```
    @property
    def face(self):
        """Return the Card's self._face   value."""
        return self._face

    @property
    def suit(self):
        """Return the Card's self._suit   value."""
        return self._suit

    @property
    def image_name(self):
        """Return the Card's image file   name."""
        return str(self).replace(' ',   '_') + '.png'

```

Methods That Return String Representations of a Card

Class `Card` provides three special methods that return string representations. As in class `Time`, method `__repr__` returns a string representation that looks like a constructor expression for creating and initializing a `Card` object:

lick here to view code image

```
    def __repr__(self):
        """Return string representation for   repr()."""
        return f"Card(face='{self.face}', suit='{self.suit}')"

```

Method `__str__` returns a string of the format `'`*face* `of` *suit*`'`, such as `'Ace of Hearts'`:

lick here to view code image

```
    def __str__(self):
        """Return string representation for   str()."""
        return f'{self.face} of {self.suit}'

```

When the preceding section's IPython session printed the entire deck, you saw that the `Card`s were displayed in four left-aligned columns. As you'll see in the `__str__` method of class `DeckOfCards`, we use f-strings to format the `Card`s in fields of 19 characters each. Class `Card`'s special method **`__format__`** is called when a `Card` object is *formatted* as a string, such as in an f-string:

lick here to view code image

```
    def __format__(self, format):
        """Return formatted string   representation for str()."""
```

C

```
39         return f'{str(self):{format}}'
```

This method's second argument is the format string used to format the object. To use the `format` parameter's value as the format specifier, enclose the parameter name in braces to the *right* of the colon. In this case, we're formatting the `Card` object's string representation returned by `str(self)`. We'll discuss `__format__` again when we present the `__str__` method in class `DeckOfCards`.

10.6.3 Class `DeckOfCards`

Class `DeckOfCards` has a class attribute `NUMBER_OF_CARDS`, representing the number of `Card`s in a deck, and creates two data attributes:

- `_current_card` keeps track of which `Card` will be dealt next (`0`–`51`) and
- `_deck` (line 12) is a list of 52 `Card` objects.

Method `__init__`

`DeckOfCards` method `__init__` initializes a `_deck` of `Card`s. The `for` statement fills the list `_deck` by appending new `Card` objects, each initialized with two strings—one from the list `Card.FACES` and one from `Card.SUITS`. The calculation `count % 13` *always* results in a value from 0 to 12 (the 13 indices of `Card.FACES`), and the calculation `count // 13` *always* results in a value from 0 to 3 (the four indices of `Card.SUITS`). When the `_deck` list is initialized, it contains the `Card`s with faces `'Ace'` through `'King'` in order for all the `Hearts`, then the `Diamonds`, then the `Clubs`, then the `Spades`.

lick here to view code image

```
1 # deck.py
2 """Deck class represents a deck of   Cards."""
3 import random
4 from card import Card
5
6 class DeckOfCards:
7     NUMBER_OF_CARDS = 52  # constant number of Cards
8
9     def __init__(self):
10         """Initialize the   deck."""
11         self._current_card   = 0
12         self._deck   = []
13
14         for count in range(DeckOfCards.NUMBER_OF_CARDS):
15             self._deck.append(Card(Card.FACES[count % 13],
16                 Card.SUITS[count // 13]))
17
```

Method `shuffle`

Method `shuffle` resets `_current_card` to `0`, then shuffles the `Card`s in `_deck` using the

C

`random` module's `shuffle` function:

lick here to view code image

```
18     def shuffle(self):
19         """Shuffle deck."""
20         self._current_card    = 0
21         random.shuffle(self._deck)
22
```

Method `deal_card`

Method `deal_card` deals one `Card` from `_deck`. Recall that `_current_card` indicates the index (`0`–`51`) of the next `Card` to be dealt (that is, the `Card` at the top of the deck). Line 26 tries to get the `_deck` element at index `_current_card`. If successful, the method increments `_current_card` by 1, then returns the `Card` being dealt; otherwise, the method returns `None` to indicate there are no more `Card`s to deal.

lick here to view code image

```
23     def deal_card(self):
24         """Return one Card."""
25         try:
26             card    = self._deck[self._current_card]
27             self._current_card    += 1
28             return card
29         except:
30             return None
31
```

Method `__str__`

Class `DeckOfCards` also defines special method `__str__` to get a string representation of the deck in four columns with each `Card` left aligned in a field of 19 characters. When line 37 formats a given `Card`, its `__format__` special method is called with format specifier `'<19'` as the method's `format` argument. Method `__format__` then uses `'<19'` to create the `Card`'s formatted string representation.

lick here to view code image

```
32     def __str__(self):
33         """Return a string representation of    the current _deck."""
34         s = ''
35
36         for index, card in    enumerate(self._deck):
37             s    += f'{self._deck[index]:<19}'
38             if (index + 1) % 4 == 0:
39                 s    += '\n'
40
41         return s
```

C

10.6.4 Displaying Card Images with Matplotlib

So far, we've displayed `Card`s as text. Now, let's display `Card` images. For this demonstration, we downloaded public-domain [5] card images from Wikimedia Commons:

[5] `ttps://creativecommons.org/publicdomain/zero/1.0/deed.en`.

```
ttps://commons.wikimedia.org/wiki/-Category:SVG_English_pattern_playing_cards
```

These are located in the `ch10` examples folder's `card_images` subfolder. First, let's create a `DeckOfCards`:

lick here to view code image

```
In [1]: from deck import DeckOfCards

In [2]: deck_of_cards = DeckOfCards()
```

Enable Matplotlib in IPython

Next, enable Matplotlib support in IPython by using the `%matplotlib` magic:

lick here to view code image

```
In [3]: %matplotlib
Using matplotlib backend: Qt5Agg
```

Create the Base `Path` for Each Image

Before displaying each image, we must load it from the `card_images` folder. We'll use the `pathlib` module's **`Path` class** to construct the full path to each image on our system. Snippet `[5]` creates a `Path` object for the current folder (the `ch10` examples folder), which is represented by `'.'`, then uses `Path` method **`joinpath`** to append the subfolder containing the card images:

lick here to view code image

```
In [4]: from pathlib import Path

In [5]: path = Path('.').joinpath('card_images')
```

Import the Matplotlib Features

Next, let's import the Matplotlib modules we'll need to display the images. We'll use a function from **`matplotlib.image`** to load the images:

lick here to view code image

```
In [6]: import matplotlib.pyplot   as plt

In [7]: import matplotlib.image as mpimg
```

Create the `Figure` and `Axes` Objects

The following snippet uses Matplotlib function **`subplots`** to create a `Figure` object in which we'll display the images as 52 *subplots* with four rows (`nrows`) and 13 columns (`ncols`). The function returns a tuple containing the `Figure` and an array of the subplots' `Axes` objects. We unpack these into variables `figure` and `axes_list`:

lick here to view code image

```
In [8]: figure, axes_list = plt.subplots(nrows=4, ncols=13)
```

When you execute this statement in IPython, the Matplotlib window appears immediately with 52 empty subplots.

Configure the `Axes` Objects and Display the Images

Next, we iterate through all the `Axes` objects in `axes_list`. Recall that `ravel` provides a one-dimensional view of a multidimensional array. For each `Axes` object, we perform the following tasks:

- We're not plotting data, so we do not need axis lines and labels for each image. The first two statements in the loop hide the *x*- and *y*-axes.
- The third statement deals a `Card` and gets its `image_name`.
- The fourth statement uses `Path` method `joinpath` to append the `image_name` to the `Path`, then calls `Path` method **`resolve`** to determine the full path to the image on our system. We pass the resulting `Path` object to the built-in `str` function to get the string representation of the image's location. Then, we pass that string to the `matplotlib.image` module's **`imread` function**, which loads the image.
- The last statement calls `Axes` method **`imshow`** to display the current image in the current subplot.

lick here to view code image

```
In [9]: for axes in axes_list.ravel():
   ...:     axes.get_xaxis().set_visible(False)
   ...:     axes.get_yaxis().set_visible(False)
   ...:     image_name =   deck_of_cards.deal_card().image_name
   ...:     img =   mpimg.imread(str(path.joinpath(image_name).resolve()))
```

C

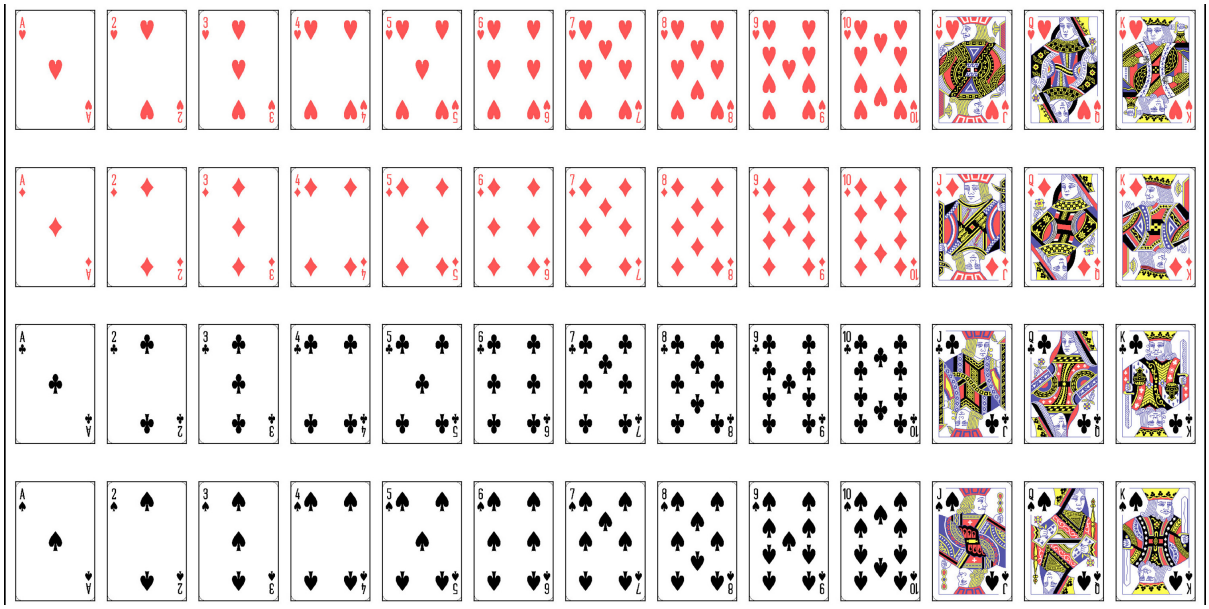
```
...:     axes.imshow(img)
...:
```

Maximize the Image Sizes

At this point, all the images are displayed. To make the cards as large as possible, you can maximize the window, then call the Matplotlib `Figure` object's **`tight_layout` method**. This removes most of the extra white space in the window:

```
In [10]: figure.tight_layout()
```

The following image shows the contents of the resulting window:



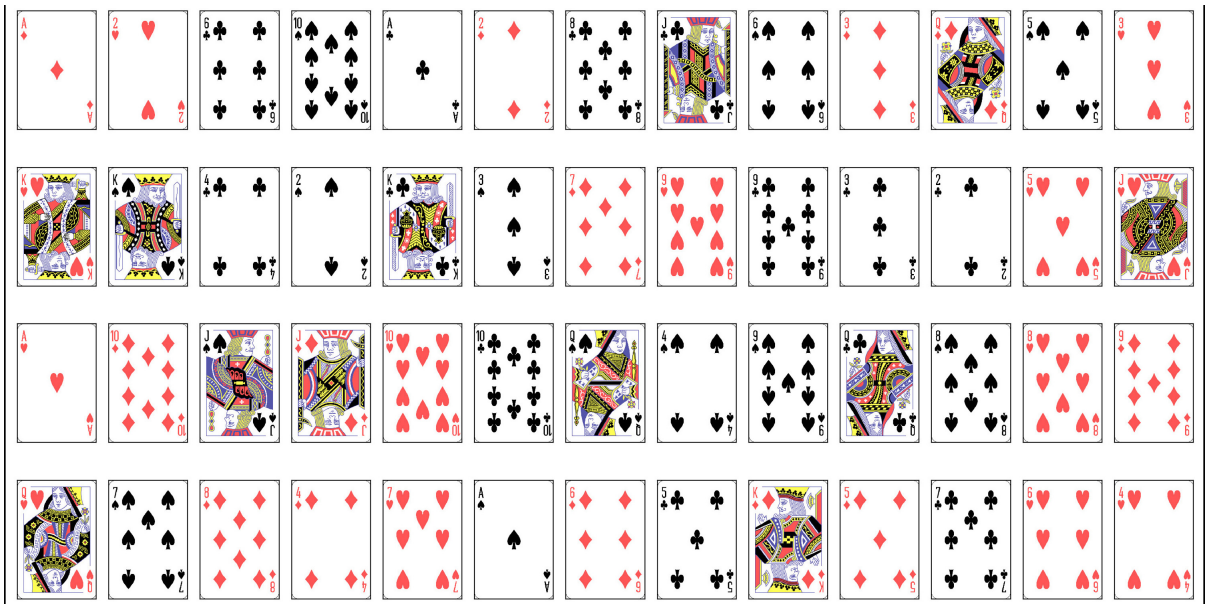
Shuffle and Re-Deal the Deck

To see the images shuffled, call method `shuffle`, then re-execute snippet `[9]`'s code:

lick here to view code image

```
In [11]: deck_of_cards.shuffle()

In [12]: for axes in axes_list.ravel():
    ...:     axes.get_xaxis().set_visible(False)
    ...:     axes.get_yaxis().set_visible(False)
    ...:     image_name =   deck_of_cards.deal_card().image_name
    ...:     img =   mpimg.imread(str(path.joinpath(image_name).resolve()))
    ...:     axes.imshow(img)
    ...:
```

C



10.7 INHERITANCE: BASE CLASSES AND SUBCLASSES

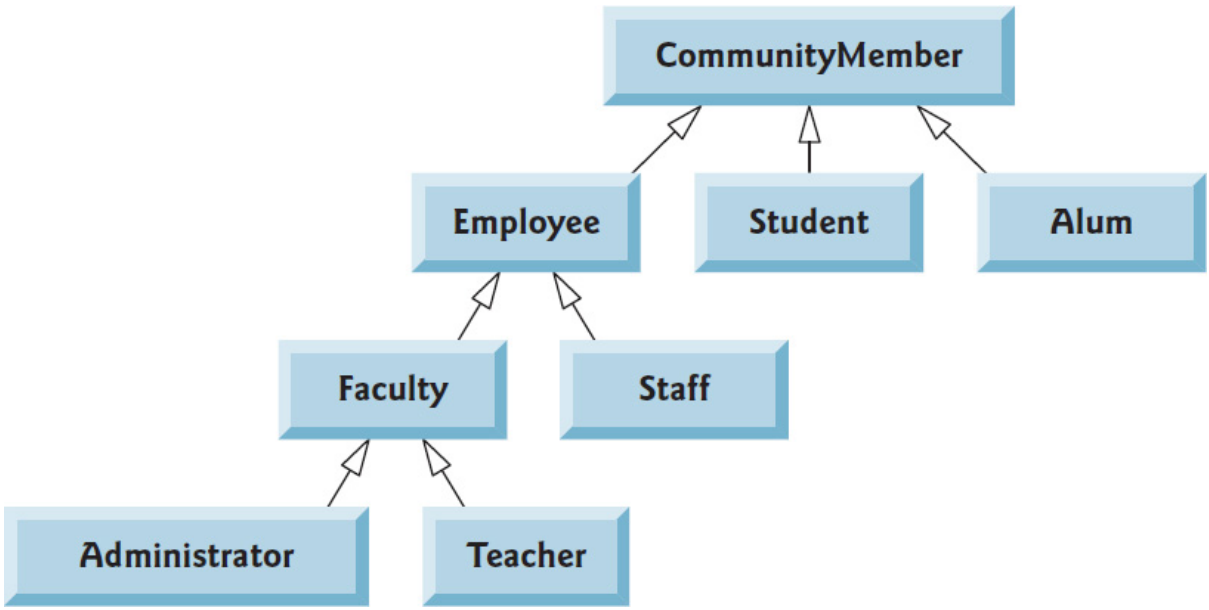
Often, an object of one class *is an* object of another class as well. For example, a `CarLoan` *is a* `Loan` as are `HomeImprovementLoan`s and `MortgageLoan`s. Class `CarLoan` can be said to inherit from class `Loan`. In this context, class `Loan` is a base class, and class `CarLoan` is a subclass. A `CarLoan` *is a* specific type of `Loan`, but it's incorrect to claim that every `Loan` *is a* `CarLoan`—the `Loan` could be of any type. The following table lists simple examples of base classes and subclasses—base classes tend to be "more general" and subclasses "more specific":

Base class	Subclasses
`Student`	`GraduateStudent`, `UndergraduateStudent`
`Shape`	`Circle`, `Triangle`, `Rectangle`, `Sphere`, `Cube`
`Loan`	`CarLoan`, `HomeImprovementLoan`, `MortgageLoan`
`Employee`	`Faculty`, `Staff`
`BankAccount`	`CheckingAccount`, `SavingsAccount`

ecause every subclass object *is an* object of its base class, and one base class can have many subclasses, the set of objects represented by a base class is often larger than the set of objects represented by any of its subclasses. For example, the base class `Vehicle` represents *all* vehicles, including cars, trucks, boats, bicycles and so on. By contrast, subclass `Car` represents a smaller, more specific subset of vehicles.

`CommunityMember` Inheritance Hierarchy

Inheritance relationships form tree-like *hierarchical* structures. A base class exists in a hierarchical relationship with its subclasses. Let's develop a sample class hierarchy (shown in the following diagram), also called an **inheritance hierarchy**. A university community has thousands of members, including employees, students and alumni. Employees are either faculty or staff members. Faculty members are either administrators (e.g., deans and department chairpersons) or teachers. The hierarchy could contain many other classes. For example, students can be graduate or undergraduate students. Undergraduate students can be freshmen, sophomores, juniors or seniors. With **single inheritance**, a class is derived from *one* base class. With **multiple inheritance**, a subclass inherits from *two or more* base classes. Single inheritance is straightforward. Multiple inheritance is beyond the scope of this book. Before you use it, search online for the "diamond problem in Python multiple inheritance."



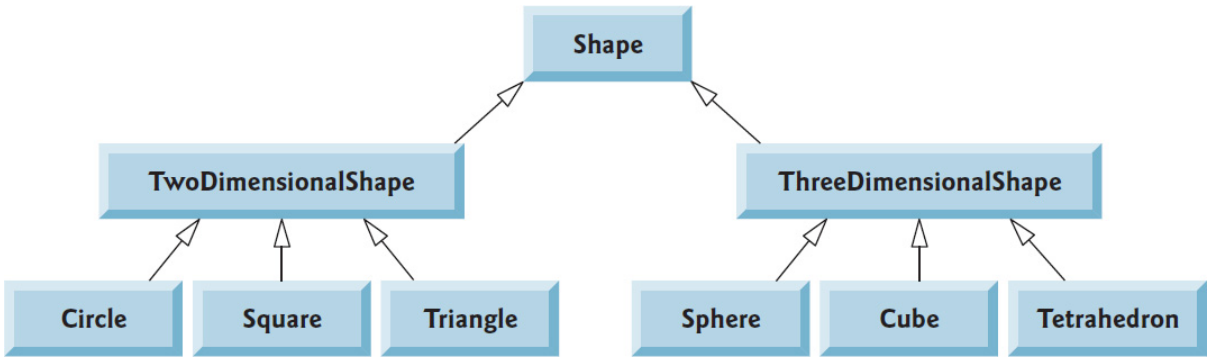
Each arrow in the hierarchy represents an *is-a* relationship. As we follow the arrows upward in this class hierarchy, we can state, for example, that "an `Employee` *is a* `Community-Member`" and "a `Teacher` *is a* `Faculty` member." `CommunityMember` is the direct base class of `Employee`, `Student` and `Alum` and is an indirect base class of all the other classes in the diagram. Starting from the bottom, you can follow the arrows and apply the *is-a* relationship up to the topmost superclass. For example, `Administrator` *is a* `Faculty` member, *is an* `Employee`, *is a* `Community-Member` and, of course, ultimately *is an* `object`.

`Shape` Inheritance Hierarchy

Now consider the `Shape` inheritance hierarchy in the following class diagram, which begins

B

ith base class `Shape`, followed by subclasses `TwoDimensionalShape` and `ThreeDimensionalShape`. Each `Shape` is either a `TwoDimensionalShape` or a `ThreeDimensionalShape`. The third level of this hierarchy contains *specific* types of `TwoDimensionalShape`s and `ThreeDimensionalShape`s. Again, we can follow the arrows from the bottom of the diagram to the topmost base class in this class hierarchy to identify several *is-a* relationships. For example, a `Triangle` *is a* `TwoDimensionalShape` and *is a* `Shape`, while a `Sphere` *is a* `ThreeDimensionalShape` and *is a* `Shape`. This hierarchy could contain many other classes. For example, ellipses and trapezoids also are `TwoDimensionalShape`s, and cones and cylinders also are `ThreeDimensionalShape`s.



"is a" vs. "has a"

Inheritance produces **"is-a" relationships** in which an object of a subclass type may also be treated as an object of the base-class type. You've also seen "has-a" (composition) relationships in which a class has references to one or more objects of other classes as members.

10.8 BUILDING AN INHERITANCE HIERARCHY; INTRODUCING POLYMORPHISM

Let's use a hierarchy containing types of employees in a company's payroll app to discuss the relationship between a base class and its subclass. All employees of the company have a lot in common, but *commission employees* (who will be represented as objects of a base class) are paid a percentage of their sales, while *salaried commission employees* (who will be represented as objects of a subclass) receive a percentage of their sales *plus* a base salary.

First, we present *base class* `CommissionEmployee`. Next, we create a *subclass* `SalariedCommissionEmployee` that inherits from class `CommissionEmployee`. Then, we use an IPython session to create a `SalariedCommissionEmployee` object and demonstrate that it has all the capabilities of the base class *and* the subclass, but calculates its earnings differently.

10.8.1 Base Class `CommissionEmployee`

Consider class `CommissionEmployee`, which provides the following features:

- Method `__init__` (lines 8–15), which creates the data attributes `_first_name`,

w

_last_name and _ssn (Social Security number), and uses the setters of properties gross_sales and commission_rate to create their corresponding data attributes.

- Read-only properties first_name (lines 17–19), last_name (lines 21–23) and ssn (line 25–27), which return the corresponding data attributes.
- Read-write properties gross_sales (lines 29–39) and commission_rate (lines 41–52), in which the setters perform data validation.
- Method earnings (lines 54–56), which calculates and returns a CommissionEmployee's earnings.
- Method __repr__ (lines 58–64), which returns a string representation of a CommissionEmployee.

lick here to view code image

```
1 # commmissionemployee.py
2 """CommissionEmployee base   class."""
3 from decimal import Decimal
4
5 class CommissionEmployee:
6     """An employee who gets paid commission   based on gross sales."""
7
8     def __init__(self, first_name, last_name, ssn,
9                  gross_sales,   commission_rate):
10         """Initialize CommissionEmployee's   attributes."""
11         self._first_name   = first_name
12         self._last_name   = last_name
13         self._ssn   = ssn
14         self.gross_sales   = gross_sales  # validate via property
15         self.commission_rate   = commission_rate  # validate via property
16
17     @property
18     def first_name(self):
19         return self._first_name
20
21     @property
22     def last_name(self):
23         return self._last_name
24
25     @property
26     def ssn(self):
27         return self._ssn
28
29     @property
30     def gross_sales(self):
31         return self._gross_sales
32
33     @gross_sales.setter
34     def gross_sales(self, sales):
35         """Set gross sales or raise ValueError   if invalid."""
36         if sales < Decimal('0.00'):
37             raise ValueError('Gross sales must   be >= to 0')
38
```

C

```
        self._gross_sales    = sales

    @property
    def commission_rate(self):
        return self._commission_rate

    @commission_rate.setter
    def commission_rate(self, rate):
        """Set commission rate or raise   ValueError if invalid."""
        if not (Decimal('0.0') <    rate < Decimal('1.0')):
            raise ValueError(
               'Interest rate must be greater than 0 and less than 1')

        self._commission_rate    = rate

    def earnings(self):
        """Calculate   earnings.""""
        return self.gross_sales * self.commission_rate

    def __repr__(self):
        """Return string representation for   repr()."""
        return ('CommissionEmployee: '    +
            f'{self.first_name} {self.last_name}\n' +
            f'social security number: {self.ssn}\n' +
            f'gross sales: {self.gross_sales:.2f}\n' +
            f'commission rate: {self.commission_rate:.2f}')
```

Properties `first_name`, `last_name` and `ssn` are read-only. We chose not to validate them, though we could have. For example, we could validate the first and last names—perhaps by ensuring that they're of a reasonable length. We could validate the Social Security number to ensure that it contains nine digits, with or without dashes (for example, to ensure that it's in the format `###-##-####` or `#########`, where each `#` is a digit).

All Classes Inherit Directly or Indirectly from Class `object`

You use inheritance to create new classes from existing ones. In fact, *every* Python class inherits from an existing class. When you do not explicitly specify the base class for a new class, Python assumes that the class inherits directly from class `object`. The Python class hierarchy begins with class `object`, the direct or indirect base class of *every* class. So, class `CommissionEmployee`'s header could have been written as

```
class CommissionEmployee(object):
```

The parentheses after `CommissionEmployee` indicate inheritance and may contain a single class for single inheritance or a comma-separated list of base classes for multiple inheritance. Once again, multiple inheritance is beyond the scope of this book.

Class `CommissionEmployee` inherits all the methods of class `object`. Class `object` does not have any data attributes. Two of the many methods inherited from `object` are `__repr__` and `__str__`. So *every* class has these methods that return string representations of the objects on which they're called. When a base-class method

implementation is inappropriate for a derived class, that method can be **overridden** (i.e., redefined) in the derived class with an appropriate implementation. Method `__repr__` (lines 58–64) overrides the default implementation inherited into class `CommissionEmployee` from class `object`. [6]

[6] ee `ttps://docs.python.org/3/reference/datamodel.html` for `objects` overridable methods.

Testing Class `CommissionEmployee`

Let's quickly test some of `CommissionEmployee`'s features. First, create and display a `CommissionEmployee`:

lick here to view code image

```
In [1]: from commissionemployee import CommissionEmployee

In [2]: from decimal import Decimal

In [3]: c = CommissionEmployee('Sue',    'Jones', '333-33-3333',
   ...:      Decimal('10000.00'), Decimal('0.06'))
   ...:

In [4]: c
Out[4]:
CommissionEmployee: Sue Jones
social security number: 333-33-3333
gross sales: 10000.00
commission rate: 0.06
```

Next, let's calculate and display the `CommissionEmployee`'s earnings:

lick here to view code image

```
In [5]: print(f'{c.earnings():,.2f}')
600.00
```

Finally, let's change the `CommissionEmployee`'s gross sales and commission rate, then recalculate the earnings:

lick here to view code image

```
In [6]: c.gross_sales = Decimal('20000.00')

In [7]: c.commission_rate = Decimal('0.1')

In [8]: print(f'{c.earnings():,.2f}')
2,000.00
```

10.8.2 Subclass `SalariedCommissionEmployee`

With single inheritance, the subclass starts essentially the same as the base class. The real strength of inheritance comes from the ability to define in the subclass additions, replacements or refinements for the features inherited from the base class.

Many of a `SalariedCommissionEmployee`'s capabilities are similar, if not identical, to those of class `CommissionEmployee`. Both types of employees have first name, last name, Social Security number, gross sales and commission rate data attributes, and properties and methods to manipulate that data. To create class `SalariedCommissionEmployee` *without* using inheritance, we could have *copied* class `CommissionEmployee`'s code and *pasted* it into class `SalariedCommissionEmployee`. Then we could have modified the new class to include a base salary data attribute, and the properties and methods that manipulate the base salary, including a new `earnings` method. This *copy-and-paste approach* is often error-prone. Worse yet, it can spread many physical copies of the same code (including errors) throughout a system, making your code less maintainable. Inheritance enables us to "absorb" the features of a class *without* duplicating code. Let's see how.

Declaring Class `SalariedCommissionEmployee`

We now declare the subclass `SalariedCommissionEmployee`, which *inherits* most of its capabilities from class `CommissionEmployee` (line 6). A `SalariedCommissionEmployee` *is a* `CommissionEmployee` (because inheritance passes on the capabilities of class `CommissionEmployee`), but class `SalariedCommissionEmployee` also has the following features:

- Method `__init__` (lines 10–15), which initializes all the data inherited from class `CommissionEmployee` (we'll say more about this momentarily), then uses the `base_salary` property's `setter` to create a `_base_salary` data attribute.
- Read-write property `base_salary` (lines 17–27), in which the `setter` performs data validation.
- A customized version of method `earnings` (lines 29–31).
- A customized version of method `__repr__` (lines 33–36).

lick here to view code image

```
1 # salariedcommissionemployee.py
2 """SalariedCommissionEmployee derived from   CommissionEmployee."""
3 from commissionemployee import   CommissionEmployee
4 from decimal import Decimal
5
6 class SalariedCommissionEmployee(CommissionEmployee):
7     """An employee who gets paid a salary plus
8     commission based on gross sales."""
9
```

C

```
10      def __init__(self, first_name, last_name, ssn,
11                   gross_sales,   commission_rate, base_salary):
12          """Initialize   SalariedCommissionEmployee's attributes."""
13          super().__init__(first_name,   last_name, ssn,
14                           gross_sales,   commission_rate)
15          self.base_salary = base_salary  #   validate via property
16
17      @property
18      def base_salary(self):
19          return self._base_salary
20
21      @base_salary.setter
22      def base_salary(self, salary):
23          """Set base salary or raise ValueError   if invalid."""
24          if salary < Decimal('0.00'):
25              raise ValueError('Base salary must   be >= to 0')
26
27          self._base_salary   = salary
28
29      def earnings(self):
30          """Calculate   earnings."""”
31          return super().earnings() + self.base_salary
32
33      def __repr__(self):
34          """Return string representation for   repr()."""
35          return ('Salaried' +   super().__repr__() +
36              f'\nbase salary: {self.base_salary:.2f}')
```

Inheriting from Class `CommissionEmployee`

To inherit from a class, you must first `import` its definition (line 3). Line 6

```
class SalariedCommissionEmployee(CommissionEmployee):
```

specifies that class `SalariedCommissionEmployee` *inherits* from `CommissionEmployee`. Though you do not see class `CommissionEmployee`'s data attributes, properties and methods in class `SalariedCommissionEmployee`, they're nevertheless part of the new class, as you'll soon see.

Method `__init__` and Built-In Function `super`

Each subclass `__init__` must explicitly call its base class's `__init__` to initialize the data attributes inherited from the base class. This call should be the first statement in the subclass's `__init__` method. `SalariedCommissionEmployee`'s `__init__` method explicitly calls class `CommissionEmployee`'s `__init__` method (lines 13–14) to initialize the base-class portion of a `SalariedCommissionEmployee` object (that is, the five inherited data attributes from class `CommissionEmployee`). The notation `super().__init__` uses the built-in function **`super`** to locate and call the base class's `__init__` method, passing the five arguments that initialize the inherited data attributes.

Overriding Method `earnings`

Class `SalariedCommissionEmployee`'s `earnings` method (lines 29–31) overrides class `CommissionEmployee`'s `earnings` method (ection 10.8.1, lines 54–56) to calculate the earnings of a `SalariedCommissionEmployee`. The new version obtains the portion of the earnings based on *commission alone* by calling `CommissionEmployee`'s `earnings` method with the expression `super().earnings()` (line 31). `SalariedCommissionEmployee`'s `earnings` method then adds the `base_salary` to this value to calculate the total earnings. By having `SalariedCommissionEmployee`'s `earnings` method invoke `CommissionEmployee`'s `earnings` method to calculate part of a `SalariedCommissionEmployee`'s earnings, we avoid duplicating the code and reduce code-maintenance problems.

Overriding Method `__repr__`

`SalariedCommissionEmployee`'s `__repr__` method (lines 33–36) overrides class `CommissionEmployee`'s `__repr__` method (ection 10.8.1, lines 58–64) to return a `String` representation that's appropriate for a `SalariedCommissionEmployee`. The subclass creates part of the string representation by concatenating `'Salaried'` and the string returned by `super().__repr__()`, which calls `Commission-Employee`'s `__repr__` method. The overridden method then concatenates the base salary information and returns the resulting string.

Testing Class `SalariedCommissionEmployee`

Let's test class `SalariedCommissionEmployee` to show that it indeed inherited capabilities from class `CommissionEmployee`. First, let's create a `SalariedCommissionEmployee` and print all of its properties:

lick here to view code image

```
In [9]: from   salariedcommissionemployee import   SalariedCommissionEmployee

In [10]: s = SalariedCommissionEmployee('Bob', 'Lewis', '444-44-4444',
    ...:          Decimal('5000.00'),   Decimal('0.04'), Decimal('300.00'))
    ...:

In [11]: print(s.first_name, s.last_name, s.ssn,   s.gross_sales,
    ...:       s.commission_rate, s.base_salary)
Bob Lewis 444-44-4444 5000.00 0.04 300.00
```

Notice that the `SalariedCommissionEmployee` object has *all* of the properties of classes `CommissionEmployee` *and* `SalariedCommissionEmployee`.

Next, let's calculate and display the `SalariedCommissionEmployee`'s earnings. Because we call method `earnings` on a `SalariedCommissionEmployee` object, the *subclass version* of the method executes:

lick here to view code image

S

```
In [12]: print(f'{s.earnings():,.2f}')
500.00
```

Now, let's modify the `gross_sales`, `commission_rate` and `base_salary` properties, then display the updated data via the `Salaried-Commission-Employee`'s `__repr__` method:

lick here to view code image

```
In [13]: s.gross_sales = Decimal('10000.00')

In [14]: s.commission_rate = Decimal('0.05')

In [15]: s.base_salary = Decimal('1000.00')

In [16]: print(s)
SalariedCommissionEmployee: Bob Lewis
social security number: 444-44-4444
gross sales: 10000.00
commission rate: 0.05
base salary: 1000.00
```

Again, because this method is called on a `SalariedCommissionEmployee` object, the *subclass version* of the method executes. Finally, let's calculate and display the `Salaried-Commission-Employee`'s updated earnings:

lick here to view code image

```
In [17]: print(f'{s.earnings():,.2f}')
1,500.00
```

Testing the "is a" Relationship

Python provides two built-in functions—**`issubclass`** and **`isinstance`**—for testing "is a" relationships. Function `issubclass` determines whether one class is derived from another:

lick here to view code image

```
In [18]: issubclass(SalariedCommissionEmployee,   CommissionEmployee)
Out[18]: True
```

Function `isinstance` determines whether an object has an "is a" relationship with a specific type. Because `SalariedCommissionEmployee` inherits from `CommissionEmployee`, both of the following snippets return `True`, confirming the "is a" relationship

lick here to view code image

```
In [19]: isinstance(s, CommissionEmployee)
```

C

```
Out[19]: True

In [20]: isinstance(s, SalariedCommissionEmployee)
Out[20]: True
```

10.8.3 Processing `CommissionEmployee`s and `SalariedCommissionEmployee`s Polymorphically

With inheritance, every object of a subclass also may be treated as an object of that subclass's base class. We can take advantage of this "subclass-object-is-a-base-class-object" relationship to perform some interesting manipulations. For example, we can place objects related through inheritance into a list, then iterate through the list and treat each element as a base-class object. This allows a variety of objects to be processed in a *general* way. Let's demonstrate this by placing the `CommissionEmployee` and `SalariedCommissionEmployee` objects in a list, then for each element displaying its string representation and earnings:

lick here to view code image

```
In [21]: employees = [c, s]

In [22]: for employee in employees:
    ...:     print(employee)
    ...:     print(f'{employee.earnings():,.2f}\n')
    ...:
CommissionEmployee: Sue Jones
social security number: 333-33-3333
gross sales: 20000.00
commission rate: 0.10
2,000.00

SalariedCommissionEmployee: Bob Lewis
social security number: 444-44-4444
gross sales: 10000.00
commission rate: 0.05
base salary: 1000.00
1,500.00
```

As you can see, the correct string representation and earnings are displayed for each employee. This is called *polymorphism*—a key capability of object-oriented programming (OOP).

10.8.4 A Note About Object-Based and Object-Oriented Programming

Inheritance with method overriding is a powerful way to build software components that are *like* existing components but need to be customized to your application's unique needs. In the Python open-source world, there are a huge number of well-developed class libraries for which your programming style is:

- know what libraries are available,

C

- know what classes are available,
- make objects of existing classes, and
- send them messages (that is, call their methods).

This style of programming is called *object-based programming (OBP)*. When you do composition with objects of known classes, you're still doing object-based programming. Adding inheritance with overriding to customize methods to the unique needs of your applications and possibly process objects polymorphically is called *object-oriented programming (OOP)*. If you do composition with objects of inherited classes, that's also object-oriented programming.

10.9 DUCK TYPING AND POLYMORPHISM

Most other object-oriented programming languages require inheritance-based "is a" relationships to achieve polymorphic behavior. Python is more flexible. It uses a concept called duck typing, which the Python documentation describes as:

A programming style which does not look at an object's type to determine if it has the right interface; instead, the method or attribute is simply called or used ("If it looks like a duck and quacks like a duck, it must be a duck."). [7]

[7] ttps://docs.python.org/3/glossary.html#term-duck-typing.

So, when processing an object at execution time, its type does not matter. As long as the object has the data attribute, property or method (with the appropriate parameters) you wish to access, the code will work.

Let's reconsider the loop at the end of ection 10.8.3, which processes a list of employees:

```
    for employee in employees:
print(employee)
print(f'{employee.earnings():,.2f}\n')
```

In Python, this loop works properly as long as `employees` contains only objects that:

- can be displayed with `print` (that is, they have a string representation) and
- have an `earnings` method which can be called with no arguments.

All classes inherit from `object` directly or indirectly, so they *all* inherit the default methods for obtaining string representations that `print` can display. If a class has an `earnings` method that can be called with no arguments, we can include objects of that class in the list `employees`, even if the object's class does not have an "is a" relationship with class `CommissionEmployee`. To demonstrate this, consider class `WellPaidDuck`:

lick here to view code image

```
In [1]: class WellPaidDuck:
   ...:     def __repr__(self):
   ...:         return 'I am a well-paid duck'
   ...:     def earnings(self):
   ...:         return Decimal('1_000_000.00')
   ...:
```

`WellPaidDuck` objects, which clearly are not meant to be employees, will work with the preceding loop. To prove this, let's create objects of our classes `CommissionEmployee`, `SalariedCommissionEmployee` and `WellPaidDuck` and place them in a list:

lick here to view code image

```
In [2]: from decimal import Decimal

In [3]: from commissionemployee import CommissionEmployee

In [4]: from   salariedcommissionemployee import   SalariedCommissionEmployee

In [5]: c = CommissionEmployee('Sue',   'Jones', '333-33-3333',
   ...:                        Decimal('10000.00'), Decimal('0.06'))
   ...:

In [6]: s = SalariedCommissionEmployee('Bob', 'Lewis', '444-44-4444',
   ...:     Decimal('5000.00'),   Decimal('0.04'), Decimal('300.00'))
   ...:

In [7]: d = WellPaidDuck()

In [8]: employees = [c, s, d]
```

Now, let's process the list using the loop from ection 10.8.3. As you can see in the output, Python is able to use duck typing to *polymorphically* process all three objects in the list:

lick here to view code image

```
In [9]: for employee in employees:
   ...:     print(employee)
   ...:     print(f'{employee.earnings():,.2f}\n')
   ...:
CommissionEmployee: Sue Jones
social security number: 333-33-3333
gross sales: 10000.00
commission rate: 0.06
600.00

SalariedCommissionEmployee: Bob Lewis
social security number: 444-44-4444
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00
```

S

```
500.00

I am a well-paid duck
1,000,000.00
```

10.10 OPERATOR OVERLOADING

You've seen that you can interact with objects by accessing their attributes and properties and by calling their methods. Method-call notation can be cumbersome for certain kinds of operations, such as arithmetic. In these cases, it would be more convenient to use Python's rich set of built-in operators.

This section shows how to use **operator overloading** to define how Python's operators should handle objects of your own types. You've already used operator overloading frequently across wide ranges of types. For example, you've used:

- the + operator for adding numeric values, concatenating lists, concatenating strings and adding a value to every element in a NumPy array.
- the `[]` operator for accessing elements in lists, tuples, strings and arrays and for accessing the value for a specific key in a dictionary.
- the * operator for multiplying numeric values, repeating a sequence and multiplying every element in a NumPy array by a specific value.

You can overload most operators. For every overloadable operator, class `object` defines a special method, such as `__add__` for the addition (+) operator or `__mul__` for the multiplication (*) operator. Overriding these methods enables you to define how a given operator works for objects of your custom class. For a complete list of special methods, see

```
ttps://docs.python.org/3/reference/datamodel.html#special-method-names
```

Operator Overloading Restrictions

There are some restrictions on operator overloading:

- The precedence of an operator cannot be changed by overloading. However, parentheses can be used to force evaluation order in an expression.
- The left-to-right or right-to-left grouping of an operator cannot be changed by overloading.
- The "arity" of an operator—that is, whether it's a unary or binary operator—cannot be changed.
- You cannot create new operators—only existing operators can be overloaded.

h

- The meaning of how an operator works on objects of built-in types cannot be changed. You cannot, for example, change + so that it subtracts two integers.
- Operator overloading works only with objects of custom classes or with a mixture of an object of a custom class and an object of a built-in type.

Complex Numbers

To demonstrate operator overloading, we'll define a class named `Complex` that represents complex numbers. [8] Complex numbers, like –3 + 4i and 6.2 – 11.73i, have the form

[8] Python has built-in support for complex values, so this class is simply for demonstration purposes.

```
realPart + imaginaryPart * i
```

where `i` is $\sqrt{-1}$. Like `int`s, `float`s and `Decimal`s, complex numbers are arithmetic types. In this section, we'll create a class `Complex` that overloads just the + addition operator and the += augmented assignment, so we can add `Complex` objects using Python's mathematical notations.

10.10.1 Test-Driving Class `Complex`

First, let's use class `Complex` to demonstrate its capabilities. We'll discuss the class's details in the next section. Import class `Complex` from `complexnumber.py`:

lick here to view code image

```
In [1]: from complexnumber import Complex
```

Next, create and display a couple of `Complex` objects. Snippets `[3]` and `[5]` implicitly call the `Complex` class's `__repr__` method to get a string representation of each object:

lick here to view code image

```
In [2]: x = Complex(real=2,   imaginary=4)

In [3]: x
Out[3]: (2 + 4i)

In [4]: y = Complex(real=5,   imaginary=-1)

In [5]: y
Out[5]: (5 - 1i)
```

We chose the `__repr__` string format shown in snippets `[3]` and `[5]` to mimic the `__repr__` strings produced by Python's built-in `complex` type. [9]

C

[9] Python uses `j` rather than `i` for . For example, `3+4j` (with no spaces around the operator) creates a complex object with `real` and `imag` attributes. The `__repr__` string for this complex value is `'(3+4j)'`.

Now, let's use the `+` operator to add the `Complex` objects `x` and `y`. This expression adds the real parts of the two operands (`2` and `5`) and the imaginary parts of the two operands (`4i` and `-1i`), then returns a new `Complex` object containing the result:

```
In [6]: x + y
Out[6]: (7 + 3i)
```

The `+` operator does not modify either of its operands:

```
In [7]: x
Out[7]: (2 + 4i)

In [8]: y
Out[8]: (5 - 1i)
```

Finally, let's use the `+=` operator to add `y` to `x` and store the result in `x`. The `+=` operator *modifies* its left operand but not its right operand:

```
In [9]: x += y

In [10]: x
Out[10]: (7 + 3i)

In [11]: y
Out[11]: (5 - 1i)
```

10.10.2 Class `Complex` Definition

Now that we've seen class `Complex` in action, let's look at its definition to see how those capabilities were implemented.

Method `__init__`

The class's `__init__` method receives parameters to initialize the `real` and `imaginary` data attributes:

lick here to view code image

```
1 # complexnumber.py
2 """Complex class with overloaded   operators."""
3
4 class Complex:
5     """Complex class that represents a complex   number
6     with real and imaginary parts."""
7
```

C

```
    def __init__(self, real, imaginary):
        """Initialize Complex class's   attributes."""
         self.real    = real
         self.imaginary    = imaginary

```

Overloaded + Operator

The following overridden special method **`__add__`** defines how to overload the + operator for use with two `Complex` objects:

lick here to view code image

```
    def __add__(self, right):
        """Overrides the +   operator."""
        return Complex(self.real + right.real,
                       self.imaginary    + right.imaginary)

```

Methods that overload binary operators must provide two parameters—the *first* (`self`) is the *left* operand and the *second* (`right`) is the *right* operand. Class `Complex`'s `__add__` method takes two `Complex` objects as arguments and returns a new `Complex` object containing the sum of the operands' `real` parts and the sum of the operands' `imaginary` parts.

We do *not* modify the contents of either of the original operands. This matches our intuitive sense of how this operator should behave. Adding two numbers does not modify either of the original values.

Overloaded += Augmented Assignment

Lines 18–22 overload special method **`__iadd__`** to define how the += operator adds two `Complex` objects:

lick here to view code image

```
    def __iadd__(self, right):
        """Overrides the +=   operator."""
        self.real    += right.real
        self.imaginary    += right.imaginary
        return self

```

Augmented assignments modify their left operands, so method `__iadd__` modifies the `self` object, which represents the left operand, then returns `self`.

Method __repr__

Lines 24–28 return the string representation of a `Complex` number.

lick here to view code image

C

```
    def __repr__(self):
        """Return string representation for   repr()."""
        return (f'({self.real} ' +
                ('+' if self.imaginary >=   0 else '-') +
                f' {abs(self.imaginary)}i)')
```

10.11 EXCEPTION CLASS HIERARCHY AND CUSTOM EXCEPTIONS

In the previous chapter, we introduced exception handling. Every exception is an object of a class in Python's exception class hierarchy [o] or an object of a class that inherits from one of those classes. Exception classes inherit directly or indirectly from base class `BaseException` and are defined in module **`exceptions`**.

[o] `ttps://docs.python.org/3/library/exceptions.html`.

Python defines four primary `BaseException` subclasses—**`SystemExit`**, **`KeyboardInterrupt`**, **`GeneratorExit`** and **`Exception`**:

- `SystemExit` terminates program execution (or terminates an interactive session) and when uncaught does not produce a traceback like other exception types.
- `KeyboardInterrupt` exceptions occur when the user types the interrupt command —*Ctrl* + *C* (or *control* + *C*) on most systems.
- `GeneratorExit` exceptions occur when a generator closes—normally when a generator finishes producing values or when its `close` method is called explicitly.
- `Exception` is the base class for most common exceptions you'll encounter. You've seen exceptions of the `Exception` subclasses `ZeroDivisionError`, `NameError`, `ValueError`, `StatisticsError`, `TypeError`, `IndexError`, `KeyError`, `RuntimeError` and `AttributeError`. Often, `StandardError`s can be caught and handled, so the program can continue running.

Catching Base-Class Exceptions

One of the benefits of the exception class hierarchy is that an `except` handler can catch exceptions of a particular type or can use a base-class type to catch those base-class exceptions and all related subclass exceptions. For example, an `except` handler that specifies the base class `Exception` can catch objects of *any* subclass of `Exception`. Placing an `except` handler that catches type `Exception` before other `except` handlers is a logic error, because all exceptions would be caught before other exception handlers could be reached. Thus, subsequent exception handlers are unreachable.

Custom Exception Classes

When you raise an exception from your code, you should generally use one of the existing

h

exception classes from the Python Standard Library. However, using the inheritance techniques presented earlier in this chapter, you can create your own custom exception classes that derive directly or indirectly from class `Exception`. Generally, that's discouraged, especially among novice programmers. Before creating custom exception classes, look for an appropriate existing exception class in the Python exception hierarchy. Define new exception classes only if you need to catch and handle the exceptions differently from other existing exception types. That should be rare.

10.12 NAMED TUPLES

You've used tuples to aggregate several data attributes into a single object. The Python Standard Library's **`collections` module** also provides **named tuples** that enable you to reference a tuple's members by name rather than by index number.

Let's create a simple named tuple that might be used to represent a card in a deck of cards. First, import function `namedtuple`:

lick here to view code image

```
In [1]: from collections import namedtuple
```

Function **`namedtuple`** creates a subclass of the built-in tuple type. The function's first argument is your new type's name and the second is a list of strings representing the identifiers you'll use to reference the new type's members:

lick here to view code image

```
In [2]: Card = namedtuple('Card',   ['face', 'suit'])
```

We now have a new tuple type named `Card` that we can use anywhere a tuple can be used. Let's create a `Card` object, access its members by name and display its string representation:

lick here to view code image

```
In [3]: card = Card(face='Ace',   suit='Spades')

In [4]: card.face
Out[4]: 'Ace'

In [5]: card.suit
Out[5]: 'Spades'

In [6]: card
Out[6]: Card(face='Ace', suit='Spades')
```

Other Named Tuple Features

C

Each named tuple type has additional methods. The type's **`_make` class method** (that is, a method called on the *class*) receives an iterable of values and returns an object of the named tuple type:

lick here to view code image

```
In [7]: values = ['Queen', 'Hearts']

In [8]: card = Card._make(values)

In [9]: card
Out[9]: Card(face='Queen', suit='Hearts')
```

This could be useful, for example, if you have a named tuple type representing records in a CSV file. As you read and tokenize CSV records, you could convert them into named tuple objects.

For a given object of a named tuple type, you can get an **`OrderedDict`** dictionary representation of the object's member names and values. An `OrderedDict` remembers the order in which its key–value pairs were inserted in the dictionary:

lick here to view code image

```
In [10]: card._asdict()
Out[10]: OrderedDict([('face', 'Queen'), ('suit',   'Hearts')])
```

For additional named tuple features see:

```
ttps://docs.python.org/3/library/collections.html#collections.namedtuple
```

10.13 A BRIEF INTRO TO PYTHON 3.7'S NEW DATA CLASSES

Though named tuples allow you to reference their members by name, they're still just tuples, not classes. For some of the benefits of named tuples, plus the capabilities that traditional Python classes provide, you can use Python 3.7's new **data classes** [1] from the Python Standard Library's **`dataclasses` module**.

[1] `ttps://www.python.org/dev/peps/pep-0557/.`

Data classes are among Python 3.7's most important new features. They help you build classes *faster* by using more *concise* notation and by *autogenerating* "boilerplate" code that's common in most classes. They could become the preferred way to define many Python classes. In this section, we'll present data-class fundamentals. At the end of the section, we'll provide links to more information.

Data Classes Autogenerate Code

Most classes you'll define provide an `__init__` method to create and initialize an object's attributes and a `__repr__` method to specify an object's custom string representation. If a class has many data attributes, creating these methods can be tedious.

Data classes *autogenerate* the data attributes and the `__init__` and `__repr__` methods for you. This can be particularly useful for classes that primarily aggregate related data items. For example, in an application that processes CSV records, you might want a class that represents each record's fields as data attributes in an object. Data classes also can be generated *dynamically* from a list of field names.

Data classes also autogenerate method **`__eq__`**, which overloads the == operator. Any class that has an `__eq__` method also implicitly supports `!=`. *All* classes inherit class `object`'s default **`__ne__`** (not equals) method implementation, which returns the opposite of `__eq__` (or `NotImplemented` if the class does not define `__eq__`). Data classes do *not* automatically generate methods for the <, <=, > and >= comparison operators, but they can.

10.13.1 Creating a `Card` Data Class

Let's reimplement class `Card` from ection 10.6.2 as a data class. The new class is defined in `carddataclass.py`. As you'll see, defining a data class requires some new syntax. In the subsequent subsections, we'll use our new `Card` data class in class `DeckOfCards` to show that it's interchangeable with the original `Card` class, then discuss some of the benefits of data classes over named tuples and traditional Python classes.

Importing from the `dataclasses` and `typing` Modules

The Python Standard Library's `dataclasses` module defines decorators and functions for implementing data classes. We'll use the **`@dataclass` decorator** (imported at line 4) to specify that a new class is a data class and causes various code to be written for you. Recall that our original `Card` class defined *class variables* `FACES` and `SUITS`, which are lists of the strings used to initialize `Card`s. We use **`ClassVar`** and **`List`** from the Python Standard Library's **`typing` module** (imported at line 5) to indicate that `FACES` and `SUITS` are *class variables* that refer to *lists*. We'll say more about these momentarily:

lick here to view code image

```
1 # carddataclass.py
2 """Card data class with class attributes, data   attributes,
3 autogenerated methods and explicitly defined   methods."""
4 from dataclasses import   dataclass
5 from typing import ClassVar,   List
6
```

Using the `@dataclass` Decorator

To specify that a class is a *data class*, precede its definition with the `@dataclass`

decorator: [2]

[2] ttps://docs.python.org/3/library/dataclasses.html#module-level-ecorators-classes--and-functions.

```
7 @dataclass
8 class Card:
```

Optionally, the `@dataclass` decorator may specify parentheses containing arguments that help the data class determine what autogenerated methods to include. For example, the decorator `@dataclass(order=True)` would cause the data class to autogenerate overloaded comparison operator methods for <, <=, > and >=. This might be useful, for example, if you need to sort your data-class objects.

Variable Annotations: Class Attributes

Unlike regular classes, data classes declare both class attributes and data attributes *inside* the class, but *outside* the class's methods. In a regular class, only *class attributes* are declared this way, and data attributes typically are created in `__init__`. Data classes require additional information, or *hints*, to distinguish class attributes from data attributes, which also affects the autogenerated methods' implementation details.

Lines 9–11 define and initialize the *class attributes* `FACES` and `SUITS`:

lick here to view code image

```
9     FACES:   ClassVar[List[str]] = ['Ace', '2', '3', '4', '5', '6', '7',
10                                     '8', '9', '10', 'Jack', 'Queen', 'King']
11     SUITS:   ClassVar[List[str]] = ['Hearts', 'Diamonds', 'Clubs', 'Spades'
12
```

In lines 9 and 11, The notation

```
: ClassVar[List[str]]
```

is a **variable annotation** [3, 4] (sometimes called a *type hint*) specifying that `FACES` is a class attribute (`ClassVar`) which refers to a *list* of strings (`List[str]`). `SUITS` also is a class attribute which refers to a list of strings.

[3] ttps://www.python.org/dev/peps/pep-0526/.

[4]Variable annotations are a recent language feature and are optional for regular classes. You will not see them in most legacy Python code.

Class variables are initialized in their definitions and are specific to the *class*, not individual *objects* of the class. Methods `__init__`, `__repr__` and `__eq__`, however, are for use with

objects of the class. When a data class generates these methods, it inspects all the variable annotations and includes only the *data attributes* in the method implementations.

Variable Annotations: Data Attributes

Normally, we create an object's data attributes in the class's `__init__` method (or methods called by `__init__`) via assignments of the form `self.`*attribute_name* = *value*. Because a data class *autogenerates* its `__init__` method, we need another way to specify data attributes in a data class's definition. We cannot simply place their names inside the class, which generates a `NameError`, as in:

lick here to view code image

```
In [1]: from dataclasses import dataclass

In [2]: @dataclass
   ...: class Demo:
   ...:     x  # attempting to   create a data attribute x
   ...:
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-2-79ffe37b1ba2> in <module>()
----> 1 @dataclass
      2 class Demo:
      3     x  # attempting to   create a data attribute x
      4

<ipython-input-2-79ffe37b1ba2> in Demo()
      1 @dataclass
      2 class Demo:
----> 3     x  # attempting to   create a data attribute x
      4

NameError: name 'x' is not   defined
```

Like class attributes, each data attribute must be declared with a variable annotation. Lines 13–14 define the data attributes `face` and `suit`. The variable annotation `": str"` indicates that each should refer to string objects:

```
13      face: str
14      suit: str
```

Defining a Property and Other Methods

Data classes are classes, so they may contain properties and methods and participate in class hierarchies. For this `Card` data class, we defined the same read-only `image_name` property and custom special methods `__str__` and `__format__` as in our original `Card` class earlier in the chapter:

lick here to view code image

C

```
    @property
    def image_name(self):
        """Return the Card's image file   name."""
        return str(self).replace(' ',   '_') + '.png'

    def __str__(self):
        """Return string representation for   str()."""
        return f'{self.face} of {self.suit}'

    def __format__(self, format):
        """Return formatted string   representation."""
        return f'{str(self):{format}}'
```

Variable Annotation Notes

You can specify variable annotations using built-in type names (like `str`, `int` and `float`), class types or types defined by the `typing` module (such as `ClassVar` and `List` shown earlier). Even with type annotations, Python is still a *dynamically typed language*. So, type annotations are *not* enforced at execution time. So, even though a `Card`'s `face` is meant to be a string, you can assign any type of object to `face`.

10.13.2 Using the `Card` Data Class

Let's demonstrate the new `Card` data class. First, create a `Card`:

lick here to view code image

```
In [1]: from carddataclass import Card

In [2]: c1 = Card(Card.FACES[0],   Card.SUITS[3])
```

Next, let's use `Card`'s autogenerated `__repr__` method to display the `Card`:

lick here to view code image

```
In [3]: c1
Out[3]: Card(face='Ace', suit='Spades')
```

Our custom `__str__` method, which `print` calls when passing it a `Card` object, returns a string of the form `'`*face* `of` *suit*`'`:

```
In [4]: print(c1)
Ace of Spades
```

Let's access our data class's attributes and read-only property:

lick here to view code image

```
In [5]: c1.face
```

C

```
Out[5]: 'Ace'

In [6]: c1.suit
Out[6]: 'Spades'

In [7]: c1.image_name
Out[7]: 'Ace_of_Spades.png'
```

Next, let's demonstrate that `Card` objects can be compared via the *autogenerated* == operator and inherited `!=` operator. First, create two additional `Card` objects—one identical to the first and one different:

lick here to view code image

```
In [8]: c2 = Card(Card.FACES[0],   Card.SUITS[3])

In [9]: c2
Out[9]: Card(face='Ace', suit='Spades')

In [10]: c3 = Card(Card.FACES[0],   Card.SUITS[0])

In [11]: c3
Out[11]: Card(face='Ace', suit='Hearts')
```

Now, compare the objects using == and `!=`:

```
In [12]: c1 == c2
Out[12]: True

In [13]: c1 == c3
Out[13]: False

In [14]: c1 != c3
Out[14]: True
```

Our `Card` data class is interchangeable with the `Card` class developed earlier in this chapter. To demonstrate this, we created the `deck2.py` file containing a copy of class `DeckOfCards` from earlier in the chapter and imported the `Card` data class into the file. The following snippets `import` class `DeckOfCards`, create an object of the class and `print` it. Recall that `print` implicitly calls the `DeckOfCards` `__str__` method, which formats each `Card` in a field of 19 characters, resulting in a call to each `Card`'s `__format__` method. Read each row left-to-right to confirm that all the `Card`s are displayed in order from each suit (`Hearts`, `Diamonds`, `Clubs` and `Spades`):

lick here to view code image

```
In [15]: from deck2 import DeckOfCards  # uses Card   data class

In [16]: deck_of_cards = DeckOfCards()

In [17]: print(deck  f  ards)
```

6

```
Ace of Hearts      2 of Hearts        3 of Hearts        4   of Hearts
5 of Hearts        6 of Hearts        7 of Hearts        8   of Hearts
9 of Hearts        10 of Hearts       Jack of Hearts       Queen of Hearts
King of Hearts     Ace of Diamonds    2 of Diamonds      3   of Diamonds
4 of Diamonds      5 of Diamonds      6 of Diamonds      7   of Diamonds
8 of Diamonds      9 of Diamonds      10 of Diamonds       Jack of Diamonds
Queen of Diamonds  King of Diamonds   Ace of Clubs       2   of Clubs
3 of Clubs         4 of Clubs         5 of Clubs         6   of Clubs
7 of Clubs         8 of Clubs         9 of Clubs         10   of Clubs
Jack of Clubs      Queen of Clubs     King of Clubs        Ace of Spades
2 of Spades        3 of Spades        4 of Spades        5   of Spades
6 of Spades        7 of Spades        8 of Spades        9   of Spades
10 of Spades       Jack of Spades     Queen of Spades      King of Spades
```

10.13.3 Data Class Advantages over Named Tuples

Data classes offer several advantages over named tuples [5]:

[5] `ttps://www.python.org/dev/peps/pep-0526/.`

- Although each named tuple technically represents a different type, a named tuple *is a* tuple and *all* tuples can be compared to one another. So, objects of *different* named tuple types could compare as equal if they have the same number of members and the same values for those members. Comparing objects of different data classes *always* returns `False`, as does comparing a data class object to a tuple object.

- If you have code that unpacks a tuple, adding more members to that tuple breaks the unpacking code. Data class objects cannot be unpacked. So you can add more data attributes to a data class without breaking existing code.

- A data class can be a base class or a subclass in an inheritance hierarchy.

10.13.4 Data Class Advantages over Traditional Classes

Data classes also offer various advantages over the traditional Python classes you saw earlier in this chapter:

- A data class autogenerates `__init__`, `__repr__` and `__eq__`, saving you time.

- A data class can autogenerate the special methods that overload the <, <=, > and >= comparison operators.

- When you change data attributes defined in a data class, then use it in a script or interactive session, the autogenerated code updates automatically. So, you have less code to maintain and debug.

- The required variable annotations for class attributes and data attributes enable you to take advantage of static code analysis tools. So, you might be able to eliminate additional errors before they can occur at execution time.

h

- Some static code analysis tools and IDEs can inspect variable annotations and issue warnings if your code uses the wrong type. This can help you locate logic errors in your code *before* you execute it.

More Information

Data classes have additional capabilities, such as creating “frozen” instances which do not allow you to assign values to a data class object’s attributes after the object is created. For a complete list of data class benefits and capabilities, see

```
ttps://www.python.org/dev/peps/pep-0557/
```

and

```
ttps://docs.python.org/3/library/dataclasses.html
```

10.14 UNIT TESTING WITH DOCSTRINGS AND `DOCTEST`

A key aspect of software development is testing your code to ensure that it works correctly. Even with extensive testing, however, your code may still contain bugs. According to the famous Dutch computer scientist Edsger Dijkstra, “Testing shows the presence, not the absence of bugs.” [6]

[6] J. N. Buxton and B. Randell, eds, *Software Engineering Techniques*, April 1970, p. 16. Report on a conference sponsored by the NATO Science Committee, Rome, Italy, 2731 October 1969

Module `doctest` and the `testmod` Function

The Python Standard Library provides the **`doctest` module** to help you test your code and conveniently retest it after you make modifications. When you execute the `doctest` module’s **`testmod` function**, it inspects your functions’, methods’ and classes' docstrings looking for sample Python statements preceded by >>>, each followed on the next line by the given statement’s expected output (if any). [7] The `testmod` function then executes those statements and confirms that they produce the expected output. If they do not, `testmod` reports errors indicating which tests failed so you can locate and fix the problems in your code. Each test you define in a docstring typically tests a specific *unit of code*, such as a function, a method or a class. Such tests are called **unit tests**.

[7] The notation >>> mimics the standard `python` interpreters input prompts.

Modified `Account` Class

The file `accountdoctest.py` contains the class `Account` from this chapter’s first example. We modified the `__init__` method’s docstring to include four tests which can be used to

h

ensure that the method works correctly:

- The test in line 11 creates a sample `Account` object named `account1`. This statement does not produce any output.
- The test in line 12 shows what the value of `account1`'s `name` attribute should be if line 11 executed successfully. The sample output is shown in line 13.
- The test in line 14 shows what the value of `account1`'s `balance` attribute should be if line 11 executed successfully. The sample output is shown in line 15.
- The test in line 18 creates an `Account` object with an invalid initial balance. The sample output shows that a `ValueError` exception should occur in this case. For exceptions, the doctest module's documentation recommends showing just the first and last lines of the traceback. [8]

[8] `ttps://docs.python.org/3/library/doctest.html?ighlight=doctest#module-doctest.`

You can intersperse your tests with descriptive text, such as line 17.

lick here to view code image

```
1 # accountdoctest.py
2 """Account class definition."""
3 from decimal import Decimal
4
5 class Account:
6     """Account class for demonstrating   doctest."""
7
8     def __init__(self, name, balance):
9         """Initialize an Account object.
10
11         >>> account1 = Account('John Green',   Decimal('50.00'))
12         >>> account1.name
13         'John Green'
14         >>> account1.balance
15         Decimal('50.00')
16
17         The balance argument must be greater than or equal to   0.
18         >>> account2 = Account('John Green',   Decimal('-50.00'))
19         Traceback (most recent call last):
20             ...
21         ValueError: Initial balance must be >= to 0.00.
22         """
23
24         # if balance is less than 0.00, raise an exception
25         if balance < Decimal('0.00'):
26             raise ValueError('Initial balance   must be >= to 0.00.')
27
28         self.name   = name
29         self.balance   = balance
30
```

```
31      def deposit(self, amount):
32          """Deposit money to the    account."""
33
34          # if amount is less than 0.00, raise an exception
35          if amount < Decimal('0.00'):
36              raise ValueError('amount must be    positive.')
37
38          self.balance    += amount
39
40  if __name__ == '__main__':
41      import doctest
42      doctest.testmod(verbose=True)
```

Module `__main__`

When you load any module, Python assigns a string containing the module's name to a global attribute of the module called `__name__`. When you execute a Python source file (such as `accountdoctest.py`) as a *script*, Python uses the string `'__main__'` as the module's name. You can use `__name__` in an `if` statement like lines 40–42 to specify code that should execute only if the source file is executed as a *script*. In this example, line 41 imports the `doctest` module and line 42 calls the module's `testmod` function to execute the docstring unit tests.

Running Tests

Run the file `accountdoctest.py` as a script to execute the tests. By default, if you call `testmod` with no arguments, it does not show test results for *successful* tests. In that case, if you get no output, all the tests executed successfully. In this example, line 42 calls `testmod` with the keyword argument `verbose=True`. This tells `testmod` to produce verbose output showing *every* test's results:

lick here to view code image

```
Trying:
    account1 = Account('John Green', Decimal('50.00'))
Expecting nothing
ok
Trying:
    account1.name
Expecting:
    'John Green'
ok
Trying:
    account1.balance
Expecting:
    Decimal('50.00')
ok
Trying:
    account2 = Account('John Green', Decimal('-50.00'))
Expecting:
    Traceback (most recent call last):
        ...
    ValueError: Initial balance must be >= to 0.00.
ok
```

C

```
3 items had no tests:
    __main__
    __main__.Account
    __main__.Account.deposit
1 items passed all tests:
   4 tests in __main__.Account.__init__
4 tests in 4 items.
4 passed and 0 failed.
Test passed.
```

In verbose mode, `testmod` shows for each test what it's `"Trying"` to do and what it's `"Expecting"` as a result, followed by `"ok"` if the test is successful. After completing the tests in verbose mode, `testmod` shows a summary of the results.

To demonstrate a *failed* test, "comment out" lines 25–26 in `accountdoctest.py` by preceding each with a `#`, then run `accountdoctest.py` as a script. To save space, we show just the portions of the doctest output indicating the failed test:

lick here to view code image

```
...
**********************************************************************
File "accountdoctest.py", line 18, in   __main__.Account.__init__
Failed example:
    account2 = Account('John Green', Decimal('-50.00'))
Expected:
    Traceback (most recent call last):
        ...
    ValueError: Initial balance must be >= to 0.00.
Got nothing
**********************************************************************
1 items had failures:
   1 of   4 in __main__.Account.__init__
4 tests in 4 items.
3 passed and 1 failed.
***Test Failed*** 1 failures.
```

In this case, we see that line 18's test failed. The `testmod` function was *expecting* a traceback indicating that a `ValueError` was raised due to the invalid initial balance. That exception did *not* occur, so the test failed. As the programmer responsible for defining this class, this failing test would be an indication that something is wrong with the validation code in your `__init__` method.

IPython `%doctest_mode` Magic

A convenient way to create doctests for existing code is to use an IPython interactive session to test your code, then copy and paste that session into a docstring. IPython's `In []` and `Out[]` prompts are not compatible with `doctest`, so IPython provides the magic **`%doctest_mode`** to display prompts in the correct `doctest` format. The magic toggles between the two prompt styles. The first time you execute `%doctest_mode`, IPython switches to >>> prompts for input and no output prompts. The second time you execute

C

`doctest_mode`, IPython switches back to `In []` and `Out[]` prompts.

10.15 NAMESPACES AND SCOPES

In the "Functions" chapter, we showed that each identifier has a scope that determines where you can use it in your program, and we introduced the local and global scopes. Here we continue our discussion of scopes with an introduction to namespaces.

Scopes are determined by **namespaces**, which associate identifiers with objects and are implemented "under the hood" as dictionaries. All namespaces are independent of one another. So, the same identifier may appear in multiple namespaces. There are three primary namespaces—local, global and built-in.

Local Namespace

Each function and method has a **local namespace** that associates local identifiers (such as, parameters and local variables) with objects. The local namespace exists from the moment the function or method is called until it terminates and is accessible *only* to that function or method. In a function's or method's suite, *assigning* to a variable that does not exist creates a local variable and adds it to the local namespace. Identifiers in the local namespace are *in scope* from the point at which you define them until the function or method terminates.

Global Namespace

Each module has a **global namespace** that associates a module's global identifiers (such as global variables, function names and class names) with objects. Python creates a module's global namespace when it loads the module. A module's global namespace exists and its identifiers are *in scope* to the code within that module until the program (or interactive session) terminates. An IPython session has its own global namespace for all the identifiers you create in that session.

Each module's global namespace also has an identifier called **`__name__`** containing the module's name, such as `'math'` for the `math` module or `'random'` for the `random` module. As you saw in the previous section's `doctest` example, `__name__` contains `'__main__'` for a `.py` file that you run as a script.

Built-In Namespace

The **built-in namespace** contains associates identifiers for Python's built-in functions (such as, `input` and `range`) and types (such as, `int`, `float` and `str`) with objects that define those functions and types. Python creates the built-in namespace when the interpreter starts executing. The built-in namespace's identifiers remain *in scope* for all code until the program (or interactive session) terminates. [9]

[9] his assumes you do not shadow the built-in functions or types by redefining their identifiers in a local or global namespace. We discussed shadowing in the Functions chapter.

Finding Identifiers in Namespaces

When you use an identifier, Python searches for that identifier in the currently accessible namespaces, proceeding from *local* to *global* to *built-in*. To help you understand the namespace search order, consider the following IPython session:

lick here to view code image

```
In [1]: z = 'global z'

In [2]: def print_variables():
   ...:     y = 'local y in   print_variables'
   ...:     print(y)
   ...:     print(z)
   ...:

In [3]: print_variables()
local y in print_variables
global z
```

The identifiers you define in an IPython session are placed in the session's *global* namespace. When snippet `[3]` calls `print_variables`, Python searches the *local*, *global* and *built-in* namespaces as follows:

- Snippet `[3]` is not in a function or method, so the session's *global* namespace and the *built-in* namespace are currently accessible. Python first searches the session's *global* namespace, which contains `print_variables`. So `print_variables` is *in scope* and Python uses the corresponding object to call `print_variables`.

- As `print_variables` begins executing, Python creates the function's *local* namespace. When function `print_variables` defines the local variable `y`, Python adds `y` to the function's *local* namespace. The variable `y` is now *in scope* until the function finishes executing.

- Next, `print_variables` calls the *built-in* function `print`, passing `y` as the argument. To execute this call, Python must resolve the identifiers `y` and `print`. The identifier `y` is defined in the *local* namespace, so it's *in scope* and Python will use the corresponding object (the string `'local y in print_variables'`) as `print`'s argument. To call the function, Python must find `print`'s corresponding object. First, it looks in the *local* namespace, which does *not* define `print`. Next, it looks in the session's *global* namespace, which does *not* define `print`. Finally, it looks in the *built-in* namespace, which *does* define `print`. So, `print` is *in scope* and Python uses the corresponding object to call `print`.

- Next, `print_variables` calls the *built-in* function `print` again with the argument `z`, which is *not* defined in the *local* namespace. So, Python looks in the *global* namespace. The argument `z` *is* defined in the *global* namespace, so `z` is *in scope* and Python will use the corresponding object (the string `'global z'`) as `print`'s argument. Again, Python

C

finds the identifier `print` in the *built-in* namespace and uses the corresponding object to call `print`.

- At this point, we reach the end of the `print_variables` function's suite, so the function terminates and its *local* namespace no longer exists, meaning the local variable `y` is now undefined.

To prove that `y` is undefined, let's try to display `y`:

lick here to view code image

```
In [4]: y
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-4-9063a9f0e032> in <module>()
----> 1 y

NameError: name 'y' is not   defined
```

In this case, there's no *local* namespace, so Python searches for `y` in the session's *global* namespace. The identifier `y` is *not* defined there, so Python searches for `y` in the *built-in* namespace. Again, Python does not find `y`. There are no more namespaces to search, so Python raises a `NameError`, indicating that `y` is not defined.

The identifiers `print_variables` and `z` still exist in the session's *global* namespace, so we can continue using them. For example, let's evaluate `z` to see its value:

```
In [5]: z
Out[5]: 'global z'
```

Nested Functions

One namespace we did not cover in the preceding discussion is the **enclosing namespace**. Python allows you to define **nested functions** inside other functions or methods. For example, if a function or method performs the same task several times, you might define a nested function to avoid repeating code in the enclosing function. When you access an identifier inside a nested function, Python searches the nested function's *local* namespace first, then the *enclosing* function's namespace, then the *global* namespace and finally the *built-in* namespace. This is sometimes referred to as the **LEGB (local, enclosing, global, built-in) rule**.

Class Namespace

A class has a namespace in which its class attributes are stored. When you access a class attribute, Python looks for that attribute first in the class's namespace, then in the base class's namespace, and so on, until either it finds the attribute or it reaches class `object`. If the attribute is not found, a `NameError` occurs.

C

Object Namespace

Each object has its own namespace containing the object's methods and data attributes. The class's `__init__` method starts with an empty object (`self`) and adds each attribute to the object's namespace. Once you define an attribute in an object's namespace, clients using the object may access the attribute's value.

10.16 INTRO TO DATA SCIENCE: TIME SERIES AND SIMPLE LINEAR REGRESSION

We've looked at sequences, such as lists, tuples and arrays. In this section, we'll discuss **time series**, which are sequences of values (called **observations**) associated with points in time. Some examples are daily closing stock prices, hourly temperature readings, the changing positions of a plane in flight, annual crop yields and quarterly company profits. Perhaps the ultimate time series is the stream of time-stamped tweets coming from Twitter users worldwide. In the "Data Mining Twitter" chapter, we'll study Twitter data in depth.

In this section, we'll use a technique called simple linear regression to make predictions from time series data. We'll use the 1895 through 2018 January average high temperatures in New York City to predict future average January high temperatures and to estimate the average January high temperatures for years preceding 1895.

In the "Machine Learning" chapter, we'll revisit this example using the scikit-learn library. In the "Deep Learning" chapter, we'll use *recurrent neural networks (RNNs)* to analyze time series.

In later chapters, we'll see that time series are popular in financial applications and with the Internet of Things (IoT), which we'll discuss in the " ig Data: Hadoop, Spark, NoSQL and oT" chapter.

In this section, we'll display graphs with Seaborn and pandas, which both use Matplotlib, so launch IPython with Matplotlib support:

```
ipython --matplotlib
```

Time Series

The data we'll use is a time series in which the observations are *ordered* by year. **Univariate time series** have *one* observation per time, such as the average of the January high temperatures in New York City for a particular year. **Multivariate time series** have *two or more* observations per time, such as temperature, humidity and barometric pressure readings in a weather application. Here, we'll analyze a univariate time series.

Two tasks often performed with time series are:

- **Time series analysis**, which looks at existing time series data for patterns, helping data analysts understand the data. A common analysis task is to look for **seasonality** in the

B

data. For example, in New York City, the monthly average high temperature varies significantly based on the seasons (winter, spring, summer or fall).

- **Time series forecasting**, which uses past data to predict the future.

We'll perform time series forecasting in this section.

Simple Linear Regression

Using a technique called **simple linear regression**, we'll make predictions by finding a linear relationship between the months (January of each year) and New York City's average January high temperatures. Given a collection of values representing an **independent variable** (the month/year combination) and a **dependent variable** (the average high temperature for that month/year), simple linear regression describes the relationship between these variables with a straight line, known as the **regression line**.

Linear Relationships

To understand the general concept of a linear relationship, consider Fahrenheit and Celsius temperatures. Given a Fahrenheit temperature, we can calculate the corresponding Celsius temperature using the following formula:

```
c = 5 / 9 * (f - 32)
```

In this formula, `f` (the Fahrenheit temperature) is the *independent variable*, and `c` (the Celsius temperature) is the *dependent variable*—each value of `c` *depends on* the value of `f` used in the calculation.

Plotting Fahrenheit temperatures and their corresponding Celsius temperatures produces a straight line. To show this, let's first create a `lambda` for the preceding formula and use it to calculate the Celsius equivalents of the Fahrenheit temperatures 0–100 in 10-degree increments. We store each Fahrenheit/Celsius pair as a tuple in `temps`:

lick here to view code image

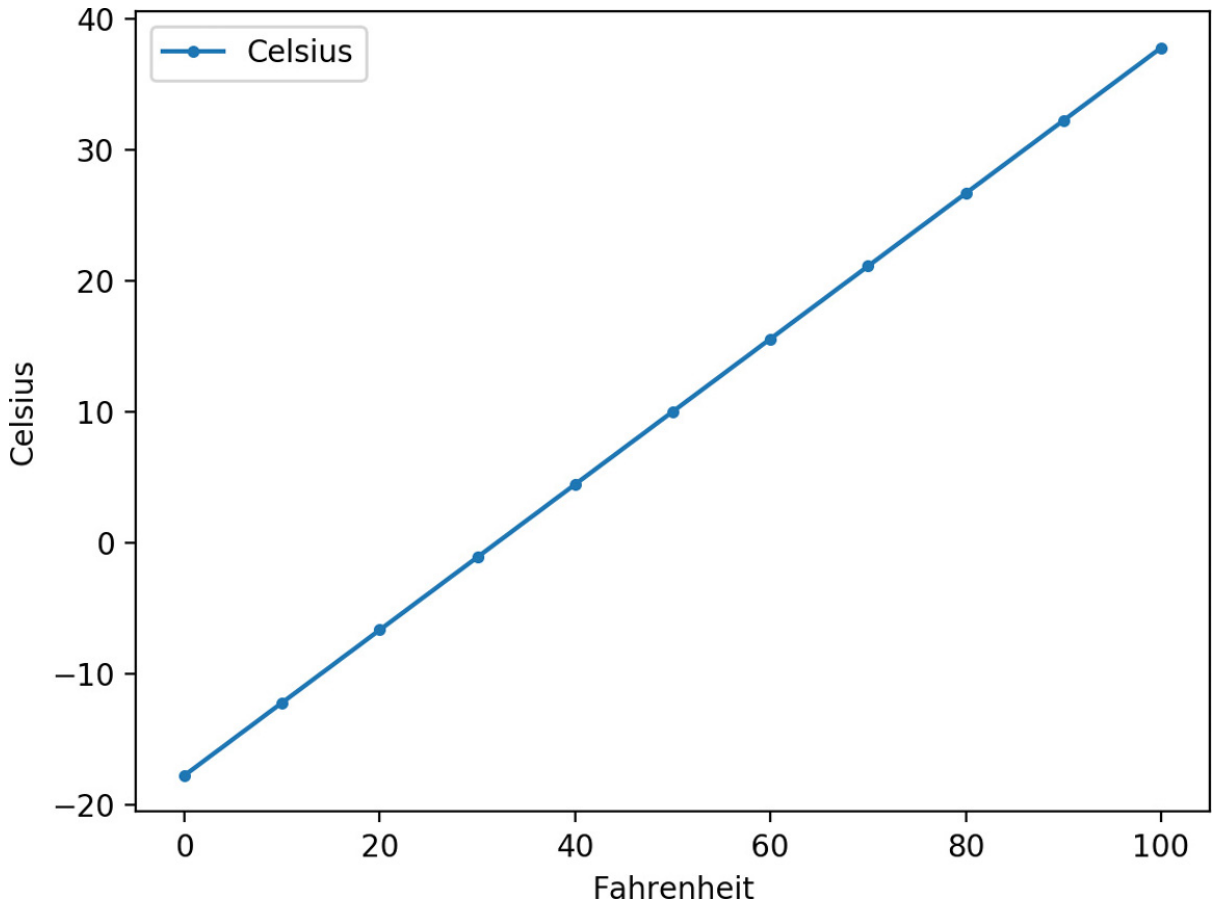
```
In [1]: c = lambda f: 5 / 9 * (f - 32)

In [2]: temps = [(f, c(f)) for f   in range(0, 101, 10)]
```

Next, let's place the data in a `DataFrame`, then use its **`plot` method** to display the linear relationship between the Fahrenheit and Celsius temperatures. The `plot` method's `style` keyword argument controls the data's appearance. The period in the string `'.-'` indicates that each point should appear as a dot, and the dash indicates that lines should connect the dots. We manually set the *y*-axis label to `'Celsius'` because the `plot` method shows `'Celsius'` only in the graph's upper-left corner legend, by default.

C

lick here to view code image

```
In [3]: import pandas as pd

In [4]: temps_df = pd.DataFrame(temps, columns=['Fahrenheit', 'Celsius'])

In [5]: axes = temps_df.plot(x='Fahrenheit',   y='Celsius', style='.-')

In [6]: y_label = axes.set_ylabel('Celsius')
```



Components of the Simple Linear Regression Equation

The points along any straight line (in two dimensions) like those shown in the preceding graph can be calculated with the equation:

y = mx + b

where

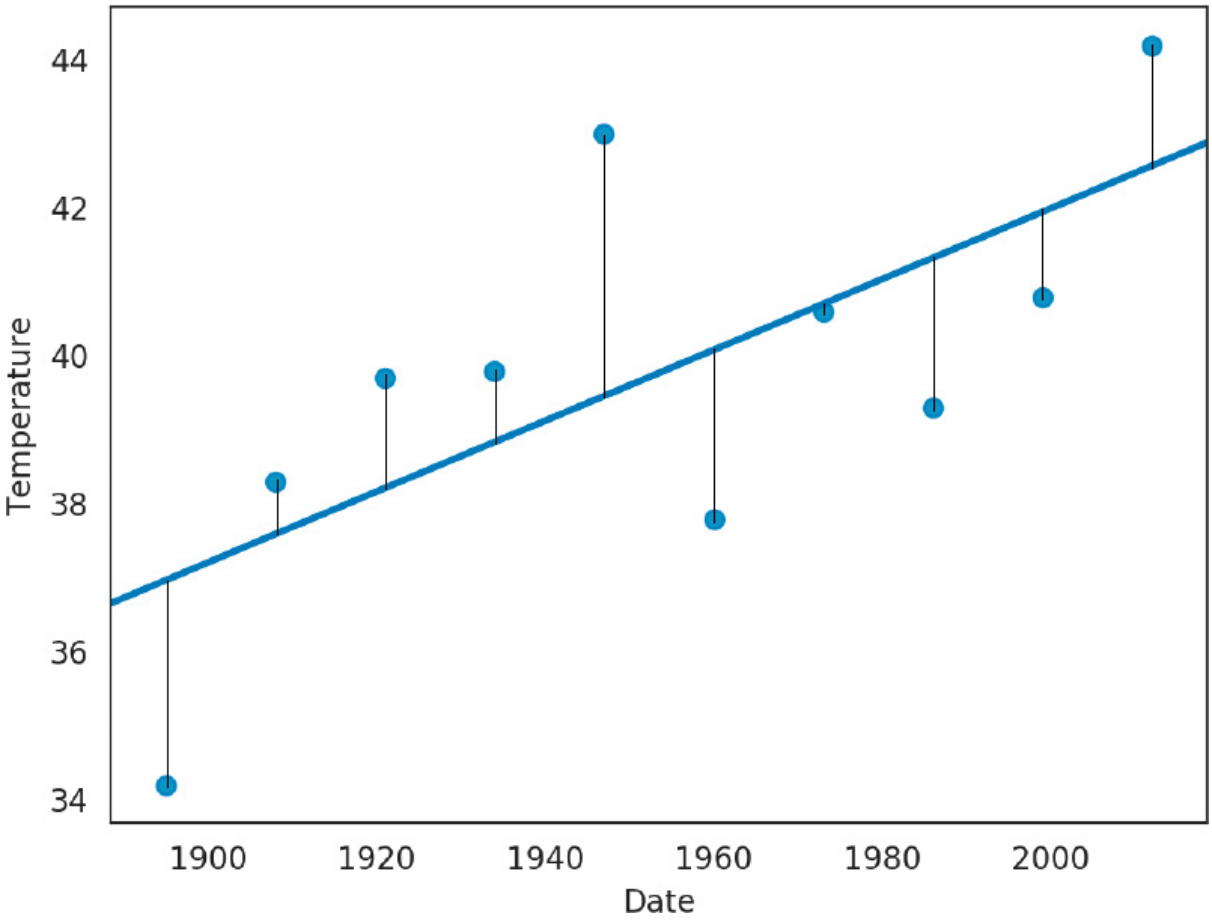
- *m* is the line's **slope**,
- *b* is the line's **intercept** with the *y*-axis (at *x* = 0),
- *x* is the independent variable (the date in this example), and
- *y* is the dependent variable (the temperature in this example).

In simple linear regression, *y* is the *predicted value* for a given *x*.

C

unction `linregress` from the SciPy's `stats` Module

Simple linear regression determines the slope (m) and intercept (b) of a straight line that best fits your data. Consider the following diagram, which shows a few of the time-series data points we'll process in this section and a corresponding regression line. We added vertical lines to indicate each data point's distance from the regression line:



The simple linear regression algorithm iteratively adjusts the slope and intercept and, for each adjustment, calculates the square of each point's distance from the line. The "best fit" occurs when the slope and intercept values minimize the sum of those squared distances. This is known as an **ordinary least squares** calculation. [o]

[o] `ttps://en.wikipedia.org/wiki/Ordinary_least_squares`.

The **SciPy (Scientific Python) library** is widely used for engineering, science and math in Python. This library's **`linregress`** function (from the **`scipy.stats` module**) performs simple linear regression for you. After calling `linregress`, you'll plug the resulting slope and intercept into the $y = mx + b$ equation to make predictions.

Pandas

In the three previous Intro to Data Science sections, you used pandas to work with data. You'll continue using pandas throughout the rest of the book. In this example, we'll load the data for New York City's 1895–2018 average January high temperatures from a CSV file into a `DataFrame`. We'll then format the data for use in this example.

Seaborn Visualization

We'll use Seaborn to plot the `DataFrame`'s data with a regression line that shows the average high-temperature trend over the period 1895–2018.

Getting Weather Data from NOAA

Let's get the data for our study. The National Oceanic and Atmospheric Administration (NOAA) [1] offers lots of public historical data including time series for average high temperatures in specific cities over various time intervals.

[1] `ttp://www.noaa.gov`.

We obtained the January average high temperatures for New York City from 1895 through 2018 from NOAA's "Climate at a Glance" time series at:

```
ttps://www.ncdc.noaa.gov/cag/
```

On that web page, you can select temperature, precipitation and other data for the entire U.S., regions within the U.S., states, cities and more. Once you've set the area and time frame, click **Plot** to display a diagram and view a table of the selected data. At the top of that table are links for downloading the data in several formats including CSV, which we discussed in the "Files and Exceptions" chapter. NOAA's maximum date range available at the time of this writing was 1895–2018. For your convenience, we provided the data in the `ch10` examples folder in the file `ave_hi_nyc_jan_1895-2018.csv`. If you download the data on your own, delete the rows above the line containing `"Date,Value,Anomaly"`.

This data contains three columns per observation:

- `Date`—A value of the form `'YYYYMM'` (such as `'201801'`). `MM` is always `01` because we downloaded data for only January of each year.
- `Value`—A floating-point Fahrenheit temperature.
- `Anomaly`—The difference between the value for the given date and average values for all dates. We do not use the `Anomaly` value in this example, so we'll ignore it.

Loading the Average High Temperatures into a `DataFrame`

Let's load and display the New York City data from `ave_hi_nyc_jan_1895-2018.csv`:

lick here to view code image

```
In [7]: nyc = pd.read_csv('ave_hi_nyc_jan_1895-2018.csv')
```

We can look at the `DataFrame`'s `head` and `tail` to get a sense of the data:

lick here to view code image

```
In [8]: nyc.head()
Out[8]:
     Date  Value  Anomaly
0  189501   34.2     -3.2
1  189601   34.7     -2.7
2  189701   35.5     -1.9
3  189801   39.6      2.2
4  189901   36.4     -1.0

In [9]: nyc.tail()
Out[9]:
       Date  Value  Anomaly
119  201401   35.5     -1.9
120  201501   36.1     -1.3
121  201601   40.8      3.4
122  201701   42.8      5.4
123  201801   38.7      1.3
```

Cleaning the Data

We'll soon use Seaborn to graph the `Date`-`Value` pairs and a regression line. When plotting data from a `DataFrame`, Seaborn labels a graph's axes using the `DataFrame`'s column names. For readability, let's rename the `'Value'` column as `'Temperature'`:

lick here to view code image

```
In [10]: nyc.columns = ['Date',   'Temperature', 'Anomaly']

In [11]: nyc.head(3)
Out[11]:
     Date  Temperature  Anomaly
0  189501         34.2     -3.2
1  189601         34.7     -2.7
2  189701         35.5     -1.9
```

Seaborn labels the tick marks on the *x*-axis with `Date` values. Since this example processes only January temperatures, the *x*-axis labels will be more readable if they do not contain `01` (for January), we'll remove it from each `Date`. First, let's check the column's type:

```
In [12]: nyc.Date.dtype
Out[12]: dtype('int64')
```

The values are integers, so we can divide by 100 to truncate the last two digits. Recall that each column in a `DataFrame` is a `Series`. Calling `Series` method `floordiv` performs *integer division* on every element of the `Series`:

lick here to view code image

```
In [13]: nyc.Date = nyc.Date.floordiv(100)

In [14]: nyc.head(3)
```

C

```
Out[14]:
   Date  Temperature  Anomaly
0  1895         34.2     -3.2
1  1896         34.7     -2.7
2  1897         35.5     -1.9
```

Calculating Basic Descriptive Statistics for the Dataset

For some quick statistics on the dataset's temperatures, call `describe` on the `Temperature` column. We can see that there are 124 observations, the mean value of the observations is 37.60, and the lowest and highest observations are 26.10 and 47.60 degrees, respectively:

lick here to view code image

```
In [15]: pd.set_option('precision',   2)

In [16]: nyc.Temperature.describe()
Out[16]:
count    124.00
mean      37.60
std        4.54
min       26.10
25%       34.58
50%       37.60
75%       40.60
max       47.60
Name: Temperature, dtype: float64
```

Forecasting Future January Average High Temperatures

The **SciPy (Scientific Python) library** is widely used for engineering, science and math in Python. Its **`stats` module** provides function **`linregress`**, which calculates a regression line's *slope* and *intercept* for a given set of data points:

lick here to view code image

```
In [17]: from scipy import stats

In [18]: linear_regression = stats.linregress(x=nyc.Date,
    ...:                                       y=nyc.Temperature)
    ...:
```

Function `linregress` receives two one-dimensional arrays [2] of the same length representing the data points' *x*- and *y*-coordinates. The keyword arguments `x` and `y` represent the independent and dependent variables, respectively. The object returned by `linregress` contains the regression line's `slope` and `intercept`:

[2]These arguments also can be one-dimensional array-like objects, such as lists or pandas `Series`.

lick here to view code image

```
In [19]: linear_regression.slope
Out[19]: 0.020147711361132966167

In [20]: linear_regression.intercept
Out[20]: 8.694845520062952
```

We can use these values with the simple linear regression equation for a straight line, $y = mx + b$, to predict the average January temperature in New York City for a given year. Let's predict the average Fahrenheit temperature for January of 2019. In the following calculation, `linear_regression.slope` is m, `2019` is x (the date value for which you'd like to predict the temperature), and `linear_regression.intercept` is b:

lick here to view code image

```
In [21]: linear_regression.slope * 2019   + linear_regression.intercept
Out[21]: 38.51837136113298
```

We also can approximate what the average temperature might have been in the years before 1895. For example, let's approximate the average temperature for January of 1890:

lick here to view code image

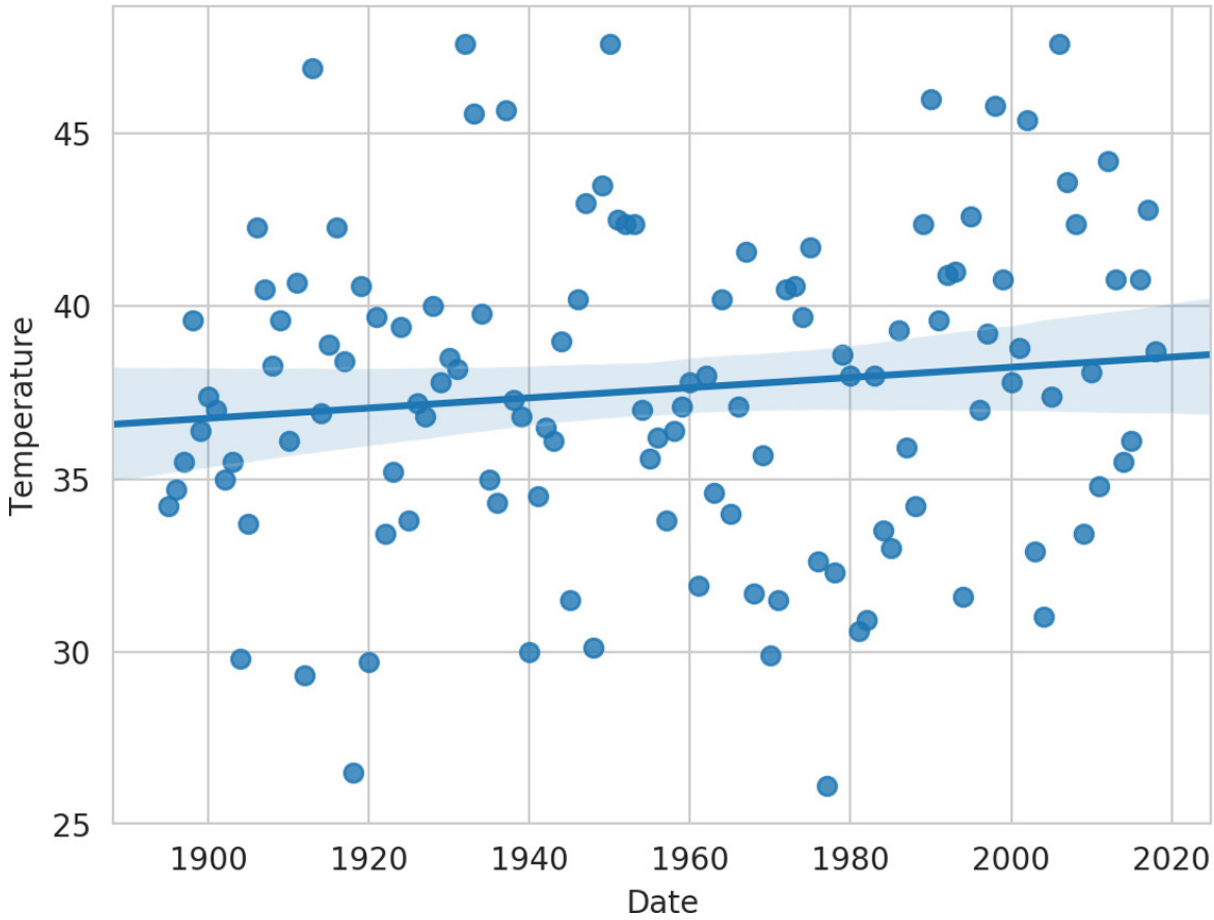
```
In [22]: linear_regression.slope * 1890   + linear_regression.intercept
Out[22]: 36.612865774980335
```

For this example, we had data for 1895–2018. You should expect that the further you go outside this range, the less reliable the predictions will be.

Plotting the Average High Temperatures and a Regression Line

Next, let's use Seaborn's **`regplot` function** to plot each data point with the dates on the x-axis and the temperatures on the y-axis. The `regplot` function creates the **scatter plot** or **scattergram** below in which the scattered dots represent the `Temperature`s for the given `Date`s, and the straight line displayed through the points is the regression line:

C



First, close the prior Matplotlib window if you have not done so already—otherwise, `regplot` will use the existing window that already contains a graph. Function `regplot`'s `x` and `y` keyword arguments are one-dimensional arrays [3] of the same length representing the *x*-*y* coordinate pairs to plot. Recall that pandas automatically creates attributes for each column name if the name can be a valid Python identifier: [4]

[3]These arguments also can be one-dimensional array-like objects, such as lists or pandas `Series`.

[4]For readers with a more statistics background, the shaded area surrounding the regression line is the 95% *confidence interval* for the regression line (`ttps://en.wikipedia.org/wiki/Simple_linear_regression#Confidence_interva` o draw the diagram without a confidence interval, add the keyword argument `ci=None` to the `regplot` functions argument list.

lick here to view code image

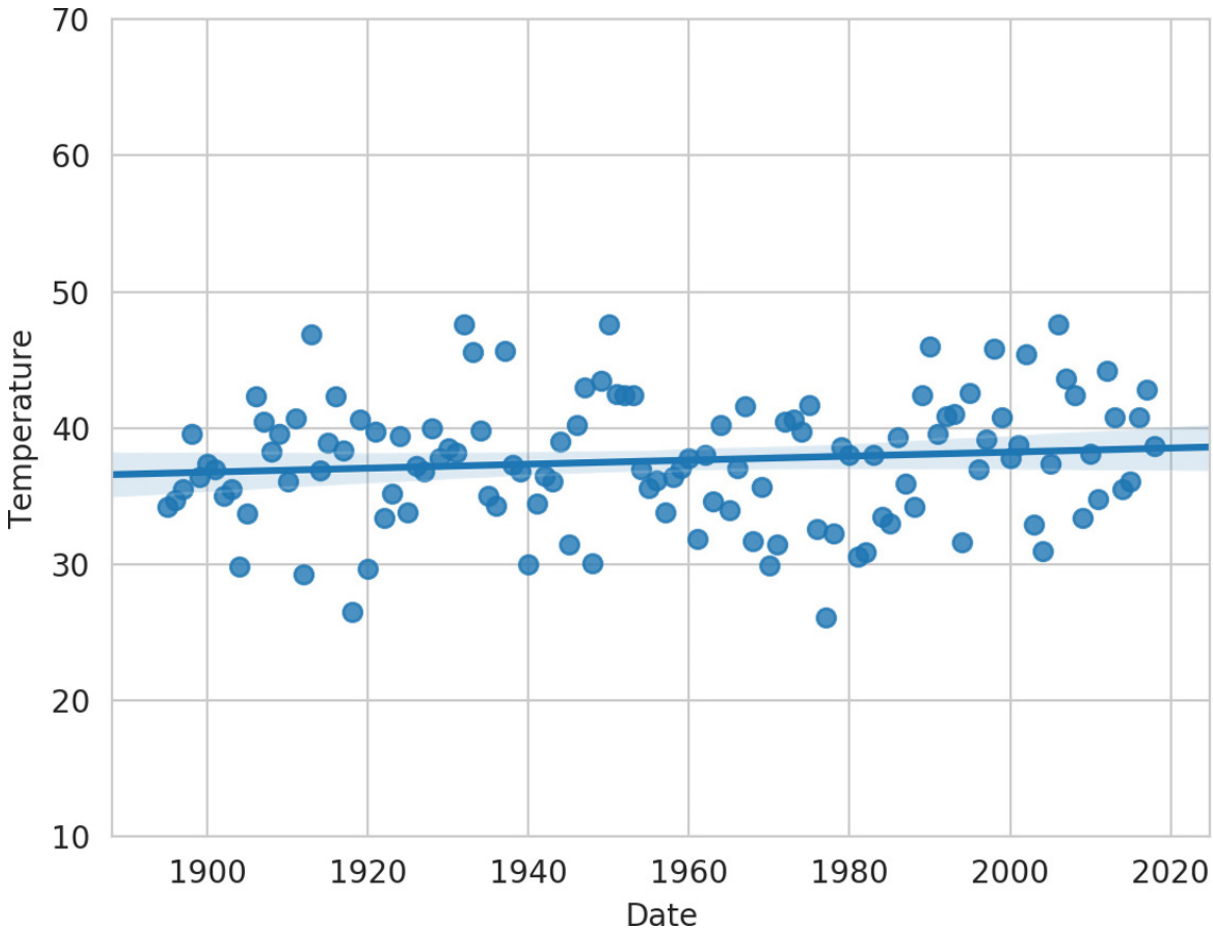
```
In [23]: import seaborn as sns

In [24]: sns.set_style('whitegrid')

In [25]: axes = sns.regplot(x=nyc.Date, y=nyc.Temperature)
```

The regression line's slope (lower at the left and higher at the right) indicates a warming trend over the last 124 years. In this graph, the *y*-axis represents a 21.5-degree temperature range between the minimum of 26.1 and the maximum of 47.6, so the data appears to be

pread significantly above and below the regression line, making it difficult to see the linear relationship. This is a common issue in data analytics visualizations. When you have axes that reflect different kinds of data (dates and temperatures in this case), how do you reasonably determine their respective scales? In the preceding graph, this is purely an issue of the graph's height—Seaborn and Matplotlib *auto-scale* the axes, based on the data's range of values. We can scale the *y*-axis range of values to emphasize the linear relationship. Here, we scaled the *y*-axis from a 21.5-degree range to a 60-degree range (from 10 to 70 degrees):

lick here to view code image

```
In [26]: axes.set_ylim(10, 70)
Out[26]: (10, 70)
```



Getting Time Series Datasets

Here are some popular sites where you can find time series to use in your studies:

Sources time-series dataset
ttps://data.gov/ This is the U.S. government's open data portal. Searching for "time series" yields over 7200 time-series datasets.

6

`ttps://www.ncdc.noaa.gov/cag/`

The National Oceanic and Atmospheric Administration (NOAA) Climate at a Glance portal provides both global and U.S. weather-related time series.

`ttps://www.esrl.noaa.gov/psd/data/timeseries/`

NOAA's Earth System Research Laboratory (ESRL) portal provides monthly and seasonal climate-related time series.

`ttps://www.quandl.com/search`

Quandl provides hundreds of free financial-related time series, as well as fee-based time series.

`ttps://datamarket.com/data/list/?q=provider:tsdl`

The Time Series Data Library (TSDL) provides links to hundreds of time series datasets across many industries.

`ttp://archive.ics.uci.edu/ml/datasets.html`

The University of California Irvine (UCI) Machine Learning Repository contains dozens of time-series datasets for a variety of topics.

`ttp://inforumweb.umd.edu/econdata/econdata.html`

The University of Maryland's EconData service provides links to thousands of economic time series from various U.S. government agencies.

10.17 WRAP-UP

In this chapter, we discussed the details of crafting valuable classes. You saw how to define a class, create objects of the class, access an object's attributes and call its methods. You defined the special method `__init__` to create and initialize a new object's data attributes.

We discussed controlling access to attributes and using properties. We showed that all object

h

ttributes may be accessed directly by a client. We discussed identifiers with single leading underscores (_), which indicate attributes that are not meant to be accessed by client code. We showed how to implement "private" attributes via the double-leading-underscore (__) naming convention, which tells Python to mangle an attribute's name.

We implemented a card shuffling and dealing simulation consisting of a `Card` class and a `DeckOfCards` class that maintained a list of `Card`s, and displayed the deck both as strings and as card images using Matplotlib. We introduced special methods `__repr__`, `__str__` and `__format__` for creating string representations of objects.

Next, we looked at Python's capabilities for creating base classes and subclasses. We showed how to create a subclass that inherits many of its capabilities from its superclass, then adds more capabilities, possibly by overriding the base class's methods. We created a list containing both base class and subclass objects to demonstrate Python's polymorphic programming capabilities.

We introduced operator overloading for defining how Python's built-in operators work with objects of custom class types. You saw that overloaded operator methods are implemented by overriding various special methods that all classes inherit from class `object`. We discussed the Python exception class hierarchy and creating custom exception classes.

We showed how to create a named tuple that enables you to access tuple elements via attribute names rather than index numbers. Next, we introduced Python 3.7's new data classes, which can autogenerate various boilerplate code commonly provided in class definitions, such as the `__init__`, `__repr__` and `__eq__` special methods.

You saw how to write unit tests for your code in docstrings, then execute those tests conveniently via the `doctest` module's `testmod` function. Finally, we discussed the various namespaces that Python uses to determine the scopes of identifiers.

In the next part of the book, we present a series of implementation case studies that use a mix of AI and big-data technologies. We explore natural language processing, data mining Twitter, IBM Watson and cognitive computing, supervised and unsupervised machine learning, and deep learning with convolutional neural networks and recurrent neural networks. We discuss big-data software and hardware infrastructure, including NoSQL databases, Hadoop and Spark with a major emphasis on performance. You're about to see some really cool stuff!

a

11. Natural Language Processing (NLP)

Objectives

In this chapter you'll:

- Perform natural language processing (NLP) tasks, which are fundamental to many of the forthcoming data science case study chapters.
- Run lots of NLP demos.
- Use the TextBlob, NLTK, Textatistic and spaCy NLP libraries and their pretrained models to perform various NLP tasks.
- Tokenize text into words and sentences.
- Use parts-of-speech tagging.
- Use sentiment analysis to determine whether text is positive, negative or neutral.
- Detect the language of text and translate between languages using TextBlob's Google Translate support.
- Get word roots via stemming and lemmatization.
- Use TextBlob's spell checking and correction capabilities.
- Get word definitions, synonyms and antonyms.
- Remove stop words from text.
- Create word clouds.
- Determine text readability with Textatistic.
- Use the spaCy library for named entity recognition and similarity detection.

Outline

11.1 INTRODUCTION

Your alarm wakes you, and you hit the "Alarm Off" button. You reach for your smartphone and read your text messages and check the latest news clips. You listen to TV hosts interviewing celebrities. You speak to family, friends and colleagues and listen to their responses. You have a hearing-impaired friend with whom you communicate via sign language and who enjoys close-captioned video programs. You have a blind colleague who reads braille, listens to books being read by a computerized book reader and listens to a screen reader speak about what's on his computer screen. You read emails, distinguishing

unk from important communications and send email. You read novels or works of non-fiction. You drive, observing road signs like “Stop,” “Speed Limit 35” and “Road Under Construction.” You give your car verbal commands, like “call home,” “play classical music” or ask questions like, “Where’s the nearest gas station?” You teach a child how to speak and read. You send a sympathy card to a friend. You read books. You read newspapers and magazines. You take notes during a class or meeting. You learn a foreign language to prepare for travel abroad. You receive a client email in Spanish and run it through a free translation program. You respond in English knowing that your client can easily translate your email back to Spanish. You are uncertain about the language of an email, but language detection software instantly figures that out for you and translates the email to English.

These are examples of **natural language** communications in text, voice, video, sign language, braille and other forms with languages like English, Spanish, French, Russian, Chinese, Japanese and hundreds more. In this chapter, you’ll master many natural language processing (NLP) capabilities through a series of hands-on demos and IPython sessions. You’ll use many of these NLP capabilities in the upcoming data science case study chapters.

Natural language processing is performed on text collections, composed of Tweets, Facebook posts, conversations, movie reviews, Shakespeare’s plays, historic documents, news items, meeting logs, and so much more. A text collection is known as a **corpus**, the plural of which is **corpora**.

Natural language lacks mathematical precision. Nuances of meaning make natural language understanding difficult. A text’s meaning can be influenced by its context and the reader’s “world view.” Search engines, for example, can get to “know you” through your prior searches. The upside is better search results. The downside could be invasion of privacy.

11.2 TEXTBLOB [1]

[1] ttps://textblob.readthedocs.io/en/latest/.

TextBlob is an object-oriented NLP text-processing library that is built on the **NLTK** and **pattern** NLP libraries and simplifies many of their capabilities. Some of the NLP tasks TextBlob can perform include:

- **Tokenization**—splitting text into pieces called **tokens**, which are meaningful units, such as words and numbers.
- **Parts-of-speech (POS) tagging**—identifying each word’s part of speech, such as noun, verb, adjective, etc.
- **Noun phrase extraction**—locating groups of words that represent nouns, such as “red brick factory.” [2]

[2] The phrase red brick factory illustrates why natural language is such a difficult subject. Is a red brick factory a factory that makes red bricks? Is it a red factory that makes bricks of any color? Is it a factory built of red bricks that makes products of any type? In todays music world, it could even be the name of a rock band or the name of a game on your smartphone.

- **Sentiment analysis**—determining whether text has positive, neutral or negative sentiment.
- **Inter-language translation** and **language detection** powered by Google Translate.
- **Inflection** [3] —pluralizing and singularizing words. There are other aspects of inflection that are not part of TextBlob.

[3] ttps://en.wikipedia.org/wiki/Inflection.

- **Spell checking** and **spelling correction**.
- **Stemming**—reducing words to their stems by removing prefixes or suffixes. For example, the stem of "varieties" is "varieti."
- **Lemmatization**—like stemming, but produces real words based on the original words' context. For example, the lemmatized form of "varieties" is "variety."
- **Word frequencies**—determining how often each word appears in a corpus.
- **WordNet integration** for finding word definitions, synonyms and antonyms.
- **Stop word elimination**—removing common words, such as a, an, the, I, we, you and more to analyze the important words in a corpus.
- **n-grams**—producing sets of consecutive words in a corpus for use in identifying words that frequently appear adjacent to one another.

Many of these capabilities are used as part of more complex NLP tasks. In this section, we'll perform these NLP tasks using TextBlob and NLTK.

Installing the TextBlob Module

To install TextBlob, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

lick here to view code image

```
conda install -c conda-forge textblob
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Once installation completes, execute the following command to download the NLTK corpora used by TextBlob:

lick here to view code image

```
ipython -m textblob.download_corpora
```

These include:

- The Brown Corpus (created at Brown University) for parts-of-speech tagging.

[4] ttps://en.wikipedia.org/wiki/Brown_Corpus.

- Punkt for English sentence tokenization.
- WordNet for word definitions, synonyms and antonyms.
- Averaged Perceptron Tagger for parts-of-speech tagging.
- conll2000 for breaking text into components, like nouns, verbs, noun phrases and more—known as **chunking** the text. The name conll2000 is from the conference that created the chunking data—Conference on Computational Natural Language Learning.
- Movie Reviews for sentiment analysis.

Project Gutenberg

A great source of text for analysis is the free e-books at Project Gutenberg:

ttps://www.gutenberg.org

The site contains over 57,000 e-books in various formats, including plain text files. These are out of copyright in the United States. For information about Project Gutenberg'sTerms of Use and copyright in other countries, see:

ttps://www.gutenberg.org/wiki/Gutenberg:Terms_of_Use

In some of this section's examples, we use the plain-text e-book file for Shakespeare's *Romeo and Juliet*, which you can find at:

ttps://www.gutenberg.org/ebooks/1513

Project Gutenberg does not allow programmatic access to its e-books. You're required to copy the books for that purpose. [5] To download *Romeo and Juliet* as a plain-text e-book, right click the **Plain Text UTF-8** link on the book's web page, then select **Save Link As...** (Chrome/FireFox), **Download Linked File As...** (Safari) or **Save target as** (Microsoft Edge) option to savethe book to your system. Save it as `RomeoAndJuliet.txt` in the `ch11` examples folder to ensure that our code examples will work correctly. For analysis purposes, we removed the Project Gutenberg text before `"THE TRAGEDY OF ROMEO AND JULIET"`, as well as the Project Guttenberg information at the end of the file starting with:

[5] ttps://www.gutenberg.org/wiki/Gutenberg:Information_About_Robot_Access_to_ou

```
End of the Project Gutenberg EBook of Romeo and Juliet, by William Shakespeare
```

11.2.1 Create a TextBlob

TextBlob is the fundamental class for NLP with the **textblob module**. Let's create a TextBlob containing two sentences:

6

ttp://textblob.readthedocs.io/en/latest/api_reference.html#textblob.blob.Tex

lick here to view code image

```
In [1]: from textblob import TextBlob

In [2]: text = 'Today is a beautiful day.   Tomorrow looks like bad weather.'

In [3]: blob = TextBlob(text)

In [4]: blob
Out[4]: TextBlob("Today is a beautiful day. Tomorrow   looks like bad weather."
```

extBlobs—and, as you'll see shortly, Sentences and Words—support string methods and can be compared with strings. They also provide methods for various NLP tasks. Sentences, Words and TextBlobs inherit from **BaseBlob**, so they have many common methods and properties.

11.2.2 Tokenizing Text into Sentences and Words

Natural language processing often requires tokenizing text before performing other NLP tasks. TextBlob provides convenient properties for accessing the sentences and words in TextBlobs. Let's use the **sentence property** to get a list of **Sentence** objects:

lick here to view code image

```
In [5]: blob.sentences
Out[5]:
[Sentence("Today is a beautiful day."),
 Sentence("Tomorrow looks like bad weather.")]
```

The **words property** returns a **WordList** object containing a list of **Word** objects, representing each word in the TextBlob with the punctuation removed:

lick here to view code image

```
In [6]: blob.words
Out[6]: WordList(['Today', 'is', 'a', 'beautiful', 'day',   'Tomorrow', 'looks'
```

1.2.3 Parts-of-Speech Tagging

Parts-of-speech (POS) tagging is the process of evaluating words based on their context to determine each word's part of speech. There are eight primary English parts of speech—nouns, pronouns, verbs, adjectives, adverbs, prepositions, conjunctions and interjections (words that express emotion and that are typically followed by punctuation, like "Yes!" or "Ha!"). Within each category there are many subcategories.

Some words have multiple meanings. For example, the words "set" and "run" have hundreds of meanings each! If you look at the `dictionary.com` definitions of the word "run," you'll see that it can be a verb, a noun, an adjective or a part of a verb phrase. An important use of POS tagging is determining a word's meaning among its possibly many meanings. This is important for helping computers "understand" natural language.

The **`tags` property** returns a list of tuples, each containing a word and a string representing its part-of-speech tag:

lick here to view code image

```
In [7]: blob
Out[7]: TextBlob("Today is a beautiful day. Tomorrow   looks like bad weather."

 n [8]: blob.tags
Out[8]:
[('Today', 'NN'),
 ('is', 'VBZ'),
 ('a', 'DT'),
 ('beautiful', 'JJ'),
 ('day', 'NN'),
 ('Tomorrow', 'NNP'),
 ('looks', 'VBZ'),
 ('like', 'IN'),
 ('bad', 'JJ'),
 ('weather', 'NN')]
```

By default, `TextBlob` uses a `PatternTagger` to determine parts-of-speech. This class uses the parts-of-speech tagging capabilities of the *pattern library*:

ttps://www.clips.uantwerpen.be/pattern

You can view the library's 63 parts-of-speech tags at

ttps://www.clips.uantwerpen.be/pages/MBSP-tags

In the preceding snippet's output:

- `Today`, `day` and `weather` are tagged as `NN`—a singular noun or mass noun.
- `is` and `looks` are tagged as `VBZ`—a third person singular present verb.
- `a` is tagged as `DT`—a determiner. [7]

  [7] ttps://en.wikipedia.org/wiki/Determiner.

- `beautiful` and `bad` are tagged as `JJ`—an adjective.
- `Tomorrow` is tagged as `NNP`—a proper singular noun.
- `like` is tagged as `IN`—a subordinating conjunction or preposition.

11.2.4 Extracting Noun Phrases

Let's say you're preparing to purchase a water ski so you're researching them online. You might search for "best water ski." In this case, "water ski" is a noun phrase. If the search engine does not parse the noun phrase properly, you probably will not get the best search results. Go online and try searching for "best water," "best ski" and "best water ski" and see what you get.

A `TextBlob`'s **`noun_phrases` property** returns a `WordList` object containing a list of `Word` objects—one for each noun phrase in the text:

lick here to view code image

```
In [9]: blob
Out[9]: TextBlob("Today is a beautiful day. Tomorrow   looks like bad weather."

 n [10]: blob.noun_phrases
Out[10]: WordList(['beautiful day', 'tomorrow', 'bad   weather'])
```

Note that a `Word` representing a noun phrase can contain multiple words. A `WordList` is an extension of Python's built-in list type. `WordList`s provide additional methods for stemming, lemmatizing, singularizing and pluralizing.

11.2.5 Sentiment Analysis with TextBlob's Default Sentiment Analyzer

One of the most common and valuable NLP tasks is **sentiment analysis**, which determines whether text is positive, neutral or negative. For instance, companies might use this to determine whether people are speaking positively or negatively online about their products. Consider the positive word "good" and the negative word "bad." Just because a sentence contains "good" or "bad" does not mean the sentence's sentiment necessarily is positive or negative. For example, the sentence

```
The food is not good.
```

clearly has negative sentiment. Similarly, the sentence

```
The movie was not bad.
```

clearly has positive sentiment, though perhaps not as positive as something like

```
The movie was excellent!
```

Sentiment analysis is a complex machine-learning problem. However, libraries like TextBlob have pretrained machine learning models for performing sentiment analysis.

Getting the Sentiment of a TextBlob

A `TextBlob`'s **`sentiment` property** returns a **`Sentiment`** object indicating whether the text is positive or negative and whether it's objective or subjective:

lick here to view code image

```
 n [11]: blob
Out[11]: TextBlob("Today is a beautiful day. Tomorrow   looks like bad weather.'
```

C

```
In [12]: blob.sentiment
Out[12]: Sentiment(polarity=0.07500000000000007,
   subjectivity=0.8333333333333333)
```

n the preceding output, the `polarity` indicates sentiment with a value from `-1.0` (negative) to `1.0` (positive) with `0.0` being neutral. The `subjectivity` is a value from 0.0 (objective) to 1.0 (subjective). Based on the values for our `TextBlob`, the overall sentiment is close to neutral, and the text is mostly subjective.

Getting the `polarity` and `subjectivity` from the Sentiment `Object`

The values displayed above probably provide more precision that you need in most cases. This can detract from numeric output's readability. The IPython magic **`%precision`** allows you to specify the default precision for *standalone* `float` objects and `float` objects in *built-in types* like lists, dictionaries and tuples. Let's use the magic to *round* the `polarity` and `subjectivity` values to three digits to the right of the decimal point:

lick here to view code image

```
In [13]: %precision 3
Out[13]: '%.3f'

In [14]: blob.sentiment.polarity
Out[14]: 0.075

In [15]: blob.sentiment.subjectivity
Out[15]: 0.833
```

Getting the Sentiment of a `Sentence`

You also can get the sentiment at the individual sentence level. Let's use the `sentence` property to get a list of **`Sentence`** objects, then iterate through them and display each `Sentence`'s `sentiment` property:

8

`ttp://textblob.readthedocs.io/en/latest/api_reference.html#textblob.blob.Sen`

lick here to view code image

```
In [16]: for sentence in blob.sentences:
    ...:     print(sentence.sentiment)
    ...:
Sentiment(polarity=0.85, subjectivity=1.0)
Sentiment(polarity=-0.6999999999999998,   subjectivity=0.6666666666666666)
```

This might explain why the entire `TextBlob`'s `sentiment` is close to `0.0` (neutral)—one sentence is positive (`0.85`) and the other negative (`-0.6999999999999998`).

11.2.6 Sentiment Analysis with the `NaiveBayesAnalyzer`

By default, a `TextBlob` and the `Sentence`s and `Word`s you get from it determine sentiment

using a `PatternAnalyzer`, which uses the same sentiment analysis techniques as in the Pattern library. The TextBlob library also comes with a **`NaiveBayesAnalyzer`** [9] (module **`text-blob.sentiments`**), which was trained on a database of movie reviews. Naive Bayes [o] is a commonly used machine learning text-classification algorithm. The following uses the `analyzer` keyword argument to specify a `TextBlob`'s sentiment analyzer. Recall from earlier in this ongoing IPython session that `text` contains `'Today is a beautiful day. Tomorrow looks like bad weather.'`:

9

ttps://textblob.readthedocs.io/en/latest/api_reference.html#module-extblob-.en.sentiments.

o ttps://en.wikipedia.org/wiki/Naive_Bayes_classifier.

lick here to view code image

```
In [17]: from   textblob.sentiments import NaiveBayesAnalyzer

In [18]: blob = TextBlob(text,   analyzer=NaiveBayesAnalyzer())

In [19]: blob
Out[19]: TextBlob("Today is a beautiful day. Tomorrow   looks like bad weather.
```

et's use the `TextBlob`'s `sentiment` property to display the text's sentiment using the `NaiveBayesAnalyzer`:

lick here to view code image

```
In [20]: blob.sentiment
Out[20]: Sentiment(classification='neg',   p_pos=0.47662917962091056, p_neg=0.5
```

n this case, the overall sentiment is classified as negative (`classification='neg'`). The `Sentiment` object's `p_pos` indicates that the `TextBlob` is 47.66% positive, and its `p_neg` indicates that the `TextBlob` is 52.34% negative. Since the overall sentiment is just slightly more negative we'd probably view this `TextBlob`'s sentiment as neutral overall.

Now, let's get the sentiment of each `Sentence`:

lick here to view code image

```
In [21]: for sentence in blob.sentences:
    ...:     print(sentence.sentiment)
    ...:
Sentiment(classification='pos', p_pos=0.8117563121751951,   p_neg=0.18824368782
 entiment(classification='neg', p_pos=0.174363226578349,   p_neg=0.825636773421
```

otice that rather than `polarity` and `subjectivity`, the Sentiment objects we get from the `NaiveBayesAnalyzer` contain a *classification*—`'pos'` (positive) or `'neg'` (negative)—and `p_pos` (percentage positive) and `p_neg` (percentage negative) values from `0.0` to `1.0`.

Once again, we see that the first sentence is positive and the second is negative.

11.2.7 Language Detection and Translation

Inter-language translation is a challenging problem in natural language processing and artificial intelligence. With advances in machine learning, artificial intelligence and natural language processing, services like Google Translate (100+ languages) and Microsoft Bing Translator (60+ languages) can translate between languages instantly.

Inter-language translation also is great for people traveling to foreign countries. They can use translation apps to translate menus, road signs and more. There are even efforts at live speech translation so that you'll be able to converse in real time with people who do not know your natural language.[1, 2] Some smartphones, can now work together with in ear headphones to provide near-live translation of many languages.[3, 4, 5] In the "IBM Watson and Cognitive Computing" chapter, we develop a script that does near real-time inter-language translation among languages supported by Watson.

[1] ttps://www.skype.com/en/features/skype-translator/.

[2] ttps://www.microsoft.com/en-us/translator/business/live/.

[3] ttps://www.telegraph.co.uk/technology/2017/10/04/googles-new-eadphones-can-translate-foreign-languages-real/.

[4] ttps://store.google.com/us/product/google_pixel_buds?hl=en-US.

[5] ttp://www.chicagotribune.com/bluesky/originals/ct-bsi-google-ixel-buds-review-20171115-story.html.

The TextBlob library uses Google Translate to detect a text's language and translate `TextBlob`s, `Sentence`s and `Word`s into other languages.[6] Let's use **`detect_language` method** to detect the language of the text we're manipulating (`'en'` is English):

[6] These features require an Internet connection.

lick here to view code image

```
In [22]: blob
Out[22]: TextBlob("Today is a beautiful day. Tomorrow   looks like bad weather.

 n [23]: blob.detect_language()
Out[23]: 'en'
```

Next, let's use the **`translate` method** to translate the text to Spanish (`'es'`) then detect the language on the result. The `to` keyword argument specifies the target language.

lick here to view code image

```
In [24]: spanish = blob.translate(to='es')

In [25]: spanish
Out[25]: TextBlob("Hoy es un hermoso dia. Mañana   parece mal tiempo.")
```

```
In [26]: spanish.detect_language()
Out[26]: 'es'
```

Next, let's translate our `TextBlob` to simplified Chinese (specified as `'zh'` or `'zh-CN'`) then detect the language on the result:

lick here to view code image

```
In [27]: chinese = blob.translate(to='zh')

In [28]: chinese
Out[28]: TextBlob("")

In [29]: chinese.detect_language()
Out[29]: 'zh-CN'
```

Method `detect_language`'s output always shows simplified Chinese as `'zh-CN'`, even though the `translate` function can receive simplified Chinese as `'zh'` or `'zh-CN'`.

In each of the preceding cases, Google Translate automatically detects the source language. You can specify a source language explicitly by passing the `from_lang` keyword argument to the `translate` method, as in

lick here to view code image

```
chinese = blob.translate(from_lang='en', to='zh')
```

Google Translate uses iso-639-1 [7] language codes listed at

[7] SO is the International Organization for Standardization (`ttps://www.iso.org/`).

ttps://en.wikipedia.org/wiki/List_of_ISO_639-1_codes

For the supported languages, you'd use these codes as the values of the `from_lang` and `to` keyword arguments. Google Translate's list of supported languages is at:

ttps://cloud.google.com/translate/docs/languages

Calling `translate` without arguments translates from the detected source language to English:

lick here to view code image

```
In [30]: spanish.translate()
Out[30]: TextBlob("Today is a beautiful day. Tomorrow   seems like bad weather.

 n [31]: chinese.translate()
Out[31]: TextBlob("Today is a beautiful day. Tomorrow   looks like bad weather.
```

ote the slight difference in the English results.

11.2.8 Inflection: Pluralization and Singularization

Inflections are different forms of the same words, such as singular and plural (like "person" and "people") and different verb tenses (like "run" and "ran"). When you're calculating word frequencies, you might first want to convert all inflected words to the same form for more accurate word frequencies. `Word`s and `WordList`s each support converting words to their singular or plural forms. Let's pluralize and singularize a couple of `Word` objects:

lick here to view code image

```
In [1]: from textblob import Word

In [2]: index = Word('index')

In [3]: index.pluralize()
Out[3]: 'indices'

In [4]: cacti = Word('cacti')

In [5]: cacti.singularize()
Out[5]: 'cactus'
```

Pluralizing and singularizing are sophisticated tasks which, as you can see above, are not as simple as adding or removing an "s" or "es" at the end of a word.

You can do the same with a `WordList`:

lick here to view code image

```
In [6]: from textblob import TextBlob

In [7]: animals = TextBlob('dog cat   fish bird').words

In [8]: animals.pluralize()
Out[8]: WordList(['dogs', 'cats', 'fish', 'birds'])
```

Note that the word "fish" is the same in both its singular and plural forms.

11.2.9 Spell Checking and Correction

For natural language processing tasks, it's important that the text be free of spelling errors. Software packages for writing and editing text, like Microsoft Word, Google Docs and others automatically check your spelling as you type and typically display a red line under misspelled words. Other tools enable you to manually invoke a spelling checker.

You can check a `Word`'s spelling with its **`spellcheck` method**, which returns a list of tuples containing possible correct spellings and a confidence value. Let's assume we meant to type the word "they" but we misspelled it as "theyr." The spell checking results show two possible corrections with the word `'they'` having the highest confidence value:

lick here to view code image

```
In [1]: from textblob import Word

In [2]: word = Word('theyr')
```

C

```
In [3]: %precision 2
Out[3]: '%.2f'

In [4]: word.spellcheck()
Out[4]: [('they', 0.57), ('their', 0.43)]
```

Note that the word with the highest confidence value might *not* be the correct word for the given context.

`TextBlob`s, `Sentence`s and `Word`s all have a **`correct` method** that you can call to correct spelling. Calling `correct` on a `Word` returns the correctly spelled word that has the highest confidence value (as returned by spellcheck):

lick here to view code image

```
In [5]: word.correct()  # chooses   word with the highest confidence value
Out[5]: 'they'
```

Calling `correct` on a `TextBlob` or `Sentence` checks the spelling of each word. For each incorrect word, `correct` replaces it with the correctly spelled one that has the highest confidence value:

lick here to view code image

```
In [6]: from textblob import Word

In [7]: sentence = TextBlob('Ths   sentense has missplled wrds.')

In [8]: sentence.correct()
Out[8]: TextBlob("The sentence has misspelled   words.")
```

11.2.10 Normalization: Stemming and Lemmatization

Stemming removes a prefix or suffix from a word leaving only a stem, which may or may not be a real word. **Lemmatization** is similar, but factors in the word's part of speech and meaning and results in a real word.

Stemming and lemmatization are **normalization** operations, in which you prepare words for analysis. For example, before calculating statistics on words in a body of text, you might convert all words to lowercase so that capitalized and lowercase words are not treated differently. Sometimes, you might want to use a word's root to represent the word's many forms. For example, in a given application, you might want to treat all of the following words as "program": program, programs, programmer, programming and programmed (and perhaps U.K. English spellings, like programmes as well).

`Word`s and `WordList`s each support stemming and lemmatization via the methods **`stem`** and **`lemmatize`**. Let's use both on a `Word`:

lick here to view code image

```
In [1]: from textblob import Word
```

C

```
In [2]: word = Word('varieties')

In [3]: word.stem()
Out[3]: 'varieti'

In [4]: word.lemmatize()
Out[4]: 'variety'
```

11.2.11 Word Frequencies

Various techniques for detecting similarity between documents rely on word frequencies. As you'll see here, `TextBlob` automatically counts word frequencies. First, let's load the e-book for Shakespeare's *Romeo and Juliet* into a `TextBlob`. To do so, we'll use the **`Path` class** from the Python Standard Library's **`pathlib` module**:

lick here to view code image

```
In [1]: from pathlib import Path

In [2]: from textblob import TextBlob

In [3]: blob = TextBlob(Path('RomeoAndJuliet.txt').read_text())
```

Use the file `RomeoAndJuliet.txt`[8] that you downloaded earlier. We assume here that you started your IPython session from that folder. When you read a file with `Path`'s **`read_text` method**, it closes the file immediately after it finishes reading the file.

[8] Each Project Gutenberg e-book includes additional text, such as their licensing information, thats not part of the e-book itself. For this example, we used a text editor to remove that text from our copy of the e-book.

You can access the word frequencies through the `TextBlob`'s **`word_counts` dictionary**. Let's get the counts of several words in the play:

lick here to view code image

```
In [4]: blob.word_counts['juliet']
Out[4]: 190

In [5]: blob.word_counts['romeo']
Out[5]: 315

In [6]: blob.word_counts['thou']
Out[6]: 278
```

If you already have tokenized a `TextBlob` into a `WordList`, you can count specific words in the list via the **`count` method**:

lick here to view code image

```
In [7]: blob.words.count('joy')
Out[7]: 14

In [8]: blob.noun_phrases.count('lady   capulet')
```

C

```
Out[8]: 46
```

11.2.12 Getting Definitions, Synonyms and Antonyms from WordNet

WordNet [9] is a word database created by Princeton University. The TextBlob library uses the NLTK library's WordNet interface, enabling you to look up word definitions, and get synonyms and antonyms. For more information, check out the NLTK WordNet interface documentation at:

[9] ttps://wordnet.princeton.edu/.

ttps://www.nltk.org/api/nltk.corpus.reader.html#module-nltk.corpus.reader.wordnet

Getting Definitions

First, let's create a `Word`:

lick here to view code image

```
In [1]: from textblob import Word

In [2]: happy = Word('happy')
```

The `Word` class's **`definitions` property** returns a list of all the word's definitions in the WordNet database:

lick here to view code image

```
In [3]: happy.definitions
Out[3]:
['enjoying or showing or marked by joy or pleasure',
 'marked by good fortune',
 'eagerly disposed to act or to be of service',
 'well expressed and to the point']
```

The database does not necessarily contain every dictionary definition of a given word. There's also a **`define` method** that enables you to pass a part of speech as an argument so you can get definitions matching only that part of speech.

Getting Synonyms

You can get a `Word`'s **synsets**—that is, its sets of synonyms—via the **`synsets` property**. The result is a list of `Synset` objects:

```
In [4]: happy.synsets
Out[4]:
[Synset('happy.a.01'),
 Synset('felicitous.s.02'),
 Synset('glad.s.02'),
 Synset('happy.s.04')]
```

Each `Synset` represents a group of synonyms. In the notation `happy.a.01`:

- `happy` is the original Word's lemmatized form (in this case, it's the same).
- `a` is the part of speech, which can be `a` for adjective, `n` for noun, `v` for verb, `r` for adverb or `s` for adjective satellite. Many adjective synsets in WordNet have satellite synsets that represent similar adjectives.
- `01` is a 0-based index number. Many words have multiple meanings, and this is the index number of the corresponding meaning in the WordNet database.

There's also a **`get_synsets` method** that enables you to pass a part of speech as an argument so you can get `Synset`s matching only that part of speech.

You can iterate through the `synsets` list to find the original word's synonyms. Each `Synset` has a **`lemmas` method** that returns a list of `Lemma` objects representing the synonyms. A `Lemma`'s `name` method returns the synonymous word as a string. In the following code, for each `Synset` in the `synsets` list, the nested `for` loop iterates through that `Synset`'s `Lemmas` (if any). Then we add the synonym to the set named `synonyms`. We used a set collection because it automatically eliminates any duplicates we add to it:

lick here to view code image

```
In [5]: synonyms = set()

In [6]: for synset in happy.synsets:
   ...:     for lemma in synset.lemmas():
   ...:         synonyms.add(lemma.name())
   ...:

In [7]: synonyms
Out[7]: {'felicitous', 'glad', 'happy', 'well-chosen'}
```

Getting Antonyms

If the word represented by a `Lemma` has antonyms in the WordNet database, invoking the `Lemma`'s `antonyms` method returns a list of `Lemma`s representing the antonyms (or an empty list if there are no antonyms in the database). In snippet `[4]` you saw there were four `Synset`s for `'happy'`. First, let's get the `Lemma`s for the `Synset` at index `0` of the `synsets` list:

lick here to view code image

```
In [8]: lemmas = happy.synsets[0].lemmas()

In [9]: lemmas
Out[9]: [Lemma('happy.a.01.happy')]
```

In this case, `lemmas` returned a list of one `Lemma` element. We can now check whether the database has any corresponding antonyms for that `Lemma`:

lick here to view code image

```
In [10]: lemmas[0].antonyms()
```

C

```
Out[10]: [Lemma('unhappy.a.01.unhappy')]
```

The result is list of `Lemma`s representing the antonym(s). Here, we see that the one antonym for `'happy'` in the database is `'unhappy'`.

11.2.13 Deleting Stop Words

Stop words are common words in text that are often removed from text before analyzing it because they typically do not provide useful information. The following table shows NLTK's list of English stop words, which is returned by the NLTK `stopwords` module's `words` function [9] (which we'll use momentarily):

[9] `ttps://www.nltk.org/book/ch02.html`.

NLTK's English stop words list

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain',
'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't",
'as', 'at', 'be', 'because', 'been', 'before', 'being',
'below', 'between', 'both', 'but', 'by', 'can', 'couldn',
"couldn't", 'd', 'did', 'didn', "didn't", 'do', 'does',
'doesn', "doesn't", 'doing', 'don', "don't", 'down', 'during',
'each', 'few', 'for', 'from', 'further', 'had', 'hadn',
"hadn't", 'has', 'hasn', "hasn't", 'have', 'haven', "haven't",
'having', 'he', 'her', 'here', 'hers', 'herself', 'him',
'himself', 'his', 'how', 'i', 'if', 'in', 'into', 'is', 'isn',
"isn't", 'it', "it's", 'its', 'itself', 'just', 'll', 'm',
'ma', 'me', 'mightn', "mightn't", 'more', 'most', 'mustn',
"mustn't", 'my', 'myself', 'needn', "needn't", 'no', 'nor',
'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or',
'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own',
're', 's', 'same', 'shan', "shan't", 'she', "she's", 'should',
"should've", 'shouldn', "shouldn't", 'so', 'some', 'such',
't', 'than', 'that', "that'll", 'the', 'their', 'theirs',
'them', 'themselves', 'then', 'there', 'these', 'they',
'this', 'those', 'through', 'to', 'too', 'under', 'until',
'up', 've', 'very', 'was', 'wasn', "wasn't", 'we', 'were',
'weren', "weren't", 'what', 'when', 'where', 'which', 'while',
'who', 'whom', 'why', 'will', 'with', 'won', "won't",
'wouldn', "wouldn't", 'y', 'you', "you'd", "you'll", "you're",
"you've", 'your', 'yours', 'yourself', 'yourselves']
```

The NLTK library has lists of stop words for several other natural languages as well. Before using NLTK's stop-words lists, you must download them, which you do with the `nltk`

module's **`download` function**:

lick here to view code image

```
In [1]: import nltk

In [2]: nltk.download('stopwords')
[nltk_data] Downloading package stopwords to
[nltk_data]       C:\Users\PaulDeitel\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
Out[2]: True
```

For this example, we'll load the `'english'` stop words list. First import `stopwords` from the `nltk.corpus` module, then use `stopwords` method `words` to load the `'english'` stop words list:

lick here to view code image

```
In [3]: from nltk.corpus import stopwords

In [4]: stops =   stopwords.words('english')
```

Next, let's create a `TextBlob` from which we'll remove stop words:

lick here to view code image

```
In [5]: from textblob import TextBlob
In [6]: blob = TextBlob('Today is a   beautiful day.')
```

Finally, to remove the stop words, let's use the `TextBlob`'s words in a list comprehension that adds each word to the resulting list only if the word is not in `stops`:

lick here to view code image

```
In [7]: [word for word in blob.words if word not in stops]
Out[7]: ['Today', 'beautiful', 'day']
```

11.2.14 n-grams

An **n-gram**[1] is a sequence of *n* text items, such as letters in words or words in a sentence. In natural language processing, *n*-grams can be used to identify letters or words that frequently appear adjacent to one another. For text-based user input, this can help predict the next letter or word a user will type—such as when completing items in IPython with tab-completion or when entering a message to a friend in your favorite smartphone messaging app. For speech-to-text, *n*-grams might be used to improve the quality of the transcription. N-grams are a form of **co-occurrence** in which words or letters appear near each other in a body of text.

[1] `ttps://en.wikipedia.org/wiki/N-gram`.

`TextBlob`'s **`ngrams` method** produces a list of `WordList` *n*-grams of length three by

default—known as *trigrams*. You can pass the keyword argument `n` to produce *n*-grams of any desired length. The output shows that the first trigram contains the first three words in the sentence (`'Today'`, `'is'` and `'a'`). Then, `ngrams` creates a trigram starting with the second word (`'is'`, `'a'` and `'beautiful'`) and so on until it creates a trigram containing the last three words in the `TextBlob`:

lick here to view code image

```
In [1]: from textblob import TextBlob

In [2]: text = 'Today is a beautiful   day. Tomorrow looks like bad weather.'

In [3]: blob = TextBlob(text)

In [4]: blob.ngrams()
Out[4]:
[WordList(['Today', 'is', 'a']),
 WordList(['is', 'a', 'beautiful']),
 WordList(['a', 'beautiful', 'day']),
 WordList(['beautiful', 'day', 'Tomorrow']),
 WordList(['day', 'Tomorrow', 'looks']),
 WordList(['Tomorrow', 'looks', 'like']),
 WordList(['looks', 'like', 'bad']),
 WordList(['like', 'bad', 'weather'])]
```

The following produces *n*-grams consisting of five words:

lick here to view code image

```
In [5]: blob.ngrams(n=5)
Out[5]:
[WordList(['Today', 'is', 'a', 'beautiful', 'day']),
 WordList(['is', 'a', 'beautiful', 'day', 'Tomorrow']),
 WordList(['a', 'beautiful', 'day', 'Tomorrow', 'looks']),
 WordList(['beautiful', 'day', 'Tomorrow', 'looks',   'like']),
 WordList(['day', 'Tomorrow', 'looks', 'like', 'bad']),
 WordList(['Tomorrow', 'looks', 'like', 'bad', 'weather'])]
```

11.3 VISUALIZING WORD FREQUENCIES WITH BAR CHARTS AND WORD CLOUDS

Earlier, we obtained frequencies for a few words in *Romeo and Juliet*. Sometimes frequency visualizations enhance your corpus analyses. There's often more than one way to visualize data, and sometimes one is superior to others. For example, you might be interested in word frequencies relative to one another, or you may just be interested in relative uses of words in a corpus. In this section, we'll look at two ways to visualize word frequencies:

- A bar chart that *quantitatively* visualizes the top 20 words in *Romeo and Juliet* as bars representing each word and its frequency.
- A **word cloud** that *qualitatively* visualizes more frequently occurring words in bigger fonts and less frequently occurring words in smaller fonts.

11.3.1 Visualizing Word Frequencies with Pandas

C

Let's visualize *Romeo and Juliet*'s top 20 words that are *not* stop words. To do this, we'll use features from TextBlob, NLTK and pandas. Pandas visualization capabilities are based on Matplotlib, so launch IPython with the following command for this session:

```
ipython --matplotlib
```

Loading the Data

First, let's load *Romeo and Juliet*. Launch IPython from the `ch11` examples folder before executing the following code so you can access the e-book file `RomeoAndJuliet.txt` that you downloaded earlier in the chapter:

lick here to view code image

```
In [1]: from pathlib import Path

In [2]: from textblob import TextBlob

In [3]: blob = TextBlob(Path('RomeoAndJuliet.txt').read_text())
```

Next, load the NLTK stopwords:

lick here to view code image

```
In [4]: from nltk.corpus import stopwords

In [5]: stop_words = stopwords.words('english')
```

Getting the Word Frequencies

To visualize the top 20 words, we need each word and its frequency. Let's call the `blob.word_counts` dictionary's `items` method to get a list of word-frequency tuples:

lick here to view code image

```
In [6]: items = blob.word_counts.items()
```

Eliminating the Stop Words

Next, let's use a list comprehension to eliminate any tuples containing stop words:

lick here to view code image

```
In [7]: items = [item for item in items if item[0] not in stop_words]
```

The expression `item[0]` gets the word from each tuple so we can check whether it's in `stop_words`.

Sorting the Words by Frequency

To determine the top 20 words, let's sort the tuples in `items` in descending order by

C

frequency. We can use built-in function `sorted` with a `key` argument to sort the tuples by the frequency element in each tuple. To specify the tuple element to sort by, use the **`itemgetter` function** from the Python Standard Library's **`operator` module**:

lick here to view code image

```
In [8]: from operator import itemgetter

In [9]: sorted_items = sorted(items, key=itemgetter(1), reverse=True)
```

As `sorted` orders `items`' elements, it accesses the element at index `1` in each tuple via the expression `itemgetter(1)`. The `reverse=True` keyword argument indicates that the tuples should be sorted in *descending* order.

Getting the Top 20 Words

Next, we use a slice to get the top 20 words from `sorted_items`. When `TextBlob` tokenizes a corpus, it splits all contractions at their apostrophes and counts the total number of apostrophes as one of the "words." *Romeo and Juliet* has many contractions. If you display `sorted_items[0]`, you'll see that they are the most frequently occurring "word" with `867` of them.[2] We want to display only words, so we ignore element `0` and get a slice containing elements `1` through `20` of `sorted_items`:

[2] In some locales this does not happen and element 0 is indeed `'romeo'`.

lick here to view code image

```
In [10]: top20 = sorted_items[1:21]
```

Convert `top20` to a `DataFrame`

Next, let's convert the `top20` list of tuples to a pandas `DataFrame` so we can visualize it conveniently:

lick here to view code image

```
In [11]: import pandas as pd

In [12]: df = pd.DataFrame(top20, columns=['word', 'count'])

In [13]: df
Out[13]:
        word  count
0      romeo    315
1       thou    278
2     juliet    190
3        thy    170
4    capulet    163
5      nurse    149
6       love    148
7       thee    138
8       lady    117
9      shall    110
10     friar    105
11      come     94
```

```
12  mercutio     88
13  lawrence     82
14      good     80
15  benvolio     79
16    tybalt     79
17     enter     75
18        go     75
19     night     73
```

Visualizing the `DataFrame`

To visualize the data, we'll use the **`bar` method** of the `DataFrame`'s **`plot` property**. The arguments indicate which column's data should be displayed along the *x*- and *y*-axes, and that we do not want to display a legend on the graph:

lick here to view code image

```
In [14]: axes = df.plot.bar(x='word',   y='count', legend=False)
```

The `bar` method creates and displays a Matplotlib bar chart.

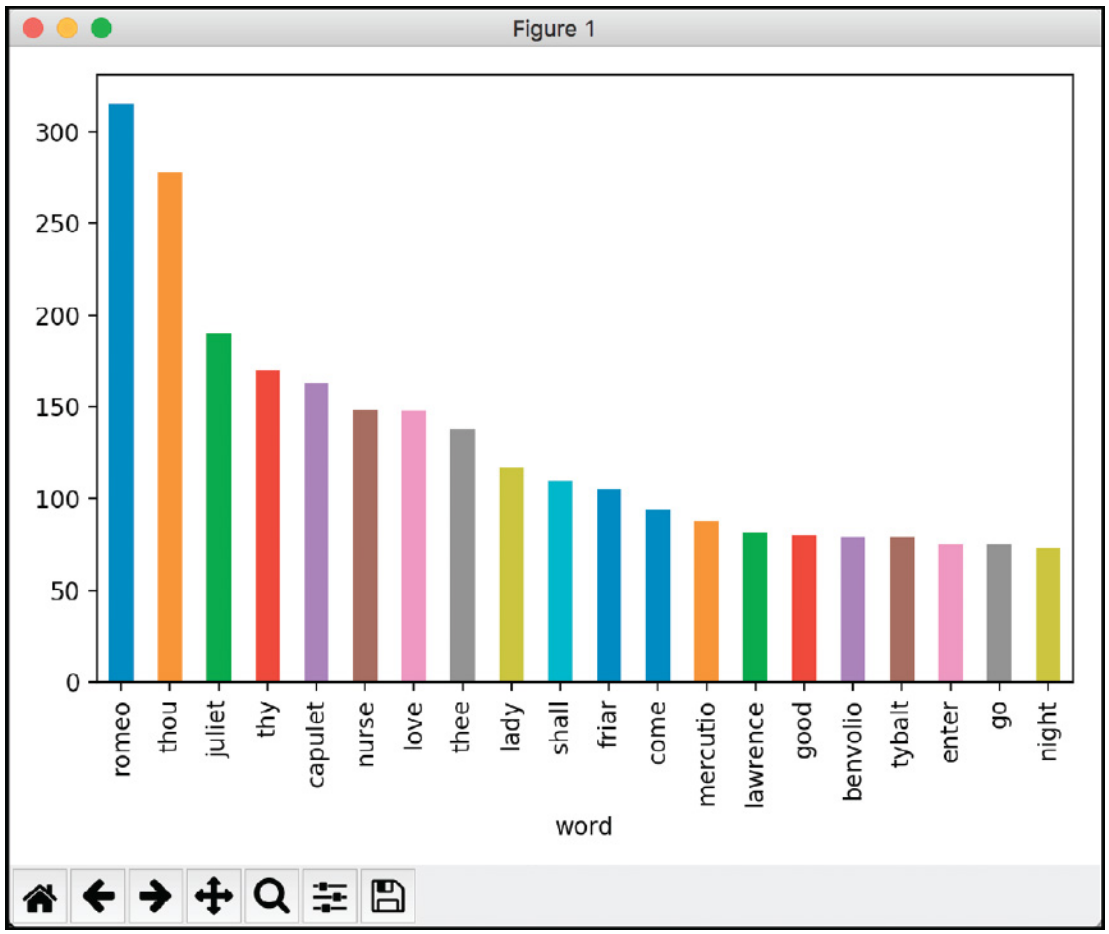
When you look at the initial bar chart that appears, you'll notice that some of the words are truncated. To fix that, use Matplotlib's `gcf` (get current figure) function to get the Matplotlib figure that pandas displayed, then call the figure's `tight_layout` method. This compresses the bar chart to ensure all its components fit:

lick here to view code image

```
In [15]: import   matplotlib.pyplot as plt
In [16]: plt.gcf().tight_layout()
```

The final graph is shown below:

C



11.3.2 Visualizing Word Frequencies with Word Clouds

Next, we'll build a word cloud that visualizes the top 200 words in *Romeo and Juliet*. You can use the open source **`wordcloud` module's** [3] **`WordCloud` class** to generate word clouds with just a few lines of code. By default, `wordcloud` creates rectangular word clouds, but as you'll see the library can create word clouds with arbitrary shapes.

[3] `ttps://github.com/amueller/word_cloud`.

Installing the `wordcloud` Module

To install `wordcloud`, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux) and enter the command:

conda install -c conda-forge wordcloud

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Loading the Text

First, let's load *Romeo and Juliet*. Launch IPython from the `ch11` examples folder before executing the following code so you can access the e-book file `RomeoAndJuliet.txt` you downloaded earlier:

lick here to view code image

```
In [1]: from pathlib import Path
```

```
In [2]: text = Path('RomeoAndJuliet.txt').read_text()
```

Loading the Mask Image that Specifies the Word Cloud's Shape

To create a word cloud of a given shape, you can initialize a `WordCloud` object with an image known as a *mask*. The `WordCloud` fills non-white areas of the mask image with text. We'll use a heart shape in this example, provided as `mask_heart.png` in the `ch11` examples folder. More complex masks require more time to create the word cloud.

Let's load the mask image by using the **`imread` function** from the `imageio` module that comes with Anaconda:

lick here to view code image

```
In [3]: import imageio

In [4]: mask_image = imageio.imread('mask_heart.png')
```

This function returns the image as a NumPy `array`, which is required by `WordCloud`.

Configuring the WordCloud Object

Next, let's create and configure the `WordCloud` object:

lick here to view code image

```
In [5]: from wordcloud import WordCloud

In [6]: wordcloud = WordCloud(colormap='prism', mask=mask_image,
   ...:     background_color='white')
   ...:
```

The default `WordCloud` width and height in pixels is 400x200, unless you specify `width` and `height` keyword arguments or a mask image. For a mask image, the `WordCloud` size is the image's size. `WordCloud` uses Matplotlib under the hood. `WordCloud` assigns random colors from a color map. You can supply the `colormap` keyword argument and use one of Matplotlib's named color maps. For a list of color map names and their colors, see:

ttps://matplotlib.org/examples/color/colormaps_reference.html

The mask keyword argument specifies the `mask_image` we loaded previously. By default, the word is drawn on a black background, but we customized this with the `background_color` keyword argument by specifying a `'white'` background. For a complete list of `WordCloud`'s keyword arguments, see

ttp://amueller.github.io/word_cloud/generated/wordcloud.WordCloud.html

Generating the Word Cloud

`WordCloud`'s **`generate` method** receives the text to use in the word cloud as an argument and creates the word cloud, which it returns as a `WordCloud` object:

lick here to view code image

```
In [7]: wordcloud = wordcloud.generate(text)
```

Before creating the word cloud, `generate` first removes stop words from the text argument using the `wordcloud` module's built-in stop-words list. Then `generate` calculates the word frequencies for the remaining words. The method uses a maximum of 200 words in the word cloud by default, but you can customize this with the `max_words` keyword argument.

Saving the Word Cloud as an Image File

Finally, we use `WordCloud`'s **`to_file` method** to save the word cloud image into the specified file:

lick here to view code image

```
In [8]: wordcloud = wordcloud.to_file('RomeoAndJulietHeart.png')
```

You can now go to the `ch11` examples folder and double-click the `RomeoAndJuliet.png` image file on your system to view it—your version might have the words in different positions and different colors:

Generating a Word Cloud from a Dictionary

If you already have a dictionary of key–value pairs representing word counts, you can pass it to `WordCloud`'s **`fit_words` method**. This method assumes you've already removed the stop words.

Displaying the Image with Matplotlib

If you'd like to display the image on the screen, you can use the IPython magic

```
%matplotlib
```

to enable interactive Matplotlib support in IPython, then execute the following statements:

C

lick here to view code image

```
import matplotlib.pyplot as plt
 plt.imshow(wordcloud)
```

11.4 READABILITY ASSESSMENT WITH TEXTATISTIC

An interesting use of natural language processing is assessing text **readability**, which is affected by the vocabulary used, sentence structure, sentence length, topic and more. While writing this book, we used the paid tool Grammarly to help tune the writing and ensure the text's readability for a wide audience.

In this section, we'll use the **Textatistic library** [4] to assess readability. [5] There are many formulas used in natural language processing to calculate readability. Textatistic uses five popular readability formulas—Flesch Reading Ease, Flesch-Kincaid, Gunning Fog, Simple Measure of Gobbledygook (SMOG) and Dale-Chall.

[4] `ttps://github.com/erinhengel/Textatistic`.

[5]Some other Python readability assessment libraries include readability-score, textstat, readability and pylinguistics.

Install Textatistic

To install Textatistic, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

```
pip install textatistic
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Calculating Statistics and Readability Scores

First, let's load *Romeo and Juliet* into the `text` variable:

lick here to view code image

```
In [1]: from pathlib import Path

In [2]: text = Path('RomeoAndJuliet.txt').read_text()
```

Calculating statistics and readability scores requires a **`Textatistic`** object that's initialized with the text you want to assess:

lick here to view code image

```
In [3]: from textatistic import Textatistic

In [4]: readability = Textatistic(text)
```

`Textatistic` method **`dict`** returns a dictionary containing various statistics and the readability scores [6]:

[6]Each Project Gutenberg e-book includes additional text, such as their licensing information, thats not part of the e-book itself. For this example, we used a text editor to remove that text from our copy of the e-book.

lick here to view code image

```
In [5]: %precision 3
Out[5]: '%.3f'

In [6]: readability.dict()
Out[6]:
{'char_count': 115141,
 'word_count': 26120,
 'sent_count': 3218,
 'sybl_count': 30166,
 'notdalechall_count': 5823,
 'polysyblword_count': 549,
 'flesch_score': 100.892,
 'fleschkincaid_score': 1.203,
 'gunningfog_score': 4.087,
 'smog_score': 5.489,
 'dalechall_score': 7.559}
```

Each of the values in the dictionary is also accessible via a `Textatistic` property of the same name as the keys shown in the preceding output. The statistics produced include:

- `char_count`—The number of characters in the text.
- `word_count`—The number of words in the text.
- `sent_count`—The number of sentences in the text.
- `sybl_count`—The number of syllables in the text.
- `notdalechall_count`—A count of the words that are not on the Dale-Chall list, which is a list of words understood by 80% of 5th graders. [7] The higher this number is compared to the total word count, the less readable the text is considered to be.

  [7] `ttp://www.readabilityformulas.com/articles/dale-chall-eadability-word-list.php`.

- `polysyblword_count`—The number of words with three or more syllables.
- `flesch_score`—The Flesch Reading Ease score, which can be mapped to a grade level. Scores over 90 are considered readable by 5th graders. Scores under 30 require a college degree. Ranges in between correspond to the other grade levels.
- `fleschkincaid_score`—The Flesch-Kincaid score, which corresponds to a specific grade level.
- `gunningfog_score`—The Gunning Fog index value, which corresponds to a specific

grade level.

- `smog_score`—The Simple Measure of Gobbledygook (SMOG), which corresponds to the years of education required to understand text. This measure is considered particularly effective for healthcare materials.[8]

[8] ttps://en.wikipedia.org/wiki/SMOG.

- `dalechall_score`—The Dale-Chall score, which can be mapped to grade levels from 4 and below to college graduate (grade 16) and above. This score considered to be most reliable for a broad range of text types.[9, o]

[9] ttps://en.wikipedia.org/wiki/Readability#The_Dale%E2%80%93Chall_formula.

[o] ttp://www.readabilityformulas.com/articles/how-do-i-decide-hich-readability-formula-to-use.php.

For more details on each of the readability scores produced here and several others, see

ttps://en.wikipedia.org/wiki/Readability

The Textatistic documentation also shows the readability formulas used:

ttp://www.erinhengel.com/software/textatistic/

11.5 NAMED ENTITY RECOGNITION WITH SPACY

NLP can determine what a text is about. A key aspect of this is **named entity recognition**, which attempts to locate and categorize items like dates, times, quantities, places, people, things, organizations and more. In this section, we'll use the named entity recognition capabilities in the **spaCy NLP library** [1], [2] to analyze text.

[1] ttps://spacy.io/.

[2]You may also want to check out Textacy (ttps://github.com/chartbeat-abs/textacy)an NLP library built on spaCy that supports additional NLP tasks.

Install spaCy

To install spaCy, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

lick here to view code image

```
conda install -c conda-forge spacy
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Once the install completes, you also need to execute the following command, so spaCy can

download additional components it needs for processing English (`en`) text:

lick here to view code image

```
python -m spacy download en
```

Loading the Language Model

The first step in using spaCy is to load the language model representing the natural language of the text you're analyzing. To do this, you'll call the `spacy` module's **`load` function**. Let's load the English model that we downloaded above:

lick here to view code image

```
In [1]: import spacy

In [2]: nlp = spacy.load('en')
```

The spaCy documentation recommends the variable name `nlp`.

Creating a spaCy Doc

Next, you use the `nlp` object to create a spaCy **`Doc`** object [3] representing the document to process. Here we used a sentence from the introduction to the World Wide Web in many of our books:

[3] `ttps://spacy.io/api/doc`.

lick here to view code image

```
In [3]: document = nlp('In 1994, Tim   Berners-Lee founded the ' +
   ...:     'World Wide Web   Consortium (W3C), devoted to ' +
   ...:     'developing web   technologies')
   ...:
```

Getting the Named Entities

The `Doc` object's **`ents` property** returns a tuple of **`Span`** objects representing the named entities found in the `Doc`. Each `Span` has many properties. [4] Let's iterate through the `Span`s and display the `text` and `label_` properties:

[4] `ttps://spacy.io/api/span`.

lick here to view code image

```
In [4]: for entity in document.ents:
   ...:     print(f'{entity.text}: {entity.label_}')
   ...:
1994: DATE
Tim Berners-Lee: PERSON
the World Wide Web Consortium: ORG
```

Each `Span`'s **`text` property** returns the entity as a string, and the **`label_` property** returns a string indicating the entity's kind. Here, spaCy found three entities representing a `DATE` (`1994`), a `PERSON` (`Tim Berners-Lee`) and an `ORG` (organization; `the World Wide Web Consortium`). For more spaCy information and to take a look at its Quickstart guide, see

```
ttps://spacy.io/usage/models#section-quickstart
```

11.6 SIMILARITY DETECTION WITH SPACY

Similarity detection is the process of analyzing documents to determine how alike they are. One possible similarity detection technique is word frequency counting. For example, some people believe that the works of William Shakespeare actually might have been written by Sir Francis Bacon, Christopher Marlowe or others. [5] Comparing the word frequencies of their works with those of Shakespeare can reveal writing-style similarities.

[5] `ttps://en.wikipedia.org/wiki/Shakespeare_authorship_question`.

Various machine-learning techniques we'll discuss in later chapters can be used to study document similarity. However, as is often the case in Python, there are libraries such as spaCy and Gensim that can do this for you. Here, we'll use spaCy's similarity detection features to compare `Doc` objects representing Shakespeare's *Romeo and Juliet* with Christopher Marlowe's *Edward the Second*. You can download *Edward the Second* from Project Gutenberg as we did for *Romeo and Juliet* earlier in the chapter. [6]

[6]Each Project Gutenberg e-book includes additional text, such as their licensing information, thats not part of the e-book itself. For this example, we used a text editor to remove that text from our copies of the e-books.

Loading the Language Model and Creating a spaCy Doc

As in the preceding section, we first load the English model:

lick here to view code image

```
In [1]: import spacy

In [2]: nlp = spacy.load('en')
```

Creating the spaCy `Doc`s

Next, we create two `Doc` objects—one for *Romeo and Juliet* and one for *Edward the Second*:

lick here to view code image

```
In [3]: from pathlib import Path

In [4]: document1 = nlp(Path('RomeoAndJuliet.txt').read_text())

In [5]: document2 = nlp(Path('EdwardTheSecond.txt').read_text())
```

Comparing the Books' Similarity

Finally, we use the `Doc` class's **`similarity` method** to get a value from 0.0 (not similar) to 1.0 (identical) indicating how similar the documents are:

lick here to view code image

```
In [6]: document1.similarity(document2)
Out[6]: 0.9349950179100041
```

spaCy believes these two documents have significant similarities. For comparison purposes, we created a `Doc` representing a current news story and compared it with *Romeo and Juliet*. As expected, spaCy returned a low value indicating little similarity between them. Try copying a current news article into a text file, then performing the similarity comparison yourself.

11.7 OTHER NLP LIBRARIES AND TOOLS

We've shown you various NLP libraries, but it's always a good idea to investigate the range of options available to you so you can leverage the best tools for your tasks. Below are some additional mostly free and open source NLP libraries and APIs:

- Gensim—Similarity detection and topic modeling.
- Google Cloud Natural Language API—Cloud-based API for NLP tasks such as named entity recognition, sentiment analysis, parts-of-speech analysis and visualization, determining content categories and more.
- Microsoft Linguistic Analysis API.
- Bing sentiment analysis—Microsoft's Bing search engine now uses sentiment in its search results. At the time of this writing, sentiment analysis in search results is available only in the United States.
- PyTorch NLP—Deep learning library for NLP.
- Stanford CoreNLP—A Java NLP library, which also provides a Python wrapper. Includes corefererence resolution, which finds all references to the same thing.
- Apache OpenNLP—Another Java-based NLP library for common tasks, including coreference resolution. Python wrappers are available.
- PyNLPl (pineapple)—Python NLP library, includes basic and more sophisticated NLP capabilities.
- SnowNLP—Python library that simplifies Chinese text processing.
- KoNLPy—Korean language NLP.
- `stop-words`—Python library with stop words for many languages. We used NLTK's stop words lists in this chapter.

C

- `TextRazor`—A paid cloud-based NLP API that provides a free tier.

11.8 MACHINE LEARNING AND DEEP LEARNING NATURAL LANGUAGE APPLICATIONS

There are many natural language applications that require machine learning and deep learning techniques. We'll discuss some of the following in our machine learning and deep learning chapters:

- Answering natural language questions—For example, our publisher Pearson Education, has a partnership with IBM Watson that uses Watson as a virtual tutor. Students ask Watson natural language questions and get answers.
- Summarizing documents—analyzing documents and producing short summaries (also called abstracts) that can, for example, be included with search results and can help you decide what to read.
- Speech synthesis (speech-to-text) and speech recognition (text-to-speech)—We use these in our "IBM Watson" chapter, along with inter-language text-to-text translation, to develop a near real-time inter-language voice-to-voice translator.
- Collaborative filtering—used to implement recommender systems ("if you liked this movie, you might also like ").
- Text classification—for example, classifying news articles by categories, such as world news, national news, local news, sports, business, entertainment, etc.
- Topic modeling—finding the topics discussed in documents.
- Sarcasm detection—often used with sentiment analysis.
- Text simplification—making text more concise and easier to read.
- Speech to sign language and vice versa—to enable a conversation with a hearing-impaired person.
- Lip reader technology—for people who can't speak, convert lip movement to text or speech to enable conversation.
- Closed captioning—adding text captions to video.

11.9 NATURAL LANGUAGE DATASETS

There's a tremendous number of text data sources available to you for working with natural language processing:

- Wikipedia—some or all of Wikipedia (`ttps://meta.wikimedia.org/wiki/Datasets`).
- IMDB (Internet Movie Database)—various movie and TV datasets are available.
- UCIs text datasets—many datasets, including the Spambase dataset.

h

- Project Gutenberg—50,000+ free e-books that are out-of-copyright in the U.S.
- Jeopardy! dataset—200,000+ questions from the Jeopardy! TV show. A milestone in AI occurred in 2011 when IBM Watson famously beat two of the world's best Jeopardy! players.
- Natural language processing datasets: `ttps://machinelearningmastery.com/datasets-natural-language-rocessing/`.
- NLTK data: `ttps://www.nltk.org/data.html`.
- Sentiment labeled sentences data set (from sources including IMDB.com, amazon.com, yelp.com.)
- Registry of Open Data on AWS—a searchable directory of datasets hosted on Amazon Web Services (`ttps://registry.opendata.aws`).
- Amazon Customer Reviews Dataset—130+ million product reviews (`ttps://registry.opendata.aws/amazon-reviews/`).
- Pitt.edu corpora (`ttp://mpqa.cs.pitt.edu/corpora/`).

11.10 WRAP-UP

In this chapter, you performed a broad range of natural language processing (NLP) tasks using several NLP libraries. You saw that NLP is performed on text collections known as corpora. We discussed nuances of meaning that make natural language understanding difficult.

We focused on the TextBlob NLP library, which is built on the NLTK and pattern libraries, but easier to use. You created `TextBlob`s and tokenized them into `Sentence`s and `Word`s. You determined the part of speech for each word in a `TextBlob`, and you extracted noun phrases.

We demonstrated how to evaluate the positive or negative sentiment of `TextBlob`s and `Sentence`s with the TextBlob library's default sentiment analyzer and with the `NaiveBayesAnalyzer`. You used the TextBlob library's integration with Google Translate to detect the language of text and perform inter-language translation.

We showed various other NLP tasks, including singularization and pluralization, spell checking and correction, normalization with stemming and lemmatization, and getting word frequencies. You obtained word definitions, synonyms and antonyms from WordNet. You also used NLTK's stop words list to eliminate stop words from text, and you created n-grams containing groups of consecutive words.

We showed how to visualize word frequencies quantitatively as a bar chart using pandas' built-in plotting capabilities. Then, we used the `wordcloud` library to visualize word frequencies qualitatively as word clouds. You performed readability assessments using the Textatistic library. Finally, you used spaCy to locate named entities and to perform similarity detection among documents. In the next chapter, you'll continue using natural language rocessing as we introduce data mining tweets using the Twitter APIs.

h

12. Data Mining Twitter

Objectives

In this chapter, you'll:

- Understand Twitter's impact on businesses, brands, reputation, sentiment analysis, predictions and more.
- Use Tweepy, one of the most popular Python Twitter API clients for data mining Twitter.
- Use the Twitter Search API to download past tweets that meet your criteria.
- Use the Twitter Streaming API to sample the stream of live tweets as they're happening.
- See that the tweet objects returned by Twitter contain valuable information beyond the tweet text.
- Use the natural language processing techniques from the last chapter to clean and preprocess tweets to prepare them for analysis.
- Perform sentiment analysis on tweets.
- Spot trends with Twitter's Trends API.
- Map tweets using folium and OpenStreetMap.
- Understand various ways to store tweets using techniques discussed throughout this book.

Outline

12.1 INTRODUCTION

We're always trying to predict the future. Will it rain on our upcoming picnic? Will the stock market or individual securities go up or down, and when and by how much? How will people vote in the next election? What's the chance that a new oil exploration venture will strike oil and if so how much would it likely produce? Will a baseball team win more games if it changes its batting philosophy to "swing for the fences?" How much customer traffic does an airline anticipate over the next many months? And hence how should the company buy oil commodity futures to guarantee that it will have the supply it needs and hopefully at a minimal cost? What track is a hurricane likely to take and how powerful will it likely become (category 1, 2, 3, 4 or 5)? That kind of information is crucial to emergency preparedness efforts. Is a financial transaction likely to be fraudulent? Will a mortgage default? Is a disease likely to spread rapidly and, if so, to what geographic area?

Prediction is a challenging and often costly process, but the potential rewards are great. With the technologies in this and the upcoming chapters, we'll see how AI, often in concert with big data, is rapidly improving prediction capabilities.

In this chapter we concentrate on data mining Twitter, looking for the sentiment in tweets. **Data mining** is the process of searching through large collections of data, often big data, to find insights that can be valuable to individuals and organizations. The sentiment that you data mine from tweets could help predict the results of an election, the revenues a new movie is likely to generate and the success of a company's marketing campaign. It could also help companies spot weaknesses in competitors' product offerings.

You'll connect to Twitter via web services. You'll use Twitter's Search API to tap into the enormous base of past tweets. You'll use Twitter's Streaming API to sample the flood of new tweets as they happen. With the Twitter Trends API, you'll see what topics are trending. You'll find that much of what we presented in the "atural Language Processing NLP)" chapter will be useful in building Twitter applications.

As you've seen throughout this book, because of powerful libraries, you'll often perform

ignificant tasks with just a few lines of code. This is why Python and its robust open-source community are appealing.

Twitter has displaced the major news organizations as the first source for newsworthy events. Most Twitter posts are public and happen in real-time as events unfold globally. People speak frankly about any subject and tweet about their personal and business lives. They comment on the social, entertainment and political scenes and whatever else comes to mind. With their mobile phones, they take and post photos and videos of events as they happen. You'll commonly hear the terms **Twitterverse** and **Twittersphere** to mean the hundreds of millions of users who have anything to do with sending, receiving and analyzing tweets.

What Is Twitter?

Twitter was founded in 2006 as a microblogging company and today is one of the most popular sites on the Internet. Its concept is simple. People write short messages called *tweets*, initially limited to 140 characters but recently increased for most languages to 280 characters. Anyone can generally choose to follow anyone else. This is different from the closed, tight communities on other social media platforms such as Facebook, LinkedIn and many others, where the "following relationships" must be reciprocal.

Twitter Statistics

Twitter has hundreds of millions of users and hundreds of millions of tweets are sent every day with many thousands sent per second. [1] Searching online for "Internet statistics" and "Twitter statistics" will help you put these numbers in perspective. Some "tweeters" have more than 100 million followers. Dedicated tweeters generally post several per day to keep their followers engaged. Tweeters with the largest followings are typically entertainers and politicians. Developers can tap into the live stream of tweets as they're happening. This has been likened to "drinking from a fire hose," because the tweets come at you so quickly.

[1] ttp://www.internetlivestats.com/twitter-statistics/.

Twitter and Big Data

Twitter has become a favorite big data source for researchers and business people worldwide. Twitter allows regular users free access to a small portion of the more recent tweets. Through special arrangements with Twitter, some third-party businesses (and Twitter itself) offer paid access to much larger portions the all-time tweets database.

Cautions

ou can't always trust everything you read on the Internet, and tweets are no exception. For example, people might use false information to try to manipulate financial markets or influence political elections. Hedge funds often trade securities based in part on the tweet streams they follow, but they're cautious. That's one of the challenges of building *business-critical* or *mission-critical* systems based on social media content.

Going forward, we use web services extensively. Internet connections can be lost, services can change and some services are not available in all countries. This is the real world of cloud-based programming. We cannot program with the same reliability as desktop apps when using web services.

12.2 OVERVIEW OF THE TWITTER APIS

Twitter's APIs are cloud-based web services, so an Internet connection is required to execute the code in this chapter. **Web services** are methods that you call in the cloud, as you'll do with the Twitter APIs in this chapter, the IBM Watson APIs in the next chapter and other APIs you'll use as computing becomes more cloud-based. Each API method has a web service **endpoint**, which is represented by a URL that's used to invoke that method over the Internet.

Twitter's APIs include many categories of functionality, some free and some paid. Most have **rate limits** that restrict the number of times you can use them in 15-minute intervals. In this chapter, you'll use the **Tweepy library** to invoke methods from the following Twitter APIs:

- **Authentication API**—Pass your Twitter credentials (discussed shortly) to Twitter so you can use the other APIs.
- **Accounts and Users API**—Access information about an account.
- **Tweets API**—Search through past tweets, access tweet streams to tap into tweets happening now and more.
- **Trends API**—Find locations of trending topics and get lists of trending topics by location.

See the extensive list of Twitter API categories, subcategories and individual methods at:

ttps://developer.twitter.com/en/docs/api-reference-index.html

Rate Limits: A Word of Caution

witter expects developers to use its services responsibly. Each Twitter API method has a **rate limit**, which is the maximum number of requests (that is, calls) you can make during a 15-minute window. Twitter may block you from using its APIs if you continue to call a given API method after that method's rate limit has been reached.

Before using any API method, read its documentation and understand its rate limits. [2] We'll configure Tweepy to wait when it encounters rate limits. This helps prevent you from exceeding the rate-limit restrictions. Some methods list both user rate limits and app rate limits. All of this chapter's examples use *app rate limits*. User rate limits are for apps that enable individual users to log into Twitter, like third-party apps that interact with Twitter on your behalf, such as smartphone apps from other vendors.

[2] Keep in mind that Twitter could change these limits in the future.

For details on rate limiting, see

ttps://developer.twitter.com/en/docs/basics/rate-limiting

For specific rate limits on individual API methods, see

ttps://developer.twitter.com/en/docs/basics/rate-limits

and each API method's documentation.

Other Restrictions

Twitter is a goldmine for data mining and they allow you to do a lot with their free services. You'll be amazed at the valuable applications you can build and how these will help you improve your personal and career endeavors. **However, if you do not follow Twitter's rules and regulations, your developer account could be terminated. You should carefully read the following and the documents they link to:**

- Terms of Service: `ttps://twitter.com/tos`
- Developer Agreement: `ttps://developer.twitter.com/en/developer-erms/agreement-and-policy.html`
- Developer Policy: `ttps://developer.twitter.com/en/developer-erms/policy.html`
- Other restrictions: `ttps://developer.twitter.com/en/developer-erms/more-on-restricted-use-cases`

ou'll see later in this chapter that you can search tweets only for the last seven days and get only a limited number of tweets using the free Twitter APIs. Some books and articles say you can get around those limits by scraping tweets directly from `twitter.com`. However, the Terms of Service explicitly say that **"scraping the Services without the prior consent of Twitter is expressly prohibited."**

12.3 CREATING A TWITTER ACCOUNT

Twitter requires you to apply for a developer account to be able to use their APIs. Go to

ttps://developer.twitter.com/en/apply-for-access

and submit your application. You'll have to register for one as part of this process if you do not already have one. You'll be asked questions about the purpose of your account. You must ***carefully*** read and agree to Twitter's terms to complete the application, then confirm your email address.

Twitter reviews every application. Approval is not guaranteed. At the time of this writing, *personal-use accounts* were approved immediately. For company accounts, the process was taking from a few days to several weeks, according to the Twitter developer forums.

12.4 GETTING TWITTER CREDENTIALS—CREATING AN APP

Once you have a Twitter developer account, you must obtain **credentials** for interacting with the Twitter APIs. To do so, you'll create an **app**. Each app has separate credentials. To create an app, log into

ttps://developer.twitter.com

and perform the following steps:

1. At the top-right of the page, click the drop-down menu for your account and select **Apps**.
2. Click **Create an app**.
3. In the **App name** field, specify your app's name. If you *send* tweets via the API, this app name will be the tweets' sender. It also will be shown to users if you create applications that require a user to log in via Twitter. We will not do either in this chapter, so a name like "*YourName* `Test App`" is fine for use with this chapter.

. In the **Application description field**, enter a description for your app. When creating Twitter-based apps that will be used by other people, this would describe what your app does. For this chapter, you can use `"Learning to use the Twitter API."`

5. In the **Website URL** field, enter your website. When creating Twitter-based apps, this is supposed to be the website where you host your app. You can use your Twitter URL: `ttps://twitter.com/`*YourUserName*, where *YourUserName* is your Twitter account screen name. For example, `ttps://twitter.com/nasa` corresponds to the NASA screen name `@nasa`.

6. The **Tell us how this app will be used** field is a description of at least 100 characters that helps Twitter employees understand what your app does. We entered `"I am new to Twitter app development and am simply learning how to use the Twitter APIs for educational purposes."`

7. Leave the remaining fields empty and click **Create**, then carefully review the (lengthy) developer terms and click **Create** again.

Getting Your Credentials

After you complete *Step 7* above, Twitter displays a web page for managing your app. At the top of the page are **App details**, **Keys and tokens** and **Permissions** tabs. Click the **Keys and tokens** tab to view your app's credentials. Initially, the page shows the **Consumer API keys**—the **API key** and the **API secret key**. Click **Create** to get an **access token** and **access token secret**. All four of these will be used to authenticate with Twitter so that you may invoke its APIs.

Storing Your Credentials

As a good practice, do not include your API keys and access tokens (or any other credentials, like usernames and passwords) directly in your source code, as that would expose them to anyone reading the code. You should store your keys in a separate file and never share that file with anyone. [3]

[3] Good practice would be to use an encryption library such as bcrypt (`ttps://github.com/pyca/bcrypt/`) to encrypt your keys, access tokens or any other credentials you use in your code, then read them in and decrypt them only as you pass them to Twitter.

The code you'll execute in subsequent sections assumes that you place your consumer key, consumer secret, access token and access token secret values into the file `keys.py` shown

below. You can find this file in the `ch12` examples folder:

lick here to view code image

```
consumer_key='YourConsumerKey'
consumer_secret='YourConsumerSecret'
access_token='YourAccessToken'
access_token_secret='YourAccessTokenSecret'
```

Edit this file, replacing `YourConsumerKey`, `YourConsumerSecret`, `YourAccessToken` and `YourAccessTokenSecret` with your consumer key, consumer secret, access token and access token secret values. Then, save the file.

OAuth 2.0

The consumer key, consumer secret, access token and access token secret are each part of the **OAuth 2.0** authentication process [4, 5] —sometimes called the *OAuth dance*—that Twitter uses to enable access to its APIs. The Tweepy library enables you to provide the consumer key, consumer secret, access token and access token secret and handles the OAuth 2.0 authentication details for you.

[4] ttps://developer.twitter.com/en/docs/basics/authentication/overview.

[5] ttps://oauth.net/.

12.5 WHAT'S IN A TWEET?

The Twitter API methods return JSON objects. **JSON (JavaScript Object Notation)** is a text-based data-interchange format used to represent objects as collections of name–value pairs. It's commonly used when invoking web services. JSON is both a human-readable and computer-readable format that makes data easy to send and receive across the Internet.

JSON objects are similar to Python dictionaries. Each JSON object contains a list of *property names* and *values*, in the following curly braced format:

{*propertyName1*: *value1*, *propertyName2*: *value2*}

As in Python, JSON lists are comma-separated values in square brackets:

[*value1*, *value2*, *value3*]

or your convenience, Tweepy handles the JSON for you behind the scenes, converting JSON to Python objects using classes defined in the Tweepy library.

Key Properties of a Tweet Object

A tweet (also called a *status update*) may contain a maximum of 280 characters, but the tweet objects returned by the Twitter APIs contain many **metadata** attributes that describe aspects of the tweet, such as:

- when it was created,
- who created it,
- lists of the hashtags, urls, @-mentions and media (such as images and videos, which are specified via their URLs) included in the tweet,
- and more.

The following table lists a few key attributes of a tweet object:

ttribute	Description
`created_at`	The creation date and time in UTC (Coordinated Universal Tim format.
`entities`	Twitter extracts `hashtags`, `urls`, `user_mentions` (that is, `@`*username* mentions), `media` (such as images and videos), `symbols` and `polls` from tweets and places them into the `entities` dictionary as lists that you can access with these key
`extended_tweet`	For tweets over 140 characters, contains details such as the twe `full_text` and `entities`
`favorite_count`	Number of times other users favorited the tweet.

F

`coordinates`	The coordinates (latitude and longitude) from which the tweet sent. This is often `null` (`None` in Python) because many users disable sending location data.
`place`	Users can associate a place with a tweet. If they do, this will be place object: `ttps://developer.twitter.com/en/docs/tweets/d ictionary/overview/geo-objects#place-dictiona` otherwise, it'll be `null` (`None` in Python).
`id`	The integer ID of the tweet. Twitter recommends using `id_str` portability.
`id_str`	The string representation of the tweet's integer ID.
`lang`	Language of the tweet, such as `'en'` for English or `'fr'` for French.
`retweet_count`	Number of times other users retweeted the tweet.
`text`	The text of the tweet. If the tweet uses the new 280-character li and contains more than 140 characters, this property will be truncated and the `truncated` property will be set to *true*. This might also occur if a 140-character tweet was retweeted and be more than 140 characters as a result.
`user`	The User object representing the user that posted the tweet. Fo User object JSON properties, see: `ttps://developer.twitter.com/en/docs/tweets/d ictionary/overview/user-object`.

Sample Tweet JSON

Let's look at sample JSON for the following tweet from the `@nasa` account:

lick here to view code image

```
@NoFear1075 Great question, Anthony! Throughout its seven-year mission,
our Parker #SolarProbe spacecraft... https://t.co/xKd6ym8waT'
```

We added line numbers and reformatted some of the JSON due to wrapping. Note that some fields in Tweet JSON are not supported in every Twitter API method; such differences are explained in the online documentation for each method.

lick here to view code image

```
 1 {'created_at':   'Wed Sep 05 18:19:34 +0000 2018',
 2  'id':   1037404890354606082,
 3  'id_str':   '1037404890354606082',
 4  'text':   '@NoFear1075 Great question, Anthony! Throughout its seven-yea
             mission, our Parker #SolarProbe spacecraft
   https://t.co/xKd6ym8waT',
 5  'truncated':   True,
 6  'entities':   {'hashtags': [{'text': 'SolarProbe', 'indices': [84, 95]}],
 7     'symbols':   [],
 8     'user_mentions':   [{'screen_name': 'NoFear1075',
 9         'name':   'Anthony Perrone',
 10         'id':   284339791,
 11         'id_str':   '284339791',
 12         'indices':   [0, 11]}],
 13     'urls':   [{'url': 'https://t.co/xKd6ym8waT',
 14         'expanded_url':   'https://twitter.com/i/web/status/
              1037404890354606082',
 15         'display_url':   'twitter.com/i/web/status/1
',
 16         'indices':   [117, 140]}]},
 17  'source':   '<a href="http://twitter.com"   rel="nofollow">Twitter Web
             Client</a>',
 18  'in_reply_to_status_id':   1037390542424956928,
 19  'in_reply_to_status_id_str':   '1037390542424956928',
 20  'in_reply_to_user_id':   284339791,
 21  'in_reply_to_user_id_str':   '284339791',
 22  'in_reply_to_screen_name':   'NoFear1075',
 23  'user':   {'id': 11348282,
 24     'id_str':   '11348282',
 25     'name':   'NASA',
 26     'screen_name':   'NASA',
 27     'location':   '',
 28     'description':   'Explore the universe and discover our home planet v
              @NASA. We usually post in EST (UTC-5)',
 29     'url':   'https://t.co/TcEE6NS8nD',
 30     'entities':   {'url': {'urls': [{'url': 'https://t.co/TcEE6NS8nD',
```

C

```
            'expanded_url':   'http://www.nasa.gov',
            'display_url':   'nasa.gov',
            'indices':   [0, 23]}]},
     'description':   {'urls': []}},
    'protected':   False,
    'followers_count':   29486081,
    'friends_count':   287,
    'listed_count':   91928,
    'created_at':   'Wed Dec 19 20:20:32 +0000 2007',
    'favourites_count':   3963,
    'time_zone':   None,
    'geo_enabled':   False,
    'verified':   True,
    'statuses_count':   53147,
    'lang':   'en',
    'contributors_enabled':   False,
    'is_translator':   False,
    'is_translation_enabled':   False,
    'profile_background_color':   '000000',
    'profile_background_image_url':   'http://abs.twimg.com/images/themes
          theme1/bg.png',
    'profile_background_image_url_https':   'https://abs.twimg.com/images
          themes/theme1/bg.png',
    'profile_image_url':   'http://pbs.twimg.com/profile_images/188302352
          nasalogo_twitter_normal.jpg',
    'profile_image_url_https':   'https://pbs.twimg.com/profile_images/
          188302352/nasalogo_twitter_normal.jpg',
    'profile_banner_url':   'https://pbs.twimg.com/profile_banners/113482
           1535145490',
    'profile_link_color':   '205BA7',
    'profile_sidebar_border_color':   '000000',
    'profile_sidebar_fill_color':   'F3F2F2',
    'profile_text_color':   '000000',
    'profile_use_background_image':   True,
    'has_extended_profile':   True,
    'default_profile':   False,
    'default_profile_image':   False,
    'following':   True,
    'follow_request_sent':   False,
    'notifications':   False,
    'translator_type':   'regular'},
'geo': None,
'coordinates':   None,
'place':   None,
'contributors':   None,
'is_quote_status':   False,
'retweet_count':   7,
'favorite_count':   19,
'favorited':   False,
'retweeted':   False,
'possibly_sensitive':   False,
'lang': 'en'}
```

Twitter JSON Object Resources

For a complete, more readable list of the tweet object attributes, see:

```
ttps://developer.twitter.com/en/docs/tweets/data-dictionary/overview/tweet-
```

or additional details that were added when Twitter moved from a limit of 140 to 280 characters per tweet, see

ttps://developer.twitter.com/en/docs/tweets/data-dictionary/overview/intro-to-tweet-son.html#extendedtweet

For a general overview of all the JSON objects that Twitter APIs return, and links to the specific object details, see

ttps://developer.twitter.com/en/docs/tweets/data-dictionary/overview/intro-to-tweet-son

12.6 TWEEPY

We'll use the Tweepy library [6] (`ttp://www.tweepy.org/`)—one of the most popular Python libraries for interacting with the Twitter APIs. Tweepy makes it easy to access Twitter's capabilities and hides from you the details of processing the JSON objects returned by the Twitter APIs. You can view Tweepy's documentation [7] at

[6] ther Python libraries recommended by Twitter include Birdy, python-twitter, Python Twitter Tools, TweetPony, TwitterAPI, twitter-gobject, TwitterSearch and twython. See `ttps://developer.twitter.com/en/docs/developer-utilities/twitter-ibraries.html` for details.

[7] The Tweepy documentation is a work in progress. At the time of this writing, Tweepy does not have documentation for their classes corresponding to the JSON objects the Twitter APIs return. Tweepys classes use the same attribute names and structure as the JSON objects. You can determine the correct attribute names to access by looking at Twitters JSON documentation. Well explain any attribute we use in our code and provide footnotes with links to the Twitter JSON descriptions.

ttp://docs.tweepy.org/en/latest/

For additional information and the Tweepy source code, visit

ttps://github.com/tweepy/tweepy

Installing Tweepy

To install Tweepy, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then execute the following command:

```
pip install tweepy==3.7
```

Windows users might need to run the Anaconda Prompt as an Administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Installing geopy

As you work with Tweepy, you'll also use functions from our `tweetutilities.py` file (provided with this chapter's example code). One of the utility functions in that file depends on the **geopy library** (`ttps://github.com/geopy/geopy`), which we'll discuss in ection 12.15. To install geopy, execute:

lick here to view code image

```
conda install -c conda-forge geopy
```

12.7 AUTHENTICATING WITH TWITTER VIA TWEEPY

In the next several sections, you'll invoke various cloud-based Twitter APIs via Tweepy. Here you'll begin by using Tweepy to authenticate with Twitter and create a **Tweepy `API` object**, which is your gateway to using the Twitter APIs over the Internet. In subsequent sections, you'll work with various Twitter APIs by invoking methods on your `API` object.

Before you can invoke any Twitter API, you must use your API key, API secret key, access token and access token secret to authenticate with Twitter. [8] Launch IPython from the `ch12` examples folder, then import the **`tweepy` module** and the `keys.py` file that you modified earlier in this chapter. You can import any `.py` file as a module by using the file's name *without* the `.py` extension in an `import` statement:

[8] You may wish to create apps that enable users to log into their Twitter accounts, manage them, post tweets, read tweets from other users, search for tweets, etc. For details on user authentication see the Tweepy Authentication tutorial at `ttp://docs.tweepy.org/en/latest/auth_tutorial.html`.

```
In [1]: import tweepy
```

```
In [2]: import keys
```

When you import `keys.py` as a module, you can individually access each of the four variables defined in that file as `keys.`*variable_name*.

Creating and Configuring an `OAuthHandler` to Authenticate with Twitter

Authenticating with Twitter via Tweepy involves two steps. First, create an object of the tweepy module's **`OAuthHandler` class**, passing your API key and API secret key to its constructor. A **constructor** is a function that has the same name as the class (in this case, `OAuthHandler`–) and receives the arguments used to configure the new object:

lick here to view code image

```
In [3]: auth = tweepy.OAuthHandler(keys.consumer_key,
In [3]: auth = tweepy.OAuthHandler(keys.consumer_key,
   ...:
```

Specify your access token and access token secret by calling the `OAuthHandler` object's **`set_access_token` method**:

lick here to view code image

```
In [4]: auth.set_access_token(keys.access_token,
   ...:                       keys.access_token_secret)
   ...:
```

Creating an API Object

Now, create the `API` object that you'll use to interact with Twitter:

lick here to view code image

```
In [5]: api = tweepy.API(auth, wait_on_rate_limit=True,
   ...:                  wait_on_rate_limit_notify=True)
   ...:
```

We specified three arguments in this call to the `API` constructor:

- `auth` is the `OAuthHandler` object containing your credentials.
- The keyword argument `wait_on_rate_limit=True` tells Tweepy to wait 15

C

minutes each time it reaches a given API method's rate limit. This ensures that you do not violate Twitter's rate-limit restrictions.

- The keyword argument `wait_on_rate_limit_notify=True` tells Tweepy that, if it needs to wait due to rate limits, it should notify you by displaying a message at the command line.

You're now ready to interact with Twitter via Tweepy. Note that the code examples in the next several sections are presented as a continuing IPython session, so the authorization process you went through here need not be repeated.

12.8 GETTING INFORMATION ABOUT A TWITTER ACCOUNT

After authenticating with Twitter, you can use the Tweepy `API` object's **`get_user` method** to get a **`tweepy.models.User` object** containing information about a user's Twitter account. Let's get a `User` object for NASA's `@nasa` Twitter account:

lick here to view code image

```
In [6]: nasa = api.get_user('nasa')
```

The `get_user` method calls the Twitter API's `users/show` method. [9] Each Twitter method you call through Tweepy has a rate limit. You can call Twitter's `users/show` method up to 900 times every 15 minutes to get information on specific user accounts. As we mention other Twitter API methods, we'll provide a footnote with a link to each method's documentation in which you can view its rate limits.

[9] ttps://developer.twitter.com/en/docs/accounts-and-users/follow-earch-get-users/api-reference/get-users-show.

The `tweepy.models` classes each correspond to the JSON that Twitter returns. For example, the `User` class corresponds to a Twitter **user object**:

ttps://developer.twitter.com/en/docs/tweets/data-dictionary/overview/user-object

Each `tweepy.models` class has a method that reads the JSON and turns it into an object of the corresponding Tweepy class.

Getting Basic Account Information

Let's access some `User` object properties to display information about the `@nasa` account:

- The **`id` property** is the account ID number created by Twitter when the user joined Twitter.
- The **`name` property** is the name associated with the user's account.
- The **`screen_name` property** is the user's Twitter handle (`@nasa`). Both the `name` and `screen_name` could be created names to protect a user's privacy.
- The **`description` property** is the description from the user's profile.

lick here to view code image

```
In [7]: nasa.id
Out[7]: 11348282

In [8]: nasa.name
Out[8]: 'NASA'

In [9]: nasa.screen_name
Out[9]: 'NASA'

In [10]: nasa.description
Out[10]: 'Explore the universe and discover our home planet   with @NASA. We
```

etting the Most Recent Status Update

The `User` object's **`status` property** returns a **`tweepy.models.Status`** object, which corresponds to a Twitter **tweet object**:

ttps://developer.twitter.com/en/docs/tweets/data-dictionary/overview/tweet-object

The `Status` object's **`text` property** contains the text of the account's most recent tweet:

lick here to view code image

```
In [11]: nasa.status.text
Out[11]: 'The interaction of a high-velocity young star with   the cloud of
```

he `text` property was originally for tweets up to 140 characters. The ... above indicates that the tweet text was *truncated*. When Twitter increased the limit to 280 characters,

they added an **`extended_tweet` property** (demonstrated later) for accessing the text and other information from tweets between 141 and 280 characters. In this case, Twitter sets `text` to a truncated version of the `extended_tweet`'s text. Also, retweeting often results in truncation because a retweet adds characters that could exceed the character limit.

Getting the Number of Followers

You can view an account's number of followers with the **`followers_count` property**:

lick here to view code image

```
In [12]: nasa.followers_count
Out[12]: 29453541
```

Though this number is large, there are accounts with over 100 million followers.[o]

[o] ttps://friendorfollow.com/twitter/most-followers/.

Getting the Number of Friends

Similarly, you can view an account's number of friends (that is, the number of accounts an account follows) with the **`friends_count` property**:

```
In [13]: nasa.friends_count
Out[13]: 287
```

Getting Your Own Account's Information

You can use the properties in this section on your own account as well. To do so, call the Tweepy `API` object's **`me` method**, as in:

me = api.me()

This returns a `User` object for the account you used to authenticate with Twitter in the preceding section.

12.9 INTRODUCTION TO TWEEPY CURSORS: GETTING AN ACCOUNT'S FOLLOWERS AND FRIENDS

When invoking Twitter API methods, you often receive as results collections of objects, such as tweets in your Twitter timeline, tweets in another account's timeline or lists of tweets that match specified search criteria. A **timeline** consists of tweets sent by that user

and by that user's friends—that is, other accounts that the user follows.

Each Twitter API method's documentation discusses the maximum number of items the method can return in one call—this is known as a **page** of results. When you request more results than a given method can return, Twitter's JSON responses indicate that there are more pages to get. Tweepy's `Cursor` class handles these details for you. A **`Cursor`** invokes a specified method and checks whether Twitter indicated that there is another page of results. If so, the Cursor automatically calls the method again to get those results. This continues, subject to the method's rate limits, until there are no more results to process. If you configure the API object to wait when rate limits are reached (as we did), the `Cursor` will adhere to the rate limits and wait as needed between calls. The following subsections discuss `Cursor` fundamentals. For more details, see the `Cursor` tutorial at:

```
ttp://docs.tweepy.org/en/latest/cursor_tutorial.html
```

12.9.1 Determining an Account's Followers

Let's use a Tweepy `Cursor` to invoke the `API` object's **`followers` method**, which calls the Twitter API's `followers/list` method[1] to obtain an account's followers. Twitter returns these in groups of 20 by default, but you can request up to 200 at a time. For demonstration purposes, we'll grab 10 of NASA's followers.

[1] `ttps://developer.twitter.com/en/docs/accounts-and-users/follow-earch-get-users/api-reference/get-followers-list`.

Method `followers` returns `tweepy.models.User` objects containing information about each follower. Let's begin by creating a list in which we'll store the `User` objects:

```
In [14]: followers = []
```

Creating a Cursor

Next, let's create a `Cursor` object that will call the `followers` method for NASA's account, which is specified with the `screen_name` keyword argument:

lick here to view code image

```
In [15]: cursor = tweepy.Cursor(api.followers, screen_name='nasa')
```

The `Cursor`'s constructor receives as its argument the name of the method to call —`api.followers` indicates that the `Cursor` will call the `api` object's `followers`

method. If the `Cursor` constructor receives any additional keyword arguments, like `screen_name`, these will be passed to the method specified in the constructor's first argument. So, this `Cursor` specifically gets followers for the `@nasa` Twitter account.

Getting Results

Now, we can use the `Cursor` to get some followers. The following `for` statement iterates through the results of the expression `cursor.items(10)`. The `Cursor`'s **`items` method** initiates the call to `api.followers` and returns the `followers` method's results. In this case, we pass `10` to the `items` method to request only 10 results:

lick here to view code image

```
In [16]: for account in cursor.items(10):
    ...:     followers.append(account.screen_name)
    ...:
In [17]: print('Followers:',
    ...:       ' '.join(sorted(followers,   key=lambda s: s.lower())))
    ...:
Followers: abhinavborra BHood1976 Eshwar12341   Harish90469614 heshamkisha H
```

he preceding snippet displays the followers in ascending order by calling the built-in `sorted` function. The function's second argument is the function used to determine how the elements of followers are sorted. In this case, we used a `lambda` that converts every follower name to lowercase letters so we can perform a case-insensitive sort.

Automatic Paging

If the number of results requested is more than can be returned by one call to `followers`, the `items` method automatically "pages" through the results by making multiple calls to `api.followers`. Recall that `followers` returns up to 20 followers at a time by default, so the preceding code needs to call `followers` only once. To get up to 200 followers at a time, we can create the `Cursor` with the `count` keyword argument, as in:

lick here to view code image

```
cursor = tweepy.Cursor(api.followers, screen_name='nasa', count=200)
```

If you do not specify an argument to the `items` method, The `Cursor` attempts to get *all* of the account's followers. For large numbers of followers, this could take a significant amount of time due to Twitter's rate limits. The Twitter API's `followers/list` method

an return a maximum of 200 followers at a time and Twitter allows a maximum of 15 calls every 15 minutes. Thus, you can only get 3000 followers every 15 minutes using Twitter's free APIs. Recall that we configured the API object to automatically wait when it hits a rate limit, so if you try to get all followers and an account has more than 3000, Tweepy will automatically pause for 15 minutes after every 3000 followers and display a message. At the time of this writing, NASA has over 29.5 million followers. At 12,000 followers per hour, it would take over 100 days to get all of NASA's followers.

Note that for this example, we could have called the `followers` method directly, rather than using a `Cursor`, since we're getting only a small number of followers. We used a `Cursor` here to show how you'll typically call `followers`. In some later examples, we'll call `API` methods directly to get just a few results, rather than using `Cursor`s.

Getting Follower IDs Rather Than Followers

Though you can get complete `User` objects for a maximum of 200 followers at a time, you can get many more Twitter ID numbers by calling the `API` object's **`followers_ids` method**. This calls the Twitter API's `followers/ids` method, which returns up to 5000 ID numbers at a time (again, these rate limits could change). [2] You can invoke this method up to 15 times every 15 minutes, so you can get 75,000 account ID numbers per rate-limit interval. This is particularly useful when combined with the API object's **`lookup_users` method**. This calls the Twitter API's `users/lookup` method [3] which can return up to 100 `User` objects at a time and can be called up to 300 times every 15 minutes. So using this combination, you could get up to 30,000 `User` objects per rate-limit interval.

[2] `ttps://developer.twitter.com/en/docs/accounts-and-users/follow-earch-get-users/api-reference/get-followers-ids`.

[3] `ttps://developer.twitter.com/en/docs/accounts-and-users/follow-earch-get-users/api-reference/get-users-lookup`.

12.9.2 Determining Whom an Account Follows

The `API` object's **`friends` method** calls the Twitter API's `friends/list` method [4] to get a list of `User` objects representing an account's friends. Twitter returns these in groups of 20 by default, but you can request up to 200 at a time, just as we discussed for method `followers`. Twitter allows you to call the `friends/list` method up to 15 times every 15 minutes. Let's get 10 of NASA's friend accounts:

[4] `ttps://developer.twitter.com/en/docs/accounts-and-users/follow-`

earch-get-users/api-reference/get-friends-list.

lick here to view code image

```
In [18]: friends = []

In [19]: cursor = tweepy.Cursor(api.friends, screen_name='nasa')

In [20]: for friend in cursor.items(10):
    ...:     friends.append(friend.screen_name)
    ...:

In [21]: print('Friends:',
    ...:       ' '.join(sorted(friends,   key=lambda s: s.lower())))
    ...:
Friends: AFSpace Astro2fish Astro_Kimiya AstroAnnimal   AstroDuke NASA3DPrin
```

2.9.3 Getting a User's Recent Tweets

The `API` method **`user_timeline`** returns tweets from the timeline of a specific account. A timeline includes that account's tweets and tweets from that account's friends. The method calls the Twitter API's `statuses/user_timeline` method [5], which returns the most recent 20 tweets, but can return up to 200 at a time. This method can return only an account's 3200 most recent tweets. Applications using this method may call it up to 1500 times every 15 minutes.

[5] ttps://developer.twitter.com/en/docs/tweets/timelines/api-eference/get-statuses-user_timeline.

Method `user_timeline` returns `Status` objects with each one representing a tweet. Each `Status`'s `user` property refers to a `tweepy.models.User` object containing information about the user who sent that tweet, such as that user's `screen_name`. A `Status`'s `text` property contains the tweet's `text`. Let's display the `screen_name` and `text` for three tweets from `@nasa`:

lick here to view code image

```
 n [22]: nasa_tweets = api.user_timeline(screen_name='nasa', count=3)

In [23]: for tweet in nasa_tweets:
    ...:     print(f'{tweet.user.screen_name}: {tweet.text}\n')
    ...:
NASA: Your Gut in Space: Microorganisms in the intestinal   tract play an esp
   https://t.co/uLOsUhwn5p
```

```
NASA: We need your help! Want to see panels at @SXSW   related to space explo
   https://t.co/ycqMMdGKUB

NASA: "You are as good as anyone in this town, but you are   no better than a
   https://t.co/nhMD4n84Nf
```

hese tweets were truncated (as indicated by), meaning that they probably use the newer 280-character tweet limit. We'll use the `extended_tweet` property shortly to access full text for such tweets.

In the preceding snippets, we chose to call the `user_timeline` method directly and use the `count` keyword argument to specify the number of tweets to retrieve. If you wish to get more than the maximum number of tweets per call (`200`), then you should use a `Cursor` to call `user_timeline` as demonstrated previously. Recall that a `Cursor` automatically pages through the results by calling the method multiple times, if necessary.

Grabbing Recent Tweets from Your Own Timeline

You can call the `API` method **`home_timeline`**, as in:

api.home_timeline()

to get tweets from *your* home timeline [6]—that is, your tweets and tweets from the people *you* follow. This method calls Twitter's `statuses/home_timeline` method. [7] By default, `home_timeline` returns the most recent 20 tweets, but can get up to 200 at a time. Again, for more than 200 tweets from your home timeline, you should use a Tweepy `Cursor` to call `home_timeline`.

[6] Specifically for the account you used to authenticate with Twitter.

[7] `ttps://developer.twitter.com/en/docs/tweets/timelines/api-eference/get-statuses-home_timeline`.

12.10 SEARCHING RECENT TWEETS

The Tweepy `API` method **`search`** returns tweets that match a query string. According to the method's documentation, Twitter maintains its search index only for the previous seven days' tweets, and a search is not guaranteed to return all matching tweets. Method search calls Twitter's `search/tweets` method [8], which returns 15 tweets at a time by default, but can return up to 100.

[8] `ttps://developer.twitter.com/en/docs/tweets/search/api-`

eference/get-search-tweets.

Utility Function `print_tweets` from `tweetutilities.py`

For this section, we created a utility function `print_tweets` that receives the results of a call to API method `search` and for each tweet displays the user's `screen_name` and the tweet's `text`. If the tweet is not in English and the `tweet.lang` is not `'und'` (undefined), we'll also translate the tweet to English using `TextBlob`, as you did in the "_atural Language Processing (NLP)" chapter. To use this function, import it from `tweetutilities.py`:

lick here to view code image

```
In [24]: from tweetutilities import print_tweets
```

Just the `print_tweets` function's definition from that file is shown below:

lick here to view code image

```
def print_tweets(tweets):
    """For each Tweepy   Status object in tweets, display the
    user's screen_name and tweet   text. If the language is not
    English, translate the text with   TextBlob."""
    for tweet in tweets:
        print(f'{tweet.screen_name}:', end=' ')

        if 'en' in tweet.lang:
            print(f'{tweet.text}\n')
        elif 'und' not in tweet.lang:  # translate to English first
            print(f'\n  ORIGINAL:   {tweet.text}')
            print(f'TRANSLATED:   {TextBlob(tweet.text).translate()}\n')
```

Searching for Specific Words

Let's search for three recent tweets about NASA's Mars Opportunity Rover. The `search` method's `q` keyword argument specifies the query string, which indicates what to search for and the `count` keyword argument specifies the number of tweets to return:

lick here to view code image

```
 n [25]: tweets = api.search(q='Mars   Opportunity Rover', count=3)

In [26]: print_tweets(tweets)
Jacker760: NASA set a deadline on the Mars Rover   opportunity! As the dust c
 https://t.co/KQ7xaFgrzr
```

```
 hivak32637174: RT @Gadgets360: NASA 'Cautiously   Optimistic' of Hearing Bac
https://t.co/O1iTTwRvFq

ladyanakina: NASA's Opportunity Rover Still Silent on   #Mars. https://t.co/r
```

s with other methods, if you plan to request more results than can be returned by one call to `search`, you should use a `Cursor` object.

Searching with Twitter Search Operators

You can use various Twitter search operators in your query strings to refine your search results. The following table shows several Twitter search operators. Multiple operators can be combined to construct more complex queries. To see all the operators, visit

```
ttps://twitter.com/search-home
```

and click the **operators** link.

Example	Finds tweets containing
`python twitter`	Implicit *logical and* operator—Finds tweets containing `python` *and* `twitter`.
`python OR twitter`	Logical `OR` operator—Finds tweets containing `python` *or* `twitter` *or both*.
`python ?`	`?` (question mark)—Finds tweets asking questions about `python`.
`planets -mars`	`-` (minus sign)—Finds tweets containing `planets` but not `mars`.
	`:)` (happy face)—Finds *positive sentiment* tweets

`python :)`	containing `python`.
`python :(`	`:(` (sad face)—Finds *negative sentiment* tweets containing `python`.
`since:2018-09-01`	Finds tweets *on or after* the specified date, which must be in the form `YYYY-MM-DD`.
`near:"New York City"`	Finds tweets that were sent near `"New York City"`.
`from:nasa`	Finds tweets from the account `@nasa`.
`to:nasa`	Finds tweets to the account `@nasa`.

Let's use the `from` and `since` operators to get three tweets from NASA since September 1, 2018—you should use a date within seven days before you execute this code:

lick here to view code image

```
In [27]: tweets = api.search(q='from:nasa   since:2018-09-01', count=3)
In [28]: print_tweets(tweets)
NASA: @WYSIW Our missions   detect active burning fires, track the transport
 https://t.co/jx2iUoMlIy
NASA: Scarring of the   landscape is evident in the wake of the Mendocino Co
 https://t.co/Nboo5GD90m
NASA: RT @NASAglenn: To   celebrate the #NASA60th anniversary, we're explori
```

earching for a Hashtag

Tweets often contain **hashtags** that begin with `#` to indicate something of importance, like a trending topic. Let's get two tweets containing the hashtag `#collegefootball`:

lick here to view code image

```
In [29]: tweets = api.search(q='#collegefootball',   count=2)

In [30]: print_tweets(tweets)
dmcreek: So much for #FAU giving #OU a game. #Oklahoma   #FloridaAtlantic #C

 heangrychef: It's game day folks! And our BBQ game is   strong. #bbq #atlan
```

2.11 SPOTTING TRENDS: TWITTER TRENDS API

If a topic "goes viral," you could have thousands or even millions of people tweeting about it at once. Twitter refers to these as **trending topics** and maintains lists of the trending topics worldwide. Via the Twitter Trends API, you can get lists of locations with trending topics and lists of the top 50 trending topics for each location.

12.11.1 Places with Trending Topics

The Tweepy `API`'s **`trends_available` method** calls the Twitter API's `trends/available` [9] method to get a list of all locations for which Twitter has trending topics. Method `trends_available` returns a *list of dictionaries* representing these locations. When we executed this code, there were 467 locations with trending topics:

[9] `ttps://developer.twitter.com/en/docs/trends/locations-with-rending-topics/api-reference/get-trends-available`.

lick here to view code image

```
In [31]: trends_available = api.trends_available()
In [32]: len(trends_available)
Out[32]: 467
```

The dictionary in each list element returned by `trends_available` has various information, including the location's `name` and `woeid` (discussed below):

lick here to view code image

```
In [33]: trends_available[0]
Out[33]:
{'name': 'Worldwide',
 'placeType': {'code': 19, 'name': 'Supername'},
 'url': 'http://where.yahooapis.com/v1/place/1',
 'parentid': 0,
```

```
 'country': '',
 'woeid': 1,
 'countryCode': None}

In [34]: trends_available[1]
Out[34]:
{'name': 'Winnipeg',
 'placeType': {'code': 7, 'name': 'Town'},
 'url': 'http://where.yahooapis.com/v1/place/2972',
 'parentid': 23424775,
 'country': 'Canada',
 'woeid': 2972,
 'countryCode': 'CA'}
```

The Twitter Trends API's `trends/place` method (discussed momentarily) uses **Yahoo! Where on Earth IDs (WOEIDs)** to look up trending topics. The WOEID `1` represents *worldwide*. Other locations have unique WOEID values greater than `1`. We'll use WOEID values in the next two subsections to get worldwide trending topics and trending topics for a specific city. The following table shows WOEID values for several landmarks, cities, states and continents. Note that although these are all valid WOEIDs, Twitter does not necessarily have trending topics for all these locations.

Place	WOEID	Place	WOEID
Statue of Liberty	23617050	Iguazu Falls	468785
Los Angeles, CA	2442047	United States	23424977
Washington, D.C.	2514815	North America	24865672
Paris, France	615702	Europe	24865675

You also can search for locations close to a location that you specify with latitude and longitude values. To do so, call the Tweepy `API`'s **`trends_closest` method**, which invokes the Twitter API's `trends/closest` method. [o]

[0] ttps://developer.twitter.com/en/docs/trends/locations-with-rending-topics/api-reference/get-trends-closest.

12.11.2 Getting a List of Trending Topics

The Tweepy `API`'s **`trends_place` method** calls the Twitter Trends API's `trends/place` method [1] to get the top 50 trending topics for the location with the specified WOEID. You can get the WOEIDs from the `woeid` attribute in each dictionary returned by the `trends_available` or `trends_closest` methods discussed in the previous section, or you can find a location's Yahoo! Where on Earth ID (WOEID) by searching for a city/town, state, country, address, zip code or landmark at

[1] ttps://developer.twitter.com/en/docs/trends/trends-for-ocation/api-reference/get-trends-place.

```
ttp://www.woeidlookup.com
```

You also can look up WOEID's programmatically using Yahoo!'s web services via Python libraries like `woeid` [2]:

[2]Youll need a Yahoo! API key as described in the `woeid` modules documentation.

```
ttps://github.com/Ray-SunR/woeid
```

Worldwide Trending Topics

Let's get today's worldwide trending topics (your results will differ):

lick here to view code image

```
In [35]: world_trends = api.trends_place(id=1)
```

Method `trends_place` returns a one-element list containing a dictionary. The dictionary's `'trends'` key refers to a list of dictionaries representing each trend:

lick here to view code image

```
In [36]: trends_list = world_trends[0]['trends']
```

Each trend dictionary has `name`, `url`, `promoted_content` (indicating the tweet is an

advertisement), `query` and `tweet_volume` keys (shown below). The following trend is in Spanish—`#BienvenidoSeptiembre` means "Welcome September":

lick here to view code image

```
In [37]: trends_list[0]
Out[37]:
{'name': '#BienvenidoSeptiembre',
 'url':   'http://twitter.com/search?q=%23BienvenidoSeptiembre',
 'promoted_content': None,
 'query': '%23BienvenidoSeptiembre',
 'tweet_volume': 15186}
```

For trends with more than 10,000 tweets, the `tweet_volume` is the number of tweets; otherwise, it's `None`. Let's use a list comprehension to filter the list so that it contains only trends with more than 10,000 tweets:

lick here to view code image

```
In [38]: trends_list = [t for t in trends_list if t['tweet_volume']]
```

Next, let's sort the trends in *descending* order by `tweet_volume`:

lick here to view code image

```
In [39]: from operator import itemgetter

In [40]: trends_list.sort(key=itemgetter('tweet_volume'), reverse=True)
```

Now, let's display the names of the top five trending topics:

lick here to view code image

```
In [41]: for trend in trends_list[:5]:
    ...:     print(trend['name'])
    ...:
#HBDJanaSenaniPawanKalyan
#BackToHogwarts
Khalil Mack
#ItalianGP
Alisson
```

New York City Trending Topics

C

Now, let's get the top five trending topics for New York City (WOEID `2459115`). The following code performs the same tasks as above, but for the different WOEID:

lick here to view code image

```
In [42]: nyc_trends = api.trends_place(id=2459115)  # New York City   WOEID

In [43]: nyc_list = nyc_trends[0]['trends']

In [44]: nyc_list = [t for t in nyc_list if t['tweet_volume']]

In [45]: nyc_list.sort(key=itemgetter('tweet_volume'), reverse=True)

In [46]: for trend in nyc_list[:5]:
    ...:     print(trend['name'])
    ...:
#IDOL100M
#TuesdayThoughts
#HappyBirthdayLiam
NAFTA
#USOpen
```

12.11.3 Create a Word Cloud from Trending Topics

In the " atural Language Processing" chapter, we used the WordCloud library to create word clouds. Let's use it again here, to visualize New York City's trending topics that have more than 10,000 tweets each. First, let's create a dictionary of key–value pairs consisting of the trending topic `name`s and `tweet_volume`s:

lick here to view code image

```
In [47]: topics = {}

In [48]: for trend in nyc_list:
    ...:     topics[trend['name']]   = trend['tweet_volume']
    ...:
```

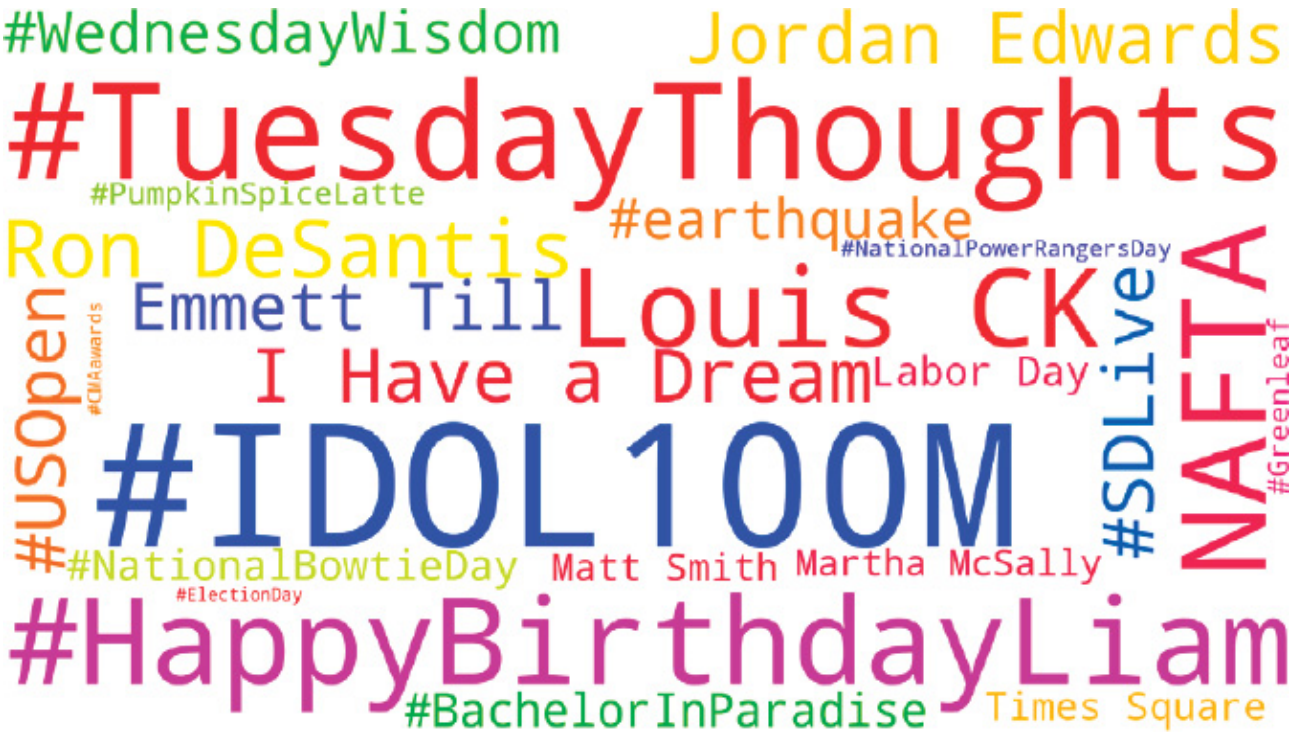
Next, let's create a `WordCloud` from the `topics` dictionary's key–value pairs, then output the word cloud to the image file `TrendingTwitter.png` (shown after the code). The argument `prefer_horizontal=0.5` *suggests* that 50% of the words should be horizontal, though the software may ignore that to fit the content:

lick here to view code image

```
In [49]: from wordcloud import WordCloud
```

```
In [50]: wordcloud = WordCloud(width=1600,   height=900,
   ...:     prefer_horizontal=0.5,   min_font_size=10, colormap='prism',
   ...:     background_color='white')
   ...:

In [51]: wordcloud = wordcloud.fit_words(topics)

In [52]: wordcloud = wordcloud.to_file('TrendingTwitter.png')
```

The resulting word cloud is shown below—yours will differ based on the trending topics the day you run the code:



12.12 CLEANING/PREPROCESSING TWEETS FOR ANALYSIS

Data cleaning is one of the most common tasks that data scientists perform. Depending on how you intend to process tweets, you'll need to use natural language processing to normalize them by performing some or all of the data cleaning tasks in the following table. Many of these can be performed using the libraries introduced in the " atural Language rocessing (NLP)" chapter:

Tweet cleaning tasks
Converting all text to the same case
Removing stop words

emoving # symbol from hashtags	Removing RT (retweet) and FAV (favorite)
Removing @-mentions	Removing URLs
Removing duplicates	Stemming
Removing excess whitespace	Lemmatization
Removing hashtags	Tokenization
Removing punctuation	

tweet-preprocessor Library and TextBlob Utility Functions

In this section, we'll use the **tweet-preprocessor library**

ttps://github.com/s/preprocessor

to perform some basic tweet cleaning. It can automatically remove any combination of:

- URLs,
- @-mentions (like @nasa),
- hashtags (like #mars),
- Twitter reserved words (like, RT for retweet and FAV for favorite, which is similar to a "like" on other social networks),
- emojis (all or just smileys) and
- numbers

The following table shows the module's constants representing each option:

Option	Option constant
@-Mentions (e.g., @nasa)	OPT.MENTION

R

Emoji	`OPT.EMOJI`
Hashtag (e.g., `#mars`)	`OPT.HASHTAG`
Number	`OPT.NUMBER`
Reserved Words (`RT` and `FAV`)	`OPT.RESERVED`
Smiley	`OPT.SMILEY`
URL	`OPT.URL`

Installing tweet-preprocessor

To install tweet-preprocessor, open your Anaconda Prompt (Windows), Terminal (macOS/Linux) or shell (Linux), then issue the following command:

pip install tweet-preprocessor

Windows users might need to run the Anaconda Prompt as an administrator for proper software installation privileges. To do so, right-click Anaconda Prompt in the start menu and select **More > Run as administrator**.

Cleaning a Tweet

Let's do some basic tweet cleaning that we'll use in a later example in this chapter. The tweet-preprocessor library's module name is `preprocessor`. Its documentation recommends that you import the module as follows:

lick here to view code image

```
In [1]: import preprocessor as p
```

C

To set the cleaning options you'd like to use call the module's **`set_options`** function. In this case, we'd like to remove URLs and Twitter reserved words:

lick here to view code image

```
In [2]: p.set_options(p.OPT.URL,   p.OPT.RESERVED)
```

Now let's clean a sample tweet containing a reserved word (RT) and a URL:

lick here to view code image

```
In [3]: tweet_text = 'RT A sample   retweet with a URL https://nasa.gov'

In [4]: p.clean(tweet_text)
Out[4]: 'A sample retweet with a URL'
```

12.13 TWITTER STREAMING API

Twitter's free Streaming API sends to your app *randomly selected* tweets dynamically as they occur—up to a maximum of one percent of the tweets per day. According to `InternetLiveStats.com`, there are approximately 6000 tweets per second, which is over 500 million tweets per day. [3] So the Streaming API gives you access to approximately five million tweets per day. Twitter used to allow free access to 10% of streaming tweets, but this service—called the *fire hose*—is now available only as a paid service. In this section, we'll use a class definition and an IPython session to walk through the steps for processing streaming tweets. Note that the code for receiving a tweet stream requires creating a *custom class* that *inherits* from another class. These topics are covered in hapter 10.

[3] ttp://www.internetlivestats.com/twitter-statistics/.

12.13.1 Creating a Subclass of `StreamListener`

The Streaming API returns tweets as they happen that match your search criteria. Rather than connecting to Twitter on each method call, a stream uses a *persistent* connection to **push** (that is, send) tweets to your app. The rate at which those tweets arrive varies tremendously, based on your search criteria. The more popular a topic is, the more likely it is that the tweets will arrive quickly.

You create a subclass of Tweepy's **`StreamListener` class** to process the tweet stream. An object of this class is the *listener* that's notified when each new tweet (or other

essage sent by Twitter [4]) arrives. Each message Twitter sends results in a call to a `StreamListener` method. The following table summarizes several such methods. `StreamListener` already defines each method, so you redefine only the methods you need—this is known as *overriding*. For additional `StreamListener` methods, see:

[4]For details on the messages, see `ttps://developer.twitter.com/en/docs/tweets/filter-realtime-guides/streaming-message-types.html`.

ttps://github.com/tweepy/tweepy/blob/master/tweepy/streaming.py

Method	Description
`on_connect(self)`	Called when you successfully connect to the Twitter stream. This is for statements that should execute only if your app is connected to the stream.
`on_status(self, status)`	Called when a tweet arrives—`status` is an object of Tweepy's `Status`.
`on_limit(self, track)`	Called when a limit notice arrives. This occurs if your search matches more tweets than Twitter can deliver based on its current streaming rate limits. In this case, the limit notice contains the number of matching tweets that could not be delivered.
`on_error(self, status_code)`	Called in response to error codes sent by Twitter.
`on_timeout(self)`	Called if the connection times out—that is, the Twitter server is not responding.
	Called if Twitter sends a disconnect warning to indicate

`on_warning(self, notice)`	that the connection might be closed. For example, Twitter maintains a queue of the tweets it's pushing to your app. If the app does not read the tweets fast enough, `on_warning`'s `notice` argument will contain a warning message indicating that the connection will terminate if the queue becomes full.

Class `TweetListener`

Our `StreamListener` subclass `TweetListener` is defined in `tweetlistener.py`. We discuss the `TweetListener`'s components here. Line 6 indicates that class `TweetListener` is a subclass of `tweepy.StreamListener`. This ensures that our new class has class `StreamListener`'s default method implementations.

lick here to view code image

```
1 # tweetlistener.py
2 """tweepy.StreamListener subclass that   processes tweets as they arrive.
3 import tweepy
4 from textblob import TextBlob
5
6 class TweetListener(tweepy.StreamListener):
7     """Handles incoming Tweet   stream."""
8
```

Class `TweetListener`: `__init__` Method

The following lines define the `TweetListener` class's `__init__` method, which is called when you create a new `TweetListener` object. The `api` parameter is the Tweepy `API` object that `TweetListener` will use to interact with `Twitter`. The `limit` parameter is the total number of tweets to process—10 by default. We added this parameter to enable you to control the number of tweets to receive. As you'll soon see, we terminate the stream when that `limit` is reached. If you set `limit` to `None`, the stream will not terminate automatically. Line 11 creates an instance variable to keep track of the number of tweets processed so far, and line 12 creates a constant to store the `limit`. If you're not familiar with `__init__` and `super()` from previous chapters, line 13 ensures that the `api` object is stored properly for use by your listener object.

lick here to view code image

C

```
    def __init__(self, api, limit=10):
        """Create instance variables for   tracking number of tweets."""
        self.tweet_count   = 0
        self.TWEET_LIMIT   = limit
        super().__init__(api)  # call   superclass's init

```

Class `TweetListener`: `on_connect` Method

Method **`on_connect`** is called when your app successfully connects to the Twitter stream. We override the default implementation to display a "Connection successful" message.

lick here to view code image

```
    def on_connect(self):
        """Called when your connection attempt   is successful, enabling
        you to perform appropriate application tasks at that   point."""
        print('Connection successful\n')

```

Class `TweetListener`: `on_status` Method

Method **`on_status`** is called by Tweepy when each tweet arrives. This method's second parameter receives a Tweepy `Status` object representing the tweet. Lines 23–26 get the tweet's text. First, we assume the tweet uses the new 280-character limit, so we attempt to access the tweet's `extended_tweet` property and get its `full_text`. An exception will occur if the tweet does not have an `extended_tweet` property. In this case, we get the `text` property instead. Lines 28–30 then display the `screen_name` of the `user` who sent the tweet, the `lang` (that is language) of the tweet and the `tweet_text`. If the language is not English (`'en'`), lines 32–33 use a `TextBlob` to translate the tweet and display it in English. We increment `self.tweet_count` (line 36), then compare it to `self.TWEET_LIMIT` in the return statement. If `on_status` returns `True`, the stream remains open. When `on_status` returns `False`, Tweepy disconnects from the stream.

lick here to view code image

```
    def on_status(self, status):
        """Called when Twitter pushes a new   tweet to you."""
        # get the tweet text
        try:
            tweet_text   = status.extended_tweet.full_text
        except:
            tweet_text   = status.text
```

C

```
27
28          print(f'Screen name: {status.user.screen_name}:')
29          print(f'   Language: {status.lang}')
30          print(f'     Status: {tweet_text}')
31
32          if status.lang != 'en':
33              print(f' Translated: {TextBlob(tweet_text).translate()}')
34
35          print()
36          self.tweet_count   += 1  # track number of   tweets processed
37
38          # if TWEET_LIMIT is reached, return False to   terminate streamin
39          return self.tweet_count != self.TWEET_LIMIT
```

2.13.2 Initiating Stream Processing

Let's use an IPython session to test our new `TweetListener`.

Authenticating

First, you must authenticate with Twitter and create a Tweepy `API` object:

lick here to view code image

```
In [1]: import tweepy

In [2]: import keys

In [3]: auth = tweepy.OAuthHandler(keys.consumer_key,
   ...:                            keys.consumer_secret)
   ...:

In [4]: auth.set_access_token(keys.access_token,
   ...:                       keys.access_token_secret)
   ...:

In [5]: api = tweepy.API(auth, wait_on_rate_limit=True,
   ...:                  wait_on_rate_limit_notify=True)
   ...:
```

Creating a `TweetListener`

Next, create an object of the `TweetListener` class and initialize it with the `api` object:

lick here to view code image

```
In [6]: from tweetlistener import TweetListener
```

```
In [7]: tweet_listener = TweetListener(api)
```

We did not specify the `limit` argument, so this `TweetListener` terminates after 10 tweets.

Creating a `Stream`

A Tweepy **`Stream`** object manages the connection to the Twitter stream and passes the messages to your `TweetListener`. The `Stream` constructor's `auth` keyword argument receives the `api` object's `auth` property, which contains the previously configured `OAuthHandler` object. The `listener` keyword argument receives your listener object:

lick here to view code image

```
In [8]: tweet_stream = tweepy.Stream(auth=api.auth,
   ...:                              listener=tweet_listener)
   ...:
```

Starting the Tweet Stream

The `Stream` object's **`filter` method** begins the streaming process. Let's track tweets about the NASA Mars rovers. Here, we use the `track` parameter to pass a list of search terms:

lick here to view code image

```
In [9]: tweet_stream.filter(track=['Mars   Rover'], is_async=True)
```

The Streaming API will return full tweet JSON objects for tweets that match any of the terms, not just in the tweet's text, but also in @-mentions, hashtags, expanded URLs and other information that Twitter maintains in a tweet object's JSON. So, you might not see the search terms you're tracking if you look only at the tweet's text.

Asynchronous vs. Synchronous Streams

The `is_async=True` argument indicates that `filter` should initiate an **asynchronous tweet stream**. This allows your code to continue executing while your listener waits to receive tweets and is useful if you decide to terminate the stream early. When you execute an asynchronous tweet stream in IPython, you'll see the next `In []` prompt and can terminate the tweet stream by setting the `Stream` object's **`running` property** to `False`, as in:

C

```
tweet_stream.running=False
```

Without the `is_async=True` argument, `filter` initiates a **synchronous tweet stream**. In this case, IPython would display the next `In []` prompt *after* the stream terminates. Asynchronous streams are particularly handy in GUI applications so your users can continue to interact with other parts of the application while tweets arrive. The following shows a portion of the output consisting of two tweets:

lick here to view code image

```
Connection successful

Screen name: bevjoy:
   Language: en
     Status: RT @SPACEdotcom: With Mars Dust Storm   Clearing, Opportunity R

 creen name: tourmaline1973:
   Language: en
     Status: RT @BennuBirdy: Our beloved Mars rover isn't   done yet, but sh


..
```

Other `filter` Method Parameters

Method `filter` also has parameters for refining your tweet searches by Twitter user ID numbers (to follow tweets from specific users) and by location. For details, see:

ttps://developer.twitter.com/en/docs/tweets/filter-realtime/guides/basic-stream-arameters

Twitter Restrictions Note

Marketers, researchers and others frequently store tweets they receive from the Streaming API. If you're storing tweets, Twitter requires you to delete any message or location data for which you receive a deletion message. This will occur if a user deletes a tweet or the tweets location data after Twitter pushes that tweet to you. In each case, your listener's **`on_delete` method** will be called. For deletion rules and message details, see

```
ttps://developer.twitter.com/en/docs/tweets/filter-realtime/guides/streamin
```

2.14 TWEET SENTIMENT ANALYSIS

In the “ atural Language Processing (NLP)” chapter, we demonstrated sentiment analysis on sentences. Many researchers and companies perform sentiment analysis on tweets. For example, political researchers might check tweet sentiment during elections season to understand how people feel about specific politicians and issues. Companies might check tweet sentiment to see what people are saying about their products and competitors’ products.

In this section, we’ll use the techniques introduced in the preceding section to create a script (`sentimentlistener.py`) that enables you to check the sentiment on a specific topic. The script will keep totals of all the positive, neutral and negative tweets it processes and display the results.

The script receives two command-line arguments representing the topic of the tweets you wish to receive and the number of tweets for which to check the sentiment—only those tweets that are not eliminated are counted. For viral topics, there are large numbers of retweets, which we are not counting, so it could take some time get the number of tweets you specify. You can run the script from the `ch12` folder as follows:

lick here to view code image

```
ipython sentimentlistener.py football 10
```

which produces output like the following. Positive tweets are preceded by a +, negative tweets by a – and neutral tweets by a space:

lick here to view code image

```
  ftblNeutral: Awful game of football. So boring slow   hoofball complete was

+ TBulmer28: I've seen 2 successful onside kicks within a   40 minute span. I

+ CMayADay12: The last normal Sunday for the next couple   months. Don't text

  rpimusic: My heart legitimately hurts for Kansas football   fans

+ DSCunningham30: @LeahShieldsWPSD It's awsome that u like   college football

  damanr: I'm bummed I don't know enough about football to   roast @samesfanc

+ jamesianosborne: @TheRochaSays @WatfordFC @JackHind   Haha.... just when yo


+ Tshanerbeer: @PennStateFball @PennStateOnBTN Ah yes,   welcome back college

- cougarhokie: @hokiehack @skiptyler I can verify the   badness of that footb
```

```
+ Unite_Reddevils: @Pablo_di_Don Well make yourself clear   it's football not

Tweet sentiment for "football"
Positive: 6
 Neutral: 2
Negative: 2
```

he script (`sentimentlistener.py`) is presented below. We focus only on the new capabilities in this example.

Imports

Lines 4–8 import the `keys.py` file and the libraries used throughout the script:

lick here to view code image

```
1 # sentimentlisener.py
2 """Script that searches for tweets that match   a search string
3 and tallies the number of positive, neutral and negative   tweets."""
4 import keys
5 import preprocessor as p
6 import sys
7 from textblob import TextBlob
8 import tweepy
9
```

Class `SentimentListener`: `__init__` Method

In addition to the `API` object that interacts with Twitter, the `__init__` method receives three additional parameters:

- `sentiment_dict`—a dictionary in which we'll keep track of the tweet sentiments,
- `topic`—the topic we're searching for so we can ensure that it appears in the tweet text and
- `limit`—the number of tweets to process (not including the ones we eliminate).

Each of these is stored in the current `SentimentListener` object (`self`).

lick here to view code image

```
10 class SentimentListener(tweepy.StreamListener):
11     """Handles incoming Tweet   stream."""
```

```
12
13      def __init__(self, api, sentiment_dict, topic, limit=10):
14          """Configure the   SentimentListener."""
15          self.sentiment_dict    = sentiment_dict
16          self.tweet_count    = 0
17          self.topic    = topic
18          self.TWEET_LIMIT = limit
19
20          # set tweet-preprocessor to remove URLs/reserved    words
21          p.set_options(p.OPT.URL, p.OPT.RESERVED)
22          super().__init__(api)  # call    superclass's init
23
```

Method `on_status`

When a tweet is received, method `on_status`:

- gets the tweet's text (lines 27–30)
- skips the tweet if it's a retweet (lines 33–34)
- cleans the tweet to remove URLs and reserved words like RT and FAV (line 36)
- skips the tweet if it does not have the topic in the tweet text (lines 39–40)
- uses a `TextBlob` to check the tweet's sentiment and updates the `sentiment_dict` accordingly (lines 43–52)
- prints the tweet text (line 55) preceded by + for positive sentiment, space for neutral sentiment or - for negative sentiment and
- checks whether we've processed the specified number of tweets yet (lines 57–60).

lick here to view code image

```
24      def on_status(self, status):
25          """Called when Twitter pushes a new   tweet to you."""
26          # get the tweet's text
27          try:
28              tweet_text    = status.extended_tweet.full_text
29          except:
30              tweet_text    = status.text
31
32          # ignore retweets
33          if tweet_text.startswith('RT'):
34              return
35
36          tweet_text    = p.clean(tweet_text)  # clean the tweet
```

C

```
37
38          # ignore tweet if the topic is not in the tweet   text
39          if self.topic.lower() not in   tweet_text.lower():
40              return
41
42          # update self.sentiment_dict with the polarity
43          blob =   TextBlob(tweet_text)
44          if blob.sentiment.polarity > 0:
45              sentiment   = '+'
46              self.sentiment_dict['positive'] += 1
47          elif blob.sentiment.polarity == 0:
48              sentiment   = ' '
49              self.sentiment_dict['neutral'] += 1
50          else:
51              sentiment   = '-'
52              self.sentiment_dict['negative'] += 1
53
54          # display the tweet
55          print(f'{sentiment} {status.user.screen_name}: {tweet_text}\n')
56
57          self.tweet_count   += 1  # track number of tweets   processed
58
59          # if TWEET_LIMIT is reached, return False to   terminate streamir
60          return self.tweet_count != self.TWEET_LIMIT
61
```

ain Application

The main application is defined in the function `main` (lines 62–87; discussed after the following code), which is called by lines 90–91 when you execute the file as a script. So `sentiment-listener.py` can be imported into IPython or other modules to use class `SentimentListener` as we did with `TweetListener` in the previous section:

lick here to view code image

```
62 def main():
63     # configure the OAuthHandler
64     auth =   tweepy.OAuthHandler(keys.consumer_key, keys.consumer_secret)
65     auth.set_access_token(keys.access_token,   keys.access_token_secret)
66
67     # get the API object
68     api =   tweepy.API(auth, wait_on_rate_limit=True,
69                        wait_on_rate_limit_notify=True)
70
71     # create the StreamListener subclass object
72     search_key   = sys.argv[1]
73     limit =   int(sys.argv[2])  #   number of tweets to tally
74     sentiment_dict   = {'positive': 0, 'neutral': 0, 'negative': 0}
75     sentiment_listener   = SentimentListener(api,
76         sentiment_dict,   search_key, limit)
```

```
77
78      # set up Stream
79      stream =    tweepy.Stream(auth=api.auth, listener=sentiment_listener)
80
81      # start filtering English tweets containing search_key
82      stream.filter(track=[search_key],    languages=['en'], is_async=False)
83
84      print(f'Tweet sentiment for "{search_key}"')
85      print('Positive:', sentiment_dict['positive'])
86      print(' Neutral:', sentiment_dict['neutral'])
87      print('Negative:', sentiment_dict['negative'])
88
89  # call main if this file is executed as a script
90  if __name__ == '__main__':
91      main()
```

ines 72–73 get the command-line arguments. Line 74 creates the `sentiment_dict` dictionary that keeps track of the tweet sentiments. Lines 75–76 create the `SentimentListener`. Line 79 creates the `Stream` object. We once again initiate the stream by calling `Stream` method `filter` (line 82). However, this example uses a synchronous stream so that lines 84–87 display the sentiment report only after the specified number of tweets (`limit`) are processed. In this call to `filter`, we also provided the keyword argument `languages`, which specifies a list of language codes. The one language code `'en'` indicates Twitter should return only English language tweets.

12.15 GEOCODING AND MAPPING

In this section, we'll collect streaming tweets, then plot the locations of those tweets. Most tweets do not include latitude and longitude coordinates, because Twitter disables this by default for all users. Those who wish to include their precise location in tweets must opt into that feature. Though most tweets do not include precise location information, a large percentage include the user's home location information; however, even that is sometimes invalid, such as "Far Away" or a fictitious location from a user's favorite movie.

In this section, for simplicity, we'll use the `location` property of the tweet's `User` object to plot that user's location on an interactive map. The map will let you zoom in and out and drag to move the map around so you can look at different areas (known as *panning*). For each tweet, we'll display a map marker that you can click to see a popup containing the user's screen name and tweet text.

We'll ignore retweets and tweets that do not contain the search topic. For other tweets, we'll track the percentage of tweets with location information. When we get the latitude and longitude information for those locations, we'll also track the percentage of those tweets that had invalid location data.

L

geopy Library

We'll use the **geopy library** (`ttps://github.com/geopy/geopy`) to translate locations into latitude and longitude coordinates—known as **geocoding**—so we can place markers on a map. The library supports dozens of geocoding web services, many of which have free or lite tiers. For this example, we'll use the **OpenMapQuest geocoding service** (discussed shortly). You installed geopy in ection 12.6.

OpenMapQuest Geocoding API

We'll use the OpenMapQuest Geocoding API to convert locations, such as Boston, MA into their latitudes and longitudes, such as 42.3602534 and -71.0582912, for plotting on maps. OpenMapQuest currently allows 15,000 transactions per month on their free tier. To use the service, first sign up at

ttps://developer.mapquest.com/

Once logged in, go to

```
ttps://developer.mapquest.com/user/me/apps
```

and click **Create a New Key**, fill in the **App Name** field with a name of your choosing, leave the **Callback URL** empty and click **Create App** to create an API key. Next, click your app's name in the web page to see your consumer key. In the `keys.py` file you used earlier in the chapter, store the consumer key by replacing *YourKeyHere* in the line

mapquest_key = '*YourKeyHere*'

As we did earlier in the chapter, we'll import `keys.py` to access this key.

Folium Library and Leaflet.js JavaScript Mapping Library

For the maps in this example, we'll use the **folium library**

ttps://github.com/python-visualization/folium

which uses the popular Leaflet.js JavaScript mapping library to display maps. The maps that folium produces are saved as HTML files that you can view in your web browser. To install folium, execute the following command:

pip install folium

Maps from OpenStreetMap.org

By default, Leaflet.js uses open source maps from `OpenStreetMap.org`. These maps are copyrighted by the OpenStreetMap.org contributors. To use these maps [5], they require the following copyright notice:

[5] `ttps://wiki.osmfoundation.org/wiki/Licence/Licence_and_Legal_FAQ.`

lick here to view code image

```
Map data © OpenStreetMap contributors
```

and they state:

You must make it clear that the data is available under the Open Database License. This can be achieved by providing a "License" or "Terms" link which links to *`ww.openstreetmap.org/copyright`* or *`ww.opendatacommons.org/licenses/odbl`*.

12.15.1 Getting and Mapping the Tweets

Let's interactively develop the code that plots tweet locations. We'll use utility functions from our `tweetutilities.py` file and class `LocationListener` in `locationlistener.py`. We'll explain the details of the utility functions and class in the subsequent sections.

Get the API Object

As in the other streaming examples, let's authenticate with Twitter and get the Tweepy `API` object. In this case, we do this via the `get_API` utility function in `tweetutilities.py`:

lick here to view code image

```
In [1]: from tweetutilities import get_API

In [2]: api = get_API()
```

Collections Required By `LocationListener`

Our `LocationListener` class requires two collections: A list (`tweets`) to store the tweets we collect and a dictionary (`counts`) to track the total number of tweets we collect and the number that have location data: